Universidad Peruana de Ciencias Aplicadas

Facultad de Ingeniería
Carrera: Ingeniería de Software

Periodo: 202620
Codigo del curso: 1ASI057
Nombre del curso: Desarrollo de Soluciones IoT
NRC: 8725

Nombre del profesor: Marco Antonio León Baca

Informe de Trabajo Final

Nombre del startup: FreshEat
Nombre del producto: FreshSense

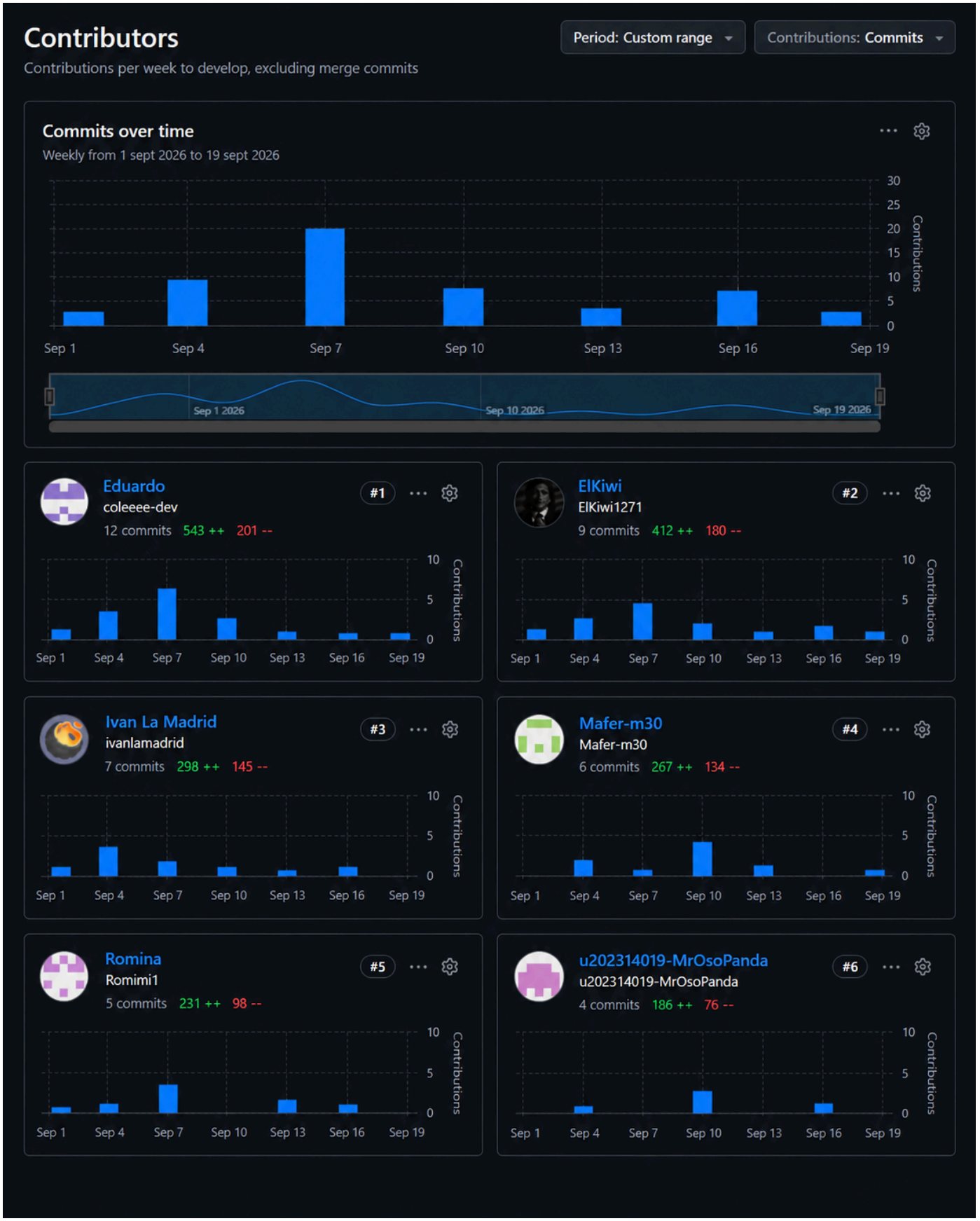
Relación de Integrantes

Apellidos y Nombres	Código
Tuesta Marin, Romina Alejandra	U202211706
Cossar Sanchez, Eduardo Jose	U202312109
Mostajo Orosco, Maria Fernanda	U202312874
Chavez Viera, Joseph Manuel	u202314019
Pastor Napa, Juan Carlos	u202217288

Septiembre 2026

Project Report Collaboration Insights

AV1:



Registro de Versiones del Informe

Versión	Fecha	Autor	Descripción de modificación
1.0	19/09/2026	Todos los Integrantes	Se realizo los capitulos del 1 - 4
1.1	06/10/2026	Todos los Integrantes	Se completaron los tres escenarios de cada historia de usuario y las secciones 6.1, 6.1.2, 6.2.1.3 y 6.2.1.6
1.2	07/10/2026	Joseph	Se agregaron las capturas del recorrido del sprint 1 en 6.2.1.6
1.3	07/10/2026	Joseph	En 6.2.1.6 se deja la landing publicada y el front publico pendiente del despliegue
1.4	07/10/2026	Joseph	En 6.2.1.6 quedan el enlace del back y el de la landing; el del front se reemplaza cuando lo pasen
1.5	07/10/2026	Joseph	Se agregó el perfil de Joseph y su aporte en el Student Outcome
1.6	08/10/2026	Joseph	Se colocaron los enlaces del front y del back, el tablero de EventStorming y las fichas nuevas de empatía y recorrido
1.7	08/10/2026	Joseph	Se integró el capítulo V, el diseño de software alineado a los segmentos, el entorno y el despliegue, las convenciones de código y las entrevistas 3
1.8	08/10/2026	Joseph	Se corrigió la figura de Impact Mapping para que se vea en el informe

Tabla de contenidos

- Student Outcome
- Capítulo I: Introducción
  - 1.1. Startup Profile
    - 1.1.1. Descripción de la Startup
    - 1.1.2. Perfiles de integrantes del equipo
  - 1.2. Solution Profile
    - 1.2.1. Antecedentes y problemática
    - 1.2.2. Lean UX Process
      - 1.2.2.1. Lean UX Problem Statements
      - 1.2.2.2. Lean UX Assumptions
      - 1.2.2.3. Lean UX Hypothesis Statements
      - 1.2.2.4. Lean UX Canvas
  - 1.3. Segmentos objetivo
- Capítulo II: Requirements Elicitation & Analysis
  - 2.1. Competidores
    - 2.1.1. Análisis competitivo
    - 2.1.2. Estrategias y tácticas frente a competidores
  - 2.2. Entrevistas
    - 2.2.1. Diseño de entrevistas
    - 2.2.2. Registro de entrevistas
    - 2.2.3. Análisis de entrevistas
  - 2.3. Needfinding
    - 2.3.1. User Personas
    - 2.3.2. User Task Matrix
    - 2.3.3. User Journey Mapping
    - 2.3.4. Empathy Mapping
  - 2.4. Big Picture EventStorming
  - 2.5. Ubiquitous Language
- Capítulo III: Requirements Specification
  - 3.1. User Stories
  - 3.2. Impact Mapping
  - 3.3. Product Backlog
- Capítulo IV: Solution Software Design
  - 4.1. Strategic-Level Domain-Driven Design
    - 4.1.1. Design-Level EventStorming
      - 4.1.1.1. Candidate Context Discovery
      - 4.1.1.2. Domain Message Flows Modeling
      - 4.1.1.3. Bounded Context Canvases
    - 4.1.2. Context Mapping

        - 4.1.3. Software Architecture
            - 4.1.3.1. Software Architecture System Landscape Diagram
            - 4.1.3.2. Software Architecture Context Level Diagrams
            - 4.1.3.2. Software Architecture Container Level Diagrams
            - 4.1.3.3. Software Architecture Deployment Diagrams
    - 4.2. Tactical-Level Domain-Driven Design
        - 4.2.X. Bounded Context: <Bounded Context Name>
            - 4.2.X.1. Domain Layer
            - 4.2.X.2. Interface Layer
            - 4.2.X.3. Application Layer
            - 4.2.X.4. Infrastructure Layer
            - 4.2.X.5. Bounded Context Software Architecture Component Level Diagrams
            - 4.2.X.6. Bounded Context Software Architecture Code Level Diagrams
                - 4.2.X.6.1. Bounded Context Domain Layer Class Diagrams
                - 4.2.X.6.2. Bounded Context Database Design Diagram

- Capítulo V: Solution UI/UX Design
    - 5.1. Style Guidelines
        - 5.1.1. General Style Guidelines
        - 5.1.2. Web, Mobile and IoT Style Guidelines
    - 5.2. Information Architecture
        - 5.2.1. Organization Systems
        - 5.2.2. Labeling Systems
        - 5.2.3. SEO Tags and Meta Tags
        - 5.2.4. Searching Systems
        - 5.2.5. Navigation Systems
    - 5.3. Landing Page UI Design
        - 5.3.1. Landing Page Wireframe
        - 5.3.2. Landing Page Mock-up
    - 5.4. Applications UX/UI Design
        - 5.4.1. Applications Wireframes
        - 5.4.2. Applications Wireflow Diagrams
        - 5.4.2. Applications Mock-ups
        - 5.4.3. Applications User Flow Diagrams
    - 5.5. Applications Prototyping
    - 5.6. IoT Device Design

- Capítulo VI: Product Implementation, Validation & Deployment
    - 6.1. Software Configuration Management
        - 6.1.1. Software Development Environment Configuration
        - 6.1.2. Source Code Management
        - 6.1.3. Source Code Style Guide & Conventions
        - 6.1.4. Software Deployment Configuration
    - 6.2. Landing Page, Services & Applications Implementation
        - 6.2.1. Sprint 1
            - 6.2.1.1. Sprint Planning 1
            - 6.2.1.2. Aspect Leaders and Collaborators
            - 6.2.1.3. Sprint Backlog 1
            - 6.2.1.4. Development Evidence for Sprint Review
            - 6.2.1.5. Testing Suite Evidence for Sprint Review
            - 6.2.1.6. Execution Evidence for Sprint Review
            - 6.2.1.7. Services Documentation Evidence for Sprint Review
            - 6.2.1.8. Software Deployment Evidence for Sprint Review
            - 6.2.1.9. Team Collaboration Insights during Sprint
    - 6.3. Validation Interviews
        - 6.3.1. Diseño de Entrevistas
        - 6.3.2. Registro de Entrevistas
        - 6.3.3. Evaluaciones según heurísticas
    - 6.4. Video About-the-Product

- Conclusiones
- Conclusiones y recomendaciones
- Video About-the-Team
- Bibliografía
- Anexos

Student Outcome

El curso contribuye al cumplimiento del Student Outcome ABET: ABET – EAC - Student Outcome 5 Criterio: La capacidad de funcionar efectivamente en un equipo cuyos miembros juntos proporcionan liderazgo, crean un entorno de colaboración e inclusivo, establecen objetivos, planifican tareas y cumplen objetivos. En el siguiente cuadro se describe las acciones realizadas y enunciados de conclusiones por parte del grupo, que permiten sustentar el haber alcanzado el logro del ABET – EAC - Student Outcome 5.

Criterio específico	Acciones realizadas	Conclusiones
Trabaja en equipo para proporcionar liderazgo en forma conjunta	**AV1:** **Romina Tuesta Marin:** Se distribuyeron responsabilidades entre los integrantes según sus habilidades, coordinando las actividades de análisis, diseño, desarrollo y documentación. Asimismo, se tomaron decisiones de manera conjunta y se realizó seguimiento al avance de cada tarea. **Eduardo Cossar:** Participó activamente en la organización y distribución de las responsabilidades del equipo, apoyando en las actividades de análisis, diseño y documentación del proyecto. Asimismo, coordinó con los demás integrantes para revisar los avances. **Maria Fernanda Mostajo:** Contribuyó al desarrollo del proyecto mediante la coordinación permanente con los integrantes del equipo, proponiendo alternativas ante los problemas encontrados y apoyando en la organización de las actividades pendientes. **Joseph Manuel Chavez Viera:** Se distribuyeron responsabilidades entre los integrantes según sus habilidades, coordinando las actividades de análisis, diseño, desarrollo y documentación. Asimismo, se tomaron decisiones de manera conjunta y se realizó seguimiento al avance de cada tarea. Además tomé la gestión de la configuración y del código (6.1 y 6.1.2), el sprint backlog 1 (6.2.1.3) y la evidencia de ejecución (6.2.1.6). Lo coordiné con el equipo para los dos segmentos, propietarios y administradores de restaurantes y encargados de negocios de distribución de alimentos en fríos, dejando el mismo recorrido para los dos. Cuando Romina indicó que la landing ya estaba publicada y que el front lo seguía desplegando Juan Carlos, ajusté esa evidencia: quedó el enlace del back y el de la landing, y el del front pendiente para cambiarlo cuando nos lo pasen. **Juan Carlos Pastor Napa:** Enriqueció las decisiones de arquitectura de software del proyecto, adaptando y ajustando el diseño propuesto según las necesidades del equipo y los requerimientos del curso. **TB1:** **Romina Tuesta Marin:** Lideró y coordinó las actividades relacionadas con el Landing Page y los componentes IoT de FreshSense, colaborando con los demás integrantes para integrar sus avances al proyecto. Asimismo, apoyó la toma de decisiones técnicas y la organización de los entregables del Sprint 1. **Eduardo Cossar:** Participó en la coordinación de las actividades del Sprint 1, contribuyendo al desarrollo de las entrevistas de los segmentos objetivo y a la documentación de las evidencias de desarrollo, pruebas y despliegue. Asimismo, colaboró con el equipo para mantener la coherencia entre los avances realizados y los requerimientos del proyecto. **Maria Fernanda Mostajo:** Colaboró en las actividades relacionadas con el monitoreo y las alertas de FreshSense, coordinando con los integrantes encargados de los distintos componentes de la solución. Asimismo, contribuyó a la revisión de avances y a la identificación de aspectos que requerían ajustes. **Joseph Manuel Chavez Viera:** Asumió responsabilidades en la organización del Sprint Backlog y la documentación de las evidencias de ejecución. Coordinó con sus compañeros para integrar la información del Landing Page, Frontend y Backend, manteniendo la coherencia de las funcionalidades con los dos segmentos empresariales definidos. **Juan Carlos Pastor Napa:** Contribuyó al liderazgo técnico del equipo mediante la preparación y configuración del entorno de despliegue de FreshSense en Google Cloud. Coordinó las actividades relacionadas con el Backend, Frontend y servidor Nginx, aportando soluciones técnicas para facilitar la publicación de los componentes.	**AV1:** El trabajo colaborativo permitió aprovechar las habilidades de cada integrante, mantener una participación activa y avanzar de manera coordinada hacia los objetivos del proyecto. **TB1:** El liderazgo compartido permitió integrar los avances de desarrollo, pruebas, documentación y despliegue de FreshSense. La coordinación entre los responsables de cada componente facilitó la toma de decisiones conjuntas y contribuyó al cumplimiento de los objetivos establecidos para el Sprint 1.
Crea un entorno colaborativo e inclusivo, establece metas, planifica tareas y cumple objetivos	**AV1:** **Romina Tuesta Marin:** Se establecieron objetivos y tareas para cada integrante, organizando las actividades de acuerdo con las prioridades del proyecto. Se mantuvo una comunicación constante para resolver dudas, compartir avances y realizar ajustes cuando fue necesario.	**AV1:** La planificación y comunicación constante facilitaron la coordinación del equipo, permitiendo cumplir las actividades asignadas y mantener el avance del

Criterio específico	Acciones realizadas	Conclusiones
	Eduardo Cossar: Colaboró en la definición de las tareas y metas del equipo, organizando sus actividades de acuerdo con las prioridades establecidas. Además, mantuvo una comunicación constante con los integrantes para compartir avances. **Maria Fernanda Mostajo:** Participó en la planificación de las actividades del equipo, definiendo tiempos de trabajo y priorizando las tareas necesarias para cada etapa del proyecto. Asimismo, promovió el intercambio de ideas entre los integrantes y realizó seguimiento a los compromisos asumidos. **Joseph Manuel Chavez Viera:** Se establecieron objetivos y tareas para cada integrante, organizando las actividades de acuerdo con las prioridades del proyecto. Se mantuvo una comunicación constante para resolver dudas, compartir avances y realizar ajustes cuando fue necesario. Además organicé las 11 historias del sprint 1 y armé la evidencia con las capturas del recorrido, el Swagger del back y la landing. Fui cerrando esa parte con lo que el equipo ya tenía definido y dejé anotado el enlace del front para reemplazarlo apenas nos lo entreguen. **Juan Carlos Pastor Napa:** Enriqueció la planificación de tareas relacionadas con el diseño de la solución, manteniendo comunicación constante con el equipo para ajustar el trabajo según los avances. **TB1:** **Romina Tuesta Marin:** Participó en la planificación de las actividades correspondientes al Landing Page y a los componentes IoT, estableciendo prioridades y coordinando los avances con el equipo. Asimismo, mantuvo una comunicación activa para identificar tareas pendientes y contribuir al cumplimiento de las metas del Sprint 1. **Eduardo Cossar:** Organizó sus actividades de acuerdo con las responsabilidades asignadas, priorizando la realización de entrevistas y la elaboración de evidencias para el informe. Además, mantuvo comunicación con sus compañeros para verificar la información disponible y coordinar los aspectos pendientes antes de la entrega TB1. **Maria Fernanda Mostajo:** Contribuyó a la organización y seguimiento de las tareas de monitoreo y alertas, participando en la coordinación de los avances y en la revisión de las funcionalidades consideradas dentro del Sprint. Asimismo, promovió el intercambio de información para mantener el trabajo alineado con los objetivos del equipo. **Joseph Manuel Chavez Viera:** Organizó las 11 historias seleccionadas para el Sprint 1 y contribuyó al seguimiento de sus avances mediante el Sprint Backlog. Asimismo, recopiló y actualizó la documentación de ejecución, incorporando las evidencias disponibles y coordinando con los responsables de cada componente para completar la información pendiente. **Juan Carlos Pastor Napa:** Planificó las actividades técnicas necesarias para el despliegue de la solución, incluyendo la preparación del servidor, la transferencia de archivos ejecutables y la configuración de Nginx. Además, coordinó con el equipo la verificación de accesibilidad de los componentes desplegados, contribuyendo al cumplimiento de las metas técnicas del Sprint.	proyecto de acuerdo con los objetivos establecidos. **TB1:** La planificación de tareas, la comunicación constante y el seguimiento de responsabilidades permitieron organizar el trabajo del Sprint 1 de manera colaborativa. Esto facilitó la integración de los avances de cada integrante, la identificación de pendientes y la preparación de los entregables correspondientes a TB1.

Capítulo I: Introducción

1.1. Startup Profilee

1.1.1. Descripción de la Startup

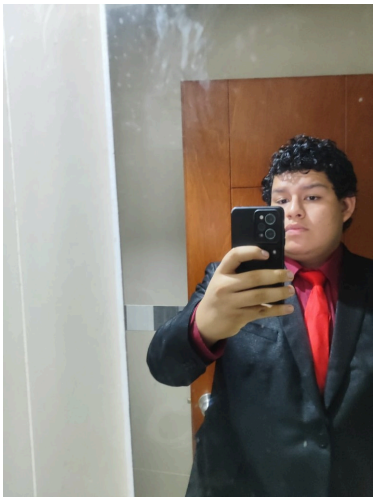
FreshSense es una startup de tecnología orientada a ayudar a restaurantes y negocios de alimentos fríos a reducir las pérdidas económicas ocasionadas por el deterioro y desperdicio de productos perecibles. La solución combina una aplicación de gestión de inventario con dispositivos de monitoreo para supervisar las condiciones de conservación de los alimentos y generar alertas oportunas ante situaciones que puedan afectar su calidad.

FreshSense permite a los negocios conocer el estado de sus productos, gestionar su inventario y recibir información que facilite la toma de decisiones sobre conservación, rotación y aprovechamiento de los alimentos. De esta manera, busca reducir las mermas, disminuir costos innecesarios y contribuir a mejorar la rentabilidad de los negocios.

La startup plantea un modelo de negocio basado en la comercialización de los dispositivos de monitoreo y una suscripción para acceder a funcionalidades avanzadas de la plataforma, como reportes, análisis del inventario y seguimiento de pérdidas.

1.1.2. Perfiles de integrantes del equipo

	Romina Alejandra Tuesta Marin - u202211706 estudio Ingeniería de Software en la UPC. Me considero una persona organizada, responsable y comprometida con el trabajo en equipo. Me interesan el desarrollo web y el diseño de soluciones tecnológicas que resuelvan problemas reales. Como integrante del equipo, me comprometo a apoyar en la planificación y coordinación del proyecto para cumplir los objetivos en los tiempos establecidos.
	Juan Carlos Pastor Napa - u202217288 Mi nombre es Juan Carlos Pastor Napa. Soy estudiante de la carrera de Ingeniería de Software en la UPC, con conocimientos en Java, Spring Boot, Angular y en el despliegue de aplicaciones en Linux y la nube. Me considero una persona responsable y analítica, con interés por la arquitectura de software. Como integrante del equipo, me comprometo a aportar en el diseño de la solución y a apoyar a mis compañeros para cumplir los objetivos del proyecto.
	Eduardo Cossar - u202312109 Mi nombre es Eduardo Cossar. Soy estudiante de la carrera de Ingeniería de Software, tengo 20 años y actualmente estoy cursando el septimo ciclo en la UPC. Me considero una persona responsable y comprometida con un gran interés por la tecnología. Como integrante de este equipo, me comprometo a brindar todo mi apoyo y participación activa para afrontar los desafíos que se presenten y dar lo mejor de mí para lograr el éxito de este proyecto.
	Maria Fernanda Mostajo - u202312874 Mi nombre es Maria Fernanda Mostajo, estoy estudiando la carrera de Ingeniería de Software en la UPC, tengo conocimientos en los lenguajes de programación C++, Python, HTML, CSS, JavaScript y SQL. Además, cuento con habilidades de trabajo en equipo, el cual me permitira realizar un buen trabajo y cumplir con los objetivos planteados en el tiempo establecido.

	**Joseph Manuel Chavez Viera - U202314019** Soy un estudiante de séptimo ciclo de Ingeniería de Software en la UPC, con manejo de C++, Python y Java. Me considero una persona adaptable, comunicativa y con gran gusto por el trabajo en equipo. Apuesto por la mejora continua y el aprendizaje constante, buscando siempre involucrarme en proyectos que me pongan a prueba y me permitan aportar valor real mientras sigo sumando experiencia.

1.2. Solution Profile

1.2.1 Antecedentes y problemática

El desperdicio de alimentos es un problema grave en el Perú tanto a nivel económico como ambiental. Según la información explicada en el II Foro Nacional de la Recuperación de Alimentos y Conmemoración del Día de Concienciación sobre la Pérdida y el Desperdicio de Alimentos en el Perú, realizado por la FOA (Organización de la Naciones Unidas para la Alimentación y la Agricultura) junto con MIDAGRI (Ministerio de Desarrollo Agrario y Riego) y PMA (Programa Mundial de Alimentos), en el país, 12.8 millones de toneladas de alimentos se pierden en la cadena de producción (FOA, 2024). Es un número muy preocupante considerando que el 51% de las familias peruanas está en situación de inseguridad alimentaria (Blas, 2022). A partir de ello, es claro que hay una relaciónestrecha entre el desperdicio de alimentos y la inseguridad alimentaria; el gobierno se está encargando de mitigar tal desperdicio mediante foros, proyectos, etc. Teniendo en cuenta este contexto, Enrique Román, representante asistente de FAO en Perú señala que los hogares peruanos representan el 16% en el desperdicio de alimentos (FAO, 2024), un porcentaje alarmante. Más aún, acorde a Alberto Huiman, doctor en Ciencias Ambientales, cada peruano genera un desperdicio de 67 kilogramos por año y los restaurantes es alrededor de 500 kilogramos por día. Esto complica gravemente la situación del país respecto a las pérdidas en el aspecto de alimentos.

Para analizar con más detalle los antecedentes y problemáticas, se realizó con anticipación la técnica 5 ‘W’s & 2 ‘H’s:

What? (¿Qué es?)

FreshSense es una aplicación web orientada a restaurantes y negocios de alimentos fríos que permite monitorear y gestionar el inventario de productos perecibles. Mediante alertas y notificaciones, ayuda a detectar oportunamente condiciones que pueden ocasionar el deterioro de los alimentos, facilitando decisiones para reducir mermas y pérdidas económicas.

Why? (¿Por qué?)

La falta de información precisa y oportuna sobre las condiciones de conservación de los alimentos dificulta detectar su deterioro antes de que se convierta en una pérdida. Esto puede generar desperdicio de productos, costos de reposición y una reducción de la rentabilidad del negocio.

Where? (¿Dónde?)

FreshSense está orientada a restaurantes y negocios de alimentos fríos, principalmente en espacios donde se almacenan productos perecibles, como refrigeradores, congeladores, cámaras de frío y almacenes.

When? (¿Cuándo?)

El problema ocurre durante el almacenamiento y conservación de los productos, especialmente cuando permanecen durante periodos prolongados sin un monitoreo adecuado de sus condiciones. FreshSense permite realizar un seguimiento continuo y recibir alertas para tomar decisiones oportunas.

Who? (¿Quién?)

Los principales usuarios de FreshSense son los propietarios y administradores de restaurantes, y los encargados de negocios de distribución de alimentos en fríos. Los dos necesitan controlar la conservación de sus productos, reducir mermas y cuidar la rentabilidad del negocio.

How? (¿Cómo?)

FreshSense busca facilitar la gestión y conservación de los productos mediante:

- Registro y gestión del inventario de alimentos.
- Monitoreo de condiciones de conservación mediante sensores.
- Alertas ante condiciones que puedan afectar los productos.
- Notificaciones sobre productos próximos a deteriorarse o vencer.
- Reportes sobre pérdidas, rotación y estado del inventario.
- Sugerencias para aprovechar productos antes de que representen una pérdida económica.
- Acceso a la información desde diferentes dispositivos.

How much? (¿Cuánto?)

El desperdicio y deterioro de alimentos representa una pérdida económica para los negocios, ya que implica desechar productos que fueron adquiridos para su comercialización o utilización. FreshSense busca reducir estas pérdidas mediante el monitoreo y gestión preventiva del inventario.

El modelo de negocio considera:

- Suscripción mensual o anual para acceder a funcionalidades avanzadas.
- Venta del dispositivo sensor como pago único.
- Funcionalidades premium orientadas al análisis del inventario y las pérdidas económicas.

1.2.2 Lean UX Process.

1.2.2.1. Lean UX Problem Statements.

Los restaurantes y negocios de distribución de alimentos fríos pueden sufrir pérdidas económicas debido al deterioro de productos perecibles que no es detectado oportunamente. La falta de monitoreo continuo de las condiciones de conservación dificulta tomar decisiones preventivas sobre el inventario, generando mermas, costos de reposición y reducción de la rentabilidad.

La gestión manual del inventario y las condiciones de conservación también dificulta conocer en tiempo real qué productos presentan mayor riesgo de deterioro y cuáles requieren atención prioritaria.

Creemos que reducir el desperdicio de alimentos permitirá a los negocios disminuir sus pérdidas económicas y mejorar su rentabilidad. **Sabremos que esto es cierto cuando** se observe una reducción medible de las pérdidas asociadas al deterioro de productos durante el periodo de evaluación.

Creemos que las alertas sobre condiciones de conservación y productos próximos a deteriorarse ayudarán a los usuarios a tomar decisiones oportunas. **Sabremos que esto es cierto cuando** los usuarios utilicen las alertas para realizar acciones preventivas sobre los productos identificados.

Creemos que permitir el acceso a la aplicación desde diferentes dispositivos facilitará el monitoreo del inventario durante la operación diaria. **Sabremos que esto es cierto cuando** los usuarios consulten regularmente el estado de sus productos desde al menos dos tipos de dispositivos.

Creemos que los reportes de pérdidas y rotación de inventario ayudarán a los administradores a identificar oportunidades para reducir mermas. **Sabremos que esto es cierto cuando** los usuarios consulten estos reportes para tomar decisiones relacionadas con su inventario.

1.2.2.2. Lean UX Assumptions.

Business Assumptions

- Creemos que nuestros usuarios necesitan una solución que les permita monitorear las condiciones de conservación de sus productos y gestionar su inventario para reducir pérdidas económicas.
- Estas necesidades pueden resolverse mediante una aplicación que centralice el inventario, las alertas de conservación y los reportes de pérdidas en una interfaz sencilla y accesible.
- Nuestros clientes iniciales son restaurantes y negocios de alimentos fríos que manejan productos perecibles y buscan reducir mermas y proteger la rentabilidad de sus operaciones.
- El principal valor que el cliente requiere de FreshSense es conocer oportunamente el estado de sus productos para tomar decisiones antes de que estos representen una pérdida económica.
- Como beneficios adicionales, el cliente podrá acceder a reportes de inventario, análisis de pérdidas, alertas preventivas y recomendaciones para aprovechar productos próximos a deteriorarse.
- Adquiriremos clientes mediante una landing page, demostraciones del producto, marketing digital y estrategias de venta directa orientadas a los segmentos objetivo.
- Generaremos ingresos mediante la venta del dispositivo sensor y suscripciones para acceder a funcionalidades avanzadas.
- Nuestra competencia incluye métodos manuales de control de inventario, hojas de cálculo y sistemas genéricos de gestión de inventario.
- Nos diferenciaremos mediante una solución especializada en la conservación de productos perecibles, que combine monitoreo, alertas y gestión del inventario.
- Nuestros principales riesgos son la resistencia al cambio tecnológico, la preferencia por métodos manuales, el costo percibido de la solución y la dificultad para incorporar el sistema en las actividades diarias del negocio.
- Reduciremos estos riesgos mediante una interfaz sencilla, procesos de registro rápidos, funcionalidades enfocadas en las necesidades principales del usuario y una implementación que requiera la menor cantidad posible de cambios en su rutina.
- Sabremos que hemos tenido éxito cuando los negocios usuarios reduzcan sus pérdidas asociadas al deterioro de alimentos y utilicen regularmente la aplicación para gestionar su inventario.

Business Outcomes

- Reducir las pérdidas económicas ocasionadas por el deterioro de alimentos.
- Disminuir la merma de productos perecibles.
- Mejorar la rentabilidad de los negocios mediante una gestión eficiente del inventario.
- Lograr la adopción y retención de usuarios mediante una solución simple y útil.
- Generar ingresos recurrentes mediante la venta del dispositivo y suscripción premium.
- Validar un modelo de negocio escalable para restaurantes y negocios de distribución de alimentos fríos.

User Assumptions

¿Quiénes serán nuestros usuarios?

Nuestros usuarios principales son:

- Propietarios y administradores de restaurantes que necesitan controlar sus insumos perecibles y reducir pérdidas económicas.
- Encargados de negocios de distribución de alimentos en fríos que necesitan monitorear la conservación de sus productos y reducir mermas.

¿Dónde encaja nuestro producto en su vida o trabajo?

Para los restaurantes, FreshSense se integra en la gestión diaria de insumos, permitiendo conocer el estado de los productos, recibir alertas y tomar decisiones para evitar pérdidas.

Para los negocios de distribución de alimentos fríos, FreshSense se integra en el proceso de almacenamiento y conservación, proporcionando información sobre las condiciones de los productos y alertando ante situaciones que puedan generar pérdidas.

¿Qué problemas tiene nuestro producto y cómo se pueden resolver?

Problemas:

- El control del inventario y conservación puede realizarse mediante inspecciones manuales o registros poco actualizados.
- Los usuarios pueden no detectar oportunamente condiciones que afectan la conservación de sus productos.
- El deterioro de alimentos genera merma, costos de reposición y reducción de la rentabilidad.
- El monitoreo manual consume tiempo durante las actividades operativas del negocio.

Soluciones:

- Implementar monitoreo de las condiciones de conservación.
- Generar alertas ante condiciones que puedan afectar los productos.
- Mostrar información del inventario de forma clara y accesible.
- Generar reportes que permitan identificar pérdidas y oportunidades de mejora.
- Facilitar decisiones preventivas para reducir la merma.

¿Cómo y cuándo es usado nuestro producto?

FreshSense es utilizada por propietarios, administradores y encargados durante las actividades de almacenamiento, conservación y gestión del inventario. Los usuarios pueden consultar el estado de sus productos, recibir alertas y revisar información sobre pérdidas y rotación durante la jornada operativa.

¿Qué características son importantes?

- Interfaz clara, sencilla y fácil de utilizar.
- Monitoreo de las condiciones de conservación.
- Alertas oportunas ante situaciones de riesgo.
- Gestión rápida del inventario.
- Reportes de pérdidas y rotación.
- Acceso desde diferentes dispositivos.

¿Cómo debe verse y comportarse nuestro producto?

FreshSense debe presentar una interfaz profesional, clara y orientada a la consulta rápida de información crítica. La navegación debe permitir acceder fácilmente al estado del inventario, alertas y reportes sin requerir múltiples pasos.

La aplicación debe estar disponible durante la jornada operativa del negocio y permitir consultas desde dispositivos móviles y computadoras.

User Outcomes

- Reducción de pérdidas económicas por deterioro de alimentos.
- Reducción de productos desperdiciados o descartados.
- Mayor frecuencia de monitoreo del inventario.
- Identificación oportuna de productos en riesgo.
- Mejor planificación de compras y reposición.
- Mayor control sobre las condiciones de conservación.
- Mejora de la rentabilidad mediante la reducción de mermas.

Features Assumptions

Desde la cuenta de un restaurante:

- Registrar alimentos e ingredientes, incluyendo cantidad, fecha de vencimiento y costo.
- Recibir alertas sobre productos próximos a deteriorarse.
- Consultar las condiciones de conservación de los productos.
- Recibir sugerencias para aprovechar productos próximos a deteriorarse.

- Generar reportes de rotación y pérdidas.

Desde la cuenta de un negocio de alimentos fríos:

- Registrar productos y cantidades disponibles.
- Monitorear las condiciones de conservación.
- Recibir alertas ante cambios que puedan afectar los productos.
- Identificar productos próximos a deteriorarse o vencer.
- Consultar reportes sobre pérdidas, rotación y estado del inventario.

1.2.2.3. Lean UX Hypothesis Statements.

Creemos que los restaurantes y negocios de distribución de alimentos fríos necesitan una solución que les permita monitorear sus productos y detectar oportunamente condiciones que puedan generar pérdidas económicas.

Creemos que las alertas de deterioro y las sugerencias de comercialización ayudarán a los negocios a reducir la merma de productos.

Creemos que la carga rápida del inventario facilitará la adopción de la aplicación sin alterar significativamente las actividades diarias de los usuarios.

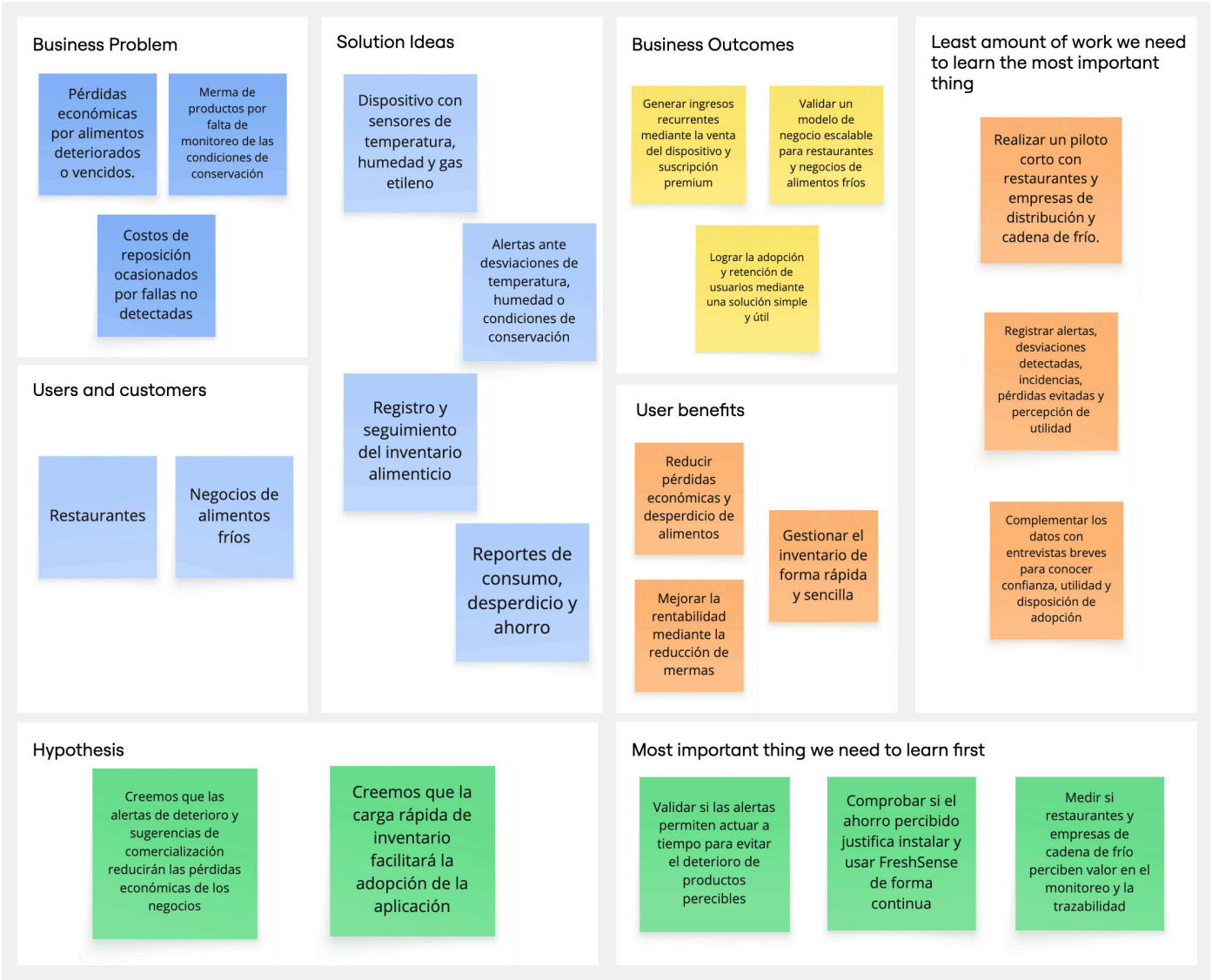
Creemos que los reportes de pérdidas y rotación ayudarán a los administradores a identificar oportunidades para reducir costos y mejorar la rentabilidad.

Creemos que una aplicación accesible desde diferentes dispositivos facilitará el monitoreo del inventario durante la operación diaria.

Sabremos que estas hipótesis son válidas cuando los usuarios utilicen regularmente las funcionalidades de monitoreo, alertas y reportes, y se observe una reducción medible de las pérdidas económicas asociadas al deterioro de alimentos.

1.2.2.4. Lean UX Canvas.

A continuación se presenta el Lean UX Canvas:



1.3. Segmentos objetivo.

Para el proyecto FreshSense se han seleccionado dos segmentos principales de usuarios a los cuales la solución aporta un valor adaptado a sus necesidades específicas:

Propietarios y administradores de restaurantes

Edad: 28 a 50 años

Perfil: Emprendedores que administran restaurantes que gestionan insumos perecibles para la preparación de alimentos.

Estilo de vida: Dinámico y enfocado en las operaciones diarias, atención al cliente y control del negocio.

Uso de tecnología: Frecuente, principalmente mediante dispositivos móviles y computadoras para realizar consultas rápidas.

Necesidad principal: Controlar y conservar adecuadamente los insumos perecibles para reducir desperdicios, pérdidas económicas y costos innecesarios.

Beneficios buscados: Alertas sobre productos en riesgo de deterioro, monitoreo de las condiciones de conservación, gestión del inventario y reportes que permitan identificar pérdidas y mejorar la rentabilidad.

Encargados de negocios de distribución de alimentos en fríos

Edad: 30 a 55 años

Perfil: Emprendedores y encargados de negocios dedicados a la distribución y conservación de productos que requieren cadena de frío.

Estilo de vida: Ocupado y enfocado en la operación diaria, supervisión de productos, atención al cliente y gestión del inventario.

Uso de tecnología: Moderado a frecuente, con disposición a utilizar herramientas que faciliten el monitoreo y control de sus productos.

Necesidad principal: Mantener las condiciones adecuadas de conservación para reducir el deterioro de productos, las mermas y las pérdidas económicas.

Beneficios buscados: Monitoreo de las condiciones de conservación, alertas ante posibles riesgos, información sobre el estado del inventario y reportes que permitan identificar pérdidas y mejorar la rentabilidad del negocio.

Capítulo II: Requirements Elicitation & Analysis

2.1. Competidores

El análisis de competidores de FreshSense se enfoca en soluciones empresariales relacionadas con monitoreo de cadena de frío, control de condiciones ambientales, trazabilidad y supervisión de productos sensibles a la temperatura.

El análisis lo hicimos pensando en los dos segmentos del proyecto, propietarios y administradores de restaurantes y encargados de negocios de distribución de alimentos en fríos. Por eso se tomaron tres competidores que ya monitorean temperatura y condiciones de conservación con dispositivos IoT:

- **Sensitech**
- **Tive**
- **Roambee**

Estas soluciones presentan similitudes con FreshSense en el monitoreo de variables ambientales, generación de alertas y visualización de información en plataformas digitales. Sin embargo, se diferencian en su nivel de especialización, mercados atendidos, alcance internacional y complejidad tecnológica.

2.1.1. Análisis competitivo

El objetivo de este análisis es responder la siguiente pregunta:

> **¿Cómo puede FreshSense diferenciarse de las soluciones empresariales existentes de monitoreo de cadena de frío mediante una propuesta enfocada en alimentos perecibles, facilidad de implementación y centralización de información de lotes, dispositivos y alertas?**

Los competidores seleccionados representan alternativas consolidadas dentro del mercado de monitoreo de condiciones y visibilidad de cadena de suministro.

Competitive Analysis Landscape

Criterio	FreshSense	Sensitech	Tive	Roambee
Tipo de competidor	Startup analizada	Directo	Directo	Directo
Overview	Solución IoT enfocada en el monitoreo de productos perecibles mediante dispositivos que recopilan temperatura y humedad y transmiten información hacia una plataforma digital.	Empresa especializada en visibilidad y monitoreo de cadena de frío, con soluciones para almacenamiento y transporte de productos sensibles.	Plataforma de visibilidad logística en tiempo real que combina rastreadores IoT con una plataforma cloud para monitorear envíos.	Plataforma de visibilidad de cadena de suministro que combina dispositivos IoT, sensores, conectividad y análisis para monitorear mercancías y activos.
Ventaja competitiva	Propuesta enfocada específicamente en empresas que trabajan con alimentos perecibles, con una arquitectura simplificada basada en dispositivos IoT, monitoreo de lotes, alertas y trazabilidad.	Amplio portafolio especializado en cadena de frío, experiencia empresarial y soluciones para almacenamiento y transporte.	Amplia capacidad de monitoreo multimodal y dispositivos capaces de medir múltiples variables en tiempo real.	Integración de monitoreo de condiciones, ubicación, analítica y visibilidad logística en una misma plataforma.
Valor ofrecido al	Centralizar información sobre condiciones de conservación,	Proteger la integridad de productos mediante monitoreo continuo,	Detectar desviaciones durante el transporte en tiempo real para	Proporcionar información en tiempo real y señales

Criterio	FreshSense	Sensitech	Tive	Roambee
cliente	dispositivos, lotes y alertas para detectar desviaciones y reducir pérdidas por deterioro.	alertas y visibilidad de extremo a extremo.	actuar antes de que el producto resulte comprometido.	relacionadas con condiciones y ubicación para mejorar la toma de decisiones logísticas.
Mercado objetivo	Propietarios y administradores de restaurantes, y encargados de negocios de distribución de alimentos en fríos.	Alimentos, ciencias de la vida, industria y organizaciones con cadenas de suministro sensibles a temperatura.	Alimentos y bebidas, productos perecibles, farmacéutica, ciencias de la vida, bienes de alto valor y operadores logísticos.	Empresas de logística, alimentos y bebidas, farmacéutica, manufactura y organizaciones con cadenas de suministro complejas.
Estrategia de marketing	Marketing B2B orientado a demostrar reducción de pérdidas, control de condiciones y facilidad de monitoreo.	Demostraciones comerciales, contenido especializado, casos de uso e información dirigida a industrias reguladas y cadenas de frío.	Demostraciones, pruebas de producto, contenido especializado y comunicación centrada en visibilidad logística y prevención de pérdidas.	Demostraciones, pruebas del servicio, contenido sobre supply chain y comunicación orientada a visibilidad y analítica logística.
Productos y servicios	Dispositivo IoT con ESP32 + DHT22, Edge API, plataforma Web/Mobile, monitoreo de temperatura y humedad, lotes, dispositivos, alertas, trazabilidad y reportes.	SensiWatch Platform, TempTale, ColdStream Site y distintos sensores/registradores para monitoreo estacionario y en tránsito.	Plataforma Tive, trackers Solo Lite, Solo 5G, Solo Pro, sensores, alertas, reportes e integraciones.	Plataforma de visibilidad, dispositivos como BeeSense, monitoreo de temperatura, humedad, ubicación y eventos de transporte.
Precios y costos	Modelo previsto basado en dispositivo IoT y servicio digital. Los precios comerciales deben validarse posteriormente con los segmentos objetivo.	No publica una lista estándar de precios; el acceso comercial se gestiona mediante contacto y demostración.	Los precios de los trackers principales no se publican de forma general y dependen del volumen y contrato. La plataforma dispone de niveles Essential, Plus y Premium.	Utiliza modelos de suscripción y planes bajo demanda definidos mediante órdenes comerciales según cantidad de dispositivos, activos o envíos.
Canales de distribución	Landing Page, Web Application y Mobile Application.	Plataforma web, aplicación móvil y contacto comercial.	Plataforma web, integraciones/API y soluciones de seguimiento accesibles digitalmente.	Plataforma web, aplicación móvil, API y contacto comercial.

Sensitech

Sensitech ofrece soluciones de monitoreo de cadena de frío tanto para productos almacenados como transportados. Su sistema de monitoreo estacionario permite controlar temperatura y humedad en almacenes e instalaciones, consultar datos históricos y en tiempo real y generar alertas cuando las condiciones salen de los rangos establecidos.

También dispone de la plataforma SensiWatch, que proporciona visibilidad de extremo a extremo de la cadena de suministro y acceso a información desde dispositivos móviles.

SWOT - Sensitech

Fortalezas

- Amplio portafolio especializado en cadena de frío.
- Monitoreo estacionario y durante transporte.
- Soporte para temperatura, humedad y otras variables.
- Alertas y datos en tiempo real.
- Experiencia en sectores como alimentos y ciencias de la vida.
- Plataforma Web y aplicación móvil.

Debilidades

- Solución empresarial de mayor complejidad que puede requerir procesos de implementación y contratación más extensos.
- No ofrece precios públicos estándar para sus principales soluciones.
- Su amplio alcance puede resultar superior a las necesidades de empresas que requieren únicamente monitoreo básico de alimentos perecibles.

Oportunidades

- Crecimiento de la digitalización en cadenas de frío.
- Mayor necesidad de trazabilidad y reducción de pérdidas de productos sensibles.
- Incremento de requisitos de control y cumplimiento en cadenas logísticas.

Amenazas

- Aparición de soluciones IoT de menor costo.
- Empresas que desarrollen sistemas internos utilizando sensores y plataformas cloud.
- Competidores especializados en soluciones más simples para sectores específicos.

Tive

Tive ofrece monitoreo de cadena de frío en tiempo real utilizando dispositivos capaces de recopilar información sobre temperatura, humedad, luz, impactos y ubicación.

Su plataforma permite supervisar envíos por transporte terrestre, marítimo, aéreo y ferroviario y generar alertas cuando las condiciones se encuentran fuera de los rangos establecidos.

SWOT - Tive

Fortalezas

- Monitoreo en tiempo real de múltiples variables.
- Seguimiento multimodal de envíos.
- Amplia cobertura de conectividad.
- Diversos modelos de trackers según el nivel de monitoreo requerido.
- Alertas automáticas e historial de los envíos.
- Integraciones mediante API y webhooks.

Debilidades

- Orientación principal hacia monitoreo de mercancías en tránsito.
- La oferta puede resultar más compleja para organizaciones que solo necesitan monitorear instalaciones o lotes específicos.
- Los precios de los dispositivos principales dependen de cotización y volumen.

Oportunidades

- Crecimiento del comercio y logística de productos sensibles.
- Mayor demanda de monitoreo en tiempo real.
- Integración con sistemas empresariales y plataformas logísticas.

Amenazas

- Competidores globales con soluciones equivalentes de seguimiento IoT.
- Reducción del costo de sensores y conectividad que facilita la entrada de nuevas soluciones.
- Desarrollo de plataformas de monitoreo propias por grandes operadores logísticos.

Roambee

Roambee desarrolla soluciones de visibilidad de cadena de suministro mediante dispositivos IoT y servicios cloud.

Sus soluciones permiten monitorear condiciones como temperatura y humedad, además de eventos relacionados con ubicación y transporte. La plataforma genera alertas y señales para apoyar la toma de decisiones durante operaciones logísticas.

SWOT - Roambee

Fortalezas

- Combina monitoreo de condiciones y ubicación.
- Dispositivos con múltiples sensores.
- Plataforma orientada a visibilidad logística empresarial.
- Integración mediante API.
- Opciones de servicio y suscripción según necesidades de monitoreo.
- Capacidad para trabajar con operaciones logísticas de gran escala.

Debilidades

- Solución de mayor alcance y complejidad que un sistema enfocado únicamente en conservación de alimentos.
- Costos dependientes de contratos, dispositivos y volumen de operaciones.
- Su propuesta está orientada a supply chain visibility de forma amplia y no exclusivamente al sector de alimentos perecibles.

Oportunidades

- Mayor adopción de IoT en transporte y logística.
- Demanda creciente de información en tiempo real.
- Necesidad empresarial de reducir interrupciones y pérdidas en cadenas de suministro.

Amenazas

- Alta competencia en plataformas de supply chain visibility.
- Evolución rápida de sensores IoT y plataformas de análisis.
- Aparición de alternativas especializadas de menor costo.

SWOT - FreshSense

Fortalezas

- Enfoque específico en productos alimenticios perecibles.
- Integración de dispositivos IoT con una plataforma digital.
- Monitoreo de temperatura y humedad.
- Gestión centralizada de dispositivos y lotes.
- Generación de alertas ante condiciones que requieren atención.

- Arquitectura preparada para Web y Mobile.
- Propuesta orientada a trazabilidad y reducción de pérdidas.

Debilidades

- Producto aún en etapa de desarrollo y validación.
- Menor cantidad de variables monitoreadas frente a competidores internacionales.
- Menor cobertura tecnológica y comercial.
- Dependencia inicial de sensores y conectividad del dispositivo.
- Modelo comercial y precios aún pendientes de validación con usuarios empresariales.

Oportunidades

- Digitalización de empresas de distribución y comercialización de alimentos.
- Necesidad de reducir pérdidas provocadas por una conservación inadecuada.
- Crecimiento de soluciones IoT accesibles.
- Posibilidad de adaptar la solución a necesidades específicas del mercado local.
- Incorporación futura de sensores adicionales y analítica avanzada.

Amenazas

- Presencia de competidores internacionales consolidados.
- Reducción de precios de soluciones comerciales existentes.
- Resistencia de algunas empresas a reemplazar procesos manuales.
- Entrada de nuevos proveedores de soluciones IoT.
- Dependencia de la correcta instalación y conectividad de los dispositivos.

Conclusión del análisis competitivo

El análisis evidencia que FreshSense participaría en un mercado en el que existen soluciones consolidadas con capacidades superiores en cobertura global, cantidad de sensores e infraestructura tecnológica.

Sensitech presenta una propuesta especialmente fuerte en monitoreo de cadena de frío tanto estacionario como en tránsito; Tive destaca por el seguimiento multimodal y la captura de múltiples variables en tiempo real; y Roambee ofrece una solución amplia de visibilidad y monitoreo de cadenas de suministro.

Frente a estos competidores, FreshSense no busca inicialmente competir por amplitud tecnológica, sino desarrollar una propuesta más específica para organizaciones que gestionan alimentos perecibles. Su diferenciación preliminar se basa en simplificar el monitoreo de temperatura y humedad y relacionarlo directamente con dispositivos, lotes, alertas y trazabilidad dentro de una experiencia centralizada.

Esta diferenciación deberá validarse posteriormente mediante entrevistas con los segmentos objetivo, principalmente para determinar qué variables necesitan monitorear, qué nivel de trazabilidad requieren y cuánto estarían dispuestos a invertir en una solución de este tipo.

2.1.2. Estrategias y tácticas frente a competidores

A partir del análisis competitivo, FreshSense plantea una estrategia de diferenciación enfocada en especialización, simplicidad y adaptación a las necesidades de empresas que gestionan alimentos perecibles.

Las principales estrategias y tácticas serán las siguientes:

- **Especialización en alimentos perecibles:** enfocar la experiencia, terminología, dashboards, lotes y alertas en procesos relacionados con almacenamiento y distribución de alimentos, evitando incorporar funcionalidades empresariales que no sean necesarias para los segmentos objetivo.
- **Implementación progresiva:** facilitar que las empresas puedan comenzar con un número reducido de dispositivos y ampliar posteriormente el monitoreo según sus necesidades.
- **Experiencia simplificada:** diferenciarse de plataformas empresariales de mayor complejidad mediante una interfaz centrada en las funciones principales: monitoreo, lotes, dispositivos, alertas y trazabilidad.
- **Adaptación al mercado objetivo:** usar las entrevistas para ver qué temperatura, reportes y alertas necesitan los propietarios y administradores de restaurantes, y los encargados de negocios de distribución de alimentos en fríos.
- **Modelo comercial flexible:** evaluar alternativas de venta o provisión de dispositivos combinadas con una suscripción por acceso a la plataforma, evitando establecer precios definitivos antes de validar la disposición de pago de los segmentos.
- **Alertas orientadas a la acción:** priorizar notificaciones que permitan identificar rápidamente desviaciones y relacionarlas con un dispositivo o lote determinado.
- **Trazabilidad centralizada:** mantener el historial de lecturas y eventos de cada lote para facilitar la revisión de condiciones durante almacenamiento o distribución.
- **Evolución tecnológica gradual:** mantener una arquitectura que permita incorporar nuevos sensores, capacidades de conectividad y funciones analíticas a medida que el producto y las necesidades del mercado evolucionen.

Estas estrategias buscan que FreshSense compita inicialmente mediante una solución especializada y comprensible para empresas del sector alimentario, en lugar de intentar replicar desde el inicio la amplitud funcional de plataformas internacionales consolidadas.

2.2. Entrevistas

Las entrevistas constituyen una de las principales fuentes de información para comprender las necesidades, problemas y expectativas de los segmentos objetivo de FreshSense.

El proceso de investigación se enfoca en representantes de empresas que participan en actividades de almacenamiento, distribución, producción y comercialización de alimentos perecibles. A través de estas entrevistas se busca conocer cómo gestionan actualmente las condiciones de conservación de sus productos, qué dificultades enfrentan, qué herramientas utilizan y qué valor podría aportar una solución basada en dispositivos IoT y monitoreo digital.

Los resultados obtenidos permitirán validar los principales supuestos de FreshSense y servirán posteriormente como base para la elaboración de User Personas, User Task Matrix, User Journey Maps y demás artefactos de análisis.

Los segmentos considerados son:

- **Segmento 1: Propietarios y administradores de restaurantes**
- **Segmento 2: Encargados de Negocio de Distribución de alimentos en frio**

2.2.1. Diseño de entrevistas

Las entrevistas serán de tipo semiestructurado, utilizando preguntas orientadas a conocer el contexto de trabajo del entrevistado, los procesos actuales de conservación de productos perecibles, los principales problemas encontrados, el uso de tecnología y la percepción de valor frente a una solución como FreshSense.

Las preguntas han sido diseñadas de acuerdo con las características de cada segmento objetivo y buscan obtener información suficiente para identificar necesidades, objetivos, frustraciones y patrones de comportamiento que posteriormente serán utilizados en la construcción de los arquetipos de usuario.

Segmento 1: Propietarios y administradores de restaurantes

1. ¿Qué tipo de alimentos manejas en tu negocio y cuáles son más propensos a perderse o malograrse?
2. ¿Con qué frecuencia enfrentas pérdidas por deterioro de productos?
3. ¿Tienes actualmente un método para controlar la frescura de tus ingredientes (listas, hojas de Excel, memoria, otro)?
4. ¿Qué impacto tiene para tu negocio perder productos perecibles? (económico, reputación, tiempo, clientes).
5. ¿Qué importancia le das a la calidad y frescura de los alimentos que vendes?
6. ¿Qué tipo de alertas o reportes te gustaría recibir para mejorar tu control de inventario?
7. ¿Te resultaría útil tener un dispositivo que monitoree automáticamente el estado de los productos dentro de tu refrigerador?
8. ¿Qué características serían imprescindibles para que confíes en este tipo de solución
9. ¿Pagarías por una herramienta que te ayude a reducir pérdidas y mantener la frescura? ¿Prefieres un pago único por el dispositivo o una suscripción mensual con funciones adicionales?
10. ¿Si el sistema pudiera darte estadísticas de consumo, desperdicio y hasta proyecciones de compras, ¿te serviría para tu negocio?

Segmento 2: Empresas productoras y comercializadoras de alimentos perecibles

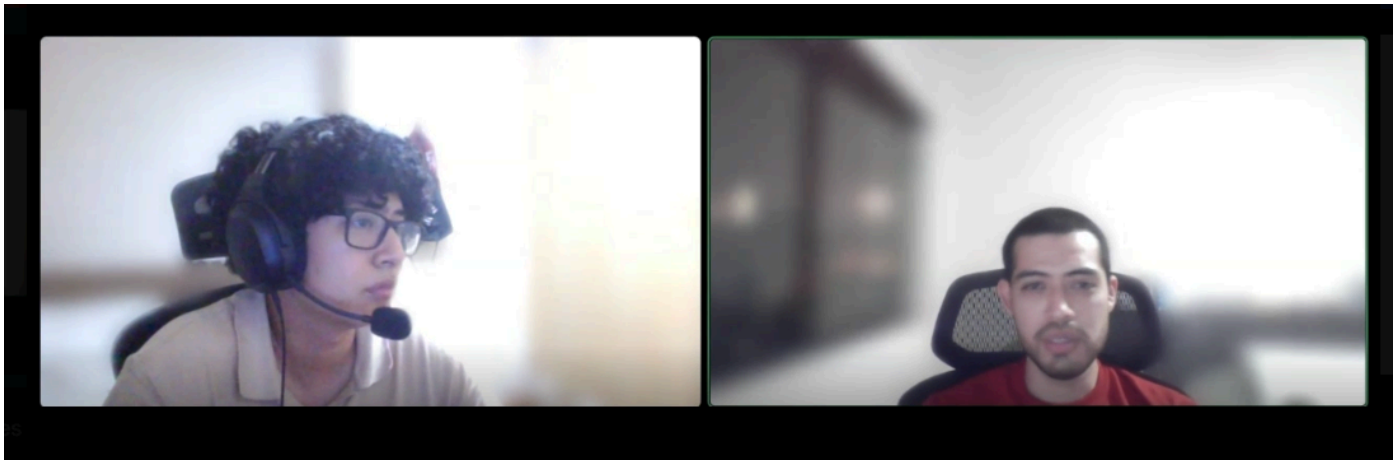
1. ¿Cuál es su cargo dentro de la empresa y qué responsabilidades tiene relacionadas con producción, almacenamiento, calidad o comercialización?
2. ¿Qué tipo de alimentos perecibles maneja la empresa y cuáles requieren un mayor control de sus condiciones de conservación?
3. ¿Cómo verifican actualmente que los productos se encuentren almacenados bajo condiciones adecuadas?
4. ¿Qué problemas de conservación o deterioro se presentan con mayor frecuencia dentro de la empresa?
5. ¿Qué impacto generan estas pérdidas o deterioros en términos económicos, operativos o de calidad?
6. ¿Utilizan sensores, termómetros, hojas de cálculo, sistemas de inventario u otras herramientas para controlar las condiciones de almacenamiento?
7. ¿Con qué frecuencia revisan actualmente variables como temperatura y humedad?
8. ¿Qué tan útil sería contar con información de estas condiciones en tiempo real y recibir alertas cuando exista una desviación?
9. ¿Cómo identifican y realizan actualmente el seguimiento de los diferentes lotes de productos?
10. Si se detecta un problema de conservación, ¿qué tan sencillo es identificar cuáles lotes podrían haber sido afectados?
11. ¿Qué características considera indispensables en una plataforma de monitoreo de productos perecibles?
12. ¿Qué factores serían más importantes al momento de adoptar una solución como FreshSense: costo, facilidad de uso, precisión, trazabilidad, alertas, reportes u otros?

2.2.2. Registro de entrevistas

Segmento 1: Propietarios y administradores de restaurantes

Entrevista 1

Dato	Información
Nombre y apellidos	Guillermo
Edad	26 años
Ubicación	Lima
Empresa / sector	Restaurante
Cargo	Propietario
Inicio de entrevista	00:00
Duración	06:18
Enlace	https://drive.google.com/file/d/1NJtkNjEPtNI9XM_wAJCINVNJYsAqRvtj/view?usp=sharing



Resumen:

Guillermo tiene 26 años y es propietario de un restaurante en Lima, donde supervisa las compras, el inventario y el estado de los alimentos. Trabaja principalmente con carnes, pescado, pollo, verduras y lácteos, siendo las carnes, el pescado y algunas verduras los productos más propensos a deteriorarse.

Actualmente, el control de frescura se realiza mediante revisión manual, fechas de vencimiento, etiquetas y registros en Excel. Sin embargo, este proceso depende bastante del personal y puede generar pérdidas cuando algún producto no se revisa a tiempo. Para Guillermo, estas pérdidas representan un impacto económico y también afectan la operatividad del restaurante.

Considera útil una solución que permita monitorear temperatura y humedad, enviar alertas ante variaciones anormales y avisar sobre productos próximos a vencer. También valora contar con reportes de desperdicio y consumo para mejorar las compras. Estaría dispuesto a pagar por una herramienta de este tipo siempre que sea precisa, fácil de usar y contribuya realmente a reducir pérdidas.

Entrevista 2

Dato	Información
Nombre y apellidos	Romy Panduro
Edad	46 años
Ubicación	San Borja
Empresa / sector	Restobar karaoke
Cargo	Propietaria
Inicio de entrevista	00:00
Duración	9:04
Enlace	https://drive.google.com/file/d/1yaP9FCvV9rKo09GO2Eu2jYF9UUS8Ujob/view?usp=sharing

Resumen:

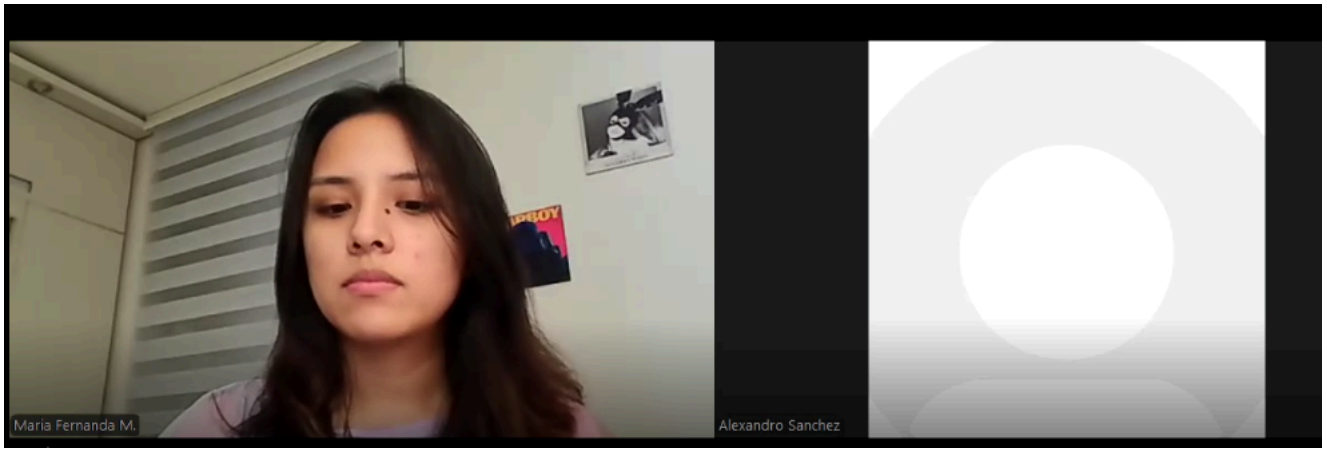
Romy Panduro tiene 46 años y administra un restobar karaoke ubicado en Tarapoto, con aproximadamente dos años de funcionamiento. En su negocio trabaja principalmente con carnes rojas, pollo y pescado, siendo este último el producto que considera más propenso a deteriorarse.

Actualmente utiliza una aplicación para registrar el stock, mientras que sus trabajadores realizan manualmente la rotación de los productos según su antigüedad. Debido a ello, considera útil contar con una solución que permita recibir alertas automáticas sobre el estado y calidad de los alimentos almacenados en las congeladoras, pudiendo consultar esta información desde cualquier lugar.

Respecto a una solución como FreshSense, valora especialmente el monitoreo automático y las notificaciones. Además, estaría interesada en un modelo de pago anual que incluya actualizaciones y mejoras continuas del sistema.

Entrevista 3

Dato	Información
Nombre y apellidos	Persona A (seudónimo)
Edad	27 años
Empresa / sector	Restaurante
Cargo	Administrador
Función	Supervisión de operaciones, compras y gestión de productos
Inicio de entrevista	00:00
Duración	4:44
Enlace	https://drive.google.com/file/d/1aCd80YB6zoBtOJcUTQrDffn5Tb8MfSop/view?usp=sharing



Resumen:

Persona A tiene 27 años y trabaja como administrador de un restaurante, donde supervisa las operaciones, las compras y el manejo de los alimentos. El negocio utiliza principalmente carnes, pollo, pescado, verduras, lácteos y productos congelados, siendo las carnes, el pescado y algunas verduras los más propensos a deteriorarse si no se almacenan correctamente.

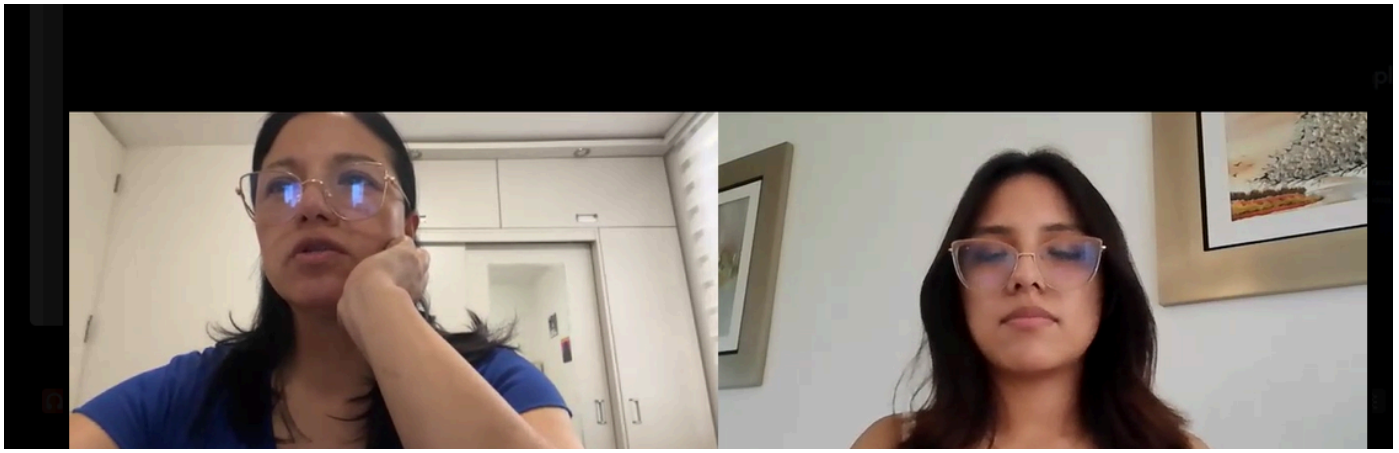
Actualmente, el control de frescura se realiza mediante revisiones manuales de fechas de vencimiento, inspecciones visuales, verificación de temperatura y registros de inventario. Sin embargo, menciona que ocasionalmente se producen pérdidas por compras excesivas o productos almacenados durante demasiado tiempo. Estas situaciones generan gastos adicionales y pueden afectar la preparación de los pedidos.

Considera útil una solución como FreshSense que permita monitorear temperatura y humedad, enviar alertas sobre cambios de temperatura o productos próximos a vencer y consultar historiales de mediciones. También valora la precisión de los sensores, la facilidad de uso y la rapidez de las notificaciones. Finalmente, estaría dispuesto a adquirir el dispositivo y pagar una suscripción económica, siempre que contribuya a reducir pérdidas y mejorar la gestión del restaurante.

Segmento 2: Empresas productoras y comercializadoras de alimentos perecibles

Entrevista 1

Dato	Información
Nombre y apellidos	Silvia
Edad	40 años
Empresa / sector	Comercialización y distribución de alimentos perecibles
Función	Supervisión del almacenamiento, control de inventario y conservación de productos
Inicio de entrevista	00:00
Duración	3:52
Enlace	https://drive.google.com/file/d/1im0wnd2XqLIU5hLwFVeBPy_K5FL50XW8/view?usp=sharing



Resumen:

Silvia tiene 40 años y trabaja en una empresa dedicada a la comercialización y distribución de alimentos perecibles, donde participa en la supervisión del almacenamiento y control del inventario. La empresa maneja principalmente carnes, lácteos y productos congelados, siendo las carnes y los lácteos los que requieren mayor cuidado debido a su sensibilidad ante cambios de temperatura.

Actualmente realizan inspecciones manuales, revisiones periódicas de los equipos de refrigeración y registros de inventario, utilizando también Excel. Sin embargo, el proceso todavía depende considerablemente de verificaciones realizadas por el personal. Silvia señala que una variación de temperatura no detectada a tiempo puede deteriorar productos, generar pérdidas económicas y afectar los pedidos planificados.

Considera útil una solución como FreshSense que permita monitorear temperatura y humedad en tiempo real, enviar alertas ante condiciones inadecuadas y consultar el historial de cada lote. Para ella, la plataforma debería ser fácil de utilizar, ofrecer mediciones confiables y generar reportes que permitan identificar problemas ocurridos durante el almacenamiento.

Entrevista 2

Dato	Información
Nombre y apellidos	Jesús Pérez
Edad	45 años
Empresa / sector	Distribución de alimentos refrigerados
Función	Supervisión de almacenamiento, control de inventario y conservación de productos
Inicio de entrevista	00:00

Dato	Información
Duración	4:57
Enlace	https://drive.google.com/file/d/1NCnw1minxYLf-k89c6vxbyryJ1yoyXz-/view?usp=sharing

Resumen:

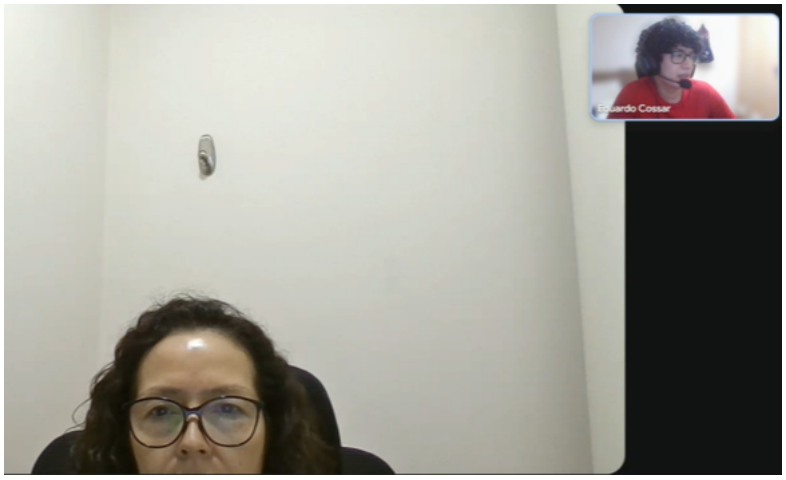
Jesús Pérez tiene 45 años y trabaja en una empresa dedicada a la distribución de alimentos refrigerados, donde supervisa el almacenamiento, controla el inventario y verifica el estado de los productos antes de su distribución. Trabajan principalmente con carnes, pollo, lácteos y productos congelados, siendo las carnes y los lácteos los que requieren mayor control.

Actualmente realizan revisiones periódicas de las cámaras de refrigeración, verificaciones de temperatura y controles visuales, además de utilizar registros internos para el seguimiento de los lotes. Sin embargo, Jesús señala que cuando una variación de temperatura no se detecta rápidamente puede ocasionar deterioro de productos y pérdidas económicas, además de dificultar la identificación exacta de los lotes afectados.

Considera útil una solución como FreshSense que permita monitorear temperatura y humedad en tiempo real, enviar alertas rápidas y relacionar las mediciones con los lotes almacenados. También valora contar con historiales y reportes, y considera importantes la precisión de los sensores, la facilidad de uso, el costo y la posibilidad de consultar la información desde una computadora o celular.

Entrevista 3

Dato	Información
Nombre y apellidos	Carmen
Edad	54 años
Empresa / sector	Comercialización y distribución de alimentos perecibles
Función	Supervisión de almacenamiento, control de inventario y conservación de productos
Inicio de entrevista	00:00
Duración	6:07
Enlace	https://drive.google.com/file/d/17b7sV7W1QHlYEh9eXrvsy_Lgdfv51viF/view?usp=sharing



Resumen:

Carmen tiene 54 años y trabaja en una empresa dedicada a la comercialización y distribución de alimentos perecibles, donde supervisa el almacenamiento, el inventario y la conservación de productos. La empresa maneja principalmente carnes, pollo, lácteos y alimentos congelados, siendo las carnes y los lácteos los que requieren mayor cuidado debido a su sensibilidad ante cambios de temperatura.

Actualmente realizan revisiones periódicas de las cámaras de refrigeración, controles visuales y registros de inventario mediante herramientas internas y Excel. Sin embargo, Carmen señala que las verificaciones todavía dependen considerablemente del personal, lo que dificulta detectar fallas de refrigeración a tiempo. Estas situaciones pueden generar pérdidas económicas, afectar los pedidos y complicar la identificación de los lotes que estuvieron expuestos a condiciones inadecuadas.

Considera útil una solución como FreshSense que permita monitorear temperatura y humedad automáticamente, recibir alertas inmediatas y consultar historiales de mediciones asociados a los lotes almacenados. También valora la facilidad de uso, la precisión de los sensores y el acceso desde computadoras o celulares. Finalmente, considera que el costo y la capacidad del sistema para reducir pérdidas serían factores importantes para decidir su implementación.

2.2.3. Análisis de entrevistas

Segmento 1: Propietarios y administradores de restaurantes

A partir de las tres entrevistas realizadas, se identificó que el **100 % de los entrevistados realiza algún control de temperatura**, aunque todos mencionaron que parte del proceso todavía depende de verificaciones manuales.

El **66.7 %** indicó que utiliza sensores o registradores digitales en determinadas etapas, pero la información no siempre se encuentra centralizada. Asimismo, el **100 % considera útil recibir alertas automáticas** cuando las condiciones salen de los rangos establecidos.

También se observó que el **100 % considera importante relacionar las condiciones registradas con los lotes monitoreados**, principalmente para identificar rápidamente posibles incidencias.

En este segmento, las principales necesidades identificadas son el monitoreo continuo, las alertas oportunas, la trazabilidad y la centralización de la información.

Segmento 2: Encargados de negocios de distribución de alimentos en fríos

En este segmento, el **100 % de los entrevistados utiliza algún mecanismo para controlar la temperatura**, aunque las tres empresas también realizan verificaciones manuales periódicas.

El **100 % considera útil contar con información en tiempo real y alertas automáticas**, especialmente para detectar desviaciones antes de que afecten la calidad de los productos.

Además, el **100 % considera relevante asociar las condiciones de almacenamiento con los lotes**, mientras que el **66.7 %** destacó la utilidad de contar con reportes e historiales de mediciones.

Las principales necesidades identificadas son el monitoreo en tiempo real, la gestión de lotes, las alertas y el acceso a información histórica.

Conclusión general

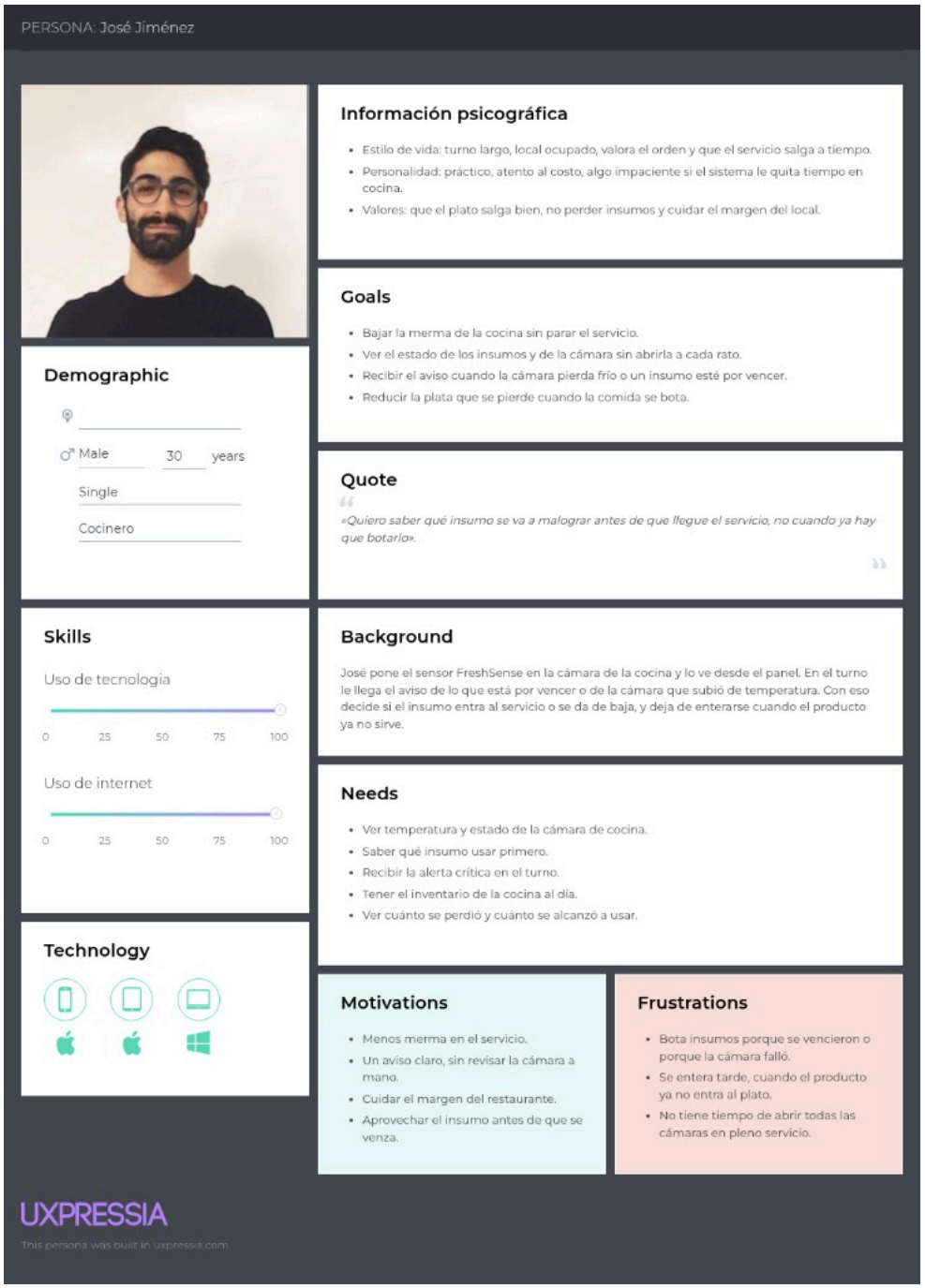
En ambos segmentos se observa una necesidad común de mejorar el control de las condiciones de conservación mediante una solución más centralizada y automatizada.

Los resultados respaldan principalmente cuatro funcionalidades de FreshSense: **monitoreo de temperatura y humedad, alertas automáticas, gestión de lotes e historial de mediciones**.

2.3. Needfinding.

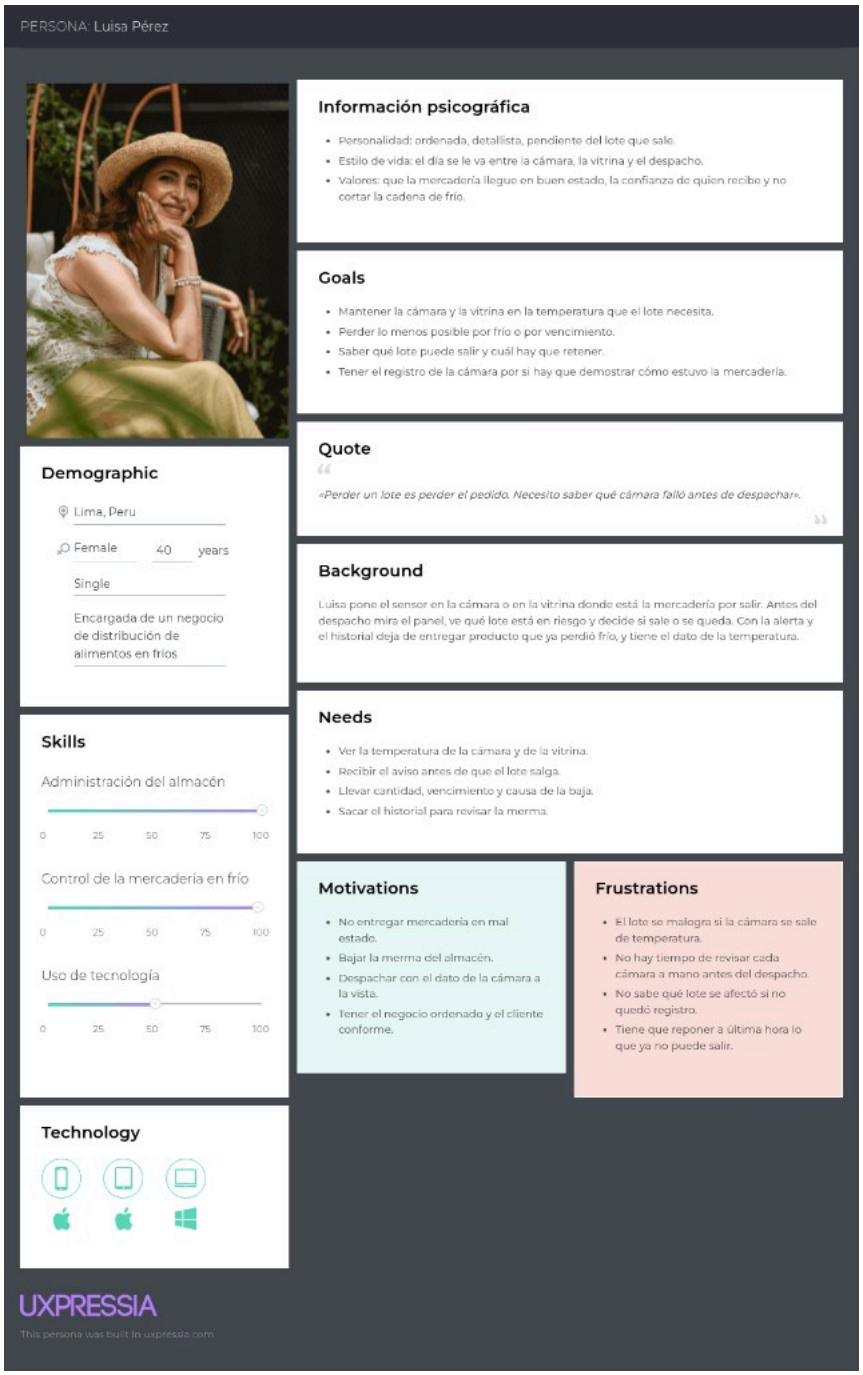
2.3.1. User Personas.

SEGMENTO 1: Propietarios y administradores de restaurantes



José Jiménez tiene 30 años y es cocinero. Representa a un propietario o administrador de restaurante. En el turno ve los insumos de la cocina y de la cámara. Si algo se vence o la cámara pierde frío, esa comida se bota y el local pierde plata. Necesita el aviso a tiempo, sin abrir la cámara a cada rato.

SEGMENTO 2: Encargados de negocios de distribución de alimentos en fríos



Luisa Pérez tiene 40 años y vive en Lima. Es encargada de un negocio de distribución de alimentos en fríos. Su problema es la mercadería que se malogra si la cámara o la vitrina se salen de temperatura, o si el lote no sale a tiempo. Quiere ver el estado de las cámaras y saber qué lote está en riesgo antes de despacharlo.

2.3.2. User Task Matrix

Tareas	**José Jimenez (Propietarios y administradores de restaurantes)**		**Luisa Pérez (Encargados de negocios de distribución de alimentos en fríos)**	
	Frecuencia	**Importancia**	**Frecuencia**	**Importancia**
Revisar estado de alimentos en la app	Casi siempre	Muy alta	Siempre	Muy alta
Recibir alertas de vencimiento	Siempre	Muy alta	Siempre	Muy alta
Consultar recetas sugeridas	A veces	Alta	Raramente	Media
Editar inventario manualmente	Raramente	Media	Casi siempre	Alta
Revisar reportes semanales (ahorro, consumo)	A veces	Media	Siempre	Alta
Compartir logros/impacto en redes sociales	A veces	Media	Raramente	Baja
Usar notificaciones configurables (hora/frecuencia)	Casi siempre	Alta	Casi siempre	Alta
Generar proyecciones de compra	Nunca	Baja	Casi siempre	Muy alta

El análisis de la matriz de tareas evidencia una clara diferencia en las prioridades operativas y patrones de uso entre ambos perfiles de usuario.

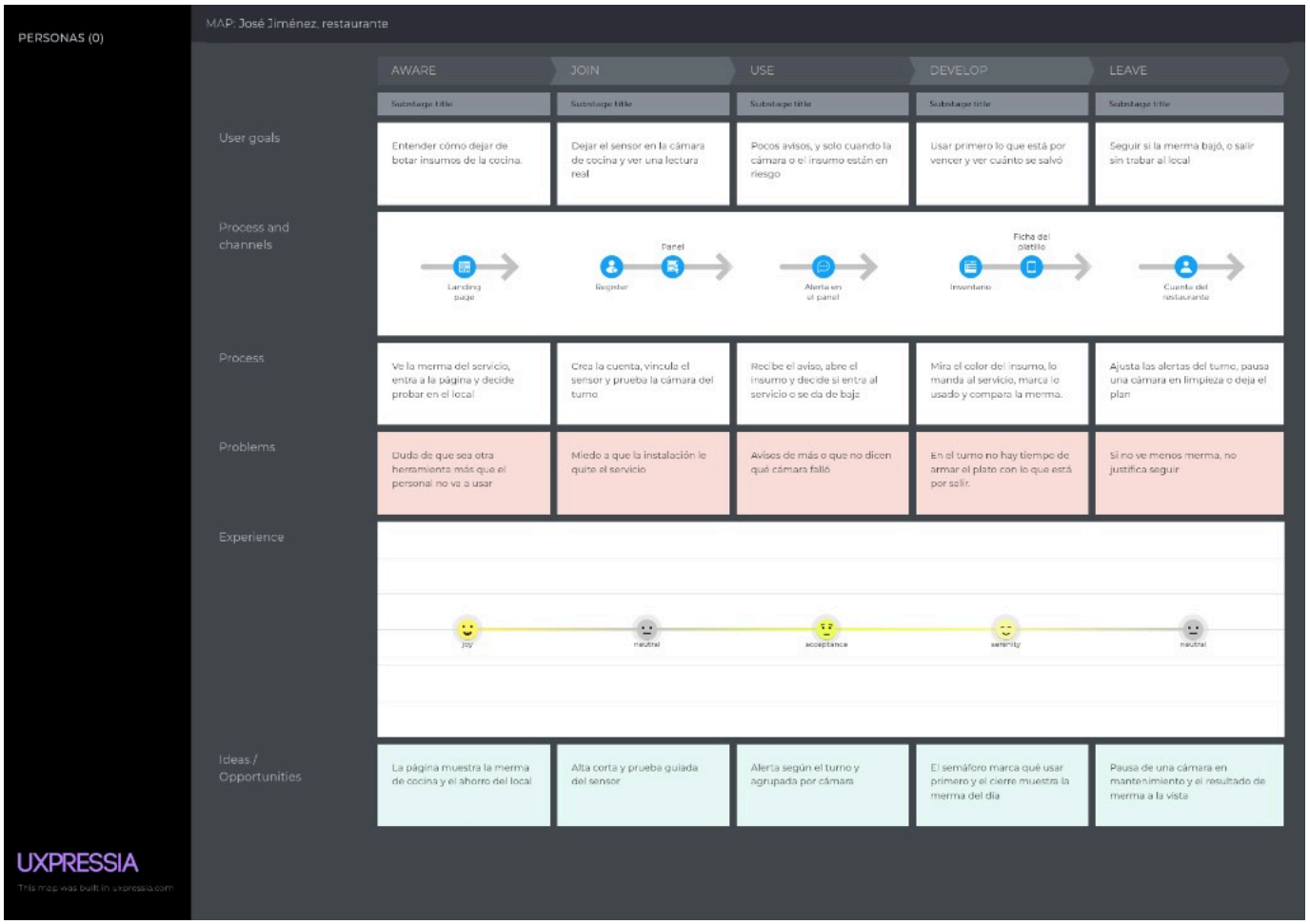
José Jiménez, del segmento de propietarios y administradores de restaurantes, usa más la revisión del estado de los alimentos y las alertas de vencimiento. Las recetas le sirven cuando tiene insumos por salir, pero no es lo primero que mira en el turno.

Luisa Pérez, del segmento de encargados de negocios de distribución de alimentos en fríos, se queda más en el inventario, las métricas y la reposición. Para ella la receta no es lo central: le importa que el lote siga en frío y salga antes de que se malogre.

2.3.3. User Journey Mapping

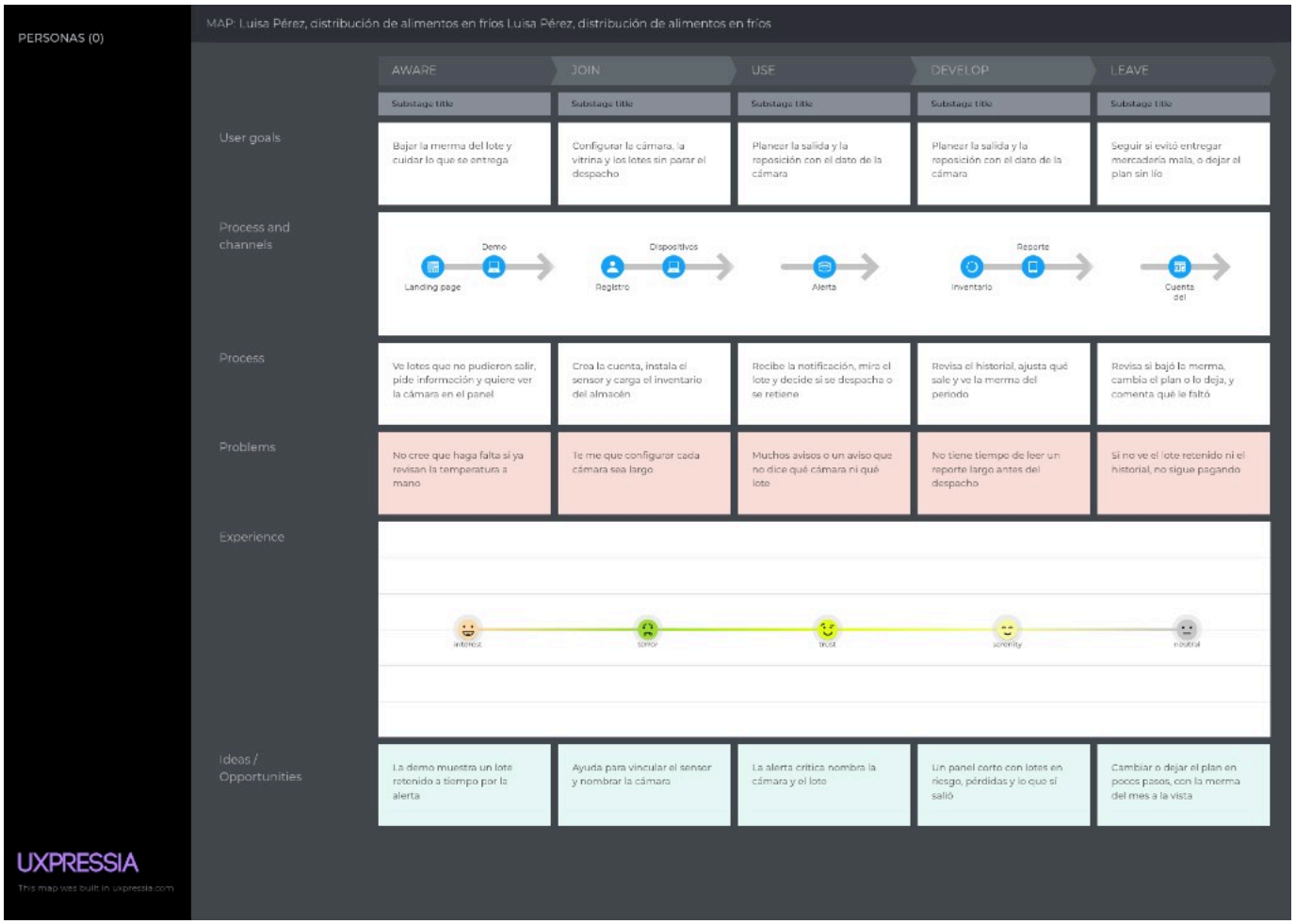
Segmento 1: Propietarios y administradores de restaurantes

En este recorrido se ve el día de José en el restaurante. Entra por la landing, crea la cuenta y vincula el sensor de la cámara de cocina. En el turno recibe el aviso, mira el color del insumo y decide si entra al servicio o se da de baja. Si la merma no baja, no justifica seguir.



Segmento 2: Encargados de negocios de distribución de alimentos en fríos

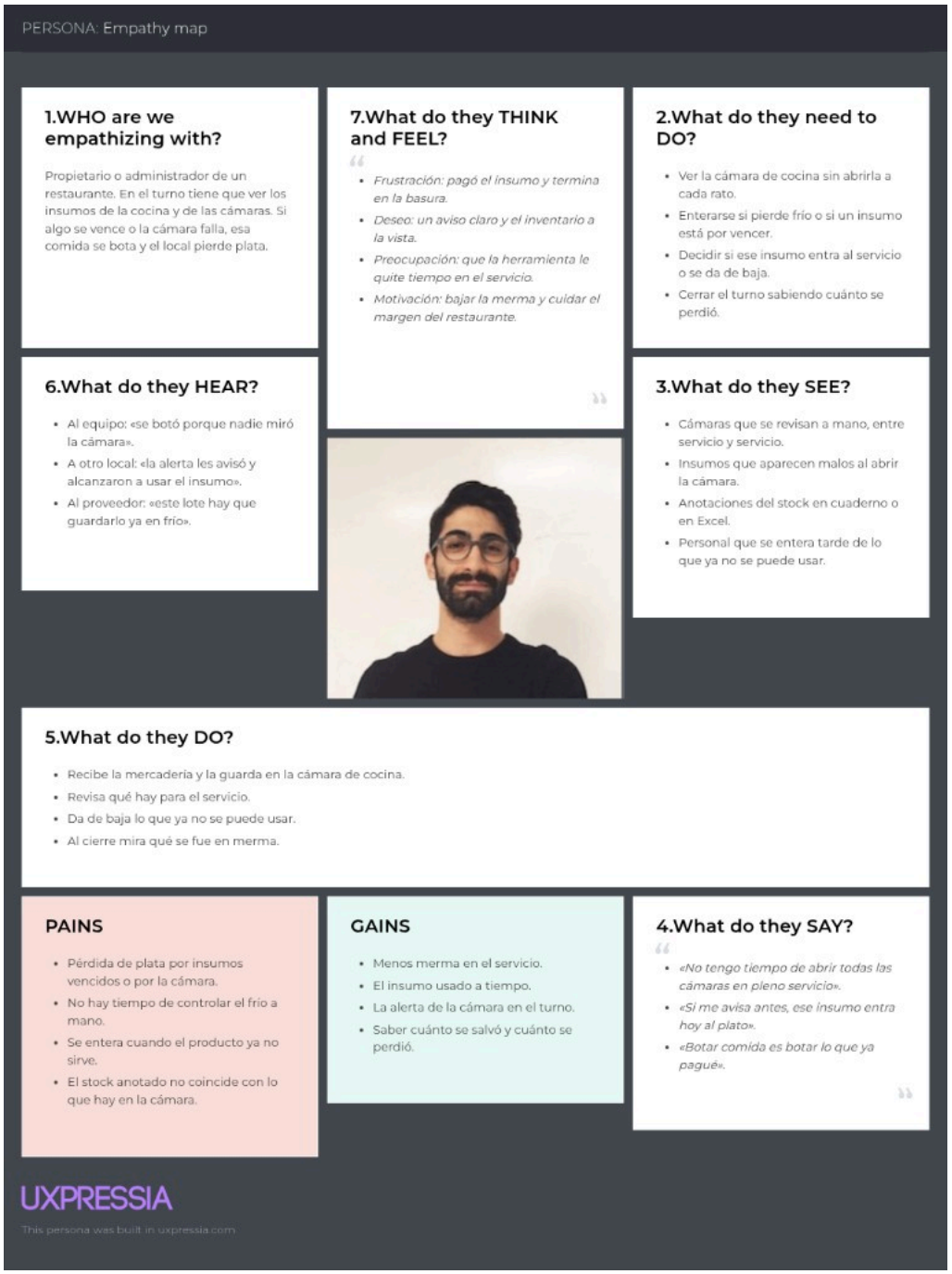
En este recorrido se ve el día de Luisa en la distribución de alimentos en fríos. Entra por la landing, crea la cuenta e instala el sensor de la cámara o la vitrina. Antes del despacho mira el lote y decide si sale o se retiene. Si no ve el historial ni la merma, no justifica seguir.



2.3.4. Empathy Mapping

Segmento 1: Propietarios y administradores de restaurantes

José representa a quien administra un restaurante. Le preocupa botar insumos porque se vencieron o porque la cámara de la cocina falló. Quiere un aviso claro y ver el inventario sin perder el turno de cocina.

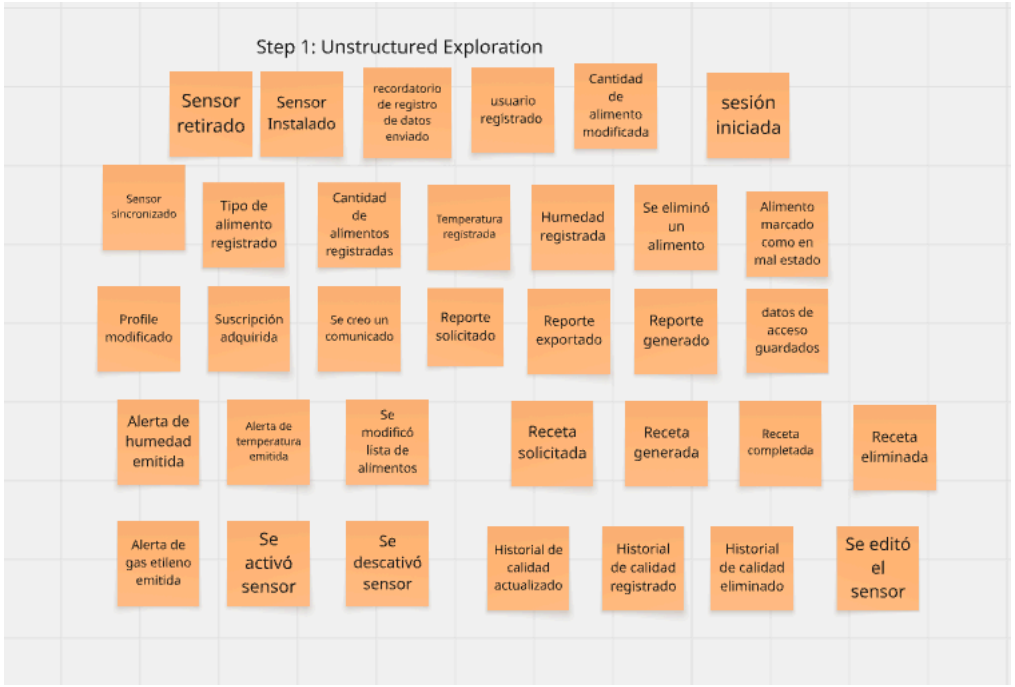


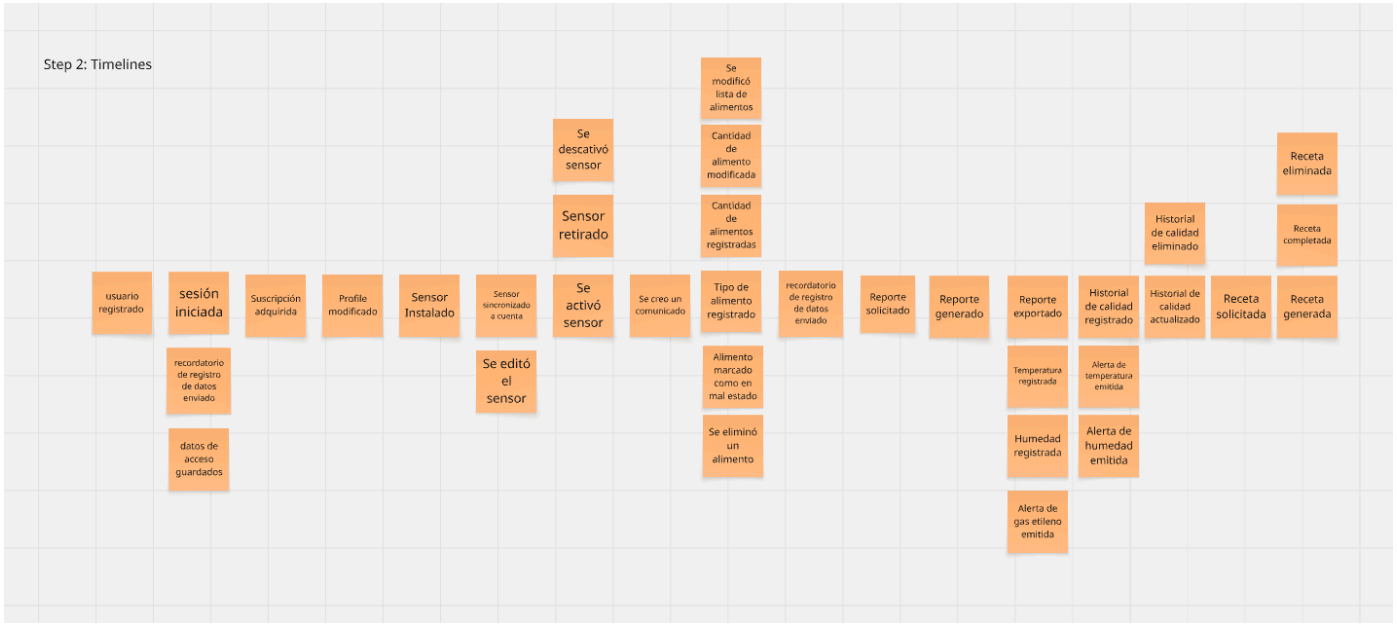
Segmento 2: Encargados de negocios de distribución de alimentos en fríos

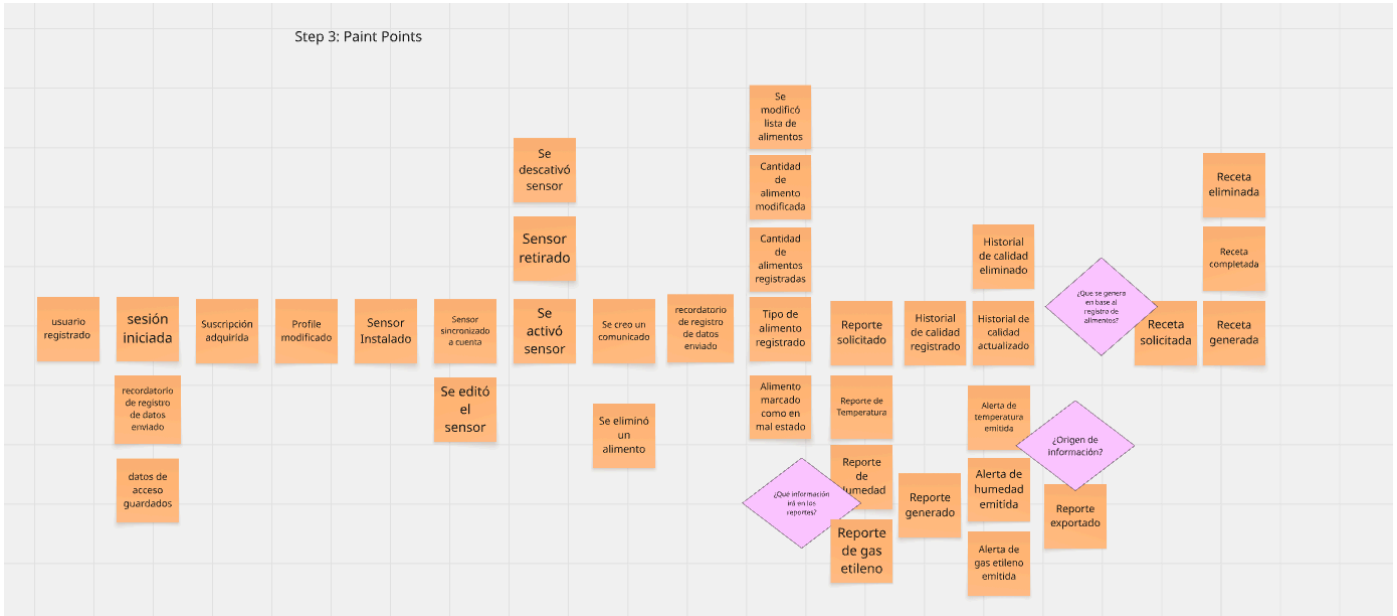
Luisa representa a quien está a cargo de un negocio de distribución de alimentos en fríos. Le preocupa entregar un lote que ya no está en buen estado y no tener cómo demostrar a qué temperatura estuvo la cámara. Necesita el historial y la alerta antes de que la mercadería salga.

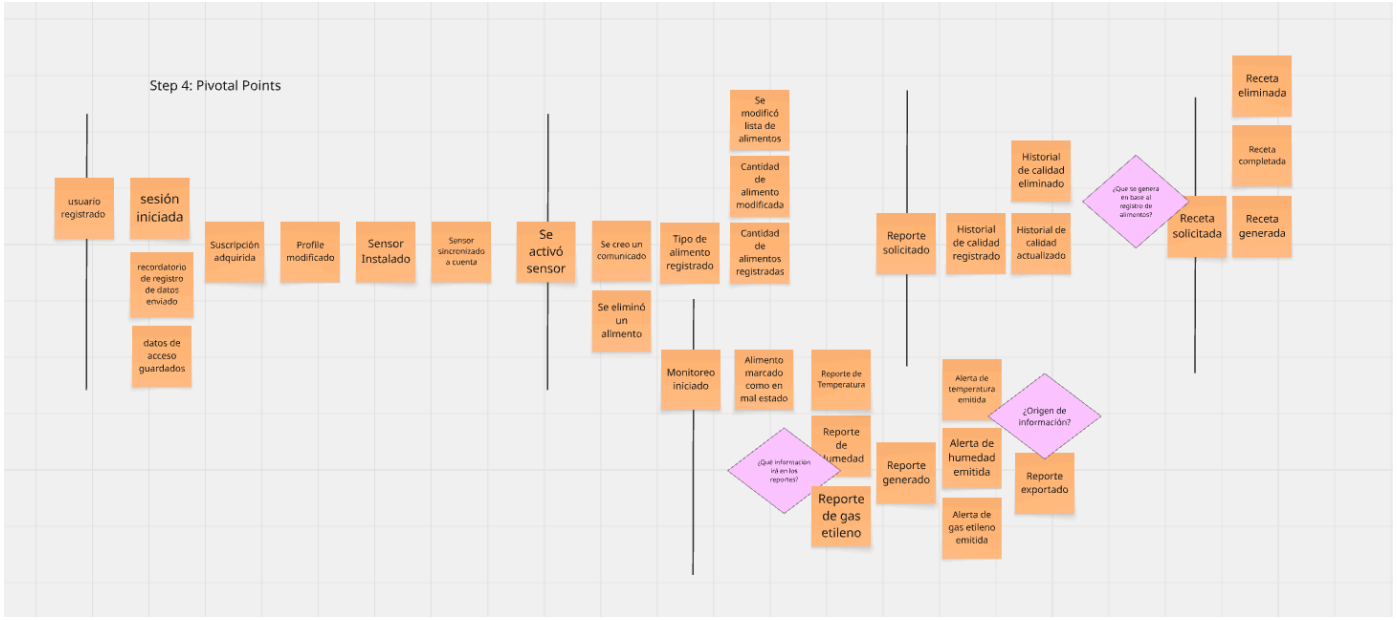
2.4. Big Picture EventStorming

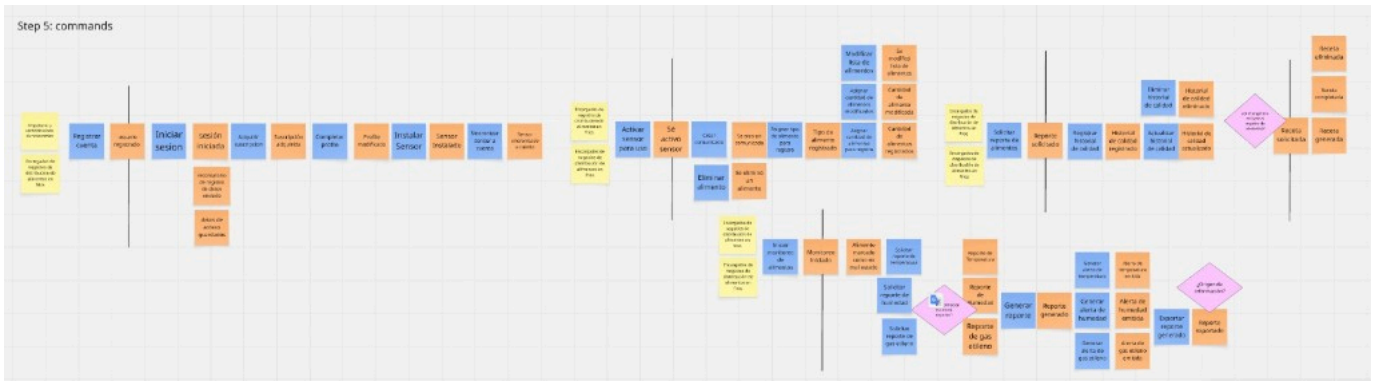
Hicimos el EventStorming pensando en los dos segmentos con los que trabajamos: **propietarios y administradores de restaurantes** (insumos de cocina, recetas y mermas) y **encargados de negocios de distribución de alimentos en fríos** (cámaras, vitrinas, lotes y cadena de frío). El tablero está en Miro.

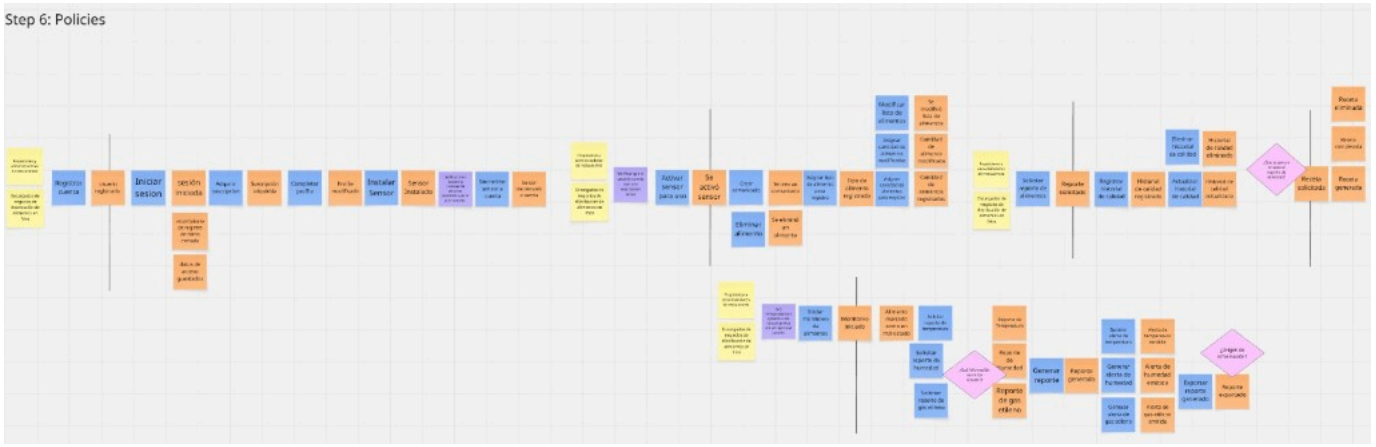

Step 1: Unstructured Exploration
Sensor retirado
Sensor Instalado
recordatorio de registro de datos enviado
usuario registrado
Cantidad de alimento modificada
sesión iniciada
Sensor sincronizado
Tipo de alimento registrado
Cantidad de alimentos registradas
Temperatura registrada
Humedad registrada
Se eliminó un alimento
Alimento marcado como en mal estado
Profile modificado
Suscripción adquirida
Se creo un comunicado
Reporte solicitado
Reporte exportado
Reporte generado
datos de acceso guardados
Alerta de humedad emitida
Alerta de temperatura emitida
Se modificó lista de alimentos
Receta solicitada
Receta generada
Receta completada
Receta eliminada
Alerta de gas etileno emitida
Se activó sensor
Se descativó sensor
Historial de calidad actualizado
Historial de calidad registrado
Historial de calidad eliminado
Se editó el sensor

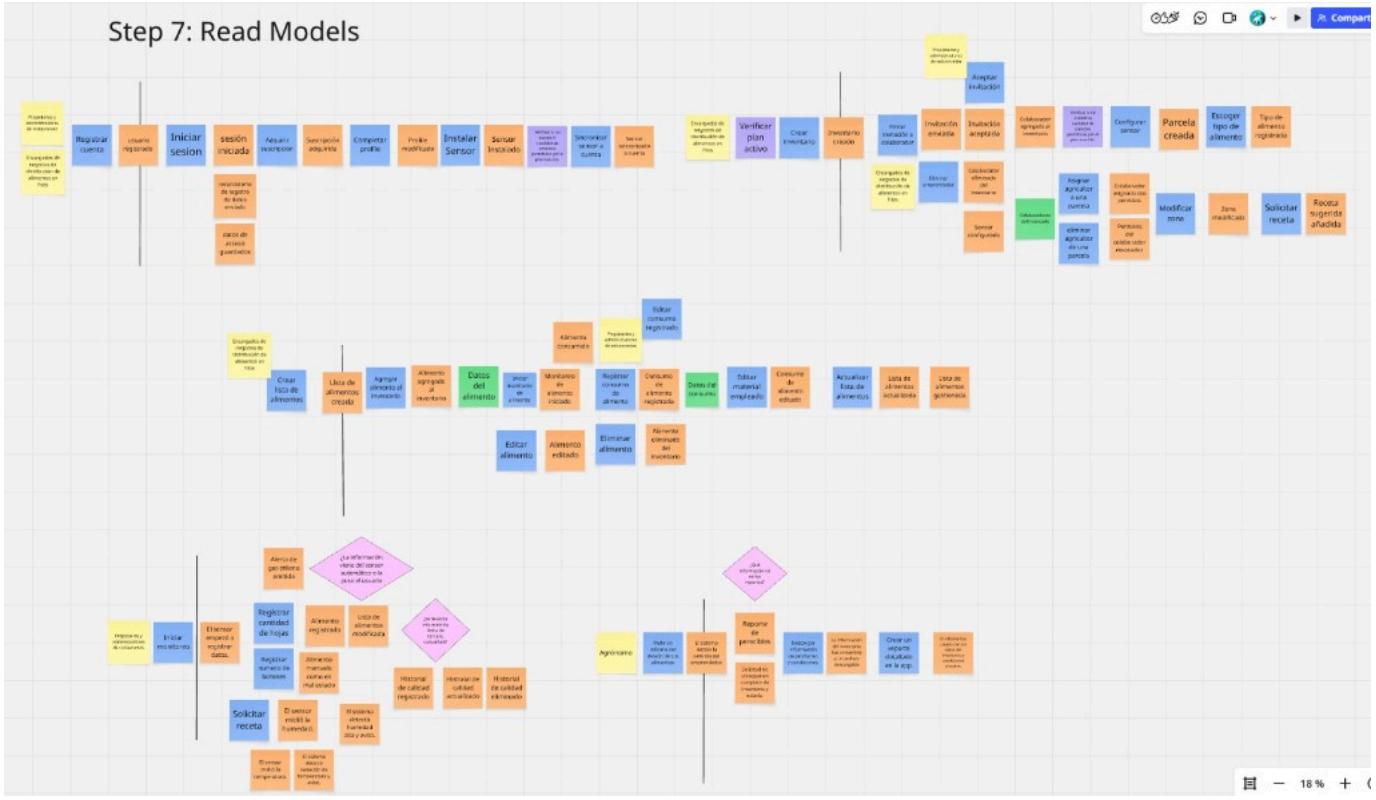

Step 2: Timelines
usuario registrado
sesión iniciada
recordatorio de registro de datos enviado
datos de acceso guardados
Suscripción adquirida
Profile modificado
Sensor Instalado
Sensor sincronizado a cuenta
Se editó el sensor
Se descativó sensor
Sensor retirado
Se activó sensor
Se creo un comunicado
Se modificó lista de alimentos
Cantidad de alimento modificada
Cantidad de alimentos registradas
Tipo de alimento registrado
Alimento marcado como en mal estado
Se eliminó un alimento
recordatorio de registro de datos enviado
Reporte solicitado
Reporte generado
Reporte exportado
Temperatura registrada
Humedad registrada
Alerta de gas etileno emitida
Historial de calidad registrado
Alerta de temperatura emitida
Alerta de humedad emitida
Historial de calidad eliminado
Historial de calidad actualizado
Receta solicitada
Receta eliminada
Receta completada
Receta generada

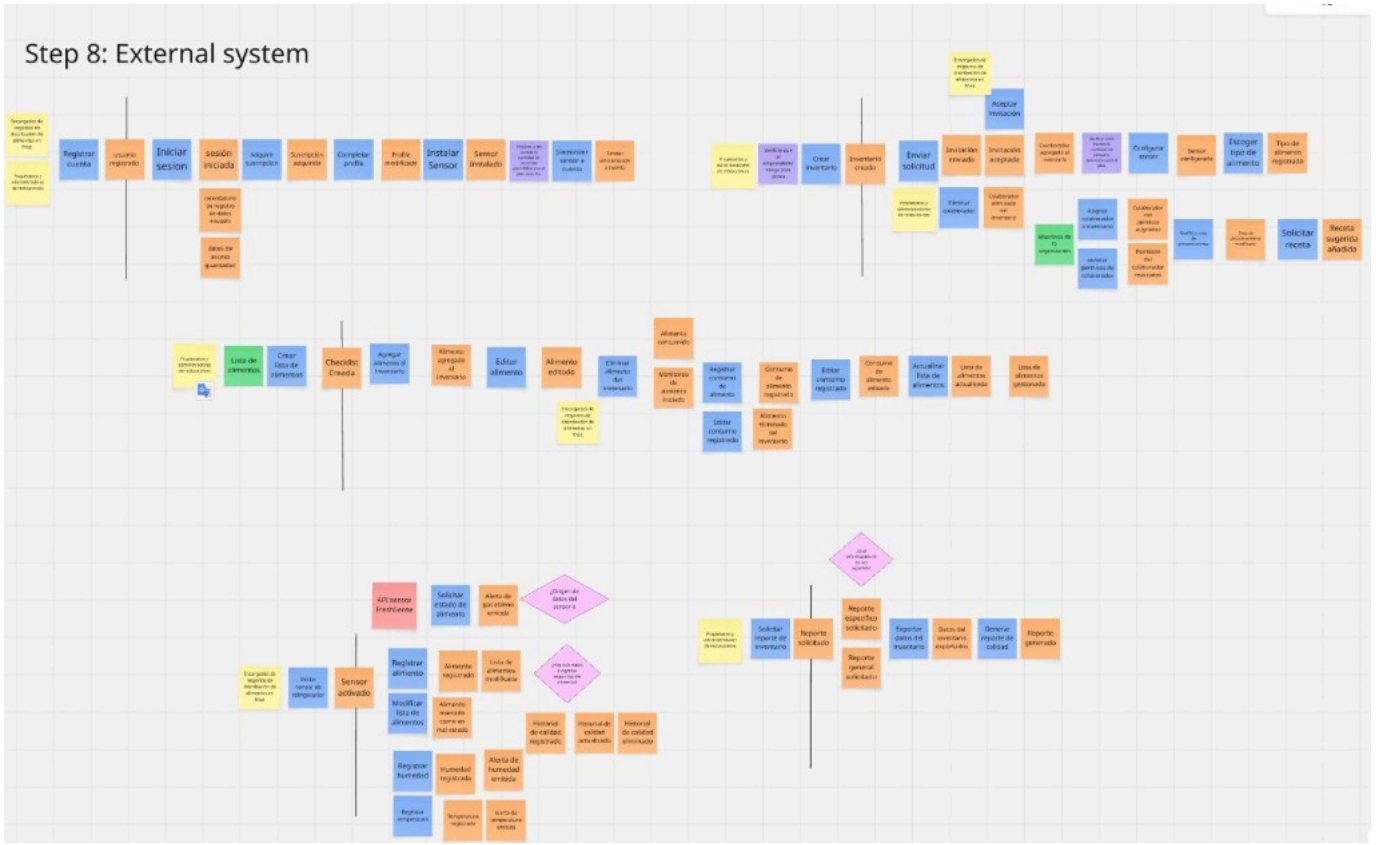

Step 3: Paint Points
usuario registrado
sesión iniciada
recordatorio de registro de datos enviado
datos de acceso guardados
Suscripción adquirida
Profile modificado
Sensor Instalado
Sensor sincronizado a cuenta
Se editó el sensor
Se descativó sensor
Sensor retirado
Se activó sensor
Se creo un comunicado
Se eliminó un alimento
recordatorio de registro de datos enviado
Se modificó lista de alimentos
Cantidad de alimento modificada
Cantidad de alimentos registradas
Tipo de alimento registrado
Alimento marcado como en mal estado
¿Qué información irá en los reportes?
Reporte solicitado
Reporte de Temperatura
Reporte de humedad
Reporte de gas etileno
Reporte generado
Historial de calidad registrado
Historial de calidad eliminado
Historial de calidad actualizado
Alerta de temperatura emitida
Alerta de humedad emitida
Alerta de gas etileno emitida
¿Origen de información?
Reporte exportado
¿Que se genera en base al registro de alimentos?
Receta solicitada
Receta eliminada
Receta completada
Receta generada

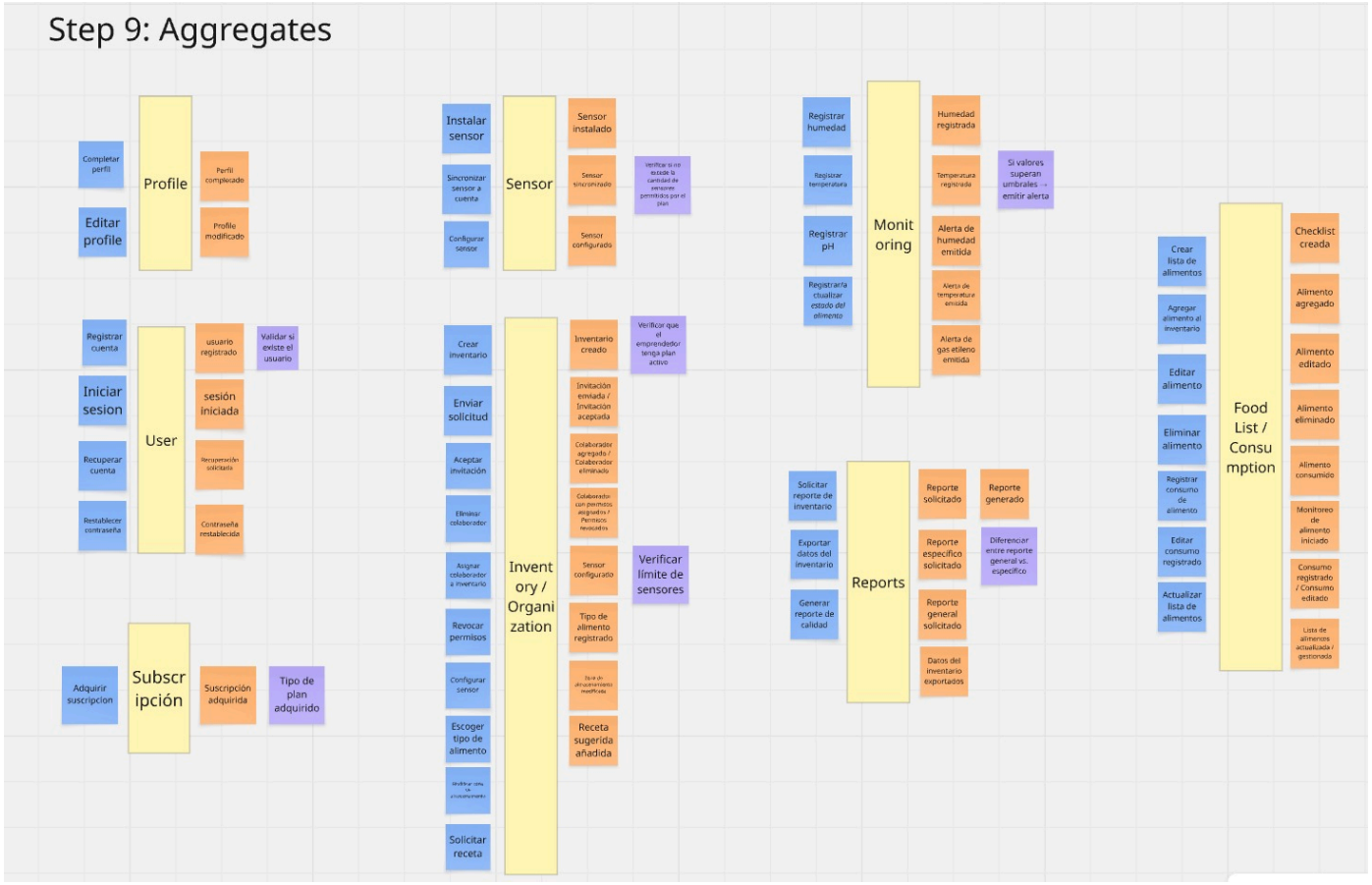

Step 4: Pivotal Points
usuario registrado
sesión iniciada
recordatorio de registro de datos enviado
datos de acceso guardados
Suscripción adquirida
Profile modificado
Sensor Instalado
Sensor sincronizado a cuenta
Se activó sensor
Se creo un comunicado
Se eliminó un alimento
Tipo de alimento registrado
Monitoreo iniciado
Se modificó lista de alimentos
Cantidad de alimento modificada
Cantidad de alimentos registradas
Alimento marcado como en mal estado
¿Qué información irá en los reportes?
Reporte de Temperatura
Reporte de humedad
Reporte de gas etileno
Reporte solicitado
Reporte generado
Historial de calidad registrado
Historial de calidad eliminado
Historial de calidad actualizado
Alerta de temperatura emitida
Alerta de humedad emitida
Alerta de gas etileno emitida
¿Origen de información?
Reporte exportado
¿Que se genera en base al registro de alimentos?
Receta solicitada
Receta eliminada
Receta completada
Receta generada

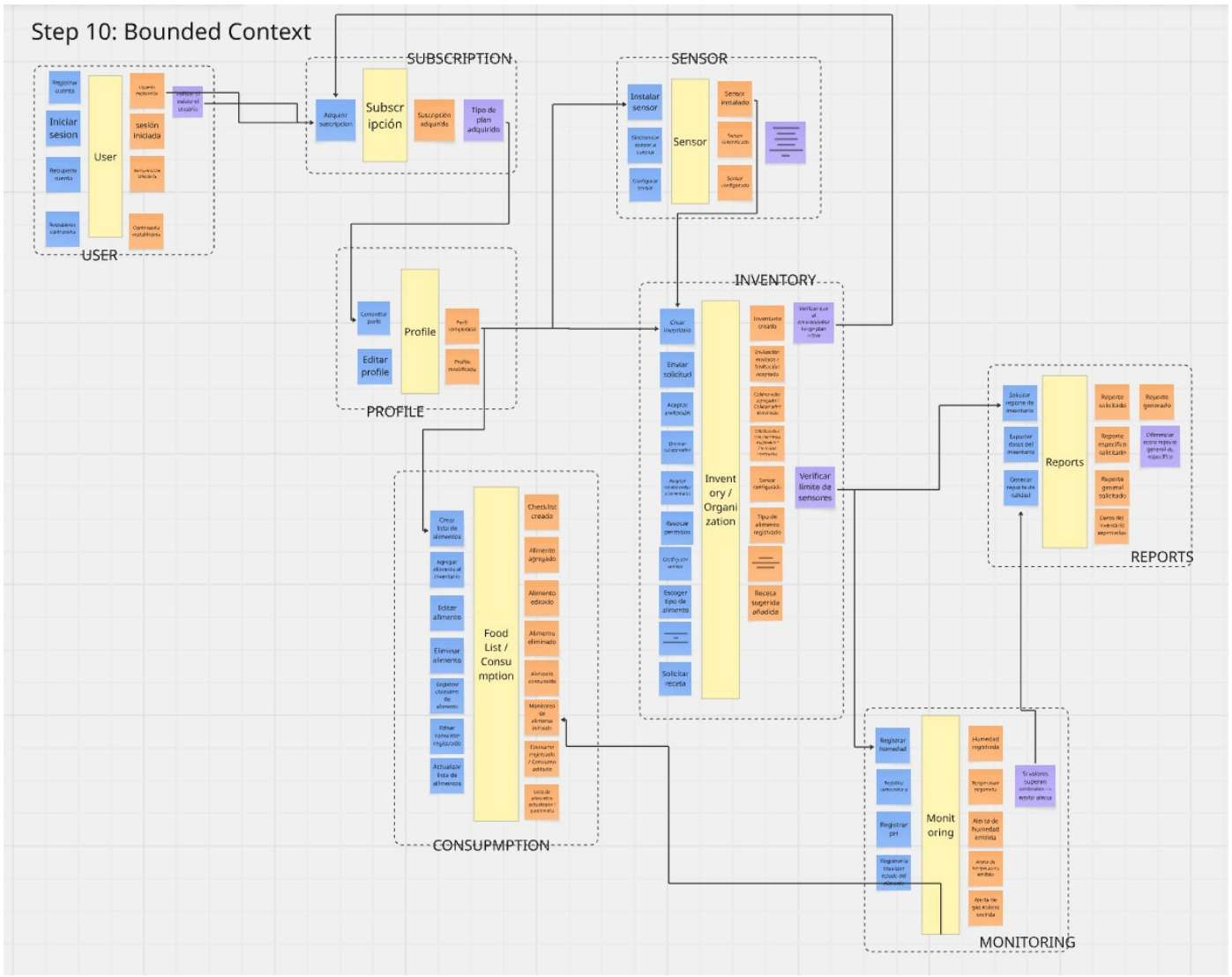

Step 5: commands


Step 6: Policies


Step 7: Read Models
Registrar cuenta
usuario registrado
Iniciar sesion
sesión iniciada
Instalar Sensor
Sensor instalado
Crear lista de alimentos
Lista de alimentos creada
Agregar alimento al inventario
Alimento agregado al inventario
Datos del alimento
Monitoreo de alimento iniciado
Registrar consumo de alimento
Consumo de alimento registrado
Editar alimento
Alimento editado
Eliminar alimento
Alimento eliminado del inventario
Verificar plan activo
Crear inventario
Inventario creado
Invitación enviada
Invitación aceptada
Parcela creada
Escoger tipo de alimento
Tipo de alimento registrado
Modificar zona
Zona modificada
Solicitar receta
Receta sugerida añadida
Editar consumo registrado
Consumo de alimento editado
Actualizar lista de alimentos
Lista de alimentos actualizada
Lista de alimentos personalizada
Solicitar receta
Registrar cantidad de hojas
Alimento registrado
Lista de alimentos modificada
Alimento marcado como en mal estado
Historial de calidad registrado
Historial de calidad actualizado
Historial de calidad eliminado
Reporte de parcelas
Agrónomo
Crear un reporte ubicado en la app
18 %


Step 8: External system
Registrar cuenta
usuario registrado
Iniciar sesion
sesión iniciada
Instalar Sensor
Sensor instalado
Crear lista de alimentos
Lista de alimentos
Checklist Creada
Agregar alimento al inventario
Alimento agregado al inventario
Editar alimento
Alimento editado
Eliminar alimento del inventario
Alimento eliminado del inventario
Monitoreo de alimento iniciado
Registrar consumo de alimento
Consumo de alimento registrado
Editar consumo registrado
Consumo de alimento editado
Actualizar lista de alimentos
Lista de alimentos actualizada
Lista de alimentos personalizada
Enviar solicitud
Invitación enviado
Invitación aceptada
Aceptar invitación
Configurar sensor
Sensor configurado
Escoger tipo de alimento
Tipo de alimento registrado
Solicitar receta
Receta sugerida añadida
Sensor activado
Registrar alimento
Alimento registrado
Modificar lista de alimentos
Alimento modificado
Alimento marcado como en mal estado
Historial de calidad registrado
Historial de calidad actualizado
Historial de calidad eliminado
Registrar humedad
Humedad registrada
Alerta de humedad emitida
Solicitar reporte de inventario
Reporte solicitado
Reporte específico solicitado
Reporte general solicitado
Reportar datos del inventario
Datos del inventario exportados
Generar reporte de calidad
Reporte generado


Step 9: Aggregates
Profile
Completar perfil
Editar profile
Perfil completado
Profile modificado
Sensor
Instalar sensor
Sincronizar sensor a cuenta
Configurar sensor
Sensor instalado
Sensor sincronizado
Sensor configurado
Verificar si no excede la cantidad de sensores permitidos por el plan
Monit oring
Registrar humedad
Registrar temperatura
Registrar pH
Registrarla cualizar estado del alimento
Humedad registrada
Temperatura registrada
Alerta de humedad emitida
Alerta de temperatura emitida
Alerta de gas etileno emitida
Si valores superan umbrales → emitir alerta
User
Registrar cuenta
Iniciar sesion
Recuperar cuenta
Restablecer contraseña
usuario registrado
sesión iniciada
Recuperación solicitada
Contraseña restablecida
Validar si existe el usuario
Invent ory / Organi zation
Crear inventario
Enviar solicitud
Aceptar invitación
Eliminar colaborador
Asignar colaborador a inventario
Revocar permisos
Configurar sensor
Escoger tipo de alimento
Solicitar receta
Inventario creado
Invitación enviada / Invitación aceptada
Colaborador agregado / Colaborador eliminado
Sensor configurado
Tipo de alimento registrado
Receta sugerida añadida
Verificar que el emprendedor tenga plan activo
Verificar límite de sensores
Reports
Solicitar reporte de inventario
Exportar datos del inventario
Generar reporte de calidad
Reporte solicitado
Reporte generado
Reporte específico solicitado
Reporte general solicitado
Datos del inventario exportados
Diferenciar entre reporte general vs. específico
Food List / Consu mption
Crear lista de alimentos
Agregar alimento al inventario
Editar alimento
Eliminar alimento
Registrar consumo de alimento
Editar consumo registrado
Actualizar lista de alimentos
Checklist creada
Alimento agregado
Alimento editado
Alimento eliminado
Alimento consumido
Monitoreo de alimento iniciado
Consumo registrado / Consumo editado
Lista de alimentos actualizada / gestionada
Subscr ipción
Adquirir suscripcion
Suscripción adquirida
Tipo de plan adquirido


Step 10: Bounded Context
SUBSCRIPTION
SENSOR
USER
PROFILE
INVENTORY
CONSUMPTION
REPORTS
MONITORING

2.5. Ubiquitous Language

Ubiquitous Term	Definition of Functional Domain
Food Waste	Loss or discard of raw materials, culinary ingredients, or commercial inventory due to poor stock rotation, cold chain disruption, or unexpected spoilage.
FreshSense Device	Sensor-based hardware placed inside commercial fridges, walk-in coolers, display cases, or cold rooms to monitor temperature, humidity, and ethylene gas levels.
Ethylene Gas	Natural plant hormone released by fruits and vegetables during ripening, monitored to prevent accelerated deterioration in kitchen storage or produce display cases.
Food Inventory	Organized tracking system for raw ingredients (for kitchens) or cold retail products (for display/storage), including expiration control and batch management.
Food Condition	Physical freshness and safety state of stored inventory monitored continuously by FreshSense (Optimal, At Risk, or Spoiled/Unusable).
Expiration Alert	Critical notification sent to staff or store managers when ingredients or retail stock are approaching expiration or entering an unviable commercial phase.
Standardized Recipe / Dish Tech Sheet	Technical recipe specification used by **Restaurants** to calculate standardized ingredient quantities, portion yields, and prevent kitchen over-consumption.
Stock Rotation / Cold Batch Management	Inventory control system (e.g., FIFO) used by **Food Cold-Sales Businesses** to prioritize the sale or removal of cold display stock prior to expiration.
Commercial Merma Report	Periodic summary detailing discarded ingredients or cold stock losses, quantifying monetary loss and supporting purchasing/stock projections.
Premium Subscription	Paid plan unlocking advanced multi-device monitoring, multi-zone/multi-cooler management, detailed merma analytics, and exportable CSV/PDF reports.
Restaurant Manager / Chef	Commercial actor (**Segment 1: Restaurants**) who uses FreshSense to optimize kitchen supply usage, track culinary mermas, and ensure dish consistency.
Cold Retail Operator / Store Manager	Commercial actor (**Segment 2: Cold-Sales Businesses**) who uses FreshSense to safeguard display cases/cold rooms, prevent merchandise loss, and audit cold chain integrity.
Cold Chain Monitoring	Uninterrupted automated tracking of storage temperatures and humidity levels in refrigeration equipment to meet safety standards and protect saleable inventory.
Smart Alert System	Priority-based notification system that routes critical alerts (e.g., fridge temperature spike) immediately to store or kitchen supervisors.
Stock & Cost Projection	Automated estimation model that predicts replenishment needs and potential financial loss based on historical mermas and current stock levels.

Capítulo III: Requirements Specification

3.1. User Stories

Las historias están escritas para los dos segmentos del proyecto: propietarios y administradores de restaurantes, y encargados de negocios de distribución de alimentos en fríos. Cada historia tiene tres escenarios.

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
US01	Visualización de propuesta B2B	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero entender la propuesta de FreshSense al entrar a la página, para ver si me ayuda a bajar la merma de mi local.	**Escenario 1: Carga limpia y mensaje claro** Dado que un visitante comercial navega a la página web Cuando la plataforma cargue completamente Entonces se mostrará el mensaje central de valor sobre reducción de mermas y conservación de la cadena de frío. **Escenario 2: Mensaje según el rubro** Dado que el visitante identifica su negocio como restaurante o como encargado de un negocio de distribución de alimentos en fríos Cuando recorra la portada Entonces verá el beneficio operativo de	**EP01** (Landing Page)

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
			su rubro: mermas de cocina o control de cámaras y vitrinas. **Escenario 3: Fallo de carga** Dado que la portada no termina de cargar Cuando el visitante espera el contenido principal Entonces verá un aviso de error con opción de reintentar, y no una pantalla vacía presentada como propuesta comercial.	
US02	Sección para Restaurantes y Negocios en Frío	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero ver la información de mi rubro, para saber qué me aporta en las cámaras.	**Escenario 1: Navegación por perfil comercial** Dado que un potencial cliente B2B está en la landing page Cuando abra la pestaña «Restaurantes» o «Distribución de alimentos en fríos» Entonces verá los módulos de mermas de cocina, control de vitrinas frías y planes corporativos de ese rubro. **Escenario 2: Cambio de rubro** Dado que el visitante está en «Restaurantes» Cuando pase a «Distribución de alimentos en fríos» Entonces el contenido cambia al segmento de encargados de negocios de distribución de alimentos en fríos sin recargar el sitio ni mezclar ambos textos. **Escenario 3: Sección no disponible** Dado que el enlace de una pestaña no resuelve a un módulo válido Cuando el visitante lo activa Entonces permanece en la vista general del portal y no ve un módulo vacío como si fuera información del rubro.	**EP01** (Landing Page)
US03	Formulario de contacto y demos B2B	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero dejar mis datos, para coordinar una demo o una prueba en el local.	**Escenario 1: Envío exitoso de consulta** Dado que el representante completa datos de contacto válidos y el tipo de negocio Cuando presione enviar Entonces el sistema registra la solicitud y muestra la confirmación en pantalla. **Escenario 2: Solicitud identificada por rubro** Dado que el gerente elige «Restaurante» o «Distribución de alimentos en fríos» Cuando la solicitud se registra Entonces la confirmación y el registro conservan ese tipo de negocio para coordinar la demo. **Escenario 3: Datos inválidos** Dado que falta el correo, el tipo de negocio o el correo no tiene formato válido Cuando presione enviar Entonces la solicitud no se registra y se señalan los campos que debe corregir.	**EP01** (Landing Page)

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
US04	Call to Action (CTA) Corporativo	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero encontrar un botón claro para pedir la demo, para no buscar el formulario.	**Escenario 1: Redirección desde el CTA** Dado que el usuario explora los planes comerciales Cuando haga clic en «Solicitar demo comercial» Entonces llega al formulario de registro de empresa. **Escenario 2: CTA en tablet o teléfono** Dado que el supervisor abre los planes desde una tablet de cocina o un teléfono Cuando pulse el mismo llamado a la acción Entonces también llega al formulario, con el botón visible y accionable. **Escenario 3: Formulario no disponible** Dado que el destino del formulario no responde Cuando pulse «Solicitar demo comercial» Entonces ve un aviso de que la solicitud no pudo abrirse y permanece en la página de planes.	**EP01** (Landing Page)
US05	Adaptabilidad en dispositivos móviles y tablets	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero abrir la plataforma en la tablet o en el celular, para revisarla mientras estoy en el local.	**Escenario 1: Ajuste a la pantalla** Dado que el supervisor ingresa desde una tablet o un navegador móvil Cuando recorra las secciones Entonces márgenes, tablas y botones se ajustan al ancho de la pantalla. **Escenario 2: Cambio de orientación** Dado que la tablet está en vertical Cuando el supervisor la gire a horizontal Entonces la sección en la que estaba sigue visible y las tablas se reacomodan sin cortar botones. **Escenario 3: Ancho mínimo** Dado que el ancho de pantalla es el mínimo soportado para operación en planta Cuando interactúe con alertas, inventario o planes Entonces el texto y los controles permanecen legibles, sin quedar fuera de la pantalla.	**EP01** (Landing Page)
US06	Telemetría y monitoreo IoT de cámaras frías	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero que el sensor mida temperatura, humedad y etileno de la cámara, para enterarme antes de que el producto se malogre.	**Escenario 1: Transmisión continua de telemetría** Dado que el dispositivo FreshSense está en línea en una cámara o vitrina Cuando detecte un cambio de temperatura, humedad o etileno Entonces envía la lectura y el panel se actualiza con esos valores. **Escenario 2: Notificación por pérdida de señal** Dado que el sensor pierde enlace con la red Cuando el servidor deja de recibir señal de esa cámara Entonces notifica al supervisor el fallo de conectividad del área afectada.	**EP02** (Monitoreo IoT)

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
			Escenario 3: Lectura incompleta Dado que el sensor envía una lectura sin temperatura, humedad o etileno Cuando el panel procese ese envío Entonces no la presenta como una medición válida de la cámara.	
US07	Dashboard de inventario con semáforo	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero ver el inventario en verde, amarillo o rojo, para saber qué está bien, qué hay que sacar pronto y qué ya está crítico.	**Escenario 1: Vista por código de colores** Dado que el usuario consulta el inventario Cuando la interfaz procese la lectura de los sensores Entonces cada insumo queda en verde (óptimo), amarillo (priorizar uso o venta) o rojo (crítico). **Escenario 2: Inspección detallada del lote** Dado que el usuario selecciona un insumo o lote Cuando se abra su ficha Entonces verá la fecha estimada de caducidad, el lote y los niveles de etileno o la variación térmica. **Escenario 3: Sin lectura utilizable** Dado que un producto no tiene una lectura reciente de su cámara Cuando el usuario consulte el inventario Entonces ese producto figura como estado desconocido y no como verde.	**EP02** (Monitoreo IoT)
US08	Sistema de alertas preventivas de temperatura	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero recibir un aviso cuando la cámara pierda frío o un producto esté en riesgo, para actuar antes de perderlo.	**Escenario 1: Notificación de quiebre de temperatura** Dado que una cámara o lote pasa a estado crítico Cuando el motor de reglas procese los datos Entonces envía una alerta push o SMS al personal responsable y la deja visible en el panel. **Escenario 2: Ausencia de falsas alarmas** Dado que la cámara se mantiene dentro de sus umbrales Cuando el sistema haga el chequeo periódico Entonces no emite ninguna notificación. **Escenario 3: Fallo del canal externo** Dado que la condición es crítica y el envío push o SMS no se entrega Cuando el motor termine de procesar el evento Entonces la alerta queda en el panel como pendiente de envío y no figura como entregada.	**EP03** (Alertas Inteligentes)
US09	Preferencias de alertas por turno de trabajo	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero configurar las alertas según el turno, para que una alerta leve no suene fuera de hora.	**Escenario 1: Alerta leve fuera de turno** Dado que se activa una alerta leve fuera del turno operativo Cuando el sistema evalúe su severidad y el horario Entonces registra el evento en el panel y no dispara alarma sonora de	**EP03** (Alertas Inteligentes)

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
			emergencia. **Escenario 2: Alerta crítica en cualquier horario** Dado que una cámara cae a estado rojo fuera del turno de cocina o de tienda Cuando el sistema evalúe la severidad Entonces notifica al responsable de guardia por el canal configurado para alertas críticas. **Escenario 3: Configuración incompleta** Dado que el administrador no indicó horario o canal Cuando intente guardar las preferencias Entonces el sistema no las aplica y señala los datos que faltan.	
US10	Alta de insumos por lector de código de barras y QR	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero dar de alta la mercadería con código de barras o QR, para no registrar el lote a mano.	**Escenario 1: Alta rápida de lote por scanner** Dado que el recepcionista escanea el código de un paquete o caja Cuando la aplicación lo identifique en el catálogo Entonces registra el insumo, la fecha de ingreso y la categoría. **Escenario 2: Nuevo ingreso de un insumo ya existente** Dado que el código corresponde a un insumo que ya está en la sucursal Cuando el recepcionista confirme la cantidad recibida Entonces suma esa cantidad al stock y conserva el registro del nuevo ingreso. **Escenario 3: Código fuera de catálogo** Dado que el código escaneado no existe en el catálogo Cuando la aplicación lo procese Entonces no crea el insumo automáticamente y ofrece completar el alta manual.	**EP04** (Gestión de Inventario)
US11	Modificación manual de stock e insumos	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero corregir las cantidades después del conteo, para que el sistema coincida con lo que hay en la cámara.	**Escenario 1: Actualización de existencias** Dado que el supervisor ajusta las unidades restantes de un insumo Cuando guarde la modificación Entonces el stock disponible queda con la nueva cantidad y se genera el registro de auditoría. **Escenario 2: Auditoría del ajuste** Dado que el ajuste se guardó Cuando un auditor consulte ese insumo Entonces ve usuario, fecha, cantidad anterior y cantidad nueva. **Escenario 3: Cantidad no válida** Dado que el supervisor ingresa una cantidad negativa o un valor no numérico Cuando intente guardar Entonces el stock no cambia y se indica que la cantidad no es válida.	**EP04** (Gestión de Inventario)

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
US12	Reporte semanal de mermas y desperdicios	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero recibir el reporte de lo descartado en la semana, para ver cuánto se perdió.	**Escenario 1: Emisión de reporte de mermas** Dado que se cumple el ciclo semanal o mensual configurado Cuando el servidor consolide las bajas y los descartes Entonces envía el resumen al correo de administración y lo publica en el panel. **Escenario 2: Consulta en el panel** Dado que el resumen del periodo ya fue consolidado Cuando el gerente lo abra en el panel antes de revisar el correo Entonces ve el mismo volumen de mermas y el mismo costo del periodo. **Escenario 3: Periodo sin mermas** Dado que en el ciclo no hubo descartes ni bajas Cuando se genere el reporte Entonces el resumen indica cero mermas y no agrega pérdidas.	**EP04** (Gestión de Inventario)
US13	Estandarización de recetas y fichas técnicas	Como propietario o administrador de un restaurante, quiero armar la ficha del platillo con sus insumos, para descontar lo que sale en cocina y usar primero lo que está por vencer.	**Escenario 1: Prioridad de insumos en alerta** Dado que existen insumos en alerta amarilla Cuando el chef consulte el módulo de recetas Entonces el sistema sugiere primero los platillos que usan esos insumos. **Escenario 2: Descuento por ficha técnica** Dado que el platillo tiene una ficha con insumos y cantidades Cuando el chef confirme la preparación Entonces el inventario descuenta las cantidades indicadas en la ficha. **Escenario 3: Stock insuficiente** Dado que un insumo de la ficha no alcanza para la preparación Cuando el chef intente confirmarla Entonces no se descuenta ningún insumo de esa preparación y se indica el faltante.	**EP05** (Recetas Personalizadas)
US14	Filtrado avanzado de insumos y lotes	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero filtrar por cámara, vencimiento, lote o categoría, para ubicar lo que tengo que mover.	**Escenario 1: Aplicación de filtros combinados** Dado que el usuario selecciona «Cámara 1», «Vegetales» y vencimiento menor a 3 días Cuando ejecute la búsqueda Entonces solo aparecen los insumos que cumplen las tres condiciones. **Escenario 2: Quitar un filtro** Dado que la lista está filtrada por cámara, categoría y vencimiento Cuando el usuario quite el filtro de categoría Entonces la lista se actualiza con los insumos que siguen cumpliendo los filtros restantes.	**EP05** (Recetas Personalizadas)

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
			Escenario 3: Sin coincidencias Dado que ningún insumo cumple la combinación elegida Cuando ejecute la búsqueda Entonces la pantalla muestra un resultado vacío y no el inventario completo.	
US15	Onboarding comercial e inducción de personal	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero dejar un tutorial para el personal nuevo, para que sepa registrar una merma y leer una alerta.	**Escenario 1: Primer ingreso** Dado que un colaborador inicia sesión por primera vez Cuando se abra la pantalla principal Entonces aparece una guía paso a paso sobre el registro de mermas y la lectura de alertas. **Escenario 2: Retomar la guía** Dado que el colaborador omitió la guía en el primer ingreso Cuando elija volver a verla desde su perfil Entonces la guía se abre en el primer paso, sin borrar su avance operativo. **Escenario 3: Ingreso posterior** Dado que el colaborador ya completó u omitió la guía Cuando vuelva a iniciar sesión Entonces entra directo al panel y la guía no bloquea la pantalla.	**EP06** (UX y Accesibilidad)
US16	Interfaz optimizada para entornos operativos	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero una pantalla clara en la tablet, para usarla en la cocina o en el almacén sin detenerme.	**Escenario 1: Lectura en planta** Dado que el trabajador abre el tablero en la tablet de cocina o almacén Cuando revise botones y alertas Entonces distingue el estado de cada insumo por texto, color y contraste suficientes para esa pantalla. **Escenario 2: Acción táctil** Dado que el trabajador necesita atender una alerta o abrir un lote Cuando pulse el control correspondiente Entonces el área táctil responde en un solo toque, sin exigir precisión de puntero. **Escenario 3: Contenido que no cabe** Dado que el nombre del insumo o el mensaje de alerta es más largo que el ancho disponible Cuando se muestre en la tablet Entonces el texto se ajusta sin tapar el semáforo ni el botón de acción.	**EP06** (UX y Accesibilidad)
US17	Analítica avanzada de rotación e inventarios	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero ver cómo rota el stock, para comprar mejor y no llenar la cámara de más.	**Escenario 1: Acceso con plan activo** Dado un usuario con plan Enterprise o B2B activo Cuando abra el menú de analítica Entonces ve gráficos de entradas, rotación y mermas del periodo. **Escenario 2: Cambio de periodo** Dado que el gerente está viendo la analítica del mes Cuando seleccione otra semana u otro mes Entonces los gráficos se recalculan solo	**EP07** (Suscripción Premium)

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
			con los movimientos de ese periodo. **Escenario 3: Plan sin analítica avanzada** Dado un usuario cuyo plan no incluye analítica Enterprise Cuando intente abrir ese menú Entonces no ve los gráficos comparativos y se informa que la función corresponde a otro plan.	
US18	Panel de impacto financiero de mermas rescatadas	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero ver cuánto dinero se salvó al usar o vender el producto antes del vencimiento, para saber si el control está sirviendo.	**Escenario 1: Ahorro del periodo** Dado que el sistema registró alimentos utilizados o vendidos antes de su vencimiento gracias a las alertas Cuando el gerente consulte el panel financiero Entonces ve la estimación en moneda local del valor rescatado. **Escenario 2: Desglose del ahorro** Dado que el periodo tiene rescates en más de una cámara o categoría Cuando abra el detalle Entonces el valor rescatado se muestra separado por categoría de insumo. **Escenario 3: Periodo sin rescates** Dado que no hubo insumos aprovechados por alerta en el periodo Cuando consulte el panel financiero Entonces el ahorro figura en cero.	**EP07** (Suscripción Premium)
US19	Módulo de fichas técnicas gourmet/avanzadas	Como propietario o administrador de un restaurante, quiero el costeo del platillo y la merma del corte, para ver si el plato deja margen.	**Escenario 1: Costeo con plan premium** Dado un restaurante con suscripción Premium o Enterprise Cuando consulte las recetas avanzadas Entonces puede calcular el rendimiento de merma por corte e insumo. **Escenario 2: Recálculo del costo** Dado que el chef modifica el rendimiento de merma de un corte Cuando guarde la ficha Entonces el costo del platillo se actualiza con ese rendimiento. **Escenario 3: Plan sin costeo avanzado** Dado un local sin suscripción Premium o Enterprise Cuando abra la sección de recetas avanzadas Entonces ve la función bloqueada y no los costos por corte.	**EP07** (Suscripción Premium)
US20	Métricas de impacto ambiental corporativo	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero ver el CO2 y el agua que se dejó de perder al rescatar producto, para tener el dato del periodo.	**Escenario 1: Reporte del periodo** Dado el registro de insumos salvados durante el periodo Cuando el usuario abra la pestaña de sostenibilidad Entonces ve el CO_2 no emitido y los litros de agua preservados de ese periodo. **Escenario 2: Otro rango de fechas** Dado que el usuario cambia el rango del reporte Cuando la pestaña se actualice	**EP08** (Sostenibilidad)

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
			Entonces las equivalencias corresponden solo a los insumos salvados en ese rango. **Escenario 3: Sin insumos salvados** Dado que el periodo no tiene insumos salvados Cuando abra la pestaña Entonces CO_2 y agua preservada figuran en cero.	
US21	Difusión de sellos de sostenibilidad	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero descargar un sello cuando cumpla la meta de merma, para mostrarlo en el local.	**Escenario 1: Meta cumplida** Dado que el local alcanzó la meta mensual de reducción de mermas Cuando presione «Generar sello verde» Entonces obtiene una imagen del sello, con el periodo y la sede, lista para difusión o impresión. **Escenario 2: Nueva descarga del mismo sello** Dado que el sello del mes ya fue generado Cuando lo vuelva a descargar Entonces obtiene la misma certificación del periodo, sin crear otra meta cumplida. **Escenario 3: Meta no alcanzada** Dado que el local no cumplió la meta mensual Cuando presione «Generar sello verde» Entonces no se emite el certificado de ese mes.	**EP08** (Sostenibilidad)
US22	Sincronización con equipos de refrigeración industrial	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero enlazar FreshSense con el equipo de frío del local, para ver sus lecturas en el mismo panel.	**Escenario 1: Emparejamiento exitoso** Dado un equipo de refrigeración compatible Cuando se complete el emparejamiento mediante API Entonces el panel central muestra la información de sus compresores y sensores. **Escenario 2: Varias cámaras del mismo equipo** Dado que el equipo vinculado controla más de una cámara Cuando el gerente abra el panel Entonces cada cámara aparece identificada con sus propias lecturas. **Escenario 3: Credenciales o equipo no compatible** Dado que la API rechaza las credenciales o el equipo no es compatible Cuando se intente el emparejamiento Entonces no se crea el vínculo y el panel no muestra lecturas de ese equipo.	**EP09** (Integración IoT)
US23	Consultas por voz en estaciones de trabajo	Como propietario o administrador de un restaurante, quiero preguntar por voz qué insumos de una cámara vencen hoy, para no soltar la preparación.	**Escenario 1: Consulta reconocida** Dado que el terminal de cocina está vinculado al asistente de voz Cuando el chef pregunte qué insumos de la Cámara 1 vencen hoy Entonces el asistente lee en voz alta los artículos en alerta de esa cámara.	**EP09** (Integración IoT)

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
			Escenario 2: Consulta de otra cámara Dado que el asistente ya respondió por la Cámara 1 Cuando el chef pregunte por otra cámara de la misma sede Entonces la respuesta corresponde solo a los insumos en alerta de esa segunda cámara. **Escenario 3: Voz no disponible o no entendida** Dado que el terminal no está vinculado o el comando no se reconoce Cuando el chef haga la consulta Entonces no se modifica el inventario y se indica que debe repetir la consulta o usar la pantalla.	
US24	Ruteo de alertas multidispositivo por jerarquía	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero elegir si la alerta crítica le llega al responsable de turno o se muestra en la pantalla del local.	**Escenario 1: Alerta roja al gerente de turno** Dado que hay una caída crítica de temperatura en la madrugada y el destino configurado es el gerente de guardia Cuando se dispare la alerta roja Entonces el aviso llega al teléfono de ese gerente. **Escenario 2: Alerta operativa a la pantalla de cocina** Dado que la regla envía las alertas de prioridad de cocina a la pantalla del turno Cuando se active una de esas alertas dentro del horario de cocina Entonces se muestra en la pantalla de cocina y no se trata como aviso de madrugada al gerente. **Escenario 3: Destinatario sin canal** Dado que la alerta es crítica y el gerente de guardia no tiene un teléfono configurado Cuando se dispare Entonces queda visible en el panel con el estado de destinatario no disponible y no figura como entregada al teléfono.	**EP09** (Integración IoT)
US25	Gamificación y metas de equipo	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero poner una meta de merma al equipo, para que registren y cuiden el stock.	**Escenario 1: Meta cumplida** Dado que la cocina o tienda cumple el periodo configurado sin mermas por negligencia Cuando el sistema cierre ese periodo Entonces otorga la insignia al perfil de la sucursal y suma los puntos de la meta. **Escenario 2: Puntos visibles para el equipo** Dado que la sucursal recibió la insignia Cuando el jefe de cocina o tienda abra el perfil de la sede Entonces ve la insignia y los puntos acumulados de esa meta. **Escenario 3: Meta no cumplida** Dado que en el periodo hubo una merma registrada por negligencia	**EP10** (Gamificación)

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
			Cuando el sistema cierre el periodo Entonces no otorga la insignia ni suma esos puntos.	
US26	Red de proveedores e intercambio de buenas prácticas	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero ver y dejar consejos de conservación con otros locales del mismo rubro.	**Escenario 1: Publicación en la red B2B** Dado que el usuario entra a la comunidad de comercios Cuando publique un consejo de calibración o conservación Entonces los demás gerentes registrados pueden verlo y responder. **Escenario 2: Guardar una sugerencia** Dado que otro gerente publicó una práctica de conservación Cuando el usuario elija guardarla Entonces queda en sus sugerencias guardadas y sigue visible en la comunidad. **Escenario 3: Publicación vacía** Dado que el mensaje no tiene contenido Cuando intente publicarlo Entonces no aparece en la comunidad.	**EP10** (Gamificación)
US27	Benchmarking entre sucursales	Como propietario o administrador de un restaurante con varias sedes, o como encargado de un negocio de distribución de alimentos en fríos con varias sedes, quiero comparar la merma entre locales, para ver cuál está perdiendo más.	**Escenario 1: Comparativa del mes** Dado que la empresa tiene 3 o más sucursales con datos del mes Cuando consulte la comparativa Entonces las sedes quedan ordenadas de menor a mayor porcentaje de mermas. **Escenario 2: Otro mes** Dado que el director cambia el mes de la comparativa Cuando el panel se actualice Entonces el orden usa solo las mermas de ese mes. **Escenario 3: Menos de tres sucursales** Dado que la empresa tiene una o dos sucursales Cuando abra la comparativa Entonces ve el aviso de que el ranking requiere al menos tres sedes y no un orden presentado como benchmarking completo.	**EP10** (Gamificación)
US28	Categorización personalizada de insumos y cámaras	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero crear mis categorías, para ordenar el stock como lo trabajo en el local.	**Escenario 1: Nueva categoría** Dado que el usuario crea la categoría «Mariscos congelados» Cuando catalogue un lote nuevo Entonces puede asignarlo a esa categoría, filtrarlo y definir alertas para ese grupo. **Escenario 2: Alerta de la categoría** Dado que «Mariscos congelados» tiene una regla de alerta propia Cuando un lote de esa categoría entre en la condición configurada Entonces la alerta se genera con esa categoría y no con una regla de otro grupo. **Escenario 3: Nombre repetido** Dado que ya existe una categoría con el	**EP11** (Gestión Avanzada)

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
			mismo nombre en la sede Cuando intente crearla otra vez Entonces no se duplica.	
US29	Historial de mermas e inventario exportable	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero descargar el historial de mermas en Excel o CSV, para pasarlo a contabilidad.	**Escenario 1: Exportación del rango** Dado que el usuario define un rango de fechas y una sucursal Cuando presione «Exportar historial comercial» Entonces descarga un CSV o Excel con lotes, cantidades, causas y costos de ese rango. **Escenario 2: Contenido del archivo** Dado que en el rango hay ingresos, consumos y descartes Cuando abra el archivo descargado Entonces cada movimiento incluye fecha, sucursal, lote, acción y costo. **Escenario 3: Rango sin movimientos** Dado que la sucursal no tiene movimientos en las fechas elegidas Cuando intente exportar Entonces no descarga un archivo con filas de movimiento e informa que no hay datos en ese rango.	**EP11** (Gestión Avanzada)
US30	Algoritmo de sugerencias de reposición de stock	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero una sugerencia de cuánto reponer según lo consumido, para no comprar de más.	**Escenario 1: Orden sugerida** Dado el historial de los últimos 30 días de la sucursal Cuando el encargado prepare la orden de compra Entonces la plataforma recomienda cantidades de reposición para los insumos con historial suficiente. **Escenario 2: Ajuste de la sugerencia** Dado que el sistema recomendó una cantidad Cuando el comprador la modifique antes de usar la orden Entonces la orden conserva la cantidad editada y deja visible la cantidad que había sugerido el sistema. **Escenario 3: Historial insuficiente** Dado que un insumo no tiene consumo suficiente en los últimos 30 días Cuando se prepare la sugerencia Entonces ese insumo no recibe una cantidad recomendada y se indica que falta historial.	**EP11** (Gestión Avanzada)
US31	Vinculación y calibración de sensores de cámaras	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero vincular el sensor leyendo su QR y guardar el límite de temperatura de esa cámara.	**Escenario 1: Vinculación QR exitosa** Dado que el supervisor abre el módulo de hardware Cuando escanee el QR del sensor de la cámara de congelación e indique el umbral Entonces el sensor queda asociado a esa zona y calibrado con la temperatura límite definida. **Escenario 2: Umbral distinto por tipo de cámara** Dado que una sede tiene cámara de congelación y vitrina de refrigeración	**EP02** (Monitoreo IoT)

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
			Cuando el supervisor calibre cada sensor con su propio límite Entonces cada zona conserva su umbral y no hereda el de la otra. **Escenario 3: QR inválido o ya usado** Dado que el código no corresponde a un sensor FreshSense o ya está vinculado a otra zona Cuando el supervisor intente asociarlo Entonces el sistema no lo vincula e indica el motivo.	
US32	Modo de mantenimiento de equipos fríos	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero pausar las alarmas de una cámara mientras se limpia o se repara.	**Escenario 1: Pausa de alertas de la cámara** Dado que el técnico activa el modo mantenimiento en la Cámara 2 por 60 minutos Cuando confirme la pausa Entonces el sistema suspende las alertas de puerta abierta o alza de temperatura solo en esa cámara. **Escenario 2: Fin del tiempo programado** Dado que el modo mantenimiento de la Cámara 2 está activo Cuando se cumplan los 60 minutos Entonces las alertas de esa cámara vuelven a evaluarse solas. **Escenario 3: Activación inválida** Dado que el técnico no eligió una cámara o indicó una duración no permitida Cuando confirme el modo mantenimiento Entonces no se pausa ninguna alerta.	**EP03** (Alertas Inteligentes)
US33	Gestión de múltiples áreas y cámaras frías	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero crear las zonas del local, como la cocina, la cámara de carnes o la vitrina, y que cada una use su regla de frío.	**Escenario 1: Zona con sensor** Dado que el usuario crea la zona «Vitrinas lácteos» Cuando vincule un sensor a esa zona Entonces los productos almacenados allí usan las reglas de frío de ese exhibidor. **Escenario 2: Cambio de zona de un producto** Dado que un producto pasa de «Vitrinas lácteos» a otra cámara con otras reglas Cuando se confirme el traslado Entonces el producto queda evaluado con las reglas de la cámara de destino. **Escenario 3: Zona duplicada o sin nombre** Dado que el nombre de la zona está vacío o ya existe en la sede Cuando intente crearla Entonces no se agrega la zona y los sensores siguen en sus zonas actuales.	**EP11** (Gestión Avanzada)
US34	Registro de causas corporativas de merma	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero anotar por qué se dio de baja un lote: falla de frío, vencimiento, contaminación o el proveedor.	**Escenario 1: Causa obligatoria** Dado que un empleado da de baja un lote Cuando la aplicación solicite el motivo Entonces guarda una causa válida, como falla de frío, caducidad, contaminación	**EP04** (Gestión de Inventario)

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
			cruzada o mal estado de proveedor. **Escenario 2: Causa disponible para auditoría** Dado que la baja quedó con una causa Cuando el auditor abra los gráficos del periodo Entonces esa baja aparece agrupada por el motivo registrado. **Escenario 3: Baja sin causa** Dado que el empleado no selecciona un motivo Cuando confirme el descarte Entonces el lote no se da de baja.	
US35	Recuperación corporativa de contraseña	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero recuperar mi clave por el correo del local cuando no pueda entrar.	**Escenario 1: Envío del enlace** Dado que el usuario solicita recuperar la clave con su correo corporativo registrado Cuando presione «Enviar enlace de recuperación» Entonces recibe un enlace de un solo uso, con vencimiento, en ese correo. **Escenario 2: Clave nueva** Dado que el enlace está vigente y no fue usado Cuando defina y confirme la nueva clave Entonces puede ingresar con la clave nueva y el enlace queda inutilizado. **Escenario 3: Enlace vencido o ya usado** Dado que el enlace expiró o ya se utilizó Cuando intente definir otra clave con ese mismo enlace Entonces la clave no cambia.	**EP12** (Core API)
US36	Panel de administración de usuarios y roles B2B	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero cambiar el rol de un trabajador y quitarle el acceso si ya no está en el local.	**Escenario 1: Cambio de rol** Dado un colaborador con rol «Cocinero» Cuando el administrador lo actualice a «Jefe de cocina» Entonces puede autorizar bajas de insumos y editar fichas técnicas desde ese momento. **Escenario 2: Suspensión de acceso** Dado un colaborador con acceso activo Cuando el administrador suspenda su cuenta Entonces no puede iniciar sesión ni operar el inventario de la sede. **Escenario 3: Reducción de permisos** Dado un jefe de cocina con permiso para autorizar bajas Cuando el administrador lo regrese al rol «Cocinero» Entonces deja de poder autorizar bajas y editar fichas desde la siguiente acción.	**EP12** (Core API)
US37	Gestión de colaboradores de local/sucursal	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero invitar a mi personal para que vea el inventario de su sede y de su turno.	**Escenario 1: Aceptación de la invitación** Dado que el gerente envía una invitación al correo del nuevo supervisor Cuando el invitado acepte el enlace Entonces accede al inventario de su	**EP04** (Gestión de Inventario)

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
			turno y su sucursal. **Escenario 2: Alcance de la sucursal** Dado que el colaborador aceptó la invitación de una sede Cuando consulte el inventario Entonces solo opera sobre la sucursal asignada. **Escenario 3: Invitación no vigente** Dado que el enlace expiró o el correo ya tiene acceso vigente a esa sede Cuando intente aceptarlo Entonces no se crea otro acceso y se informa que la invitación no es válida.	
US38	Descarga de facturación y comprobantes B2B	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero descargar el comprobante de la suscripción, para entregarlo a contabilidad.	**Escenario 1: Descarga del comprobante** Dado que el cobro mensual de la suscripción corporativa ya fue procesado Cuando el contador entre a «Facturación B2B» Entonces descarga el comprobante en PDF y los datos fiscales. **Escenario 2: Comprobante del periodo correcto** Dado que existen cobros de varios meses Cuando el contador elija un mes Entonces descarga solo el comprobante de ese periodo. **Escenario 3: Cobro aún no procesado** Dado que el periodo seleccionado no tiene un cobro confirmado Cuando intente descargarlo Entonces no se genera un comprobante de ese mes.	**EP07** (Suscripción Premium)
US39	Búsqueda global de inventario y lotes	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero buscar un insumo o un lote por nombre y ver en qué cámara está y cuándo vence.	**Escenario 1: Búsqueda de un insumo** Dado que el usuario escribe «Lomo» en el buscador Cuando el sistema procese el texto Entonces muestra la cámara, la fecha de caducidad y los lotes asociados de su sucursal. **Escenario 2: Sugerencias mientras escribe** Dado que el usuario escribió al menos los caracteres mínimos de búsqueda Cuando pause un instante sobre el buscador Entonces aparecen sugerencias de insumos, lotes y recetas coincidentes antes de confirmar. **Escenario 3: Sin resultados en la sucursal** Dado que el texto no coincide con insumos, lotes ni recetas de su sede Cuando ejecute la búsqueda Entonces ve un resultado vacío y no registros de otra sucursal.	**EP06** (UX y Accesibilidad)

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
US40	Evaluación de calidad de proveedores de insumos	Como propietario o administrador de un restaurante, o como encargado de un negocio de distribución de alimentos en fríos, quiero anotar si la mercadería de un proveedor se malogró antes de tiempo, para tener su historial.	**Escenario 1: Incidencia sobre un lote** Dado que un lote de verduras se deteriora antes del tiempo previsto Cuando el usuario registre la incidencia en la ficha de la compra Entonces se actualiza la puntuación histórica de frescura de ese proveedor. **Escenario 2: Historial del proveedor** Dado que el proveedor ya tiene evaluaciones anteriores Cuando el jefe de compras abra su ficha Entonces ve las notas, los lotes afectados y la puntuación acumulada. **Escenario 3: Evaluación sin lote** Dado que la incidencia no está asociada a un lote recibido Cuando intente guardarla Entonces la puntuación del proveedor no cambia.	**EP05** (Recetas Personalizadas)
TS41	API Ingesta de Sensores IoT B2B	Como desarrollador del backend, quiero guardar las lecturas de temperatura, humedad y etileno de las cámaras, para que los propietarios y administradores de restaurantes y los encargados de negocios de distribución de alimentos en fríos vean el estado en el panel.	**Escenario 1: Lectura aceptada** Dado que un sensor autenticado envía temperatura, humedad y etileno Cuando el backend valide su token Entonces guarda la lectura y responde HTTP 201. **Escenario 2: Lectura duplicada** Dado que esa misma lectura ya fue almacenada con su identificador Cuando el sensor la reenvíe Entonces el backend no crea otro registro y responde que la lectura ya existe. **Escenario 3: Token inválido** Dado que la petición no trae un token válido del hardware Cuando llegue al endpoint de ingesta Entonces no guarda la lectura y responde HTTP 401.	**EP12** (Core API)
TS42	Microservicio de Notificaciones de Emergencia	Como desarrollador del backend, quiero un servicio aparte que envíe la alerta de falla de frío por push, SMS o correo a los dos segmentos.	**Escenario 1: Alerta crítica despachada** Dado un alza crítica de temperatura en una cámara Cuando el microservicio reciba el evento Entonces procesa la plantilla de emergencia y la entrega al proveedor de mensajería configurado. **Escenario 2: Evento que no es de emergencia** Dado un evento informativo que no es falla de frío Cuando llegue al microservicio Entonces no usa la plantilla de emergencia ni el canal de aviso crítico. **Escenario 3: Proveedor no disponible** Dado un evento crítico y una falla del proveedor de mensajería Cuando el microservicio intente el despacho Entonces deja el aviso en reintento y no lo marca como entregado.	**EP12** (Core API)

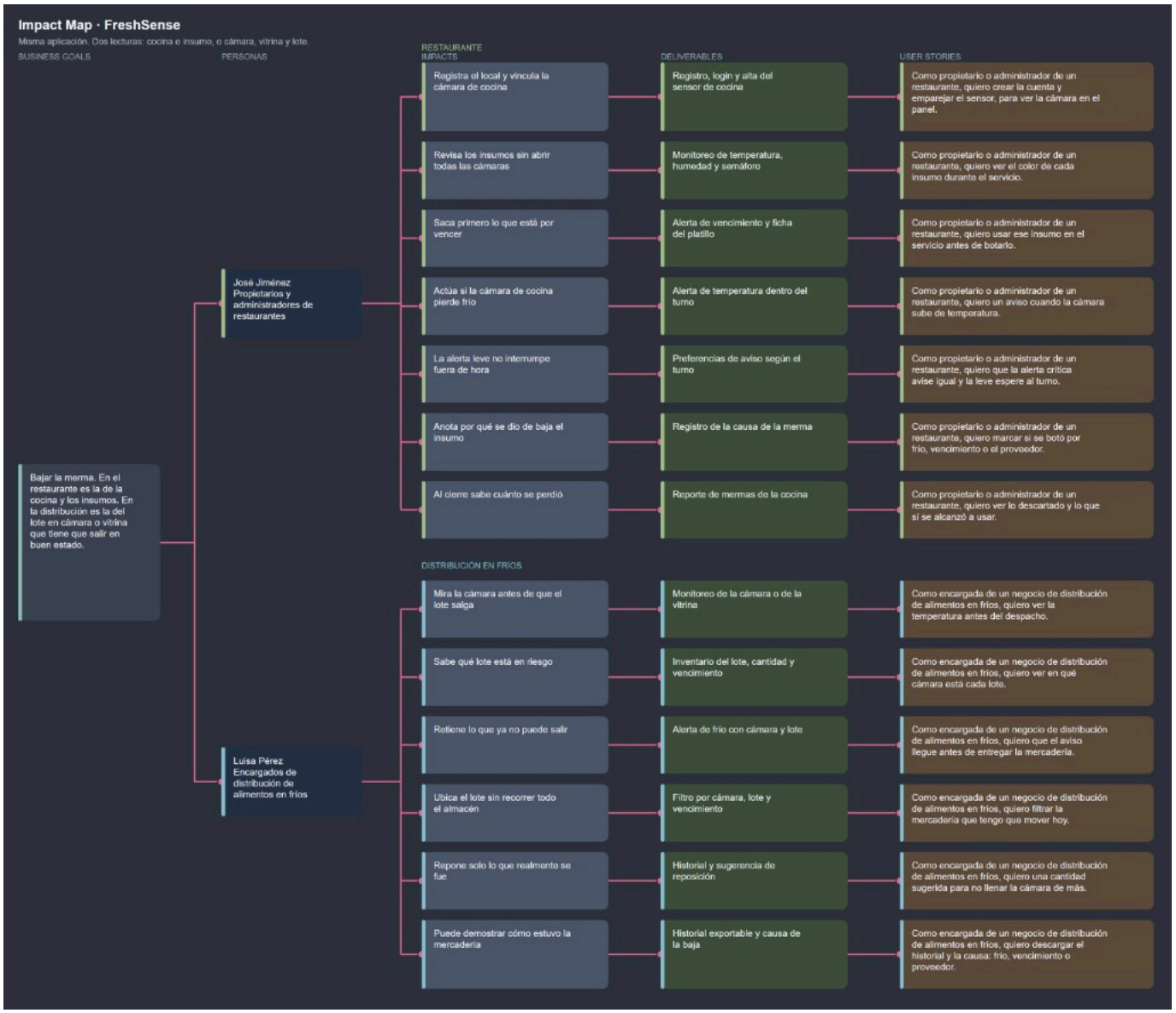
User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
TS43	API Gestor de Autenticación y RBAC B2B	Como desarrollador del backend, quiero controlar el acceso por rol, para que el personal del restaurante o del negocio de distribución en fríos solo entre a lo que le corresponde.	**Escenario 1: Rol sin permiso** Dado un usuario con rol «Cocinero» y un token vigente Cuando solicite facturación o analítica avanzada Entonces el middleware responde HTTP 403 y no entrega esos datos. **Escenario 2: Rol con permiso** Dado un usuario cuyo rol sí incluye analítica avanzada y un token vigente Cuando solicite ese recurso Entonces el middleware permite la consulta y responde HTTP 200. **Escenario 3: Token ausente o vencido** Dado que la petición no incluye token o el token expiró Cuando llegue a un endpoint protegido Entonces responde HTTP 401 y no evalúa el rol como si la sesión fuera válida.	**EP12** (Core API)
TS44	Integración de Pasarela de Pagos B2B (Stripe/PayPal)	Como desarrollador del backend, quiero que el pago del plan active la suscripción del restaurante o del negocio de distribución en fríos y deje el comprobante.	**Escenario 1: Pago confirmado** Dado que la pasarela procesa el pago del plan Enterprise Cuando el servidor reciba el webhook de confirmación Entonces extiende la vigencia de la suscripción y deja disponible el comprobante. **Escenario 2: Pago rechazado** Dado que la pasarela notifica un pago rechazado Cuando el servidor reciba ese webhook Entonces la vigencia actual no se extiende y el intento queda registrado. **Escenario 3: Webhook repetido** Dado que el mismo pago confirmado se notifica otra vez Cuando el servidor reciba el webhook duplicado Entonces no extiende de nuevo el mismo periodo ni emite otro comprobante por ese cobro.	**EP07** (Suscripción Premium)
TS45	Caching de Consultas Frecuentes e Inventarios en Tiempo Real	Como desarrollador del backend, quiero que el inventario de la sede cargue rápido, para que el personal de los dos segmentos no espere al abrir el stock.	**Escenario 1: Respuesta desde caché** Dado que el inventario de la sucursal ya fue consultado y sigue vigente en caché Cuando otro usuario de la misma sucursal lo abra Entonces la API responde desde Redis en menos de 50 ms con ese inventario. **Escenario 2: Inventario modificado** Dado que se registró un alta, consumo o descarte en la sucursal Cuando otro usuario abra el inventario después de ese cambio Entonces la respuesta refleja la cantidad nueva y no la copia anterior de Redis. **Escenario 3: Redis no disponible** Dado que Redis no responde Cuando un usuario abra el inventario	**EP12** (Core API)

User Story ID	Título	Descripción	Criterios de Aceptación (Gherkin)	Relacionado con (Epic)
			Entonces la API consulta el almacén principal y devuelve el inventario vigente, sin entregar una lista vacía por la falla de caché.	

Epics (Épicas)

Epic ID	Título de la Épica	Alcance y Descripción Corta
EP01	Portal Informativo y Landing Page B2B	Estructuración y despliegue del sitio web principal enfocado en la conversión de clientes comerciales, destacando la propuesta de valor para **Restaurantes** y **Negocios en Frío**, demostraciones interactivas, retorno de inversión (ROI) y captura de leads B2B.
EP02	Telemetría e Ingesta IoT Comercial	Desarrollo del sistema continuo de captura, calibración y procesamiento en tiempo real de las lecturas de sensores (temperatura, porcentaje de humedad y niveles de gas etileno) instalados en cámaras industriales, walk-in coolers y vitrinas comerciales.
EP03	Motor de Alertas e Indicadores de Riesgo de Frío	Algoritmo inteligente encargado de evaluar las desviaciones métricas frente a los umbrales de seguridad de la cadena de frío, gestionando notificaciones de emergencia (SMS/Push), ruteo por turnos y modo de mantenimiento técnico.
EP04	Gestión Dinámica de Inventario y Lotes Commerciales	Módulo para la organización integral de insumos y productos en frío, incluyendo altas rápidas por código de barras/QR, control de lotes y fechas de vencimiento, arqueos manuales de stock y gestión de colaboradores por sucursal.
EP05	Fichas Técnicas de Cocina y Rotación Comercial	Sistema dinámico que gestiona la estandarización de recetas/costeo para **Restaurantes** (salida automática de insumos) y la rotación FIFO de productos para **Negocios en Frío** (evaluación de proveedores e insumos próximos a vencer).
EP06	Experiencia de Usuario, Accesibilidad y Navegación Operativa	Diseño UX/UI optimizado para entornos de alta operatividad (tablets de cocina/tienda) bajo estándares WCAG AA, que contempla búsquedas globales de lotes, vistas responsivas y onboarding guiado para el personal operativo.
EP07	Modelo de Suscripción SaaS y Facturación B2B	Módulo de monetización encargado del cobro recurrente mediante webhooks (Stripe/PayPal), emisión de comprobantes y facturación electrónica, y el desbloqueo de capacidades corporativas Enterprise.
EP08	Analítica de Sostenibilidad y Reportes ESG	Sistema de cálculo enfocado en cuantificar el volumen de insumos y mercadería salvada, la reducción estimada de la huella de carbono (kg de CO_2 evitado), el agua preservada y la emisión de sellos/certificados de sostenibilidad para empresas.
EP09	Conectividad y Ecosistemas Smart Industrial	Módulos de integración mediante API con equipos de refrigeración comercial/industrial, asistentes de voz para estaciones de trabajo (manos libres en cocina) y ruteo jerárquico de alertas hacia relojes y teléfonos del personal de turno.
EP10	Gamificación, Metas de Equipo y Red B2B	Funcionalidades de interacción comercial que incentivan el desperdicio cero mediante metas por sucursal, foros B2B para intercambio de buenas prácticas de conservación y rankings comparativos entre sedes de la empresa.
EP11	Analítica Comercial y Gestión Avanzada B2B	Herramientas orientadas a la gestión por cámaras frías, registro de causas corporativas de merma (falla de frío, vencimiento, mala recepción), exportación de reportes contables (Excel/PDF) y sugerencias automáticas de reposición de stock.
EP12	Arquitectura Backend, Seguridad y Core API B2B	Construcción de la infraestructura del servidor, ingestión asíncrona de alta velocidad para sensores IoT, capa de almacenamiento en caché con Redis, recuperación segura de claves y control de acceso basado en roles corporativos (RBAC JWT).

3.2. Impact Mapping



El Impact Mapping junta lo que quiere el negocio con lo que vamos a construir, para los dos segmentos: propietarios y administradores de restaurantes, y encargados de negocios de distribución de alimentos en fríos. La idea es que cada función sirva para bajar la merma de la cocina o para cuidar la mercadería que está en frío.

3.3. Product Backlog

El Product Backlog junta las 45 historias, ordenadas por prioridad. El esfuerzo está en story points con la escala de Fibonacci (1, 2, 3, 5, 8). Las historias sirven a los dos segmentos: propietarios y administradores de restaurantes, y encargados de negocios de distribución de alimentos en fríos.

Orden (#)	User Story ID	Título de la Historia / Tarea Técnica	Descripción Sintetizada	Story Points
1	**US01**	Visualización de propuesta B2B	Explicación clara de la propuesta de valor comercial de FreshSense en la landing page.	**2**
2	**US02**	Sección para Restaurantes y Negocios en Frío	Módulo web orientado a gerentes gastronómicos y comercios de frío para destacar ventajas del ROI.	**2**
3	**US03**	Formulario de contacto y demos B2B	Formulario dinámico para la recepción de solicitudes de información y coordinar pruebas piloto.	**2**
4	**US04**	Call to Action (CTA) Corporativo	Botón de llamado a la acción para la conversión rápida hacia la solicitud de demo o prueba comercial.	**2**
5	**US05**	Adaptabilidad en móviles y tablets	Configuración de estilos responsivos para lectura en tablets de cocina y teléfonos de supervisores.	**3**
6	**TS43**	API Gestor de Autenticación y RBAC B2B	Configuración del servidor de identidad con seguridad basada en tokens JWT y roles comerciales.	**5**
7	**TS41**	API Ingesta de Sensores IoT B2B	Endpoint RESTful/MQTT para recibir y almacenar telemetría de cámaras frías en tiempo real.	**5**
8	**US06**	Telemetría y monitoreo IoT de cámaras frías	Recolección en tiempo real de métricas enviadas por sensores instalados en refrigeración comercial.	**5**

Orden (#)	User Story ID	Título de la Historia / Tarea Técnica	Descripción Sintetizada	Story Points
9	**US07**	Dashboard con semáforo comercial	Módulo visual que clasifica la frescura de insumos y stock mediante colores verde, amarillo y rojo.	**3**
10	**US08**	Alertas preventivas de temperatura	Notificaciones automáticas ante el riesgo de ruptura de cadena de frío o descomposición.	**5**
11	**TS42**	Microservicio de Notificaciones de Emergencia	Módulo asíncrono para emisión prioritaria de avisos push, SMS y correo sobre fallas de frío.	**3**
12	**US09**	Preferencias de alertas por turno	Panel para parametrizar horarios de notificación y canales según el turno del personal operativo.	**3**
13	**US31**	Vinculación y calibración de sensores	Módulo de emparejamiento mediante QR y calibración de rangos de temperatura para cámaras.	**3**
14	**US10**	Alta de insumos por lector de código/QR	Registro rápido de lotes e ingredientes escaneando códigos de barra o empaques recibidos.	**5**
15	**US11**	Modificación manual de stock e insumos	Ajuste y corrección de cantidades y categorías tras realizar arqueos de inventario.	**2**
16	**US37**	Gestión de colaboradores por local	Invitación y asignación de permisos a empleados del restaurante o tienda sobre el inventario.	**3**
17	**US12**	Reporte semanal de mermas	Consolidado periódico que detalla el volumen y costo de insumos consumidos y descartados.	**3**
18	**US13**	Fichas técnicas y rotación de insumos	Algoritmo que sugiere uso prioritario en recetas (Restaurantes) o salida FIFO (encargados de negocios de distribución de alimentos en fríos).	**5**
19	**US14**	Filtrado avanzado de insumos y lotes	Búsqueda y filtrado por cámara fría, fecha de caducidad, lote o categoría de producto.	**3**
20	**US39**	Búsqueda global de inventario y lotes	Barra de búsqueda centralizada para ubicar insumos, lotes y recetas con sugerencias en vivo.	**3**
21	**US15**	Onboarding comercial interactivo	Guía interactiva para capacitar al nuevo personal operativo en el uso del sistema.	**2**
22	**US16**	Interfaz optimizada para operación	Optimización UX/UI con alto contraste para uso fluido en tablets de cocina y almacén.	**3**
23	**TS44**	Pasarela de Pagos Recurrentes B2B	Integración de webhooks para cobros periódicos de planes corporativos (Stripe/PayPal).	**5**
24	**US17**	Analítica avanzada de rotación e inventario	Dashboard corporativo con gráficos de velocidad de rotación (FIFO) e insumos críticos.	**5**
25	**US18**	Panel de impacto financiero de mermas	Métricas que cuantifican en moneda local el valor monetario rescatado al evitar mermas.	**3**
26	**US19**	Fichas técnicas gourmet y costeo	Módulo avanzado de costeo de platillos y cálculo de mermas por corte para cocinas.	**2**
27	**US38**	Descarga de facturación B2B	Módulo de consulta y descarga de comprobantes contables y facturas electrónicas en PDF/XML.	**2**
28	**US20**	Métricas de impacto ambiental corporativo	Estimación del equivalente en CO_2 evitado y agua preservada para auditorías ESG.	**3**
29	**US21**	Difusión de sellos de sostenibilidad	Generador de certificados y sellos de "Desperdicio Cero" para difusión en redes y locales.	**2**
30	**US32**	Modo de mantenimiento de equipos fríos	Pausa temporal de alarmas durante periodos de limpieza o mantenimiento técnico de cámaras.	**2**
31	**US33**	Gestión de cámaras y áreas de frío	Configuración y jerarquización de múltiples cámaras frías y vitrinas dentro de la sede.	**3**
32	**US34**	Registro de causas corporativas de merma	Clasificación obligatoria del motivo de baja (falla de frío, vencimiento, mala recepción).	**2**
33	**US28**	Categorización personalizada de insumos	Creación de etiquetas y grupos custom para clasificar stock según la dinámica del local.	**3**
34	**US29**	Historial de mermas e inventario exportable	Funcionalidad para descargar informes detallados de mermas e inventarios en CSV y Excel.	**5**
35	**US30**	Algoritmo de sugerencias de reposición	Proyección automatizada de compras de insumos según histórico de consumo y ventas.	**5**

Orden (#)	User Story ID	Título de la Historia / Tarea Técnica	Descripción Sintetizada	Story Points
36	**US25**	Gamificación y metas de equipo	Otorgamiento de insignias y reconocimientos a las sucursales que cumplan metas de cero mermas.	**3**
37	**US26**	Red B2B e intercambio de prácticas	Espacio para interacción entre comercios para compartir técnicas de conservación y proveedores.	**2**
38	**US27**	Benchmarking entre sucursales	Tabla comparativa de rendimiento en reducción de mermas entre distintas sedes de la empresa.	**5**
39	**US40**	Evaluación de calidad de proveedores	Sistema de calificación e historial sobre la frescura y vida útil de los insumos entregados.	**2**
40	**US35**	Recuperación corporativa de contraseña	Flujo seguro de restablecimiento de clave vía enlace temporal al correo corporativo.	**2**
41	**US36**	Panel de administración de usuarios B2B	Módulo interno para la supervisión de cuentas corporativas, gestión de roles y accesos.	**3**
42	**US24**	Ruteo de notificaciones por jerarquía	Enrutamiento de alertas críticas hacia los teléfonos de los gerentes o supervisores de turno.	**3**
43	**TS45**	Caching de Consultas e Inventario	Capa de memoria en caché con Redis para servir inventarios de alta concurrencia en <50ms.	**3**
44	**US22**	Sincronización con frio industrial	Enlace e integración API directa con sistemas de refrigeración comercial e industrial.	**8**
45	**US23**	Consultas por voz en estaciones de trabajo	Desarrollo de skill de voz para consultar el estado de insumos con las manos libres en cocina.	**5**

Capítulo IV: Solution Software Design

4.1. Strategic-Level Domain-Driven Design

Para el diseño estratégico de FreshSense se emplea Domain-Driven Design (DDD) con el objetivo de organizar la solución de acuerdo con las principales responsabilidades del dominio y mantener una separación clara entre sus capacidades de negocio.

A partir del análisis realizado sobre FreshSense, la solución se organiza en bounded contexts que mantienen sus propias reglas de negocio, modelos de dominio y mecanismos de persistencia. Los principales contextos identificados son Accounts, Monitoring, Inventory, Alerts, Recipes y Billing.

Esta separación permite reducir el acoplamiento entre las funcionalidades del sistema y facilita la evolución independiente de los diferentes módulos. La comunicación entre contextos se realiza principalmente mediante eventos de dominio, evitando dependencias directas entre sus implementaciones internas.

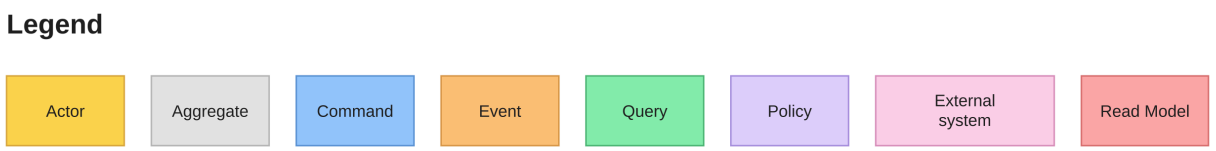
4.1.1. Design-Level EventStorming

El Design-Level EventStorming de FreshSense permite representar el comportamiento del sistema a partir de los eventos que ocurren dentro del dominio y de las acciones que los generan.

En el trabajo previo del proyecto se desarrolló un EventStorming que incluye elementos como eventos de dominio, comandos, actores, read models, sistemas externos, políticas y aggregates. El análisis de estos elementos permitió identificar responsabilidades relacionadas con la gestión de usuarios, dispositivos y sensores, inventario de alimentos, monitoreo, alertas, recetas, suscripciones y reportes.

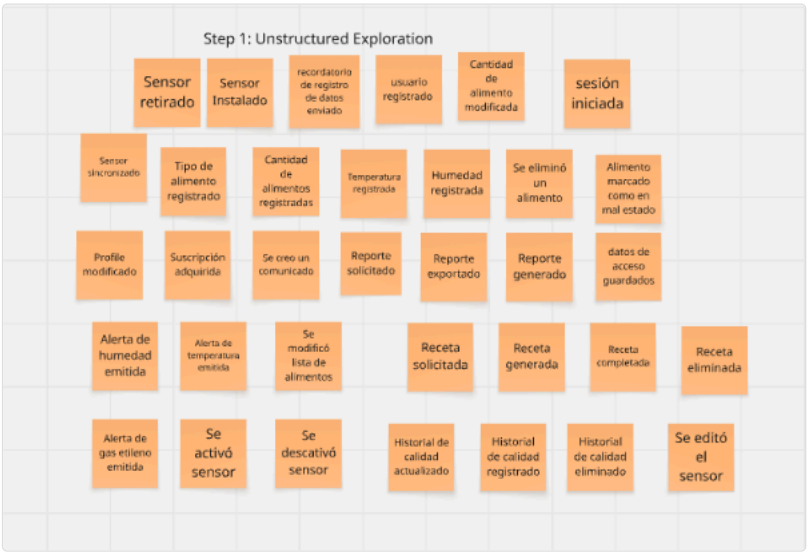
El resultado del EventStorming sirve como base para identificar los límites entre los diferentes contextos del dominio y analizar posteriormente la comunicación existente entre ellos.

A continuación, se presentan los artefactos obtenidos durante las últimas etapas del Design-Level EventStorming de FreshSense.



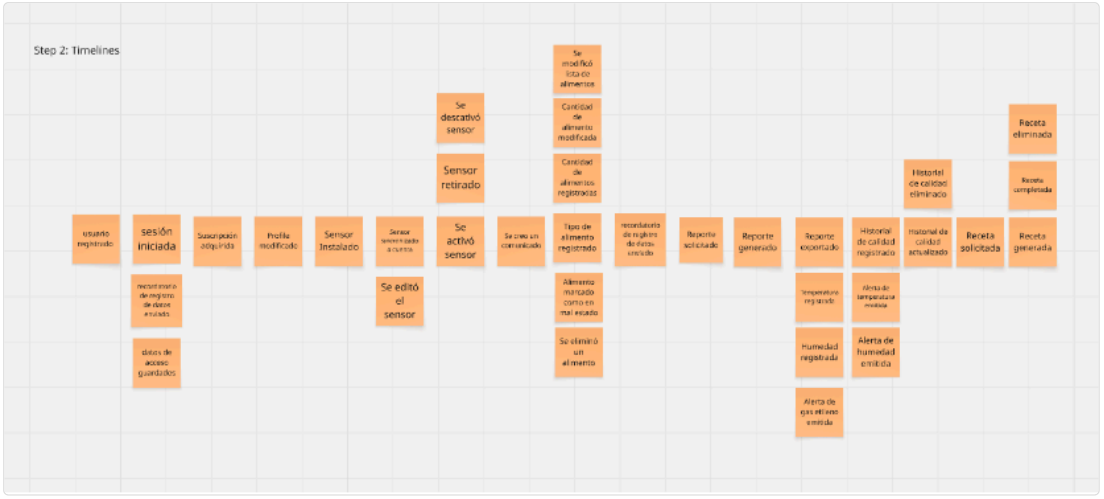
Legend — notación de colores empleada en el Event Storming.

Step 1 — Unstructured Exploration. Lluvia de eventos de dominio sin orden ni agrupación, identificando el vocabulario inicial del negocio (sensores, alertas, reportes, recetas, suscripción).



Step 1: Unstructured Exploration — lluvia inicial de eventos de dominio.

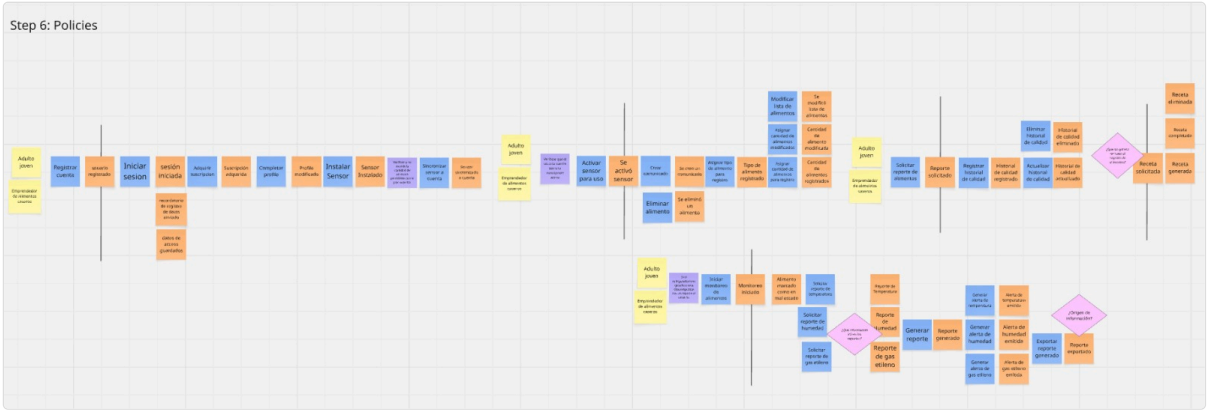
Step 2 — Timelines. Los eventos se ordenan cronológicamente según el ciclo de vida del usuario y del dispositivo, desde el registro hasta la generación de reportes y recetas.



Step 2: Timelines — eventos ordenados cronológicamente.

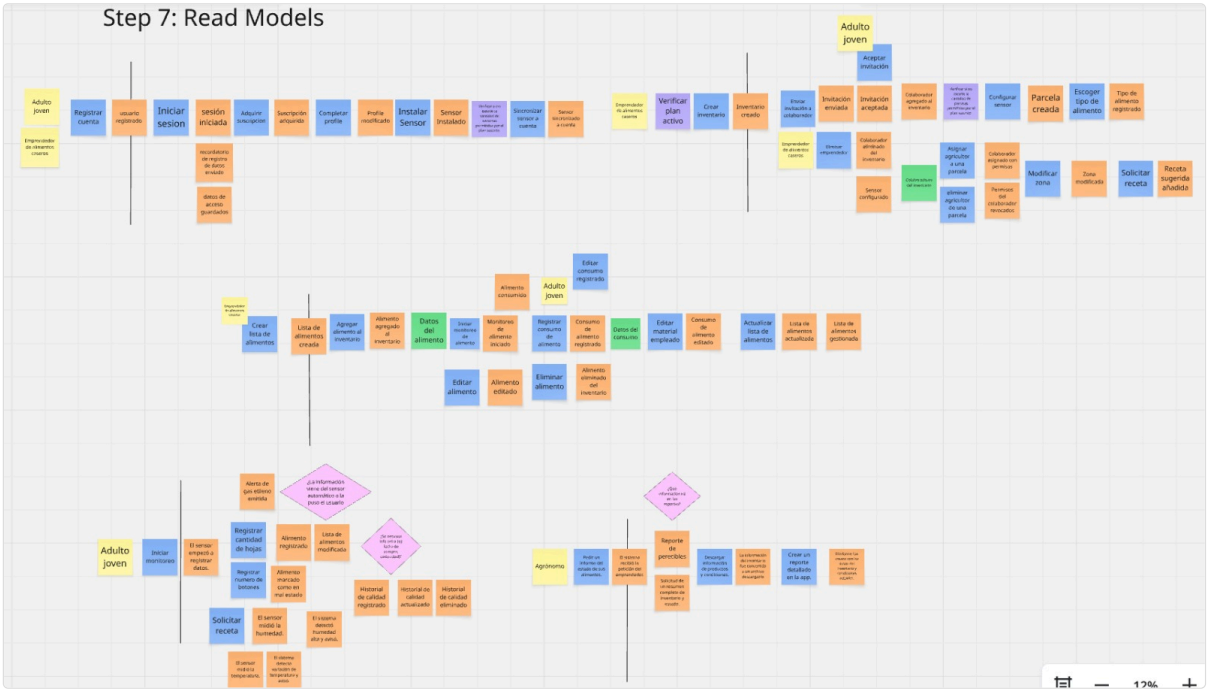
Steps 3–5 (Actors, Commands, Aggregates preliminares) se trabajaron de forma iterativa junto con el resto de pasos; su resultado consolidado se refleja directamente en los Steps 6 a 10 que se muestran a continuación.

Step 6 — Policies. Se identificaron las políticas ("cuando ocurre X, automáticamente se hace Y") que conectan eventos con nuevos comandos sin intervención directa del usuario — por ejemplo, "*si los valores del sensor superan los umbrales configurados, se emite una alerta*".



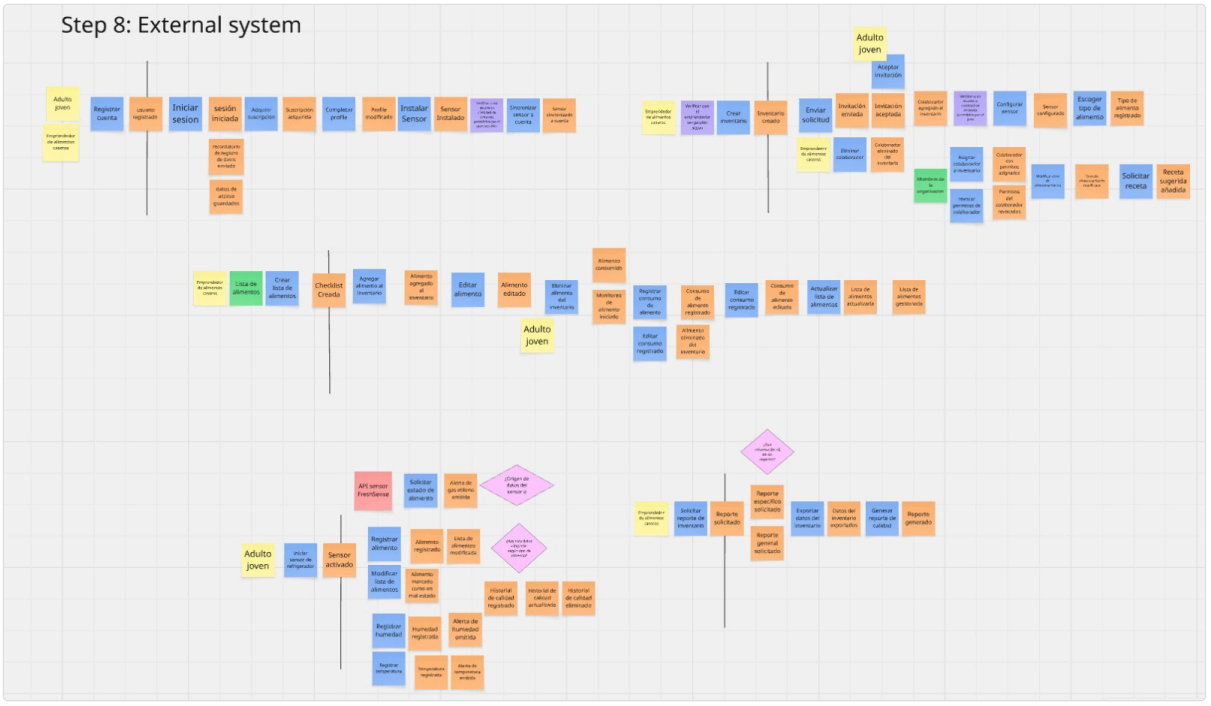
Step 6: Policies del dominio FreshSense.

Step 7 — Read Models. Se definieron las vistas de solo lectura que necesita cada actor: la lista de alimentos, el historial de calidad, el inventario y los reportes.



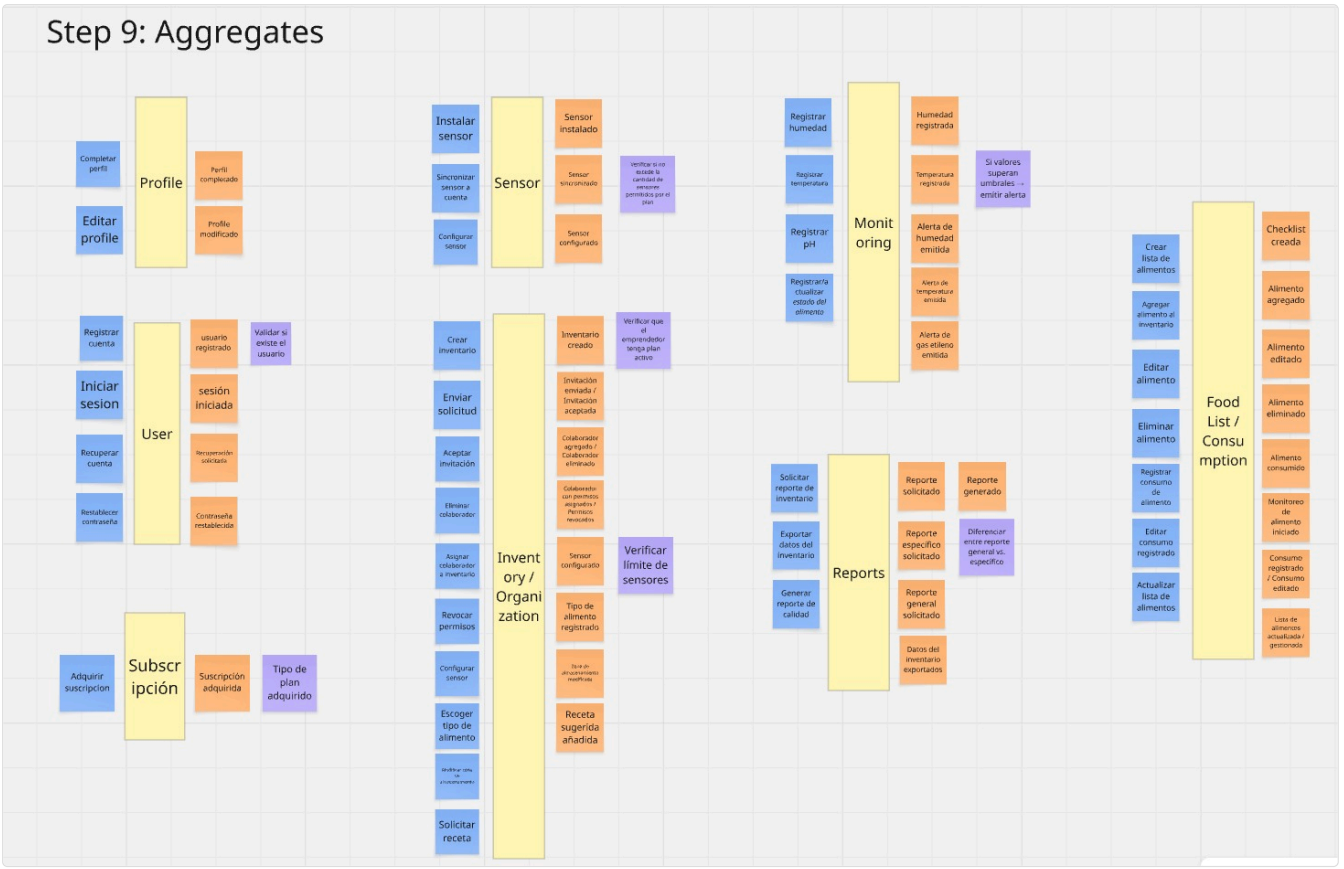
Step 7: Read Models identificados.

Step 8 — External Systems. Se ubicó al dispositivo FreshSense (sensor físico) y a los servicios de notificación como sistemas externos que interactúan con el dominio mediante comandos y eventos.



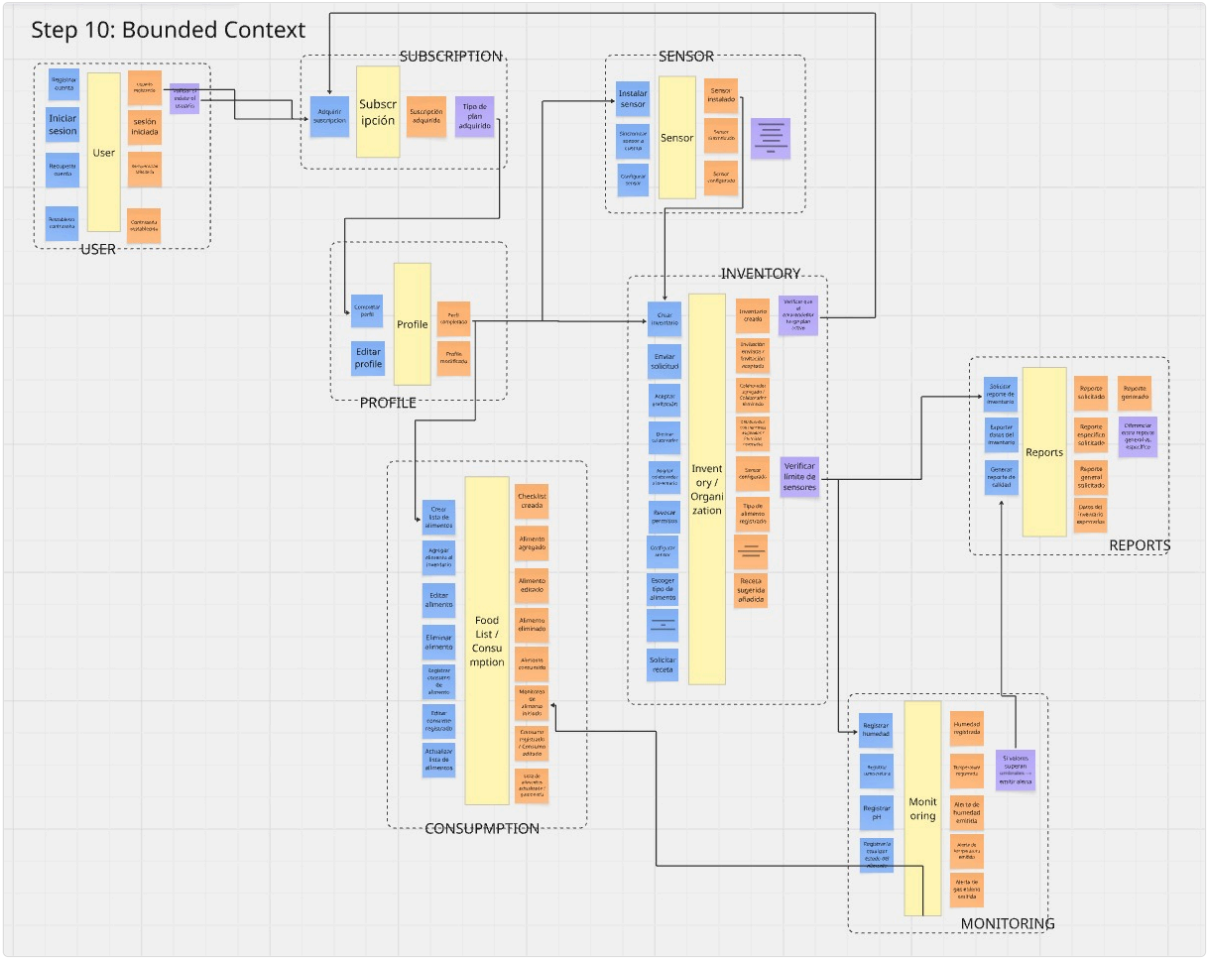
Step 8: External Systems del dominio.

Step 9 — Aggregates. Los comandos y eventos se agruparon alrededor de siete agregados candidatos: `User`, `Profile`, `Subscription`, `Sensor`, `Inventory/Organization`, `Food List/Consumption`, `Monitoring` y `Reports`.



Step 9: Aggregates — agrupación de comandos y eventos.

Step 10 — Bounded Context. Finalmente, los agregados se agruparon en ocho *bounded contexts*, trazando las primeras fronteras del dominio que se detallan en la sección 4.1.1.1.



Step 10: Bounded Context — fronteras candidatas del dominio.

4.1.1.1 Candidate Context Discovery

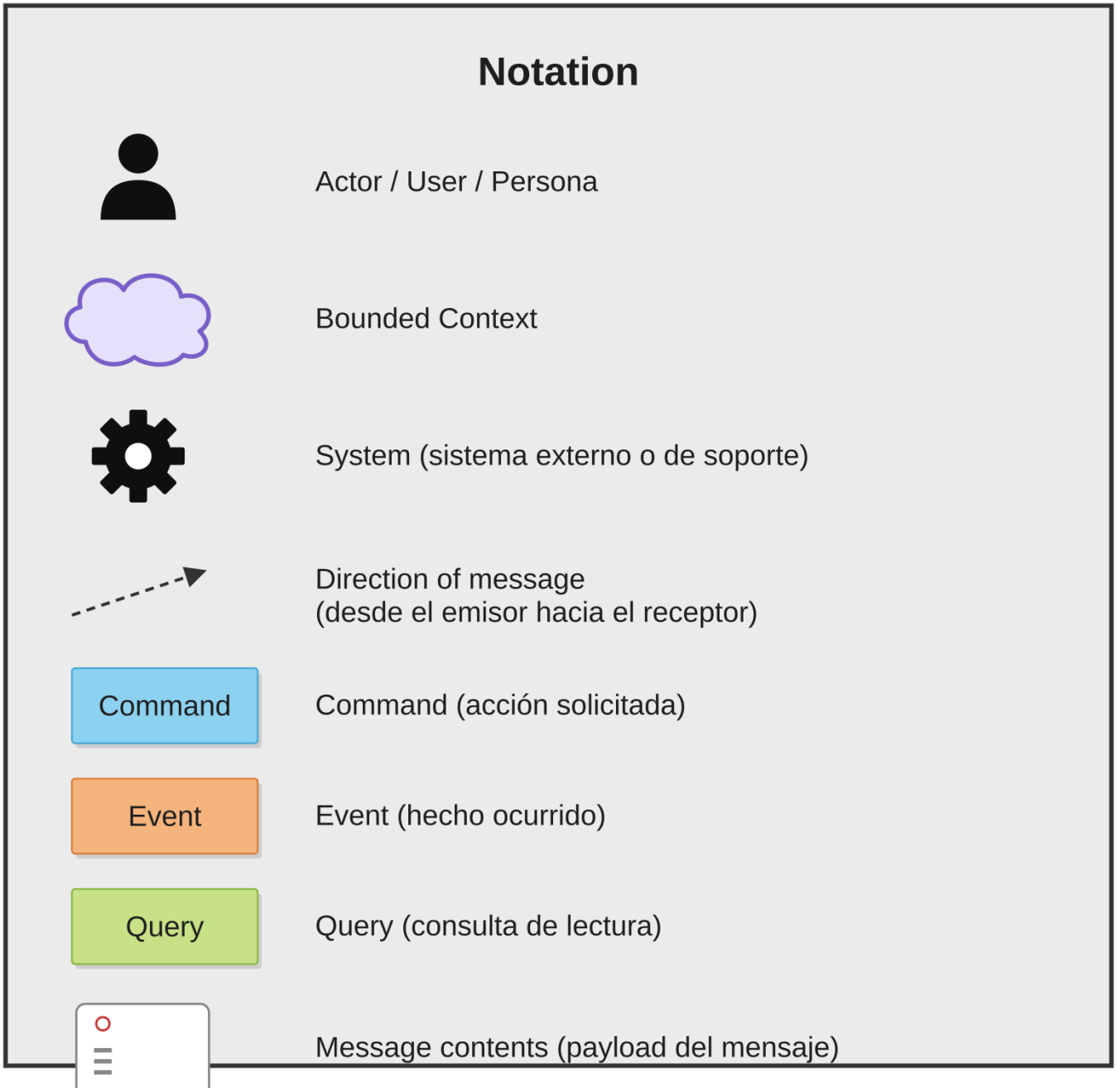
Identificar contextos candidatos es un paso clave para gestionar la complejidad del sistema. A partir de los agregados del Step 9 y las fronteras trazadas en el Step 10, se agruparon los conceptos del dominio FreshSense en **ocho bounded contexts** lógicos y coherentes, cada uno con una responsabilidad de negocio propia:

Bounded Context candidato	Responsabilidad principal
User Management	Registro, autenticación y recuperación de cuentas.
Profile Management	Negocio (restaurante o distribuidora en frío), su personal y las preferencias de notificación por turno.
Subscription Management	Plan contratado (FREE/PREMIUM) y límites asociados.
Sensor Management	Instalación, sincronización y configuración del dispositivo FreshSense.
Inventory Management	Inventario de insumos y lotes por cámara o zona, con alta por código de barras/QR.
Consumption Management	Consumo y merma, fichas técnicas del restaurante, rotación FIFO de la distribuidora y proveedores.
IoT Monitoring	Lecturas del sensor, umbrales, alertas y calidad del alimento.
Report Management	Reportes de inventario, calidad y consumo con métricas de sostenibilidad.

Esta separación facilita el diseño e implementación del sistema y potencia su escalabilidad, desempeño y mantenibilidad — por ejemplo, permite que el equipo evolucione el algoritmo de alertas de **IoT Monitoring** sin afectar la lógica de **Subscription Management**, o que se sustituya el proveedor de notificaciones sin tocar el dominio de inventario.

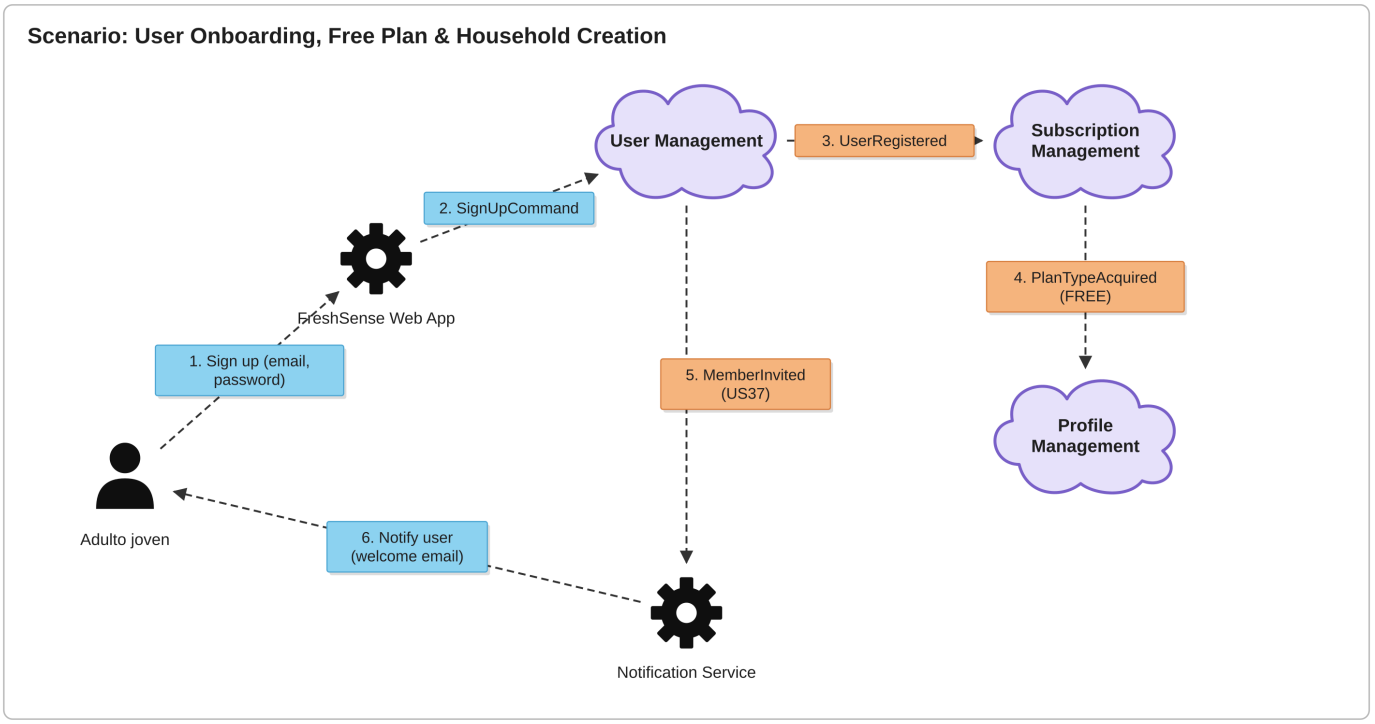
4.1.1.2 Domain Message Flows Modeling

Para analizar y diseñar sistemas de software, se usa el Modelado de Flujos de Mensajes de Dominio, un método que ilustra la transferencia de información entre componentes mediante mensajes. Este proceso se centra en especificar los mensajes enviados y recibidos por los diferentes actores del sistema y en descifrar sus relaciones. El uso de esta metodología aporta claridad para entender y representar las vías de información del sistema, permitiendo detectar problemas potenciales más fácilmente y optimizar la estructura del diseño. A modo de ejemplo, se muestran a continuación algunos diagramas aplicados a nuestro sistema.



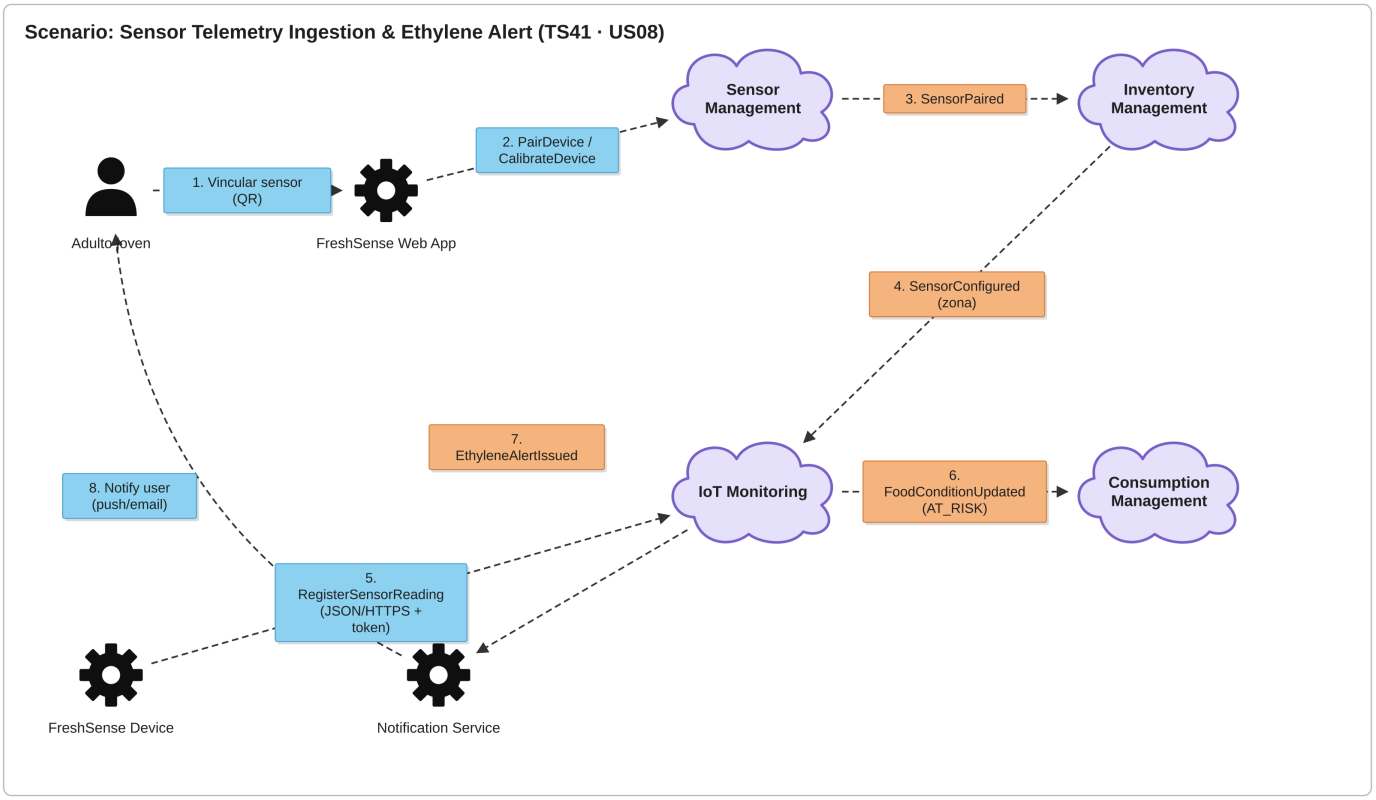
Notación utilizada en los Domain Message Flows.

Escenario 1 — Registro de usuario y asignación del plan gratuito. El propietario o administrador del restaurante (o el encargado del negocio de distribución en frío) se registra en la app; **User Management** valida y crea la cuenta, publica `UserRegistered`, y **Subscription Management** reacciona asignando automáticamente el plan FREE, que a su vez habilita la creación del negocio en **Profile Management**.



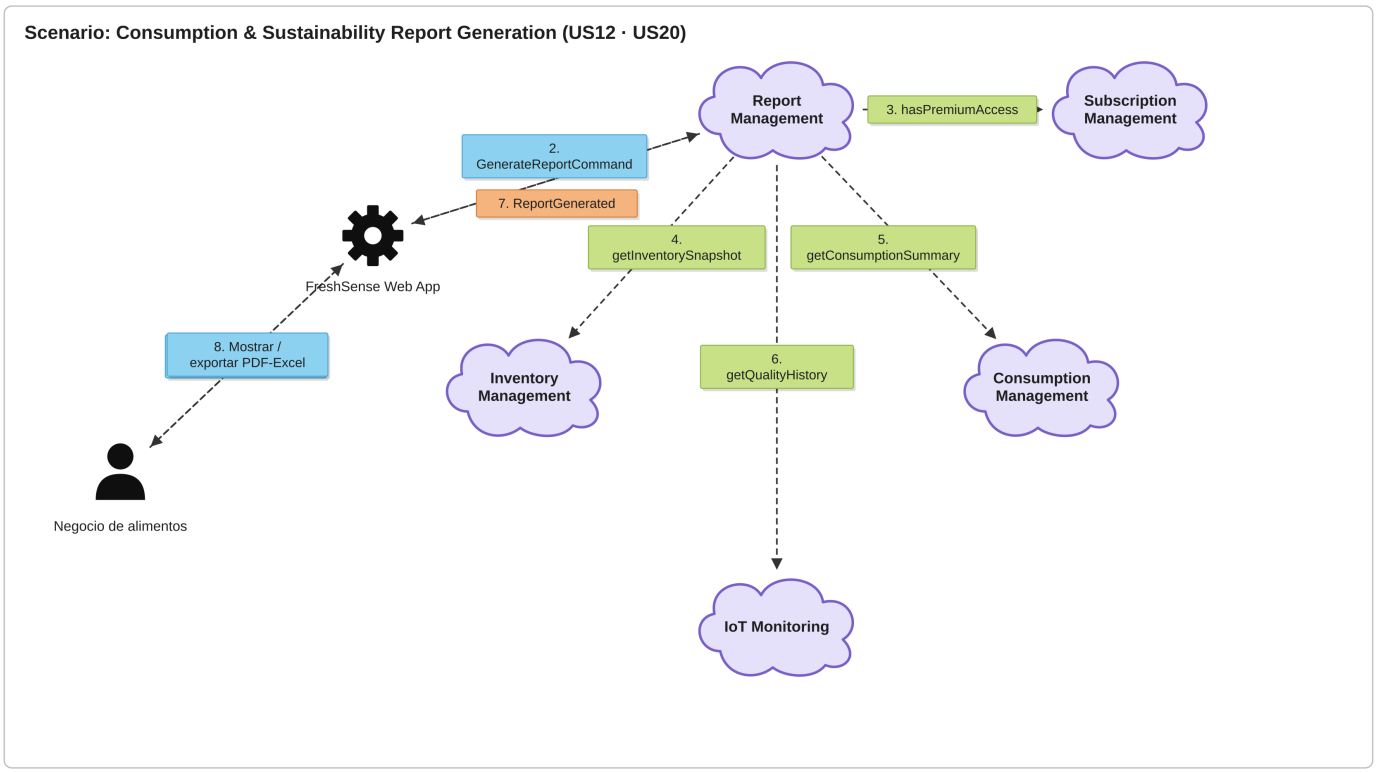
Escenario: User Onboarding & Free Plan Assignment.

Escenario 2 — Lectura del sensor y alerta de etileno. El dispositivo físico envía una lectura; **IoT Monitoring** la traduce, evalúa los umbrales configurados, actualiza la condición del alimento en **Consumption Management** y notifica al usuario cuando el nivel de gas etileno es crítico.



Escenario: Sensor Reading & Ethylene Alert.

Escenario 3 — Generación de un reporte de inventario. El encargado del negocio de distribución en frío solicita un reporte general; **Report Management** verifica el acceso premium y consulta a los contextos de Inventory, Consumption y Monitoring antes de generar y entregar el reporte.

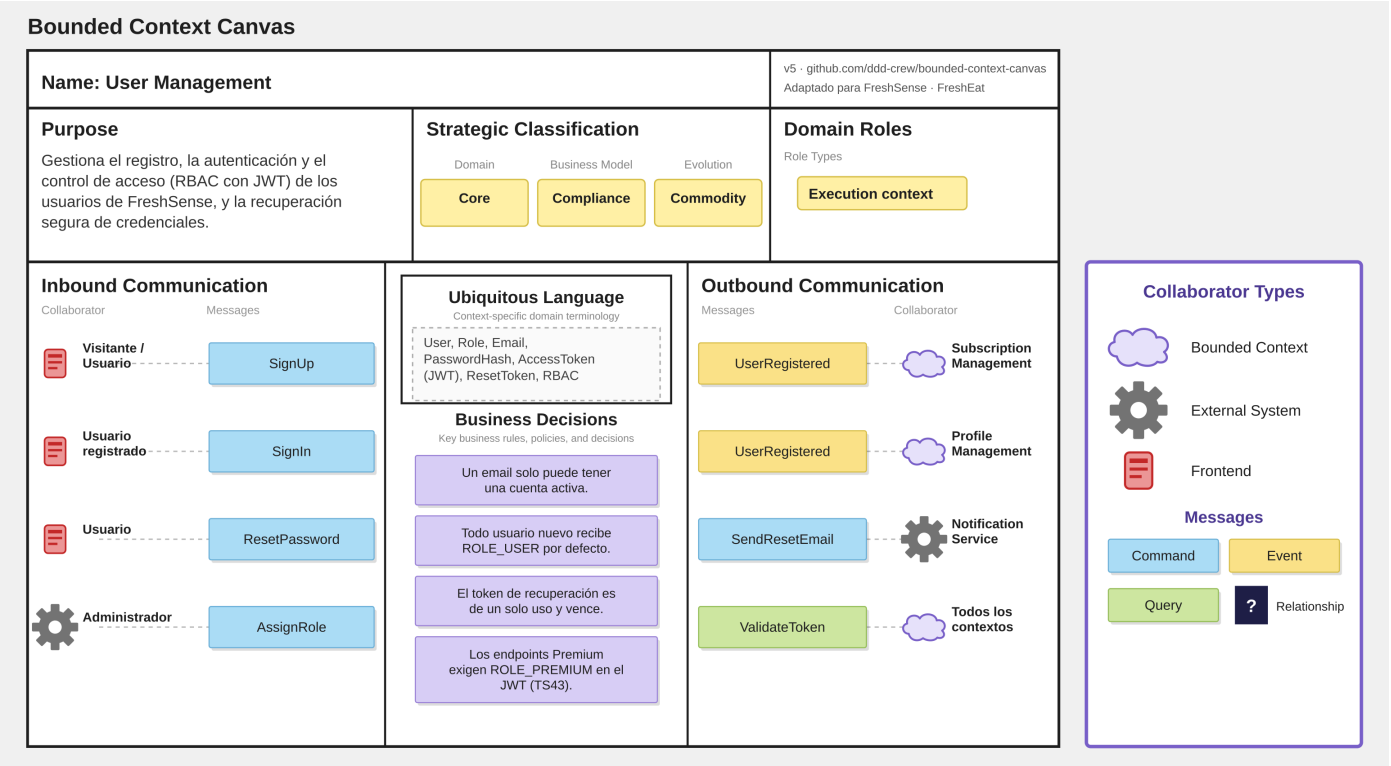


Escenario: Inventory Report Generation.

4.1.1.3 Bounded Context Canvases

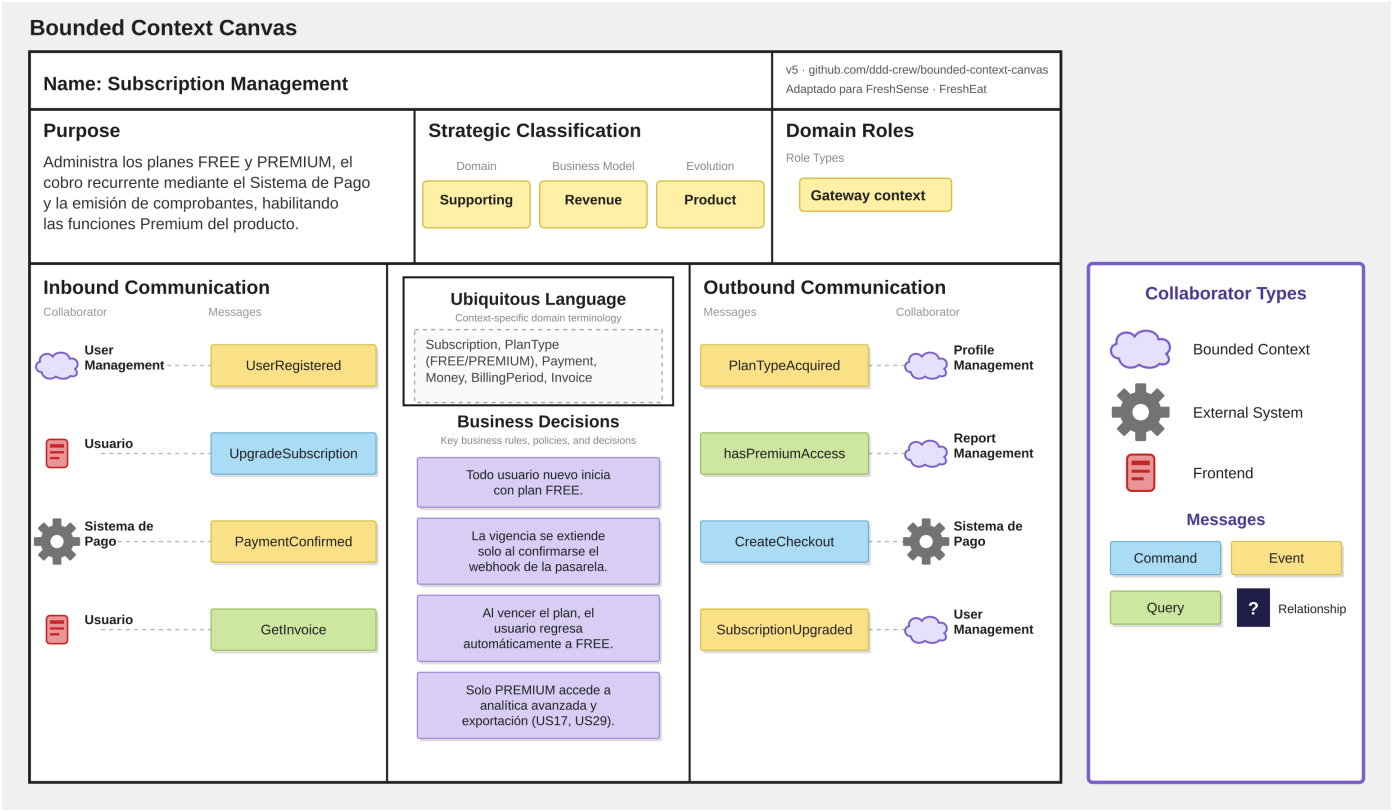
Cada bounded context candidato se documentó con un *Bounded Context Canvas*, que resume su propósito, su clasificación estratégica, sus roles de dominio, su lenguaje ubicuo, sus decisiones de negocio y sus comunicaciones entrantes y salientes.

User Management



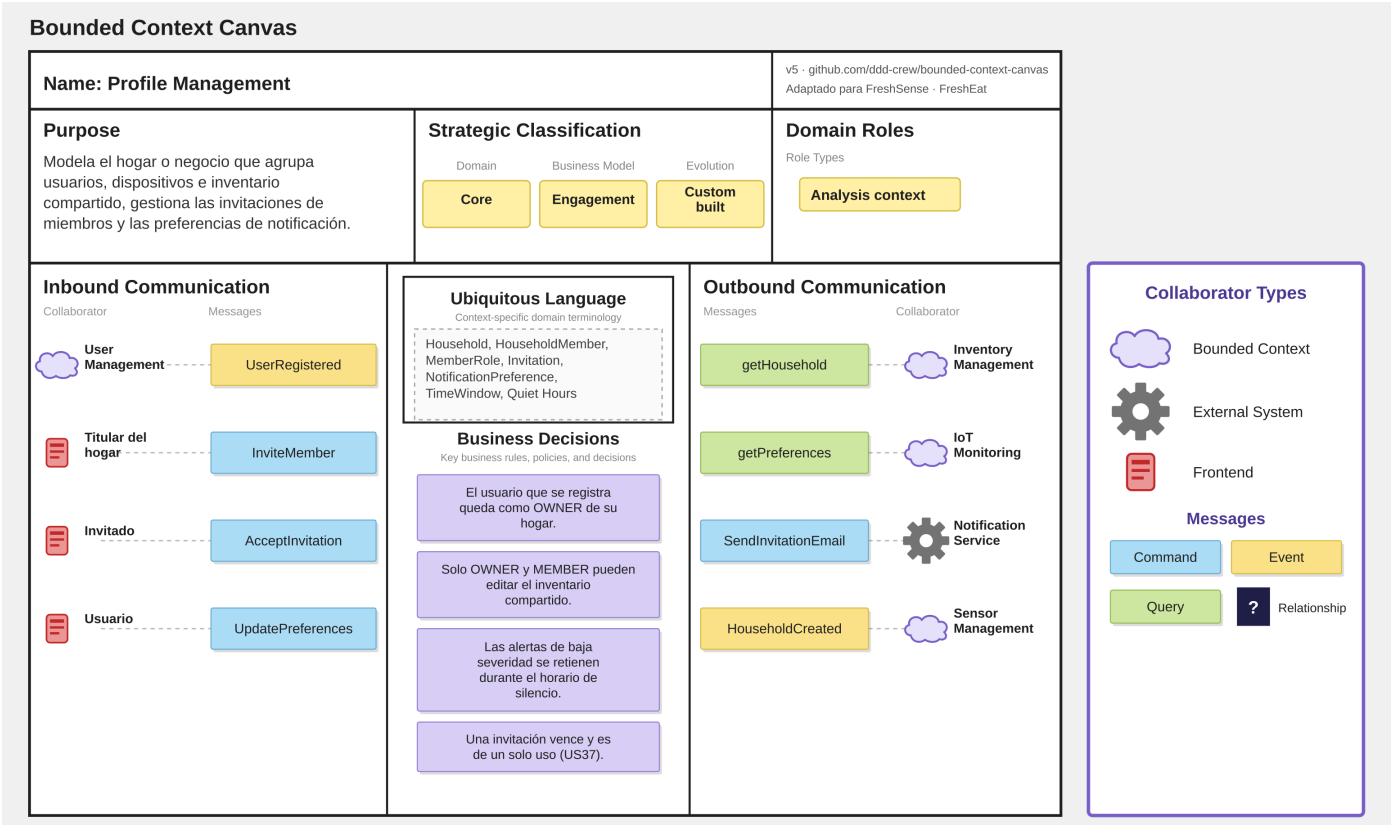
Bounded Context Canvas — User Management.

Subscription Management



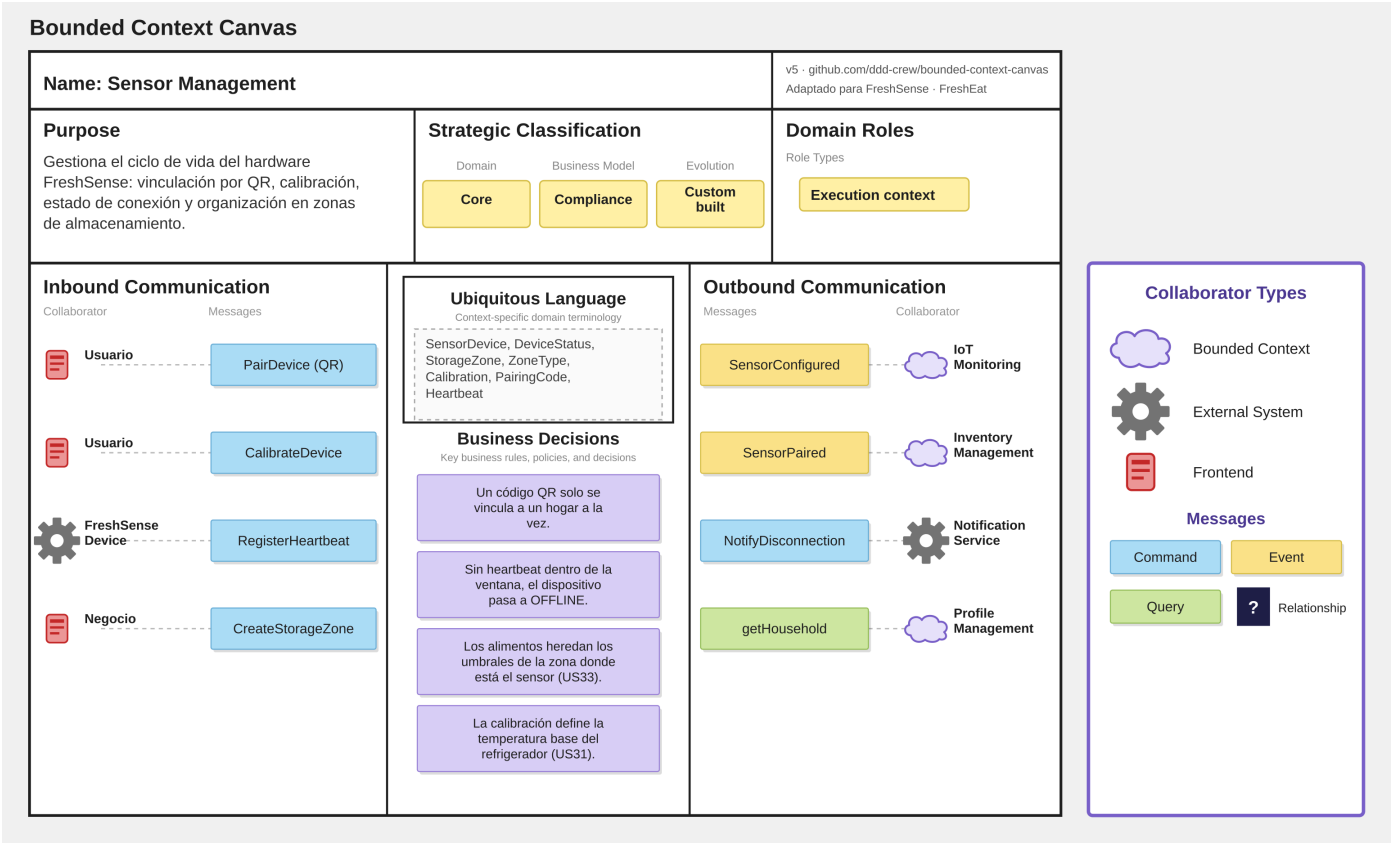
Bounded Context Canvas — Subscription Management.

Profile Management



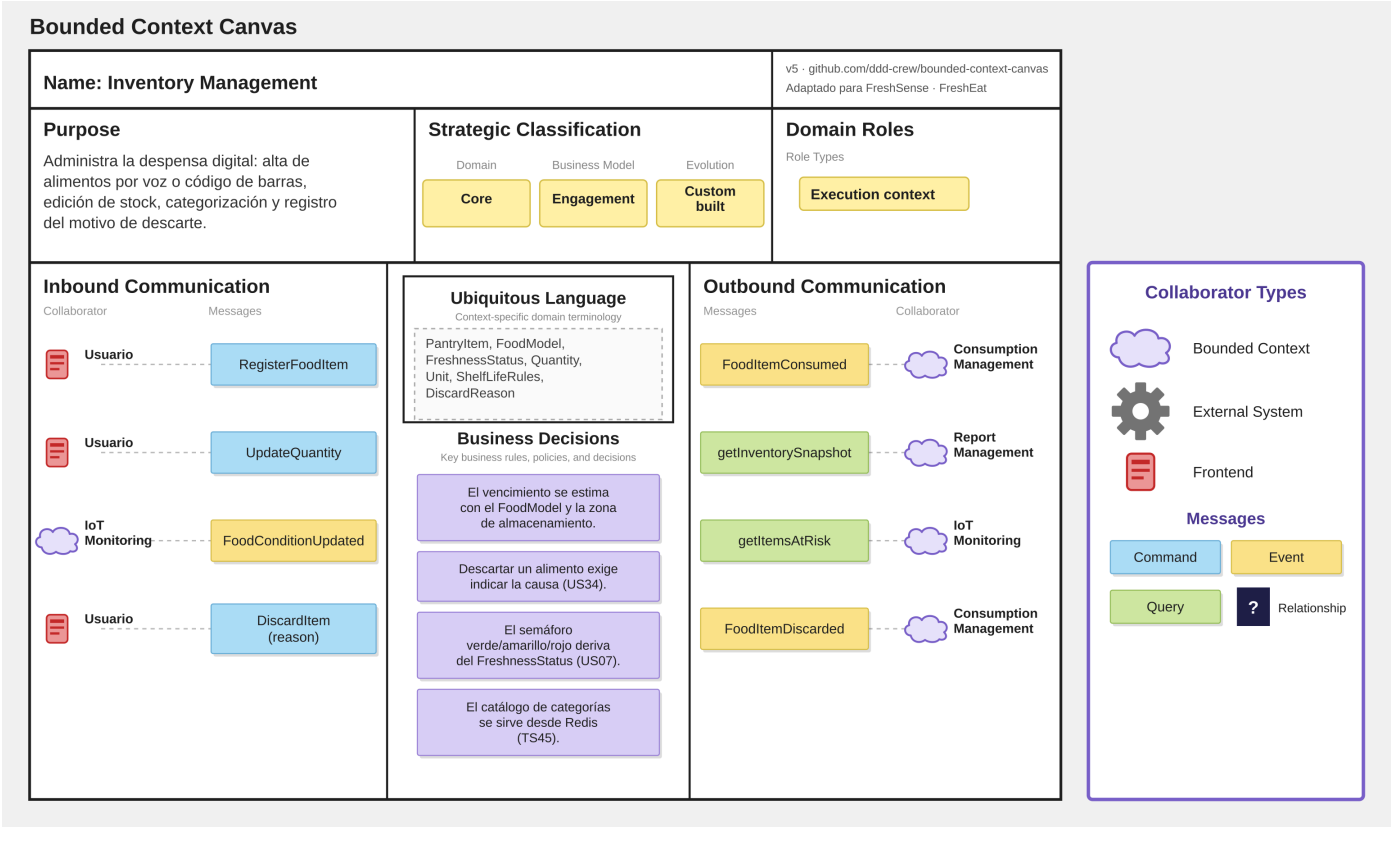
Bounded Context Canvas — Profile Management.

Sensor Management



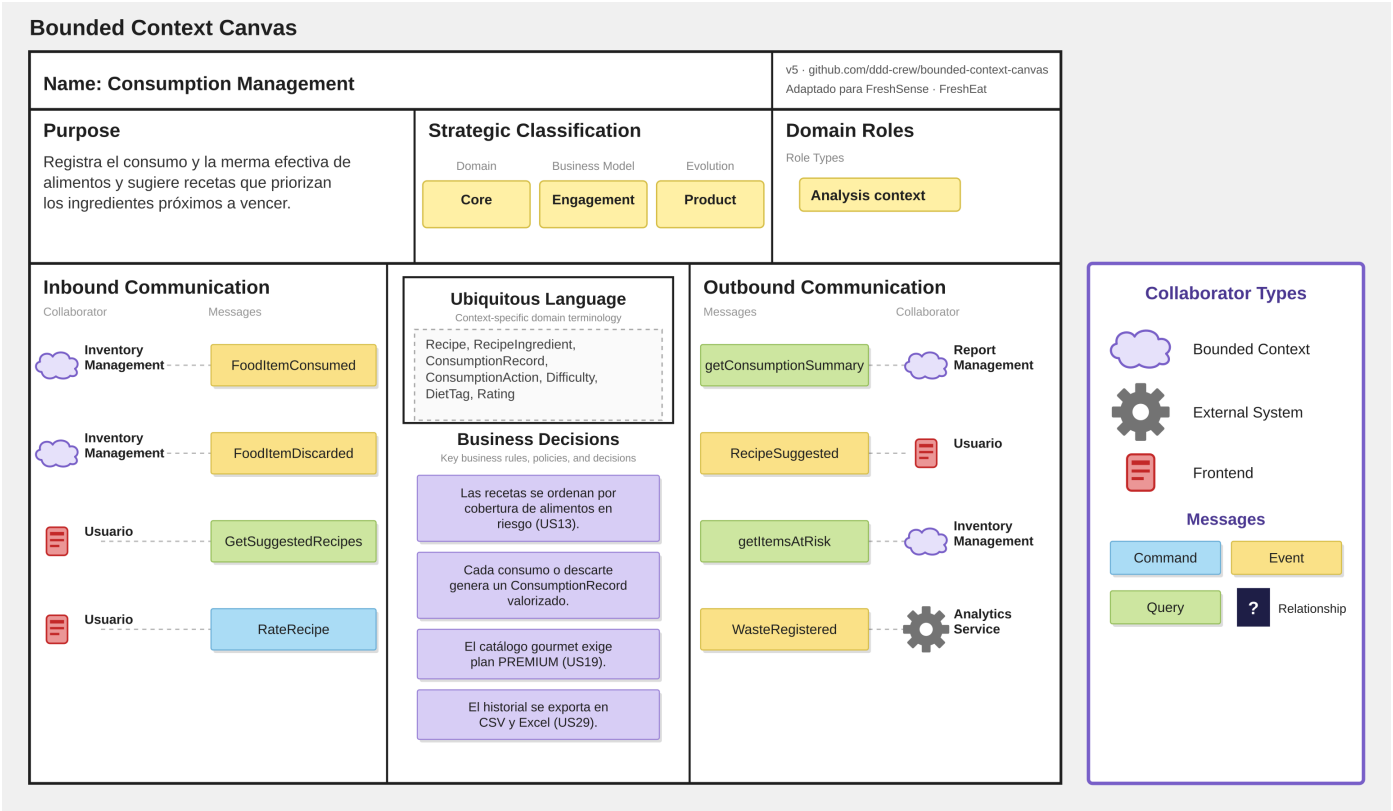
Bounded Context Canvas — Sensor Management.

Inventory Management



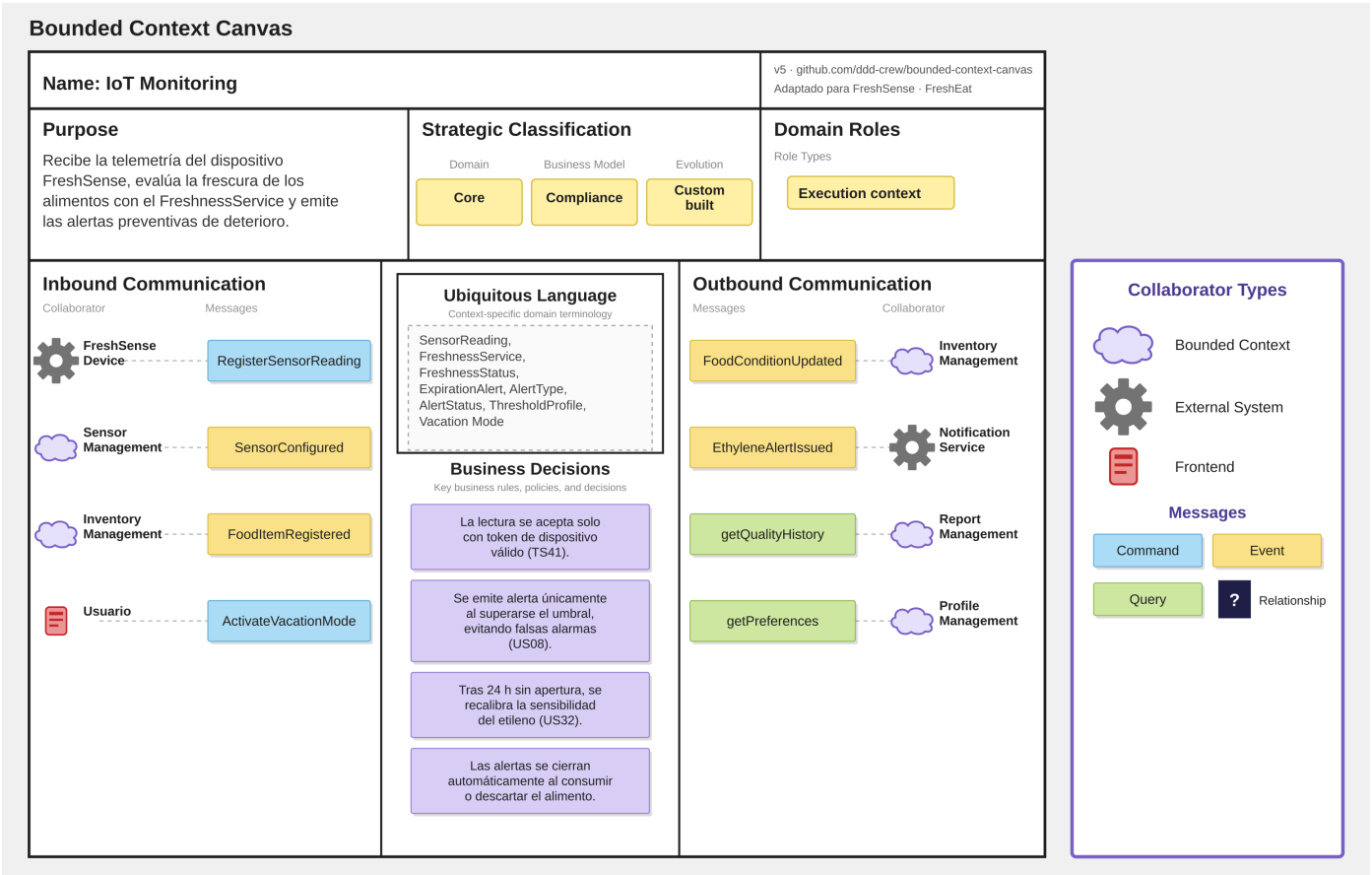
Bounded Context Canvas — Inventory Management.

Consumption Management



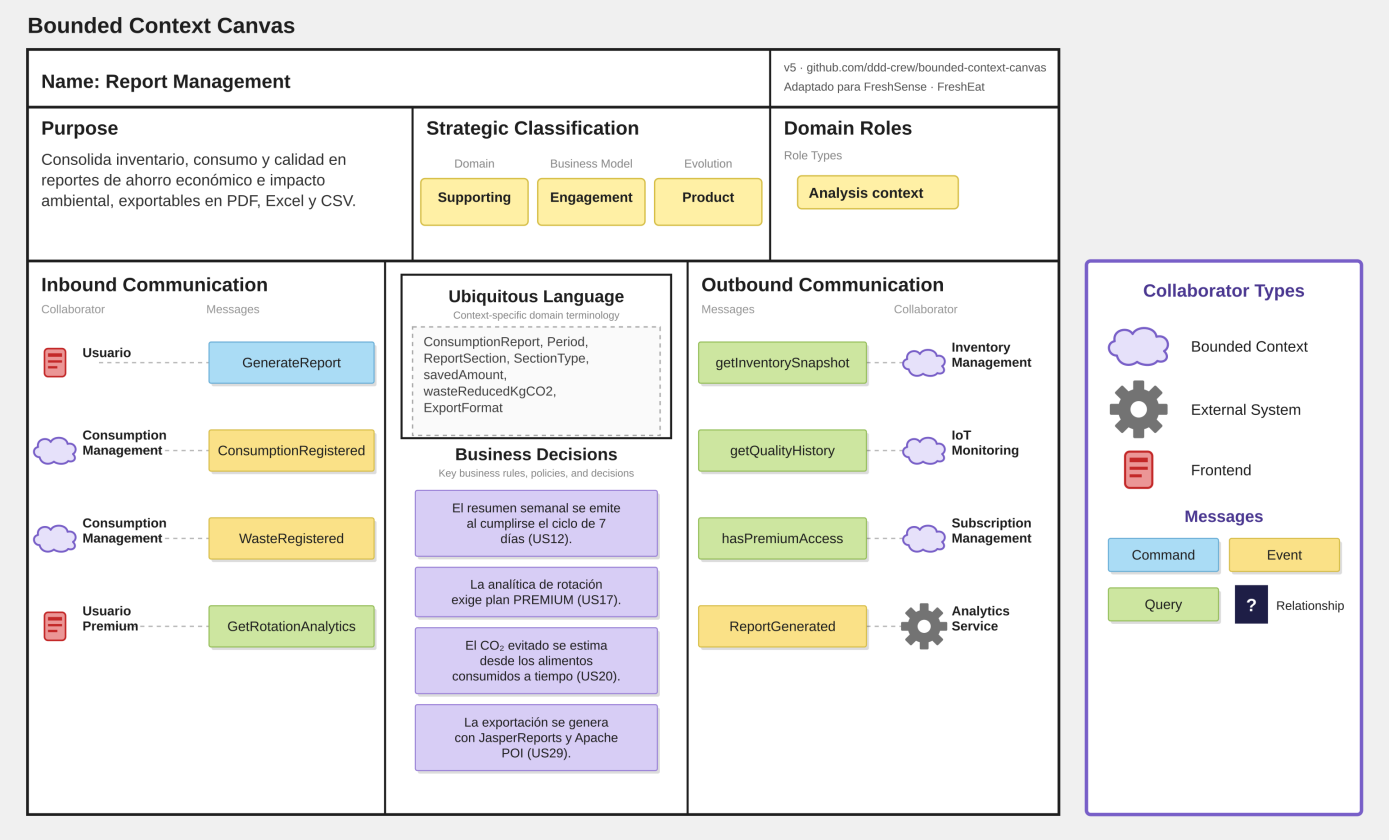
Bounded Context Canvas — Consumption Management.

IoT Monitoring



Bounded Context Canvas — IoT Monitoring.

Report Management



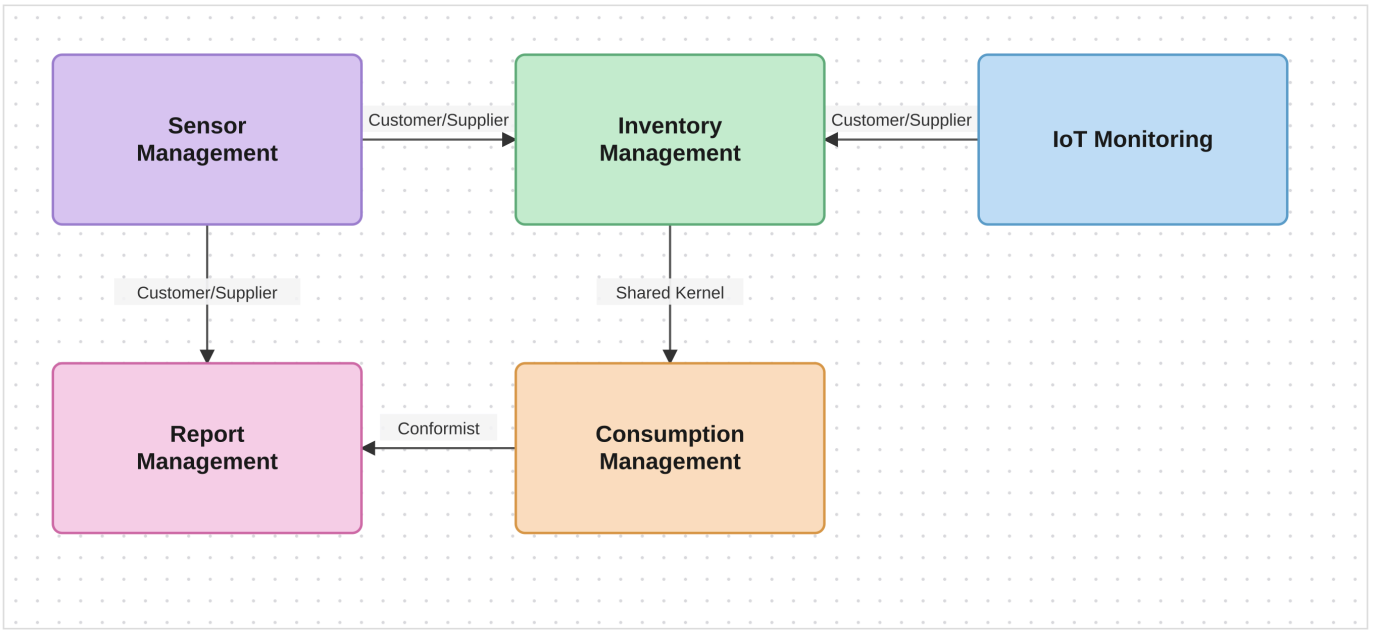
Bounded Context Canvas — Report Management.

4.1.2. Context Mapping

En esta sección desarrollamos un conjunto de *context maps* para visualizar las relaciones entre los *bounded contexts* del sistema. A partir de la información recolectada en el Event Storming, exploramos distintas alternativas de diseño, cuestionando cómo cambiaría la estructura si reubicamos, dividimos o agrupamos *capabilities*. Finalmente, evaluamos cada propuesta considerando patrones como *Anti-corruption Layer*, *Conformist*, *Customer/Supplier* y *Shared Kernel*, con el fin de definir la mejor aproximación para la arquitectura del dominio. A continuación presentaremos las opciones que contemplamos para FreshSense y la estructura final.

Opción 1

En esta estructura mantenemos los cinco bounded contexts del núcleo operativo separados, con relaciones claramente definidas. Las ventajas de este tipo de contexto son, por un lado, la clara separación de responsabilidades y, por otro, que cada contexto se enfoca en una funcionalidad específica. Una de las principales desventajas es que hay una mayor complejidad en la sincronización entre contextos, especialmente entre Sensor Management e IoT Monitoring, que comparten el mismo origen de datos.

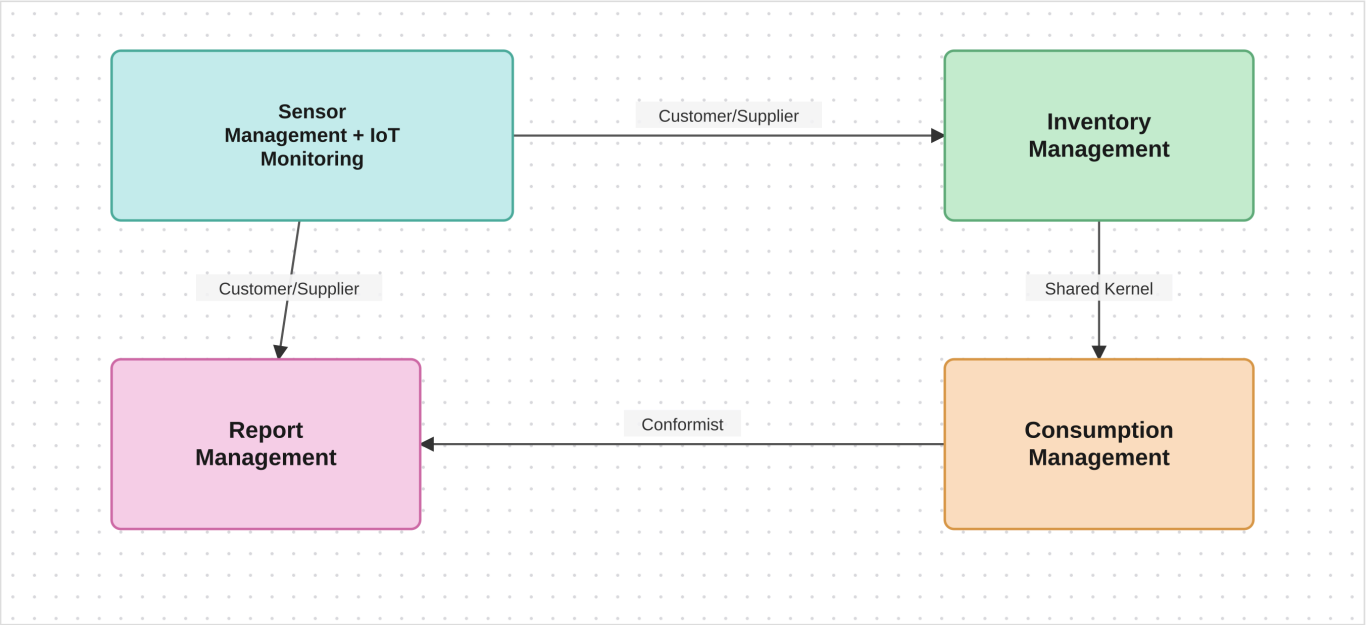


Opción 1 — cinco bounded contexts separados.

Opción 2

Esta alternativa propone unir los contextos de Sensor Management e IoT Monitoring en un solo bounded context. Al hacerlo, se elimina la necesidad de sincronización externa entre ambos, manteniendo relaciones similares con los demás contextos del sistema.

Esta combinación presenta ventajas como la simplificación de la arquitectura al disminuir la cantidad de bounded contexts y una comunicación más directa entre la lectura del sensor y la interpretación de su estado. No obstante, tiene desventajas en la combinación de responsabilidades distintas, ya que una parte se enfoca en la infraestructura de sensores y la otra en la interpretación de datos y generación de alertas. Esto podría dificultar que el usuario realice la instalación del sistema sin ayuda técnica, y además genera el riesgo de que un solo contexto asuma demasiadas funciones.

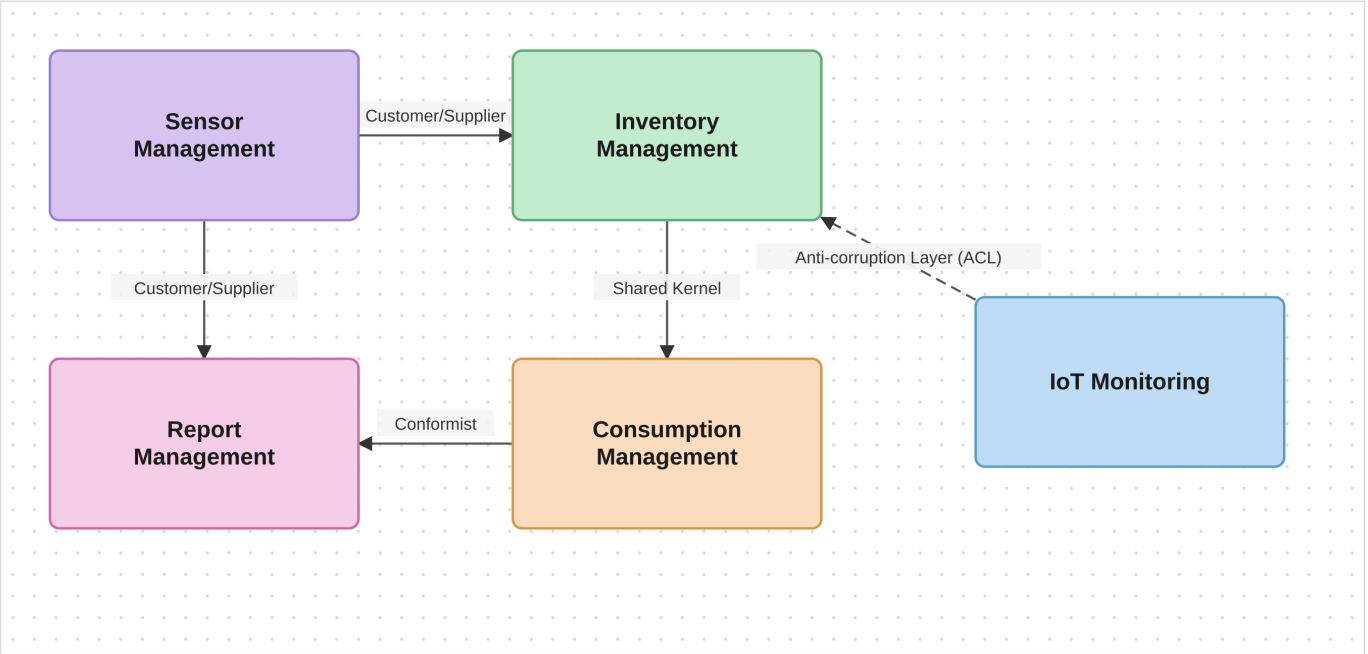


Opción 2 — Sensor Management e IoT Monitoring fusionados.

Opción 3

Esta alternativa propone una arquitectura compuesta por cinco bounded contexts bien definidos, con relaciones claras entre ellos. La estructura busca equilibrar la separación de responsabilidades, para permitir que el sistema escale y se mantenga con facilidad. Además, facilita la instalación sin asistencia técnica y asegura tiempos de respuesta adecuados, lo que contribuye directamente a mejorar la experiencia del cliente en el uso del sistema FreshSense.

- Sensor Management se comunica con Inventory Management y Report Management, proporcionando la información de los sensores instalados y su configuración. En ambos casos, la relación es del tipo Customer/Supplier, donde Sensor Management es el proveedor.
- Inventory Management y Consumption Management comparten el modelo de "alimento" y su estado de frescura. Por eso, tienen una relación de tipo Shared Kernel, lo que asegura que ambos usen los mismos conceptos para evitar errores o confusión.
- Consumption Management también se relaciona con Report Management, pero en este caso la relación es Conformist. Report utiliza información de consumo, pero se adapta a su estructura sin modificarla.
- IoT Monitoring se conecta con Inventory Management mediante una Anti-corruption Layer. Esta capa traduce los datos que vienen de los sensores (temperatura, humedad y etileno) a un formato que Inventory Management pueda entender. Así, se protege el sistema de los detalles técnicos del IoT y se facilita la instalación del sistema sin ayuda especializada.



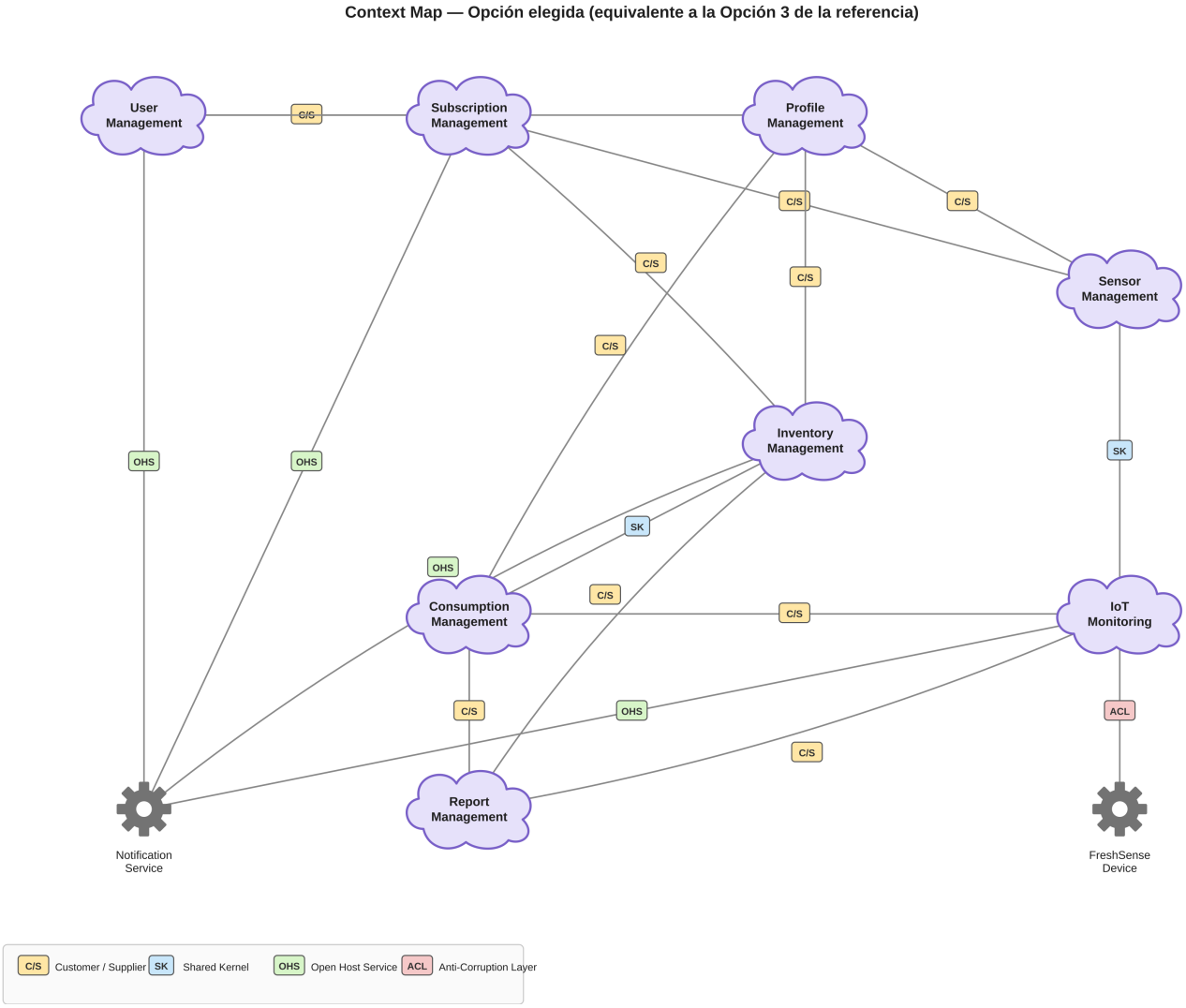
Opción 3 (elegida) — cinco bounded contexts con Anti-corruption Layer.

Elección

Elegimos la **opción 3**, ya que proporciona el mejor equilibrio entre la separación de responsabilidades, la facilidad de implementación y el cumplimiento de los requisitos del sistema.

Al definir cinco bounded contexts con relaciones claras, se facilita la evolución independiente de cada parte del sistema, lo que mejora su escalabilidad y mantenibilidad. Además, al separar la gestión del hardware (Sensor Management) de la interpretación de las lecturas (IoT Monitoring) mediante una Anti-corruption Layer, se simplifica la instalación y el reemplazo del dispositivo. Asimismo, esta estructura garantiza tiempos de respuesta rápidos ante alertas de frescura y optimiza la gestión de inventario, consumo y reportes, brindando una experiencia más fluida y eficiente para los usuarios.

Finalmente, el siguiente *context map* consolidado muestra la totalidad de los ocho bounded contexts de FreshSense — incluyendo los contextos de cuenta (User, Subscription y Profile Management) — con las relaciones resultantes de la opción elegida.



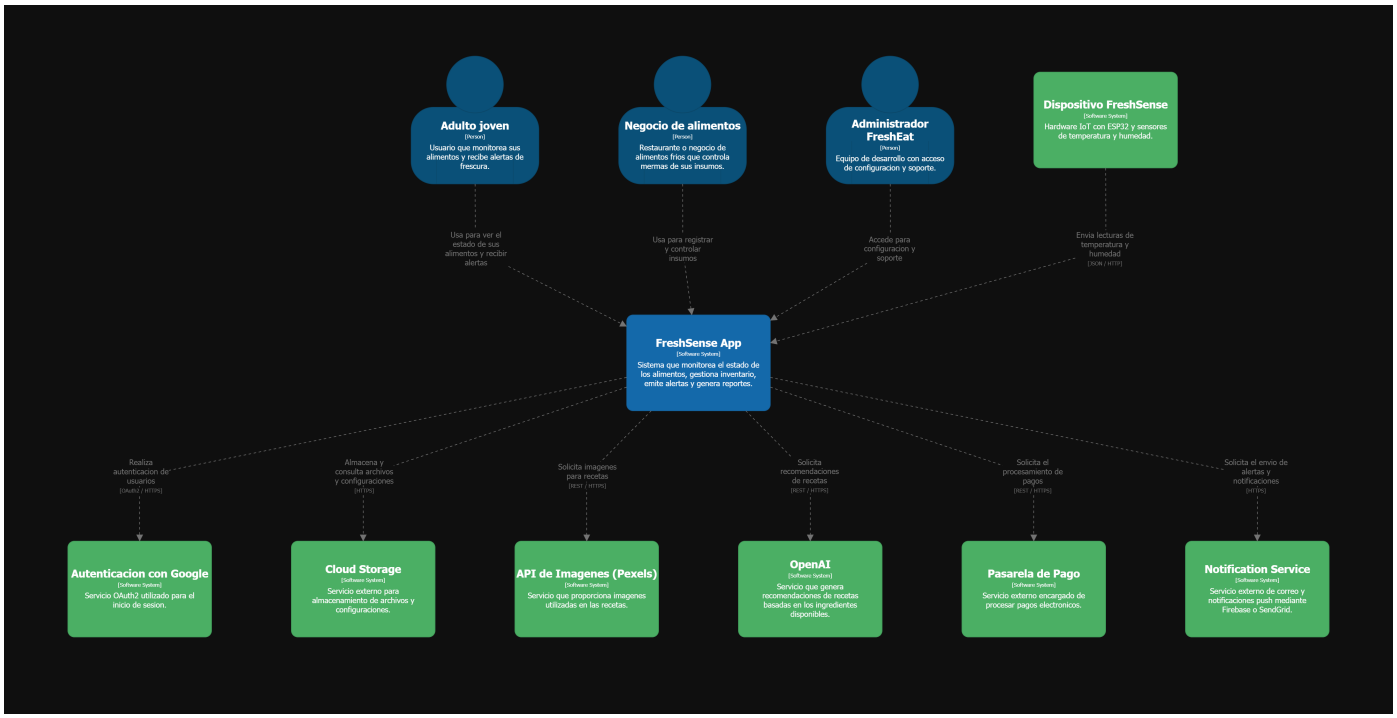
Context Map consolidado — los ocho bounded contexts de FreshSense.

4.1.3. Software Architecture

Se utiliza el **C4 Model** para representar la arquitectura de software de FreshSense en tres niveles de abstracción: contexto, contenedores y despliegue físico.

4.1.3.1. Software Architecture Context Level Diagrams

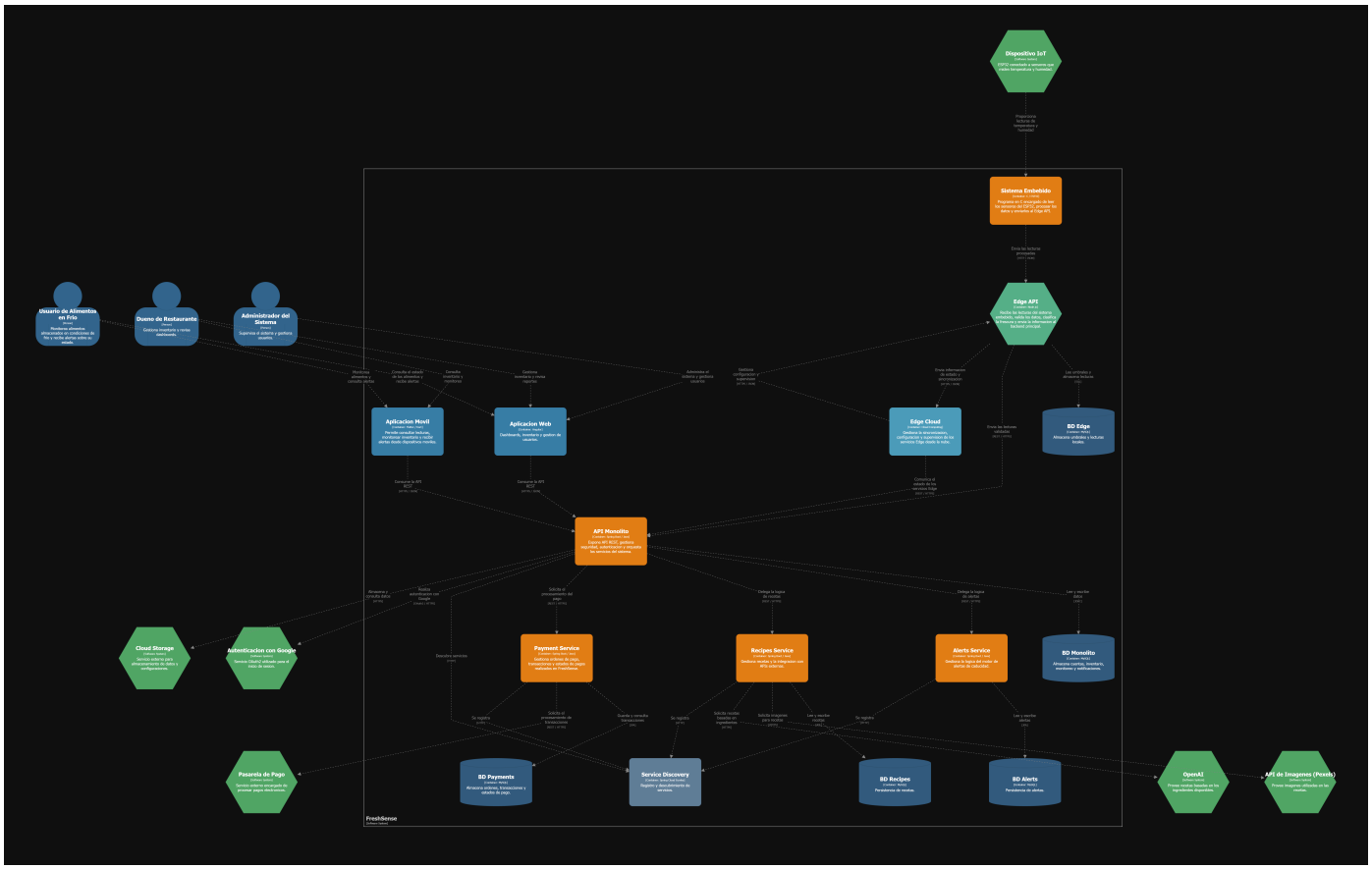
Este diagrama muestra al sistema FreshSense (representado como una única entidad) interactuando con sus tres tipos de usuario y con el **Dispositivo FreshSense**, el hardware con sensores de temperatura, humedad y gas etileno que se instala en el refrigerador o cámara fría y envía su telemetría al endpoint REST seguro definido en la **TS41**. El sistema se comunica además con el Notification Service para las alertas.



Software Architecture Context Level Diagram.

4.1.3.2. Software Architecture Container Level Diagrams

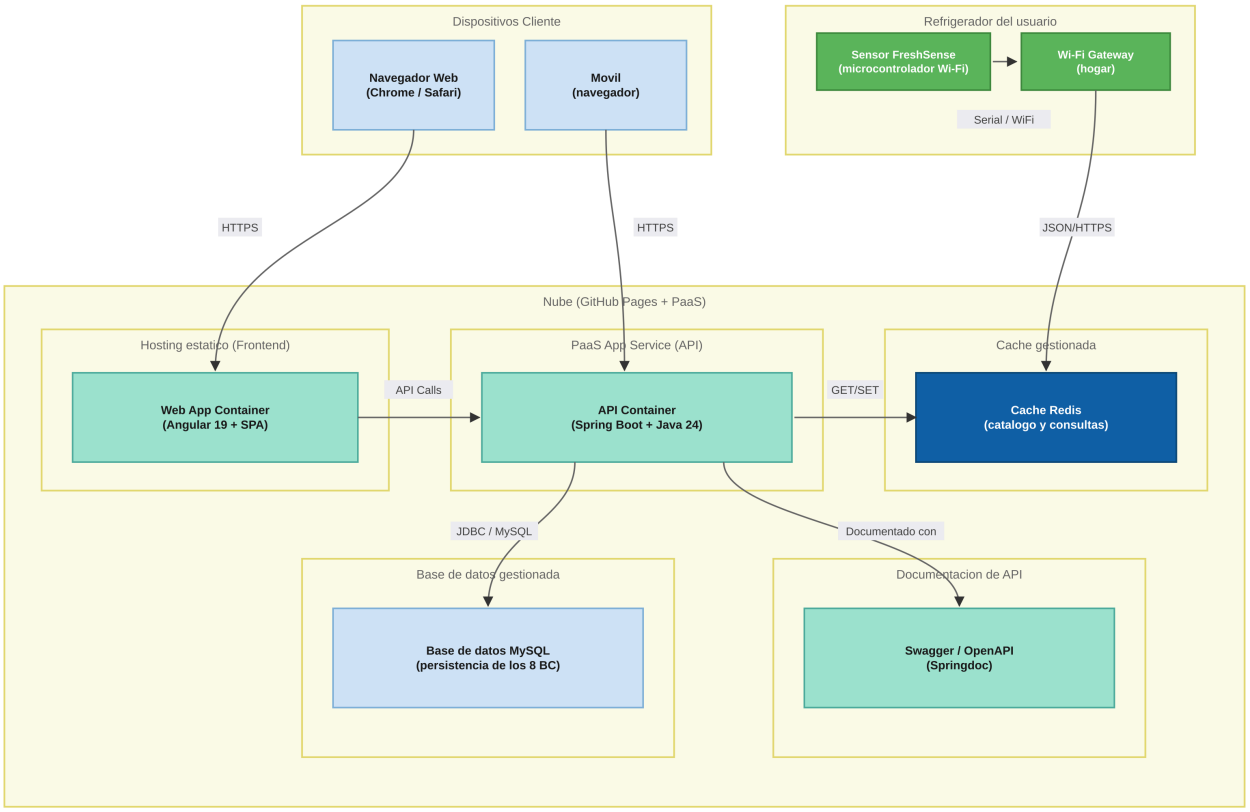
Este diagrama muestra que el sistema FreshSense está compuesto por una **Landing Page** (HTML, CSS, JS), una **Single Page Application** desarrollada en **Angular 19** con NGX-Translate y Angular Material, y una **API REST en Spring Boot + Java 24** que organiza los ocho bounded contexts. La API persiste en **MySQL** y documenta sus endpoints con **Swagger/OpenAPI**; estos cinco elementos son los únicos containers dentro del límite del sistema, ya que representan unidades de software desplegables. Conforme al modelo C4 y a la norma ISO/IEC/IEEE 42010 sobre vistas y puntos de vista arquitectónicos, el **hardware no se modela como container**: el **Dispositivo FreshSense** se representa como sistema externo que envía su telemetría a la API mediante JSON sobre HTTPS con token de dispositivo (**TS41**). Del mismo modo, la API **consume** tres sistemas de terceros externos al límite: el **Sistema de Pago**, para crear el cobro de las suscripciones Premium y recibir su confirmación de forma asíncrona (**TS44**); el **Analytics Service**, al que publica métricas para generar reportes; y el **Notification Service** (Firebase/SendGrid), al que invoca para despachar las alertas por correo y push (**TS42**).



Software Architecture Container Level Diagram.

4.1.3.3. Software Architecture Deployment Diagrams

Este diagrama muestra que el sistema FreshSense se despliega en tres entornos principales: los **dispositivos cliente** (navegador web y móvil), el **refrigerador o cámara fría del usuario**, donde el dispositivo FreshSense mide temperatura, humedad y gas etileno y transmite por la red Wi-Fi local, y la **nube**, que aloja el frontend estático, el contenedor de la API (Spring Boot + Java 24), la base de datos MySQL administrada, la caché Redis exigida por la **TS45** y la documentación Swagger/OpenAPI. Todas las comunicaciones utilizan HTTPS, y la ingesta de telemetría se autentica con token de dispositivo (**TS41**).



Software Architecture Deployment Diagram.

4.2. Tactical-Level Domain-Driven Design

Esta sección detalla el diseño táctico de cada uno de los ocho bounded contexts de FreshSense, siguiendo la arquitectura en capas (Domain, Interface, Application, Infrastructure) y el patrón CQS (Command/Query Separation) usado de forma consistente en todo el backend.

4.2.1 Bounded Context: User Management

El **User Management (IAM — Identity and Access Management) Bounded Context** es responsable de gestionar la autenticación y la creación de usuarios en el sistema. Este contexto asegura que los usuarios puedan registrarse, iniciar sesión y gestionar sus credenciales de manera segura, cumpliendo con los estándares de seguridad y privacidad. Implementa el control de acceso basado en roles (RBAC) con tokens JWT exigido por la **TS43**, y el flujo de recuperación de contraseña de la **US35**.

4.2.1.1. Domain Layer

La Domain Layer del User Management Bounded Context encapsula la lógica de negocio del contexto. En esta capa se definen los agregados, entidades, objetos de valor, servicios de dominio y políticas que representan los conceptos clave del sistema.

Aggregates

1. **User**
   - *Propósito:* Agregado raíz que representa a una persona con cuenta en FreshSense. Encapsula credenciales, datos de identidad y roles de acceso.
   - *Atributos:*

Atributo	Tipo	Descripción
id	UUID	Identificador único del usuario.
email	Email (VO)	Correo electrónico único, usado como nombre de usuario.
fullName	String	Nombre completo del usuario.
passwordHash	PasswordHash (VO)	Contraseña almacenada con hash BCrypt (nunca en texto plano).

Atributo	Tipo	Descripción
roles	Set<Role>	Roles asignados (ROLE_USER, ROLE_PREMIUM, ROLE_ADMIN).
createdAt / updatedAt	DateTime	Campos de auditoría.

- *Métodos:*

Método	Descripción
User(SignUpCommand, Set<Role>)	Crea un usuario validando unicidad del email y la política de contraseña.
addRole(Role) / addRoles(List<Role>)	Asigna uno o varios roles válidos al usuario (US36).
requestPasswordReset()	Genera un `ResetToken` temporal y único para la recuperación (US35).
resetPassword(token, newHash)	Valida el token vigente, reemplaza el hash y lo invalida (uso único).
registerSignIn()	Registra el último acceso y habilita la emisión del JWT.

Entities

1. **Role** — Rol que puede asignarse a un usuario para el control de acceso RBAC (TS43, US36).

Atributo	Tipo	Descripción
id	UUID	Identificador del rol.
name	Roles (enum)	ROLE_USER, ROLE_PREMIUM o ROLE_ADMIN.

Método	Descripción
getDefaultRole()	Devuelve ROLE_USER.
toRoleFromName(String)	Convierte un nombre en instancia de Role.
validateRoleSet(List<Role>)	Asigna el rol por defecto si el conjunto es nulo o vacío.

2. **ResetToken** — Token de un solo uso para restablecer la contraseña (US35).

Atributo	Tipo	Descripción
id	UUID	Identificador.
tokenHash	String	Hash del token enviado por correo.
expiresAt	DateTime	Vencimiento del enlace de recuperación.
used	boolean	Indica si ya fue consumido.

Método	Descripción
isExpired()	Evalúa si el token venció.
markUsed()	Invalida el token.

Value Objects

Value Object	Descripción
Email	Encapsula y valida el formato del correo electrónico.
PasswordHash	Encapsula el hash BCrypt; impide exponer la contraseña en texto plano.
Roles	Enumeración inmutable: `ROLE_USER`, `ROLE_PREMIUM`, `ROLE_ADMIN`.
AccessToken	Token JWT emitido tras la autenticación, con los claims de rol (TS43).

Domain Services

Servicio	Descripción
PasswordPolicyService	Valida longitud mínima, mayúscula, minúscula y número antes de aceptar una contraseña.
TokenService	Genera y valida los tokens JWT que protegen los endpoints, incluidos los Premium (TS43).

Domain Events

Evento	Descripción
UserRegistered	Se creó la cuenta (ES: *cuenta registrada*). Dispara la creación del plan FREE y del perfil.
SessionStarted	El usuario se autenticó correctamente (ES: *sesión iniciada*).
PasswordResetRequested	Se solicitó el enlace de recuperación (US35).
PasswordReset	Se restableció la contraseña.

Evento	Descripción
UserRoleChanged	Un administrador modificó el rol o plan del usuario (US36).

Policies

- **Validar si existe el usuario**: no se permite registrar dos cuentas con el mismo email.
- **Asignar rol por defecto**: cuando se registra un usuario sin roles, el sistema le asigna automáticamente `ROLE_USER`.
- **Bloquear endpoints Premium**: cuando un JWT sin `ROLE_PREMIUM` invoca un endpoint exclusivo, el middleware responde `403 Forbidden` (TS43).

4.2.1.2. Interface Layer

La Interface Layer del User Management expone los puntos de entrada del contexto mediante controladores REST, recursos y *assemblers* que traducen entre el modelo HTTP y el modelo de dominio.

Controllers

1. **AuthenticationController** — Registro, inicio de sesión y recuperación de credenciales.

Método	Endpoint	Descripción
POST	/api/v1/authentication/sign-up	Registra un nuevo usuario en el sistema.
POST	/api/v1/authentication/sign-in	Autentica al usuario y genera un token de acceso.
POST	/api/v1/authentication/forgot-password	Envía el enlace de restablecimiento (US35).
POST	/api/v1/authentication/reset-password	Restablece la contraseña con el token recibido.

- *Dependencias:* UserCommandService

2. **UsersController** — Gestiona las operaciones relacionadas con los usuarios.

Método	Endpoint	Descripción
GET	/api/v1/users	Obtiene la lista de todos los usuarios (solo ROLE_ADMIN).
GET	/api/v1/users/{userId}	Obtiene los detalles de un usuario específico por su ID.
PATCH	/api/v1/users/{userId}/roles	Modifica los roles del usuario (US36).

- *Dependencias:* UserQueryService, UserCommandService

3. **RolesController** — Gestiona las consultas relacionadas con los roles.

Método	Endpoint	Descripción
GET	/api/v1/roles	Obtiene la lista de todos los roles disponibles.

- *Dependencias:* RoleQueryService

Resources

Resource	Campos
UserResource	id, email, fullName, roles
AuthenticatedUserResource	id, email, token (JWT)
SignUpResource	email, fullName, password, roles
SignInResource	email, password
ResetPasswordResource	token, newPassword
RoleResource	id, name

Transformers / Assemblers

Assembler	Transformación
UserResourceFromEntityAssembler	Convierte una entidad `User` en un recurso `UserResource`.
AuthenticatedUserResourceFromEntityAssembler	Convierte un `User` y su token en `AuthenticatedUserResource`.
SignUpCommandFromResourceAssembler	Convierte `SignUpResource` en el comando `SignUpCommand`.
SignInCommandFromResourceAssembler	Convierte `SignInResource` en el comando `SignInCommand`.
RoleResourceFromEntityAssembler	Convierte una entidad `Role` en un recurso `RoleResource`.

4.2.1.3 Application Layer

La Application Layer del User Management coordina las operaciones entre la Interface Layer y la Domain Layer, orquestando comandos, consultas y la reacción a eventos de otros contextos.

Command Services

Servicio	Comandos que maneja	Descripción
UserCommandServiceImpl	SignUpCommand, SignInCommand, RequestPasswordResetCommand, ResetPasswordCommand	Registra usuarios, autentica credenciales, emite el JWT y gestiona la recuperación de contraseña.
RoleCommandServiceImpl	SeedRolesCommand, AssignRoleCommand	Crea los roles predefinidos al iniciar el sistema y asigna roles a los usuarios (US36).

Query Services / Facades

Servicio	Consultas	Descripción
UserQueryServiceImpl	GetAllUsersQuery, GetUserByIdQuery, GetUserByEmailQuery	Recupera usuarios registrados sin modificar el estado del sistema.
RoleQueryServiceImpl	GetAllRolesQuery, GetRoleByNameQuery	Recupera los roles disponibles.
IamContextFacade	existsUserById, getRolesOf	Open Host Service que exponen los demás bounded contexts para validar identidad.

Event Handlers

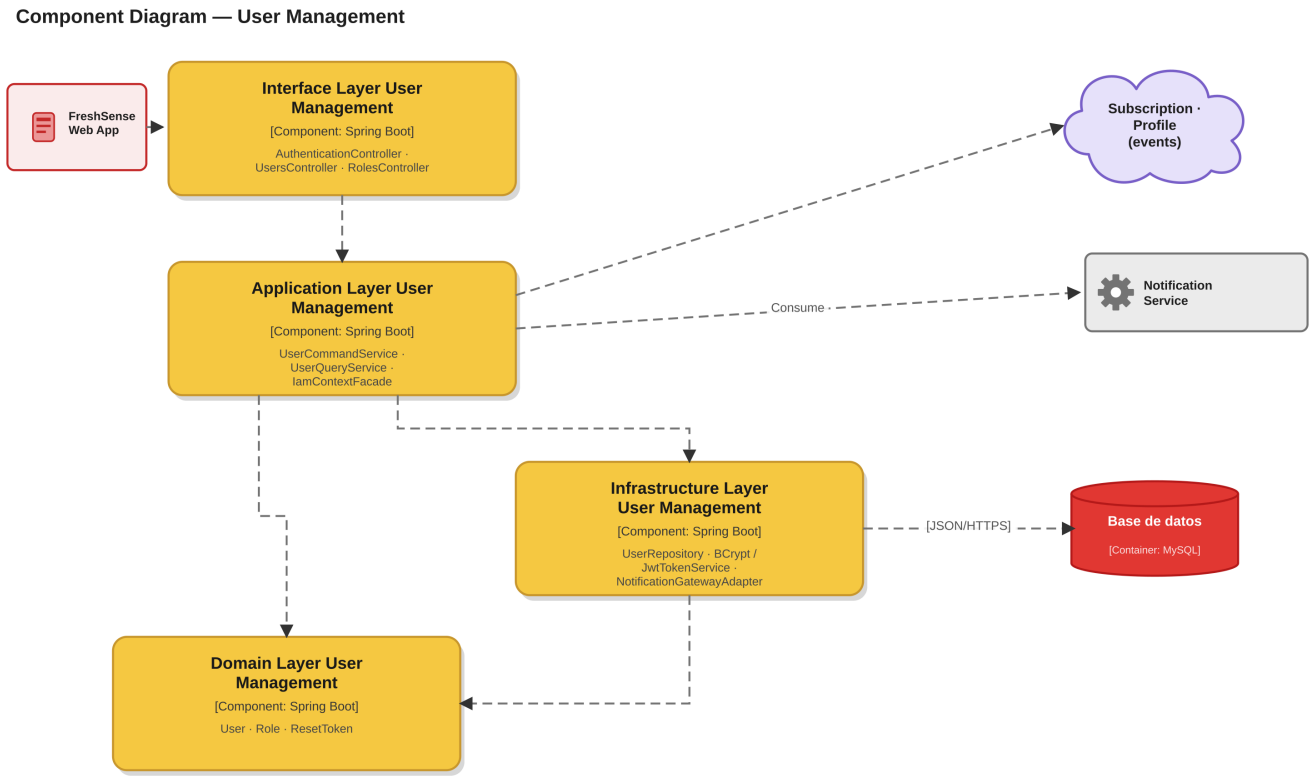
Handler	Evento que escucha	Reacción
ApplicationReadyEventHandler	ApplicationReadyEvent (Spring Boot)	Ejecuta `SeedRolesCommand` para crear ROLE_USER, ROLE_PREMIUM y ROLE_ADMIN si no existen.
SubscriptionChangedEventHandler	PlanTypeAcquired (Subscription Management)	Sincroniza el rol `ROLE_PREMIUM` del usuario cuando su plan cambia.

4.2.1.4. Infrastructure Layer

La Infrastructure Layer del User Management implementa la persistencia y las integraciones técnicas (ACL) hacia otros contextos y sistemas externos.

Componente	Tipo	Responsabilidad
UserRepository	JPA Repository	Persistencia de usuarios; `findByEmail`, `existsByEmail`.
RoleRepository	JPA Repository	Persistencia de roles; `findByName`, `existsByName`.
ResetTokenRepository	JPA Repository	Persistencia de los tokens de recuperación (US35).
BCryptHashingService	Servicio técnico	Codifica y verifica contraseñas con BCrypt.
JwtTokenService	Servicio técnico	Genera y valida los tokens JWT con los claims de rol (TS43).
NotificationGatewayAdapter	ACL saliente	Solicita al Notification Service el envío del correo de recuperación (TS42).

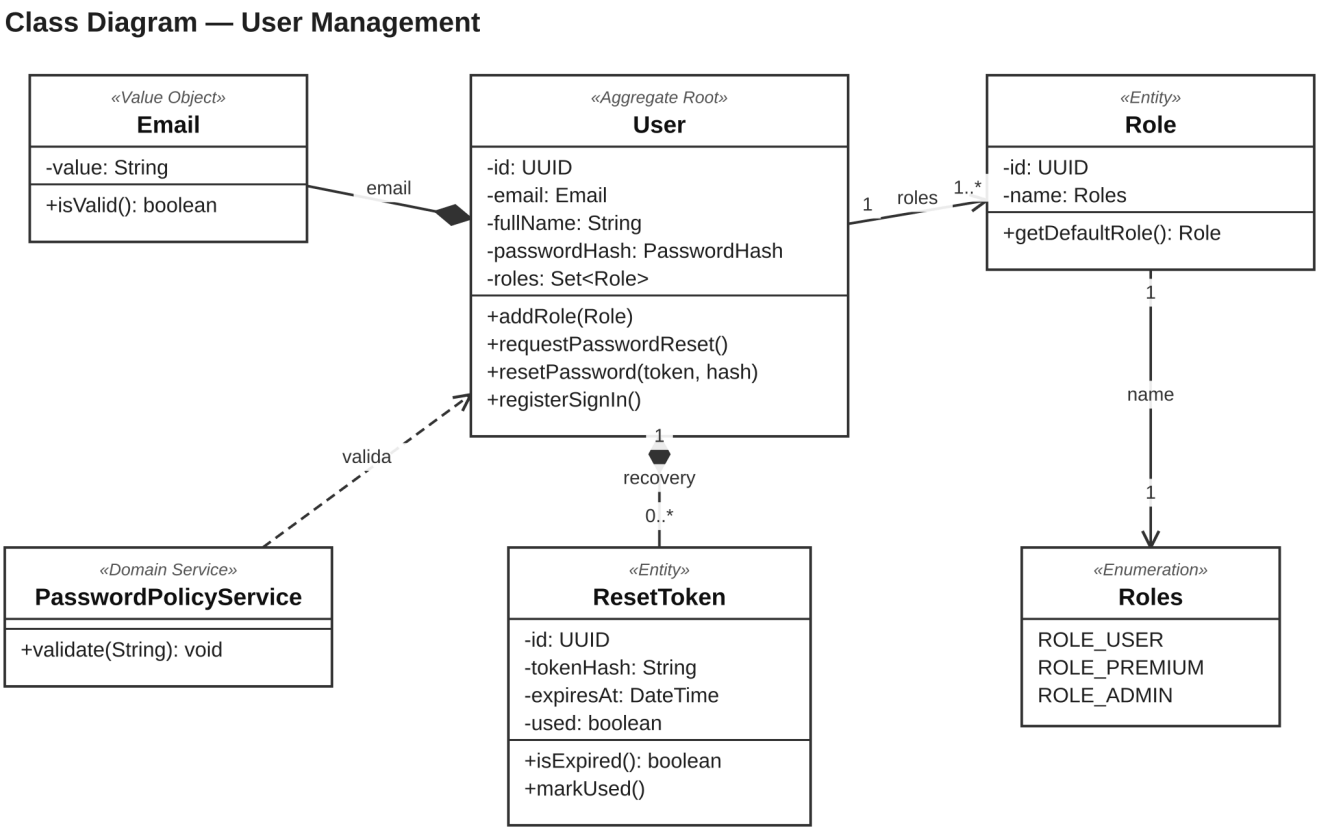
4.2.1.5. Bounded Context Software Architecture Component Level Diagrams



Component Level Diagram — User Management.

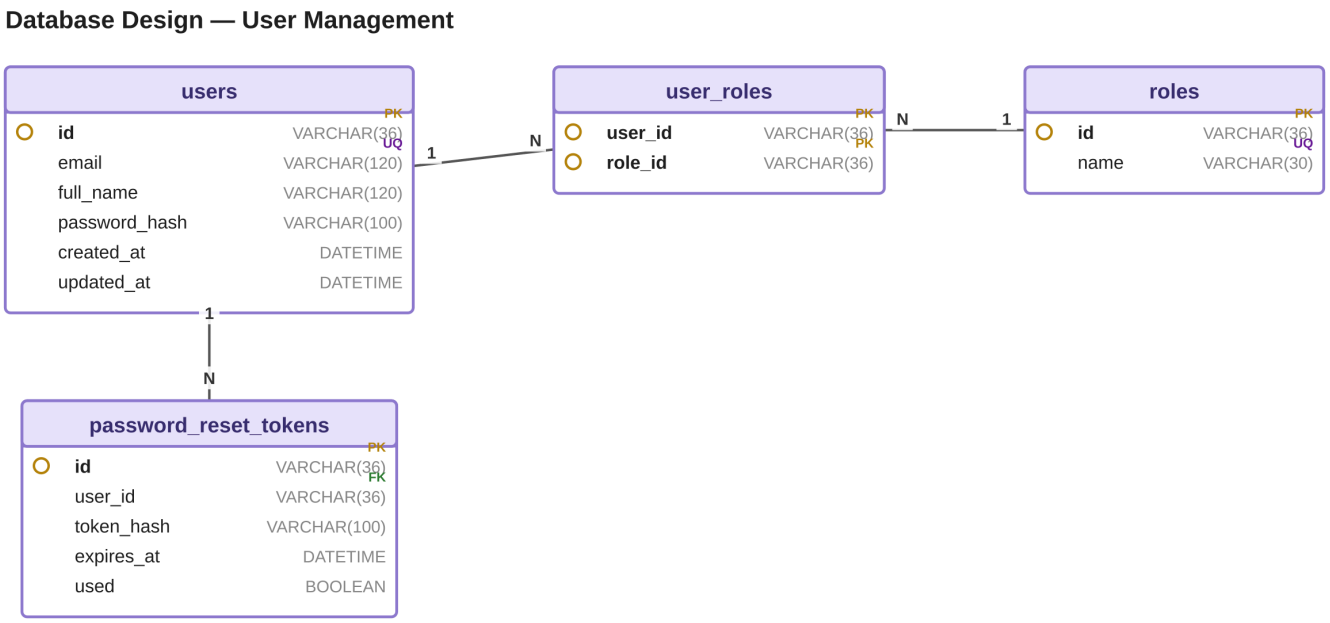
4.2.1.6. Bounded Context Software Architecture Code Level Diagrams

4.2.1.6.1. Bounded Context Domain Layer Class Diagrams



Class Diagram (Domain Layer) — User Management.

4.2.1.6.2. Bounded Context Database Design Diagram



Database Design Diagram — User Management.

4.2.2 Bounded Context: Subscription Management

El **Subscription Management Bounded Context** administra el ciclo de vida de los planes de FreshSense (FREE y PREMIUM), el cobro recurrente a través de la pasarela de pagos y la emisión de comprobantes. Implementa la integración con el **Sistema de Pago** exigida por la **TS44** y la descarga de facturas de la **US38**. Es el contexto que habilita las funcionalidades Premium (US17-US19).

4.2.2.1. Domain Layer

La Domain Layer del Subscription Management Bounded Context encapsula la lógica de negocio del contexto. En esta capa se definen los agregados, entidades, objetos de valor, servicios de dominio y políticas que representan los conceptos clave del sistema.

Aggregates

1. **Subscription**
    - *Propósito:* Agregado raíz que representa el plan contratado por un usuario y su vigencia.
    - *Atributos:*

Atributo	Tipo	Descripción
id	UUID	Identificador de la suscripción.
userId	UUID	Usuario propietario del plan.
plan	PlanType (VO)	FREE o PREMIUM.
startedAt	DateTime	Fecha de inicio de la vigencia.
endsAt	DateTime?	Fecha de término; nula para el plan FREE.
active	boolean	Indica si la suscripción está vigente.

- *Métodos:*

Método	Descripción
isActive()	Indica si el plan está vigente a la fecha actual.
upgradeToPremium(period)	Cambia el plan a PREMIUM y fija la fecha de término.
renew(days)	Extiende la vigencia al confirmarse el webhook de pago (TS44).
cancel()	Desactiva la renovación automática y marca el fin del periodo.

Entities

1. **Payment** — Cobro procesado por la pasarela para una suscripción (TS44, US38).

Atributo	Tipo	Descripción
id	UUID	Identificador del pago.
subscriptionId	UUID	Suscripción asociada.
amount	Money (VO)	Importe cobrado.
provider	String	Identificador del medio de pago procesado por el Sistema de Pago.
externalRef	String	Identificador del cobro en la pasarela.
paidAt	DateTime	Fecha de confirmación.
invoiceUrl	String	Enlace al comprobante en PDF (US38).

Método	Descripción
confirm(externalRef)	Marca el pago como confirmado tras el webhook.
generateInvoice()	Produce el comprobante descargable en PDF.

Value Objects

Value Object	Descripción
PlanType	Enumeración inmutable: `FREE`, `PREMIUM`.
Money	Importe con moneda; evita operar con números sueltos.
BillingPeriod	Periodo de facturación: `MONTHLY` o `ANNUAL`.

Domain Services

Servicio	Descripción
PlanEntitlementService	Determina qué funcionalidades (analítica avanzada, costeo de fichas técnicas, exportación) habilita cada plan.

Domain Events

Evento	Descripción
PlanTypeAcquired	Se asignó un plan al usuario (ES: *tipo de plan adquirido*).
SubscriptionUpgraded	El usuario pasó de FREE a PREMIUM.
SubscriptionRenewed	La pasarela confirmó la renovación y se extendió la vigencia (TS44).
SubscriptionExpired	Venció la suscripción y el usuario regresó al plan FREE.
PaymentConfirmed	Se registró un cobro exitoso y su comprobante (US38).

Policies

- **Asignar plan gratuito**: cuando ocurre `UserRegistered`, se crea automáticamente una suscripción FREE.
- **Extender vigencia por webhook**: cuando la pasarela confirma la renovación mensual, se extiende la fecha de vigencia 30 días (TS44).
- **Degradar al vencer**: cuando `endsAt` es anterior a la fecha actual, la suscripción se marca inactiva y el usuario vuelve a FREE.

4.2.2.2. Interface Layer

La Interface Layer del Subscription Management expone los puntos de entrada del contexto mediante controladores REST, recursos y *assemblers* que traducen entre el modelo HTTP y el modelo de dominio.

Controllers

1. **SubscriptionsController** — Gestiona la consulta y el cambio de plan del usuario.

Método	Endpoint	Descripción
GET	/api/v1/subscriptions/me	Obtiene la suscripción vigente del usuario autenticado.
POST	/api/v1/subscriptions/upgrade	Inicia el cambio a plan PREMIUM consumiendo el Sistema de Pago para crear el cobro (TS44).
POST	/api/v1/subscriptions/cancel	Cancela la renovación automática.

- *Dependencias:* SubscriptionCommandService, SubscriptionQueryService

2. **PaymentsController** — Expone el historial de pagos y los comprobantes (US38).

Método	Endpoint	Descripción
GET	/api/v1/payments	Lista los cobros del usuario.
GET	/api/v1/payments/{paymentId}/invoice	Descarga el comprobante en PDF.

- *Dependencias:* PaymentQueryService

3. **BillingWebhookController** — Recibe las notificaciones de la pasarela de pagos (TS44).

Método	Endpoint	Descripción
POST	/api/v1/billing/webhook	Procesa la notificación del Sistema de Pago y extiende la vigencia.

- *Dependencias:* SubscriptionCommandService

Resources

Resource	Campos
SubscriptionResource	id, plan, startedAt, endsAt, active
UpgradeResource	plan, billingPeriod
PaymentResource	id, amount, provider, paidAt, invoiceUrl
WebhookPayloadResource	eventType, externalRef, amount, status

Transformers / Assemblers

Assembler	Transformación
SubscriptionResourceFromEntityAssembler	Convierte `Subscription` en `SubscriptionResource`.
PaymentResourceFromEntityAssembler	Convierte `Payment` en `PaymentResource`.
UpgradeCommandFromResourceAssembler	Convierte `UpgradeResource` en `UpgradeSubscriptionCommand`.

4.2.2.3 Application Layer

La Application Layer del Subscription Management coordina las operaciones entre la Interface Layer y la Domain Layer, orquestando comandos, consultas y la reacción a eventos de otros contextos.

Command Services

Servicio	Comandos que maneja	Descripción
SubscriptionCommandServiceImpl	CreateFreeSubscriptionCommand, UpgradeSubscriptionCommand, RenewSubscriptionCommand, CancelSubscriptionCommand	Crea, actualiza y cancela suscripciones; al ejecutar `UpgradeSubscriptionCommand` consume el `PaymentGatewayAdapter` para iniciar el cobro en el Sistema de Pago.
PaymentCommandServiceImpl	ConfirmPaymentCommand, GenerateInvoiceCommand	Registra los cobros confirmados y genera los comprobantes en PDF (US38).

Query Services / Facades

Servicio	Consultas	Descripción
SubscriptionQueryServiceImpl	GetSubscriptionByUserQuery, GetActivePlanQuery	Recupera el plan vigente del usuario.
PaymentQueryServiceImpl	GetPaymentsByUserQuery, GetInvoiceQuery	Recupera el historial de cobros y comprobantes.
SubscriptionsContextFacade	hasPremiumAccess(userId)	Open Host Service consultado por Report e Inventory para habilitar funciones Premium.

Event Handlers

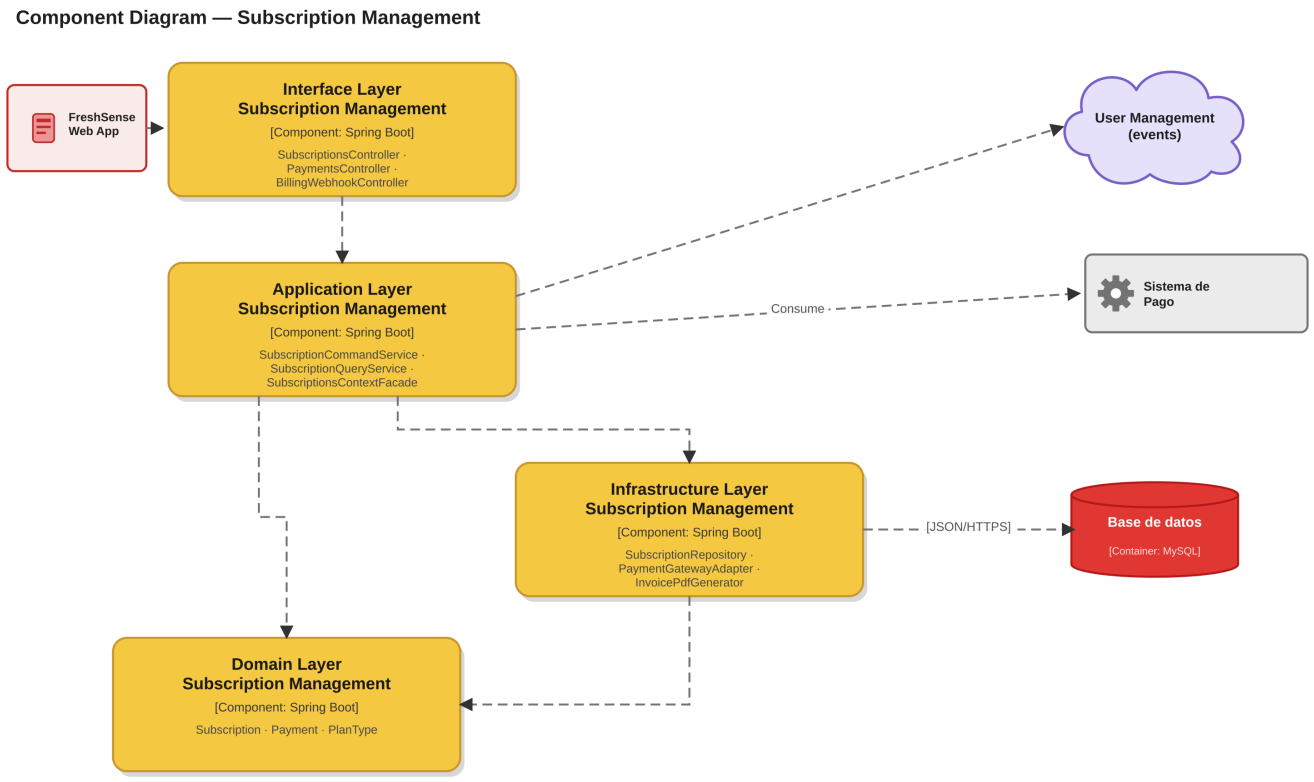
Handler	Evento que escucha	Reacción
UserRegisteredEventHandler	UserRegistered (User Management)	Crea automáticamente una suscripción FREE para el usuario recién registrado.
BillingWebhookEventHandler	PaymentConfirmed (pasarela externa)	Extiende la vigencia de la suscripción 30 días y emite `SubscriptionRenewed` (TS44).

4.2.2.4. Infrastructure Layer

La Infrastructure Layer del Subscription Management implementa la persistencia y las integraciones técnicas (ACL) hacia otros contextos y sistemas externos.

Componente	Tipo	Responsabilidad
SubscriptionRepository	JPA Repository	Persistencia de suscripciones; `findByUserId`, `findActiveByUserId`.
PaymentRepository	JPA Repository	Persistencia de los cobros y sus comprobantes.
PaymentGatewayAdapter	ACL saliente	Consume la API del Sistema de Pago para crear el cobro y traduce su confirmación al modelo de dominio (TS44).
InvoicePdfGenerator	Servicio técnico	Genera el comprobante en PDF con JasperReports (US38).
ExternalUserService	ACL entrante	Consulta el `IamContextFacade` para validar la existencia del usuario.

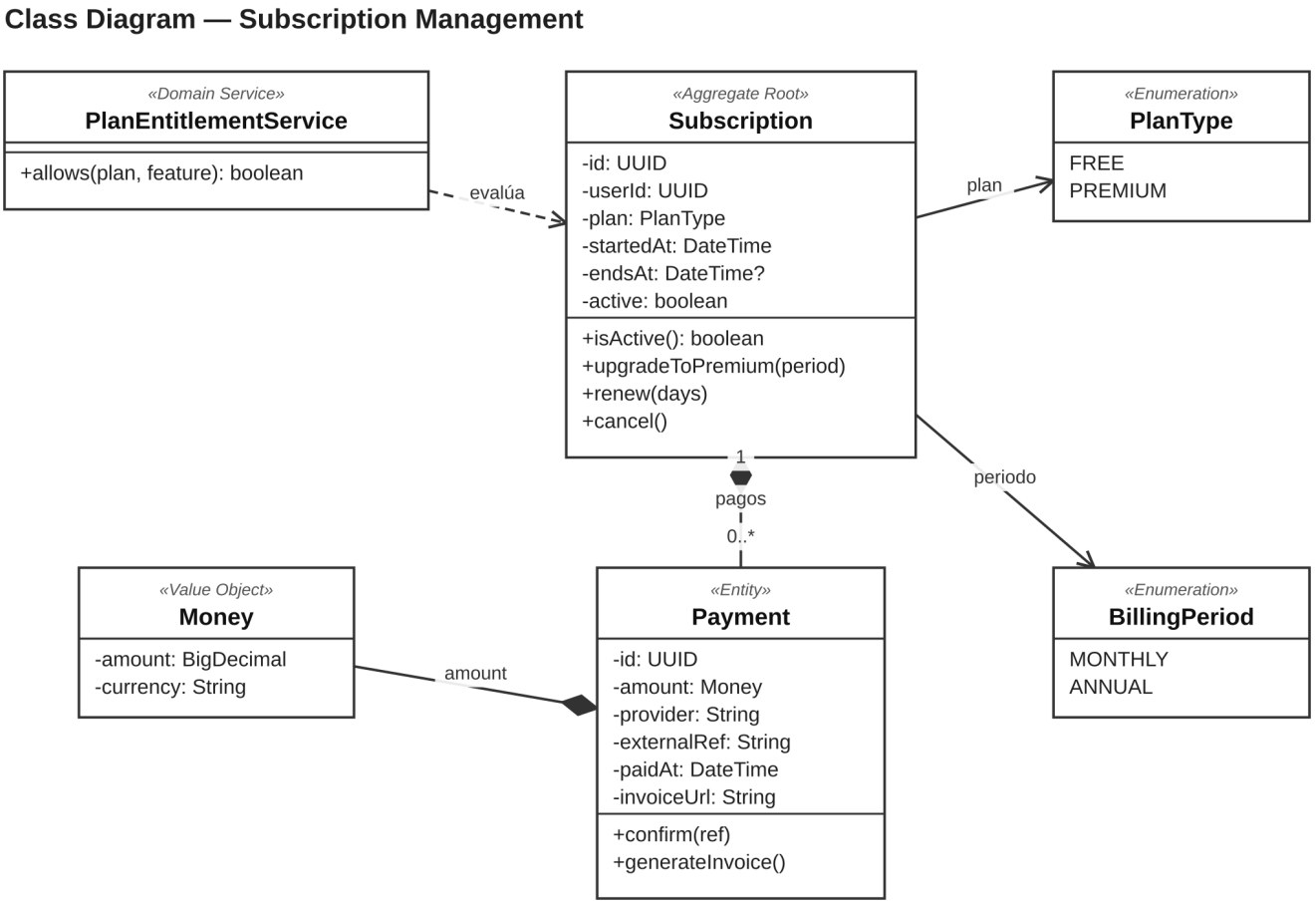
4.2.2.5. Bounded Context Software Architecture Component Level Diagrams



Component Level Diagram — Subscription Management.

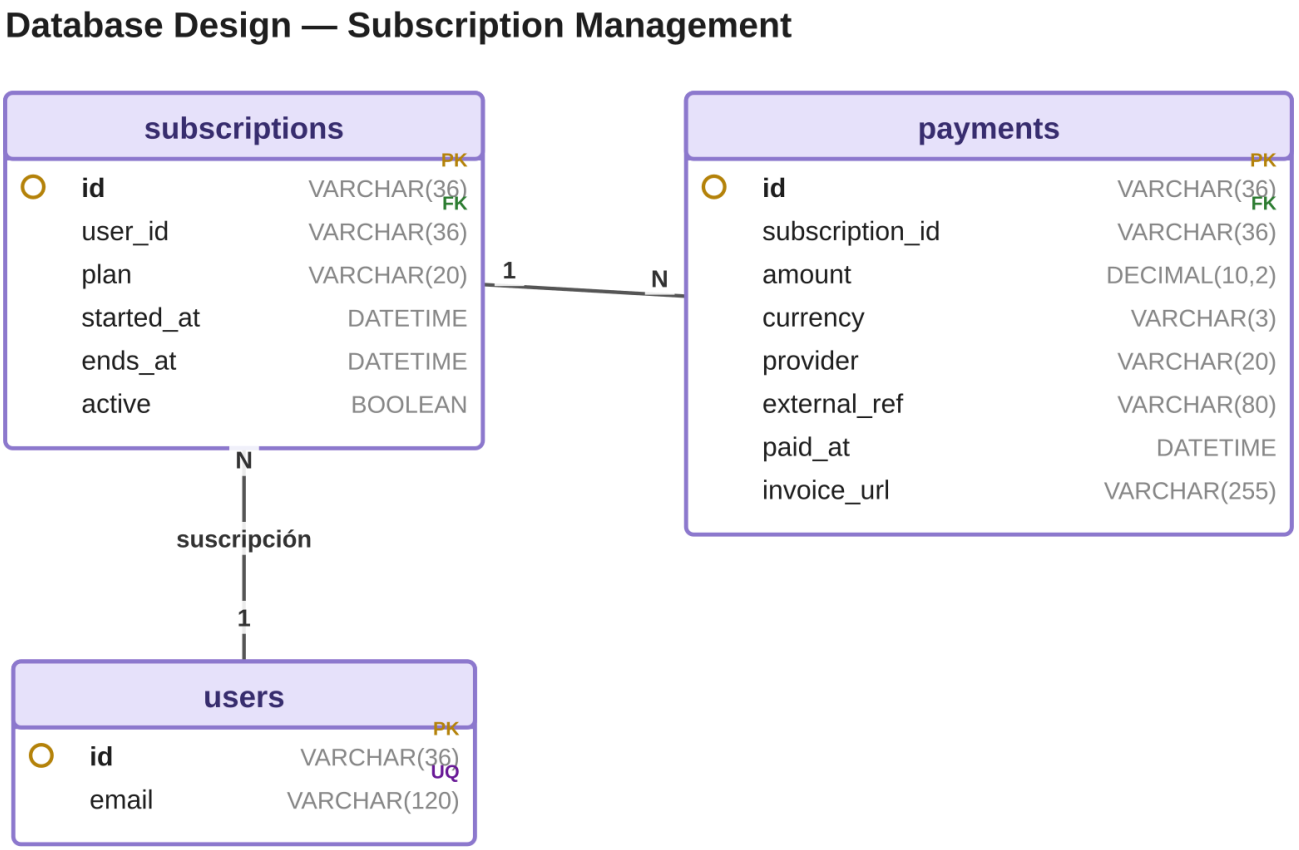
4.2.2.6. Bounded Context Software Architecture Code Level Diagrams

4.2.2.6.1. Bounded Context Domain Layer Class Diagrams



Class Diagram (Domain Layer) — Subscription Management.

4.2.2.6.2. Bounded Context Database Design Diagram



Database Design Diagram — Subscription Management.

4.2.3 Bounded Context: Profile Management

El **Profile Management Bounded Context** modela el **negocio** (`Business`) que agrupa a los usuarios y sus dispositivos, la gestión de miembros exigida por la **US37**, y las preferencias de notificación de la **US09** y **US24**. Representa a los dos segmentos objetivo: el **restaurante** y el **negocio de distribución de alimentos en frío**. Es el contexto que permite que el propietario, el administrador y el personal de cocina o almacén compartan un mismo inventario, y que cada uno configure cómo y en qué turno recibir alertas.

4.2.3.1. Domain Layer

La Domain Layer del Profile Management Bounded Context encapsula la lógica de negocio del contexto. En esta capa se definen los agregados, entidades, objetos de valor, servicios de dominio y políticas que representan los conceptos clave del sistema.

Aggregates

1. **Business**
    - *Propósito:* Agregado raíz que representa el negocio (restaurante o distribuidora en frío) al que pertenecen los usuarios, los dispositivos y el inventario compartido.
    - *Atributos:*

Atributo	Tipo	Descripción
id	UUID	Identificador del negocio.
name	String	Nombre comercial (por ejemplo, «Restaurante El Sabor» o «Distribuidora Frío Norte»).
segment	BusinessSegment (VO)	RESTAURANT o COLD_DISTRIBUTION: segmento objetivo al que pertenece.
members	List<BusinessMember>	Miembros vinculados con su rol.
createdAt / updatedAt	DateTime	Campos de auditoría.

- *Métodos:*

Método	Descripción
invite(email, role)	Emite una invitación para incorporar a un miembro (US37).
addMember(userId, role)	Vincula un usuario al negocio cuando acepta la invitación.
removeMember(userId)	Desvincula a un miembro y revoca su acceso al inventario.
changeMemberRole(userId, role)	Modifica el rol del miembro dentro del negocio.

Entities

1. **BusinessMember** — Vínculo entre un usuario y un negocio, con el rol que desempeña (US37).

Atributo	Tipo	Descripción
id	UUID	Identificador del vínculo.
businessId	UUID	Negocio al que pertenece.
userId	UUID	Usuario vinculado.
role	MemberRole (VO)	OWNER, MANAGER o STAFF.
joinedAt	DateTime	Fecha de incorporación.

Método	Descripción
canManageInventory()	Indica si el rol permite editar el inventario compartido.

2. **Invitation** — Invitación pendiente para incorporar a un nuevo miembro (US37).

Atributo	Tipo	Descripción
id	UUID	Identificador de la invitación.
email	Email (VO)	Correo del invitado.
token	String	Token único del enlace de vinculación.
expiresAt	DateTime	Vencimiento de la invitación.
status	InvitationStatus	PENDING, ACCEPTED o EXPIRED.

Método	Descripción
accept(userId)	Convierte la invitación en un `BusinessMember`.

Método	Descripción
isExpired()	Evalúa el vencimiento.

3. **NotificationPreference** — Preferencias de canal y turno operativo de cada usuario (US09, US24).

Atributo	Tipo	Descripción
id	UUID	Identificador.
userId	UUID	Usuario propietario.
emailEnabled	boolean	Habilita el envío por correo.
pushEnabled	boolean	Habilita las notificaciones push.
operatingShift	TimeWindow (VO)	Turno operativo del usuario (por ejemplo, 06:00–23:00).
updatedAt	DateTime	Última modificación.

Método	Descripción
isOffShift(at)	Indica si un instante cae fuera del turno operativo.
routeFor(severity)	Determina el canal de destino según la severidad (US24).

Value Objects

Value Object	Descripción
MemberRole	Enumeración: `OWNER` (propietario), `MANAGER` (administrador o encargado), `STAFF` (personal de cocina o almacén).
BusinessSegment	Enumeración: `RESTAURANT`, `COLD_DISTRIBUTION`.
TimeWindow	Rango horario con inicio y fin, usado para el turno operativo.
InvitationStatus	Enumeración: `PENDING`, `ACCEPTED`, `EXPIRED`.

Domain Services

Servicio	Descripción
NotificationRoutingService	Decide el canal y el destinatario (responsable de turno o pantalla del local) según severidad y preferencias (US24).

Domain Events

Evento	Descripción
BusinessCreated	Se creó el negocio tras registrarse el usuario.
MemberInvited	Se envió una invitación a un nuevo miembro (US37).
MemberJoined	Un invitado aceptó y quedó vinculado al negocio.
NotificationPreferencesUpdated	El usuario cambió sus preferencias de alerta (US09).
OperatingShiftConfigured	Se definió el turno operativo para las notificaciones (US09).

Policies

- **Crear negocio al registrarse**: cuando ocurre `UserRegistered`, se crea un `Business` con el usuario como `OWNER`.
- **Retener alertas leves fuera de turno**: cuando se dispara una alerta de baja severidad fuera del turno operativo, se retiene hasta el inicio del siguiente turno (US09).
- **Invitación de un solo uso**: una invitación aceptada o vencida no puede volver a utilizarse.

4.2.3.2. Interface Layer

La Interface Layer del Profile Management expone los puntos de entrada del contexto mediante controladores REST, recursos y *assemblers* que traducen entre el modelo HTTP y el modelo de dominio.

Controllers

1. **BusinessesController** — Gestiona el negocio y sus miembros.

Método	Endpoint	Descripción
GET	/api/v1/businesses/me	Obtiene el negocio del usuario autenticado y sus miembros.
POST	/api/v1/businesses/{id}/invitations	Envía una invitación a un nuevo miembro (US37).
POST	/api/v1/invitations/{token}/accept	Acepta la invitación y vincula al usuario.
DELETE	/api/v1/businesses/{id}/members/{userId}	Desvincula a un miembro.

- *Dependencias:* BusinessCommandService, BusinessQueryService

2. **NotificationPreferencesController** — Gestiona las preferencias de notificación (US09, US24).

Método	Endpoint	Descripción
GET	/api/v1/notification-preferences	Obtiene las preferencias del usuario.
PUT	/api/v1/notification-preferences	Actualiza canales y turno operativo.

- *Dependencias:* NotificationPreferenceCommandService

Resources

Resource	Campos
BusinessResource	id, name, segment, members[]
BusinessMemberResource	userId, fullName, role, joinedAt
InvitationResource	email, role
NotificationPreferenceResource	emailEnabled, pushEnabled, shiftStart, shiftEnd

Transformers / Assemblers

Assembler	Transformación
BusinessResourceFromEntityAssembler	Convierte `Business` y sus miembros en `BusinessResource`.
NotificationPreferenceResourceFromEntityAssembler	Convierte `NotificationPreference` en su recurso.
InviteMemberCommandFromResourceAssembler	Convierte `InvitationResource` en `InviteMemberCommand`.

4.2.3.3 Application Layer

La Application Layer del Profile Management coordina las operaciones entre la Interface Layer y la Domain Layer, orquestando comandos, consultas y la reacción a eventos de otros contextos.

Command Services

Servicio	Comandos que maneja	Descripción
BusinessCommandServiceImpl	CreateBusinessCommand, InviteMemberCommand, AcceptInvitationCommand, RemoveMemberCommand	Crea el negocio, gestiona invitaciones y administra la membresía compartida (US37).
NotificationPreferenceCommandServiceImpl	UpdatePreferencesCommand, SetOperatingShiftCommand	Actualiza canales y turno operativo de las alertas (US09).

Query Services / Facades

Servicio	Consultas	Descripción
BusinessQueryServiceImpl	GetBusinessByUserQuery, GetMembersQuery	Recupera el negocio y sus miembros.
ProfileContextFacade	getBusinessOf(userId), getPreferencesOf(userId)	Open Host Service consultado por Inventory (inventario compartido) y por IoT Monitoring (envío de alertas).

Event Handlers

Handler	Evento que escucha	Reacción
UserRegisteredEventHandler	UserRegistered (User Management)	Crea el `Business` inicial, con el segmento elegido en el registro y el usuario como OWNER, y sus preferencias de notificación por defecto.
PlanTypeAcquiredEventHandler	PlanTypeAcquired (Subscription Management)	Ajusta el número máximo de miembros permitidos según el plan contratado.

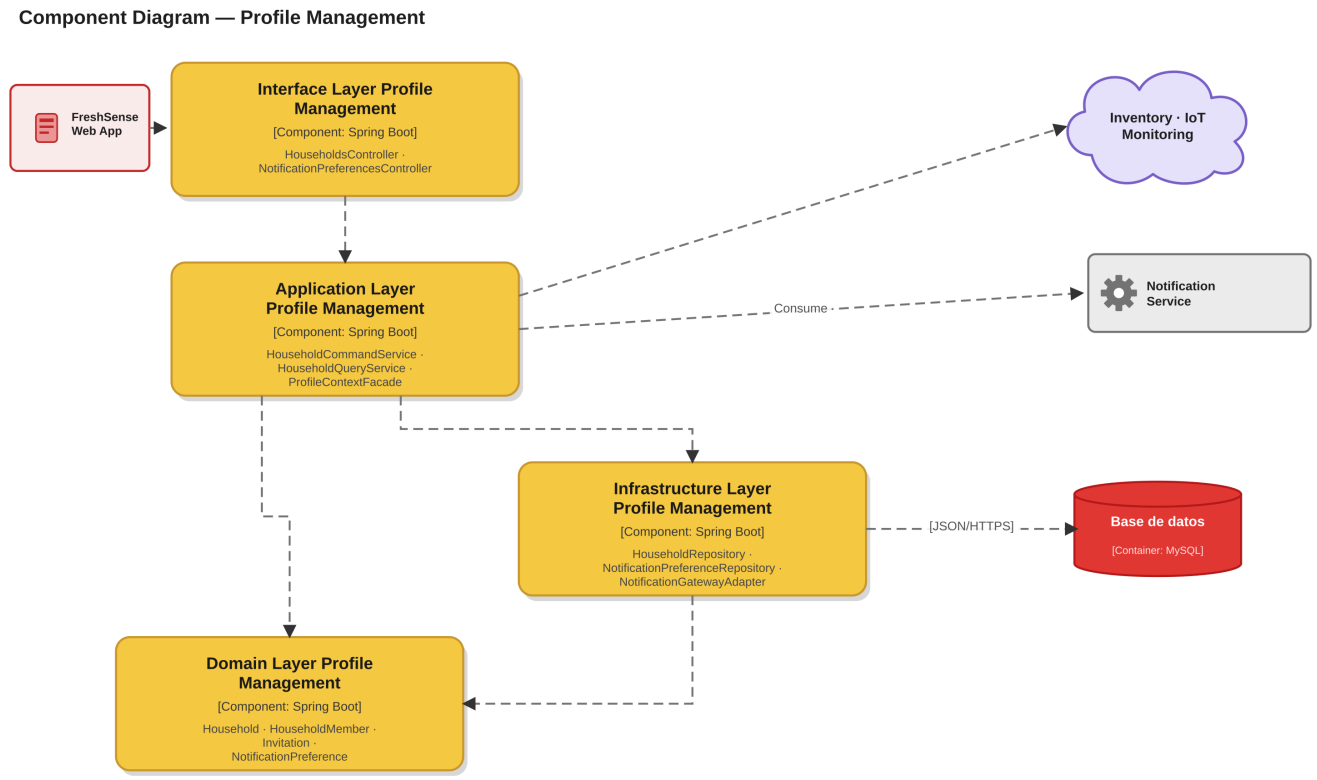
4.2.3.4. Infrastructure Layer

La Infrastructure Layer del Profile Management implementa la persistencia y las integraciones técnicas (ACL) hacia otros contextos y sistemas externos.

Componente	Tipo	Responsabilidad
BusinessRepository	JPA Repository	Persistencia de negocios; `findByMemberUserId`.
BusinessMemberRepository	JPA Repository	Persistencia de los vínculos usuario–negocio.
InvitationRepository	JPA Repository	Persistencia de invitaciones pendientes (US37).
NotificationPreferenceRepository	JPA Repository	Persistencia de las preferencias de alerta.
ExternalUserService	ACL entrante	Consulta el `IamContextFacade` para resolver los datos del usuario.

Componente	Tipo	Responsabilidad
NotificationGatewayAdapter	ACL saliente	Envía el correo de invitación mediante el Notification Service (TS42).

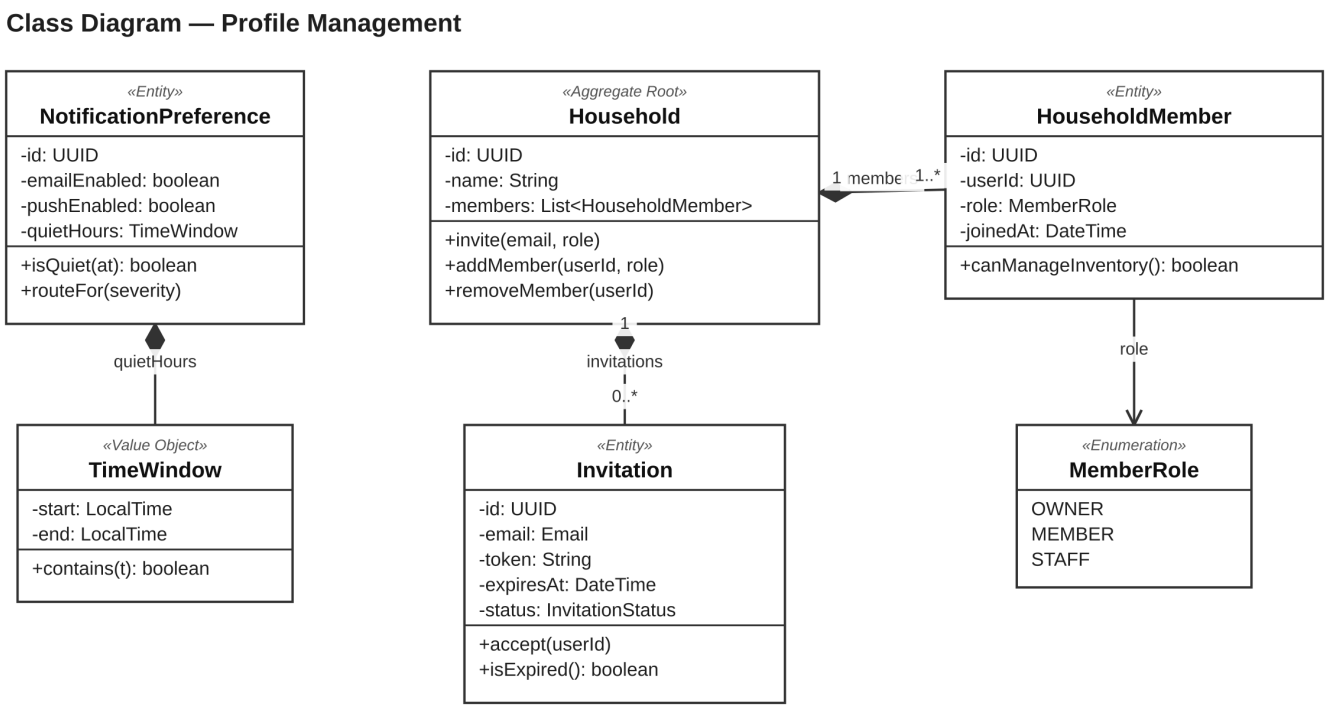
4.2.3.5. Bounded Context Software Architecture Component Level Diagrams



Component Level Diagram — Profile Management.

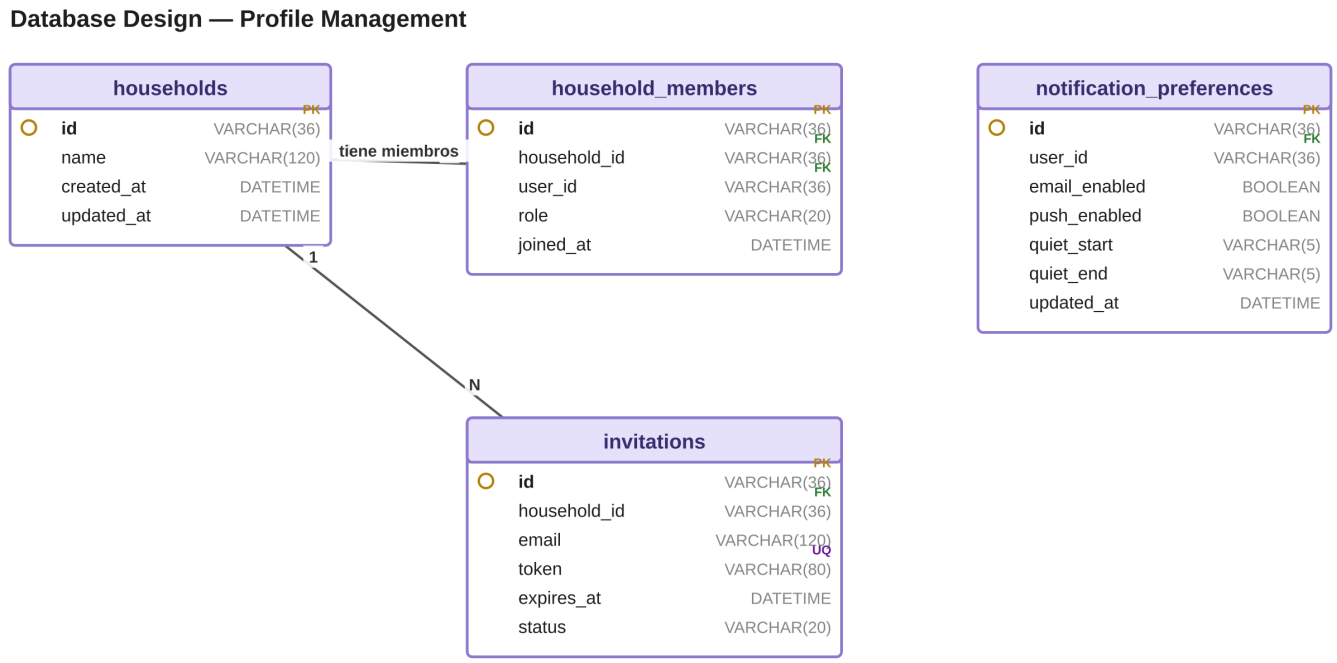
4.2.3.6. Bounded Context Software Architecture Code Level Diagrams

4.2.3.6.1. Bounded Context Domain Layer Class Diagrams



Class Diagram (Domain Layer) — Profile Management.

4.2.3.6.2. Bounded Context Database Design Diagram



Database Design Diagram — Profile Management.

4.2.4 Bounded Context: Sensor Management

El **Sensor Management Bounded Context** gestiona el ciclo de vida del hardware **FreshSense Device**: la vinculación mediante código QR y la calibración exigidas por la **US31**, el control del estado de conexión de la **US06**, y la organización en **zonas de almacenamiento** (cámaras frías, congeladores) que pide la **US33**. Este contexto conoce el dispositivo como equipo físico; la interpretación de sus lecturas corresponde a IoT Monitoring.

4.2.4.1. Domain Layer

La Domain Layer del Sensor Management Bounded Context encapsula la lógica de negocio del contexto. En esta capa se definen los agregados, entidades, objetos de valor, servicios de dominio y políticas que representan los conceptos clave del sistema.

Aggregates

1. **SensorDevice**
    - *Propósito:* Agregado raíz que representa un dispositivo FreshSense instalado en una cámara fría, congelador o vitrina del negocio.
    - *Atributos:*

Atributo	Tipo	Descripción
id	UUID	Identificador del dispositivo.
businessId	UUID	Negocio propietario.
model	String	Modelo del hardware.
firmwareVersion	String	Versión de firmware instalada.
status	DeviceStatus (VO)	ACTIVE, OFFLINE o UNPAIRED.
zoneId	UUID?	Zona de almacenamiento asignada (US33).
calibration	Calibration (VO)	Temperatura base configurada durante la instalación (US31).
registeredAt / updatedAt	DateTime	Campos de auditoría.

- *Métodos:*

Método	Descripción
register()	Da de alta el dispositivo en el sistema.
pairToBusiness(Business)	Vincula el dispositivo al negocio tras escanear su código QR (US31).
calibrate(baseTemperature)	Registra la temperatura base de la cámara para ajustar los umbrales.
assignToZone(zoneId)	Asigna el dispositivo a una cámara o congelador concreto (US33).
markOffline()	Marca el dispositivo como desconectado al no recibirse el *heartbeat* (US06).

Entities

1. **StorageZone** — Área de almacenamiento del negocio, con sus propias reglas de temperatura (US33).

Atributo	Tipo	Descripción
id	UUID	Identificador de la zona.
businessId	UUID	Negocio propietario.
name	String	Nombre asignado (por ejemplo, «Cámara Fría 1» o «Congelador Carnes»).
type	ZoneType (VO)	FRIDGE, FREEZER, COLD_ROOM o DISPLAY_CASE (vitrina).
targetTempC	float	Temperatura objetivo de la zona.

Método	Descripción
appliesRulesTo(InventoryItem)	Indica si un alimento hereda las reglas de temperatura de la zona.

Value Objects

Value Object	Descripción
DeviceStatus	Enumeración: `ACTIVE`, `OFFLINE`, `UNPAIRED`.
ZoneType	Enumeración: `FRIDGE`, `FREEZER`, `COLD_ROOM`, `DISPLAY_CASE`.
Calibration	Temperatura base y desviación aceptada, definidas al instalar el equipo (US31).
PairingCode	Código QR impreso en el hardware que identifica al dispositivo durante la vinculación.

Domain Services

Servicio	Descripción
DeviceHealthService	Evalúa el *heartbeat* recibido y determina cuándo un dispositivo pasa a OFFLINE (US06).

Domain Events

Evento	Descripción
SensorRegistered	Se dio de alta un dispositivo en el sistema.
SensorPaired	El dispositivo quedó vinculado al negocio tras escanear el QR (ES: *sensor instalado*).
SensorCalibrated	Se registró la temperatura base de la cámara (US31).
SensorConfigured	Se asignó el dispositivo a una zona de almacenamiento (ES: *sensor configurado*).
SensorDisconnected	El dispositivo perdió el enlace Wi-Fi y se notificó al usuario (US06).

Policies

- **Vinculación única**: un código QR solo puede vincularse a un negocio a la vez.
- **Notificar desconexión**: cuando el servidor no recibe *heartbeat* dentro de la ventana esperada, se marca el dispositivo OFFLINE y se avisa al usuario (US06).
- **Heredar reglas de zona**: cuando un dispositivo se asigna a una zona, los alimentos asociados adoptan sus umbrales de temperatura (US33).

4.2.4.2. Interface Layer

La Interface Layer del Sensor Management expone los puntos de entrada del contexto mediante controladores REST, recursos y *assemblers* que traducen entre el modelo HTTP y el modelo de dominio.

Controllers

1. **DevicesController** — Gestiona el alta, vinculación y calibración de los dispositivos.

Método	Endpoint	Descripción
GET	/api/v1/devices	Lista los dispositivos del negocio.
POST	/api/v1/devices/pair	Vincula un dispositivo escaneando su código QR (US31).
PATCH	/api/v1/devices/{id}/calibration	Registra la temperatura base de calibración.
PATCH	/api/v1/devices/{id}/zone	Asigna el dispositivo a una zona de almacenamiento.

- *Dependencias:* DeviceCommandService, DeviceQueryService

2. **StorageZonesController** — Administra las zonas de almacenamiento del negocio (US33).

Método	Endpoint	Descripción
GET	/api/v1/storage-zones	Lista las zonas configuradas.

Método	Endpoint	Descripción
POST	/api/v1/storage-zones	Crea una nueva cámara, congelador o refrigerador.
DELETE	/api/v1/storage-zones/{id}	Elimina una zona sin dispositivos asociados.

- *Dependencias:* StorageZoneCommandService

3. **DeviceHeartbeatController** — Recibe la señal periódica de vida del dispositivo (US06).

Método	Endpoint	Descripción
POST	/api/v1/devices/{id}/heartbeat	Registra el *ping* del dispositivo y actualiza su estado.

- *Dependencias:* DeviceCommandService

Resources

Resource	Campos
DeviceResource	id, model, firmwareVersion, status, zoneId, calibration
PairDeviceResource	pairingCode (QR), businessId
CalibrationResource	baseTemperatureC
StorageZoneResource	id, name, type, targetTempC

Transformers / Assemblers

Assembler	Transformación
DeviceResourceFromEntityAssembler	Convierte `SensorDevice` en `DeviceResource`.
StorageZoneResourceFromEntityAssembler	Convierte `StorageZone` en su recurso.
PairDeviceCommandFromResourceAssembler	Convierte `PairDeviceResource` en `PairDeviceCommand`.

4.2.4.3 Application Layer

La Application Layer del Sensor Management coordina las operaciones entre la Interface Layer y la Domain Layer, orquestando comandos, consultas y la reacción a eventos de otros contextos.

Command Services

Servicio	Comandos que maneja	Descripción
DeviceCommandServiceImpl	RegisterDeviceCommand, PairDeviceCommand, CalibrateDeviceCommand, AssignZoneCommand, RegisterHeartbeatCommand	Da de alta, vincula, calibra y monitorea el estado de conexión de los dispositivos.
StorageZoneCommandServiceImpl	CreateZoneCommand, DeleteZoneCommand	Administra las áreas de almacenamiento del negocio (US33).

Query Services / Facades

Servicio	Consultas	Descripción
DeviceQueryServiceImpl	GetDevicesByBusinessQuery, GetDeviceByIdQuery	Recupera los dispositivos y su estado.
StorageZoneQueryServiceImpl	GetZonesByBusinessQuery	Recupera las zonas configuradas.
SensorContextFacade	existsDevice(id), getZoneOf(deviceId), getCalibration(deviceId)	Open Host Service consultado por IoT Monitoring para validar el origen de las lecturas.

Event Handlers

Handler	Evento que escucha	Reacción
BusinessCreatedEventHandler	BusinessCreated (Profile Management)	Crea la zona de almacenamiento por defecto («Cámara principal») para el nuevo negocio.
PlanTypeAcquiredEventHandler	PlanTypeAcquired (Subscription Management)	Ajusta el número máximo de dispositivos y zonas permitidos según el plan.

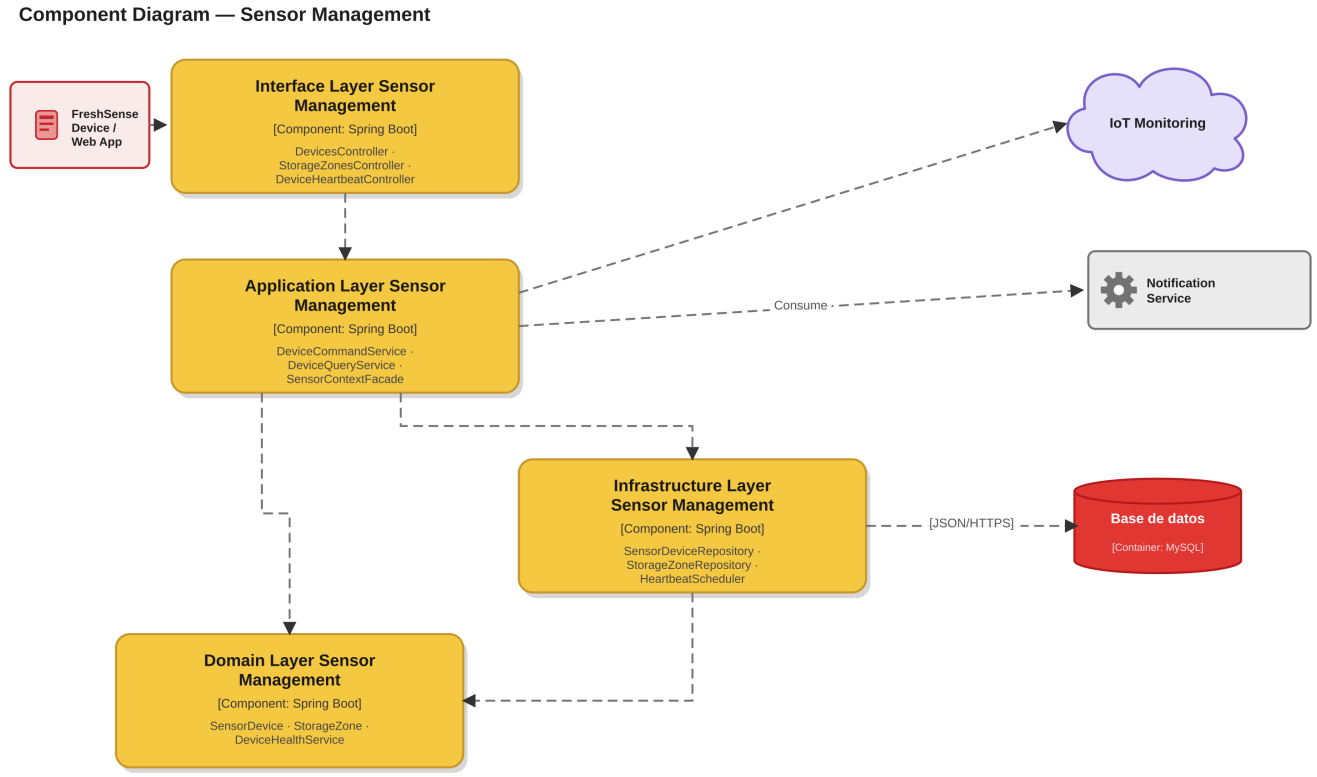
4.2.4.4. Infrastructure Layer

La Infrastructure Layer del Sensor Management implementa la persistencia y las integraciones técnicas (ACL) hacia otros contextos y sistemas externos.

Componente	Tipo	Responsabilidad
SensorDeviceRepository	JPA Repository	Persistencia de dispositivos; `findByBusinessId`, `findByPairingCode`.

Componente	Tipo	Responsabilidad
StorageZoneRepository	JPA Repository	Persistencia de las zonas de almacenamiento.
HeartbeatScheduler	Spring Scheduler	Revisa periódicamente los *heartbeats* y marca los dispositivos OFFLINE (US06).
ExternalBusinessService	ACL entrante	Consulta el `ProfileContextFacade` para validar el negocio propietario.
NotificationGatewayAdapter	ACL saliente	Notifica al usuario la desconexión del dispositivo (TS42).

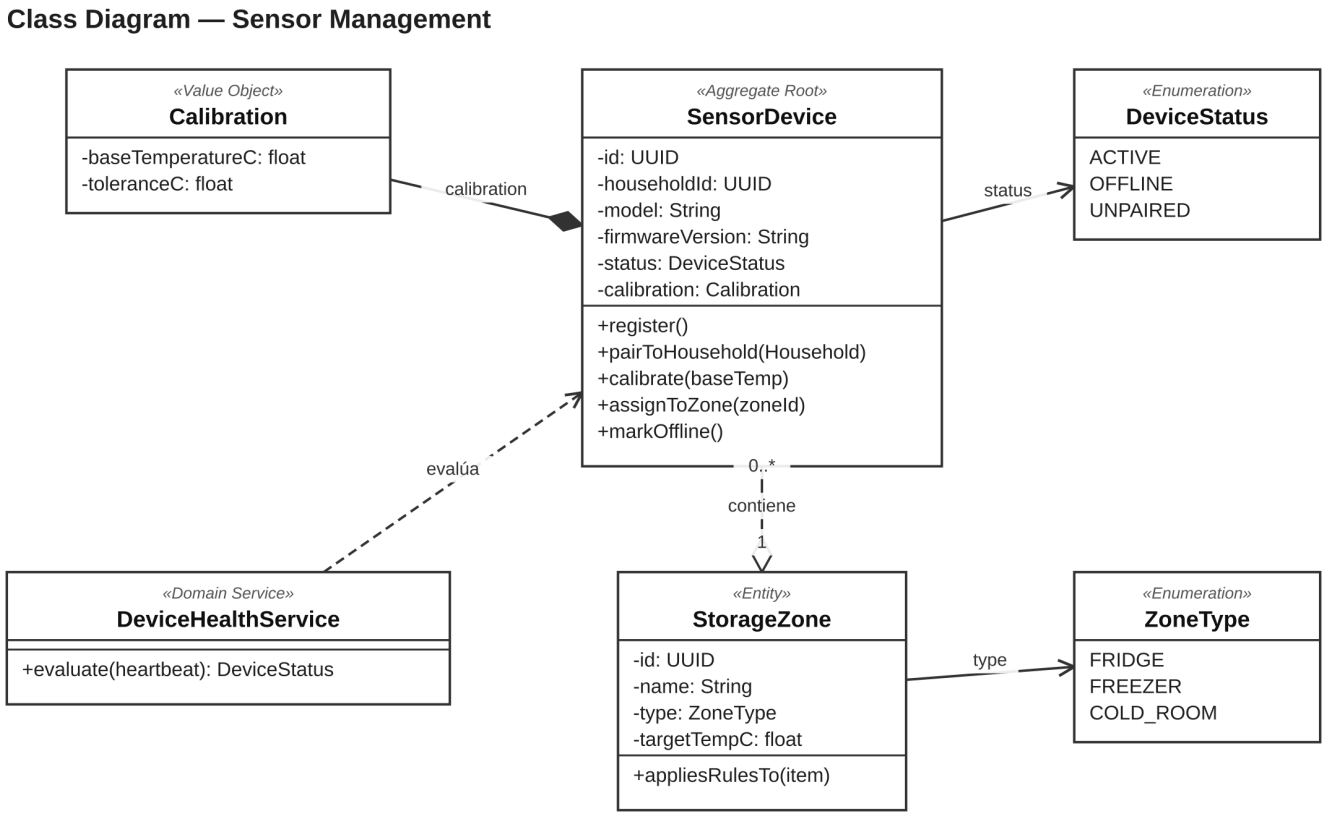
4.2.4.5. Bounded Context Software Architecture Component Level Diagrams



Component Level Diagram — Sensor Management.

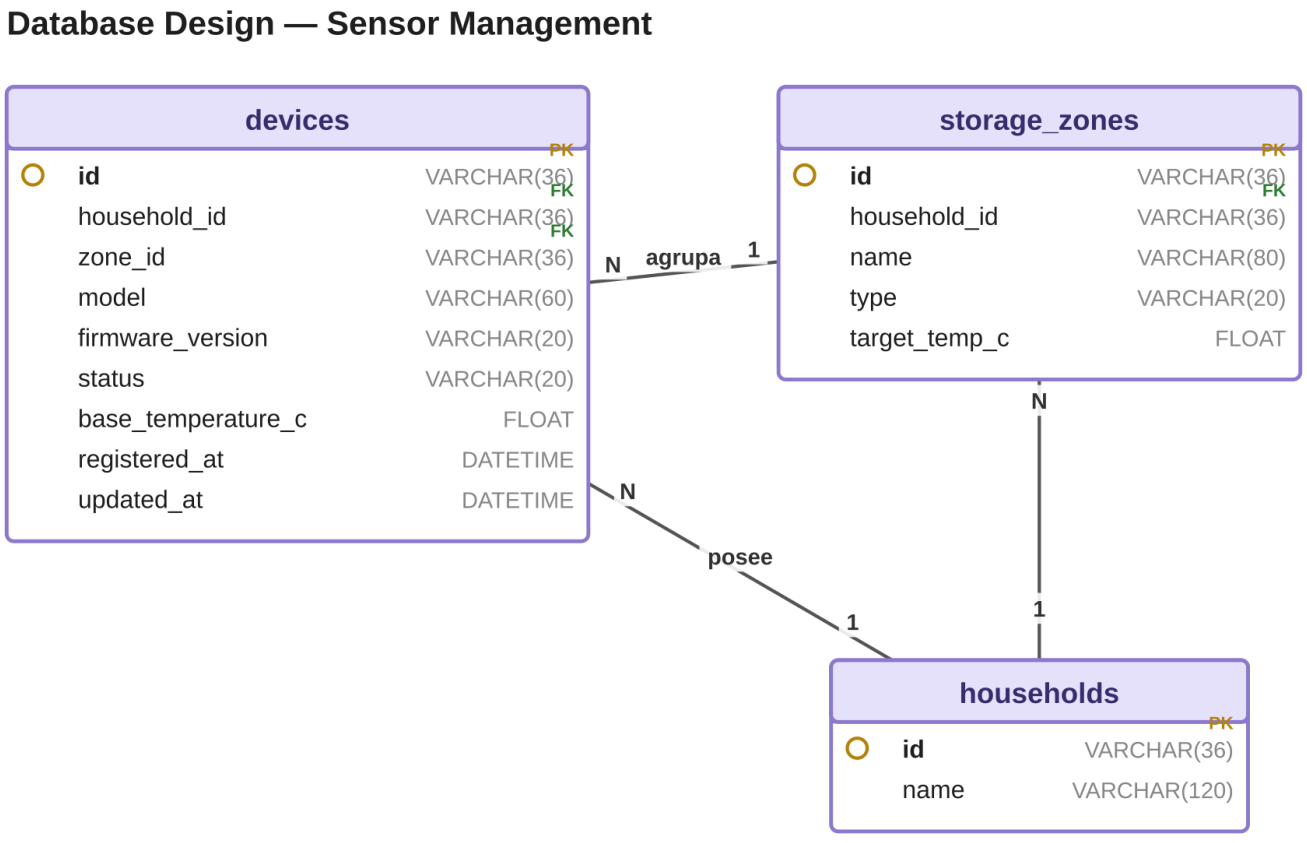
4.2.4.6. Bounded Context Software Architecture Code Level Diagrams

4.2.4.6.1. Bounded Context Domain Layer Class Diagrams



Class Diagram (Domain Layer) — Sensor Management.

4.2.4.6.2. Bounded Context Database Design Diagram



Database Design Diagram — Sensor Management.

4.2.5 Bounded Context: Inventory Management

El **Inventory Management Bounded Context** administra el inventario de insumos y lotes del negocio, ya sea la cocina de un restaurante o el almacén de una distribuidora en frío: el alta por código de barras o QR (**US10**), el filtrado por cámara, lote y vencimiento (**US14**), la edición manual del stock (**US11**), la categorización personalizada (**US28**) y el registro del motivo de descarte (**US34**). Contiene el agregado `InventoryItem` y el catálogo `FoodModel`, que define la vida útil estimada por categoría de alimento y las reglas usadas para estimar la fecha de consumo preferente.

4.2.5.1. Domain Layer

La Domain Layer del Inventory Management Bounded Context encapsula la lógica de negocio del contexto. En esta capa se definen los agregados, entidades, objetos de valor, servicios de dominio y políticas que representan los conceptos clave del sistema.

Aggregates

1. **InventoryItem**
    - *Propósito:* Agregado raíz que representa un insumo o lote registrado en el inventario del negocio, con su cantidad, vencimiento estimado y estado de frescura.
    - *Atributos:*

Atributo	Tipo	Descripción
id	UUID	Identificador del alimento.
businessId	UUID	Negocio propietario del inventario.
userId	UUID	Miembro del personal que lo registró.
zoneId	UUID	Cámara, congelador o vitrina donde se almacena (US14, US33).
lotCode	String	Código de lote leído del código de barras o QR (US10, US14).
supplier	String	Proveedor que entregó el lote (US40).
foodModelId	UUID	Categoría del catálogo que define su vida útil.
name	String	Nombre del alimento.
quantity	Quantity (VO)	Cantidad disponible.
unit	Unit (VO)	Unidad de medida (kg, unidades, litros).
addedAt	DateTime	Fecha de registro en el inventario.
bestBefore	DateTime?	Fecha estimada de consumo preferente.
status	FreshnessStatus (VO)	FRESH, AT_RISK o SPOILED.
notes	String	Observaciones y etiquetas personalizadas (US28).

- *Métodos:*

Método	Descripción
markConsumed()	Registra el consumo del alimento y descuenta la cantidad.
markDiscarded(reason)	Registra la baja indicando la causa: falla de frío, vencimiento, contaminación o proveedor (US34).
updateQuantity(qty)	Corrige manualmente el stock disponible (US11).
updateStatus(FreshnessStatus)	Actualiza el estado de frescura según la evaluación del dominio.
estimateBestBefore(FoodModel)	Calcula la fecha de consumo preferente a partir de la vida útil del catálogo.

Entities

1. **FoodModel** — Catálogo que define la vida útil y las reglas de conservación de cada categoría de alimento.

Atributo	Tipo	Descripción
id	UUID	Identificador del modelo.
name	String	Nombre de la categoría.
category	String	Agrupación (Fruta, Verdura, Lácteo, Carne).
shelfLifeDays	int	Vida útil estimada en días.
rulesJson	ShelfLifeRules (VO)	Umbrales de temperatura, humedad y etileno aplicables.

Método	Descripción
rulesFor(zoneType)	Devuelve los umbrales ajustados al tipo de zona de almacenamiento.

Value Objects

Value Object	Descripción
FreshnessStatus	Enumeración: `FRESH`, `AT_RISK`, `SPOILED`.
Quantity	Cantidad con validación de no negatividad.
Unit	Unidad de medida del alimento (kg, g, L, unidades).
ShelfLifeRules	Reglas de vida útil y umbrales por categoría, serializadas en JSON.
DiscardReason	Enumeración: `COLD_FAILURE`, `EXPIRED`, `CONTAMINATED`, `SUPPLIER_DEFECT` (US34).

Domain Services

Servicio	Descripción
ShelfLifeEstimationService	Estima la fecha de consumo preferente combinando el `FoodModel` y la zona de almacenamiento.
BarcodeLookupService	Resuelve el producto, la categoría y el lote a partir del código de barras o QR escaneado (US10).

Domain Events

Evento	Descripción
FoodItemRegistered	Se agregó un alimento al inventario (ES: *alimento agregado*).
FoodItemUpdated	Se modificó manualmente la cantidad o los datos del alimento (US11).
FoodItemConsumed	El alimento fue consumido (ES: *alimento consumido*).
FoodItemDiscarded	El alimento fue descartado indicando la causa (US34).
FoodConditionUpdated	Cambió el estado de frescura del alimento tras una evaluación.

Policies

- **Estimar vencimiento al registrar**: cuando se agrega un alimento, el sistema calcula automáticamente su `bestBefore` a partir del `FoodModel`.
- **Heredar reglas de la zona**: cuando el alimento se asocia a una zona de almacenamiento, adopta sus umbrales de temperatura (US33).
- **Solicitar motivo al descartar**: cuando el usuario elimina un producto marcándolo como desechado, la aplicación exige el motivo para el informe de mermas (US34).

4.2.5.2. Interface Layer

La Interface Layer del Inventory Management expone los puntos de entrada del contexto mediante controladores REST, recursos y *assemblers* que traducen entre el modelo HTTP y el modelo de dominio.

Controllers

1. **InventoryItemsController** — Gestiona el inventario de alimentos del negocio.

Método	Endpoint	Descripción
GET	/api/v1/inventory-items	Lista los insumos con su estado de frescura (semáforo, US07) y filtros por cámara, lote y vencimiento (US14).
POST	/api/v1/inventory-items	Registra un insumo o lote por código de barras, QR o manualmente (US10).
PUT	/api/v1/inventory-items/{id}	Actualiza cantidad, unidad o etiquetas (US11, US28).
POST	/api/v1/inventory-items/{id}/consume	Marca el alimento como consumido.
POST	/api/v1/inventory-items/{id}/discard	Marca el alimento como descartado con su causa (US34).

- *Dependencias:* InventoryItemCommandService, InventoryItemQueryService

2. **FoodModelsController** — Expone el catálogo de categorías y vida útil.

Método	Endpoint	Descripción
GET	/api/v1/food-models	Lista las categorías disponibles.
GET	/api/v1/food-models/{id}	Obtiene las reglas de vida útil de una categoría.

- *Dependencias:* FoodModelQueryService

3. **InventorySearchController** — Búsqueda global con autocompletado sobre el inventario (US39).

Método	Endpoint	Descripción
GET	/api/v1/inventory/search	Devuelve coincidencias en vivo de insumos, lotes y fichas técnicas.

- *Dependencias:* InventoryItemQueryService

Resources

Resource	Campos
InventoryItemResource	id, name, lotCode, zoneId, supplier, quantity, unit, addedAt, bestBefore, status, notes
CreateInventoryItemResource	name, quantity, unit, foodModelId, zoneId, lotCode?, supplier?, barcode?
DiscardResource	reason (COLD_FAILURE
FoodModelResource	id, name, category, shelfLifeDays

Transformers / Assemblers

Assembler	Transformación
InventoryItemResourceFromEntityAssembler	Convierte `InventoryItem` en `InventoryItemResource`.
FoodModelResourceFromEntityAssembler	Convierte `FoodModel` en su recurso.
CreateInventoryItemCommandFromResourceAssembler	Convierte el recurso de alta en `RegisterFoodItemCommand`.

4.2.5.3 Application Layer

La Application Layer del Inventory Management coordina las operaciones entre la Interface Layer y la Domain Layer, orquestando comandos, consultas y la reacción a eventos de otros contextos.

Command Services

Servicio	Comandos que maneja	Descripción
InventoryItemCommandServiceImpl	RegisterFoodItemCommand, UpdateFoodItemCommand, ConsumeFoodItemCommand, DiscardFoodItemCommand	Registra, actualiza, consume y descarta alimentos, publicando los eventos correspondientes.
FoodModelCommandServiceImpl	SeedFoodModelsCommand	Carga el catálogo inicial de categorías y vidas útiles al iniciar el sistema.

Query Services / Facades

Servicio	Consultas	Descripción
InventoryItemQueryServiceImpl	GetInventoryItemsQuery, GetItemsAtRiskQuery, SearchInventoryQuery	Recupera el inventario, los alimentos en riesgo y las coincidencias de búsqueda (US39).
FoodModelQueryServiceImpl	GetAllFoodModelsQuery, GetFoodModelByIdQuery	Recupera el catálogo de categorías.
InventoryContextFacade	getInventorySnapshot(businessId), getItemsAtRisk(userId)	Open Host Service consultado por Consumption y Report Management.

Event Handlers

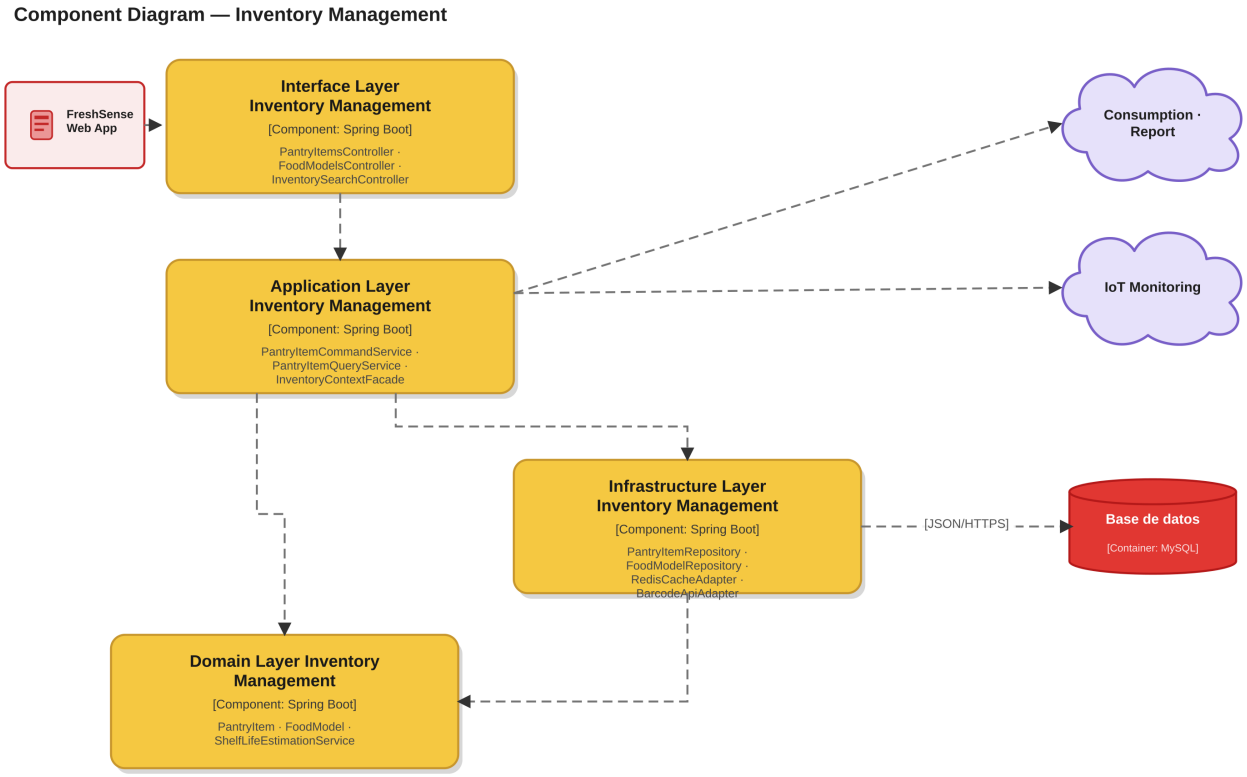
Handler	Evento que escucha	Reacción
FoodConditionUpdatedEventHandler	FoodConditionUpdated (IoT Monitoring)	Actualiza el `status` del `InventoryItem` a AT_RISK o SPOILED según la evaluación del FreshnessService.
SensorConfiguredEventHandler	SensorConfigured (Sensor Management)	Asocia los alimentos de la zona con el dispositivo que la monitorea.

4.2.5.4. Infrastructure Layer

La Infrastructure Layer del Inventory Management implementa la persistencia y las integraciones técnicas (ACL) hacia otros contextos y sistemas externos.

Componente	Tipo	Responsabilidad
InventoryItemRepository	JPA Repository	Persistencia del inventario; `findByBusinessId`, `findByZoneId`, `findByStatus`.
FoodModelRepository	JPA Repository	Persistencia del catálogo de categorías.
BarcodeApiAdapter	ACL saliente	Consulta el catálogo externo de códigos de barras y QR (US10).
RedisCacheAdapter	Caché	Almacena el catálogo y las consultas frecuentes del inventario (TS45).
ExternalSensorService	ACL entrante	Consulta el `SensorContextFacade` para conocer la zona del alimento.

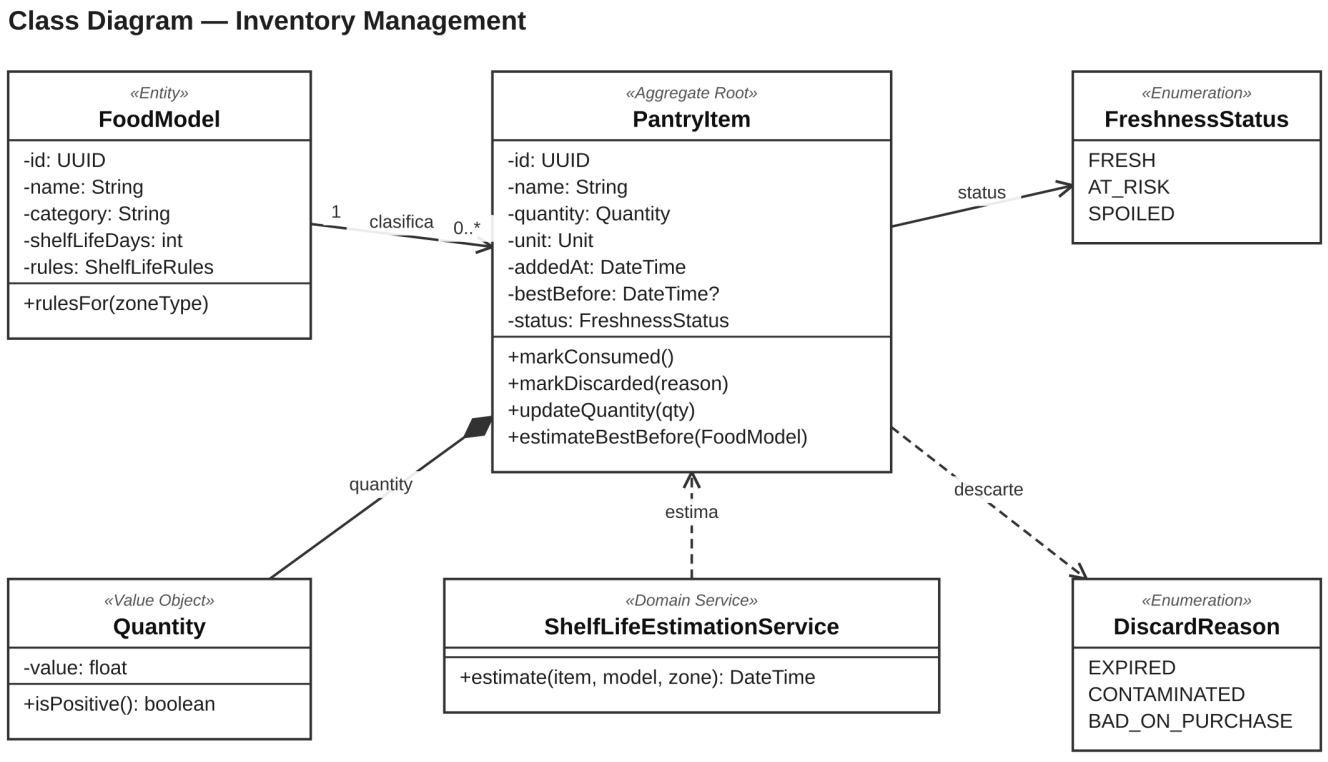
4.2.5.5. Bounded Context Software Architecture Component Level Diagrams



Component Level Diagram — Inventory Management.

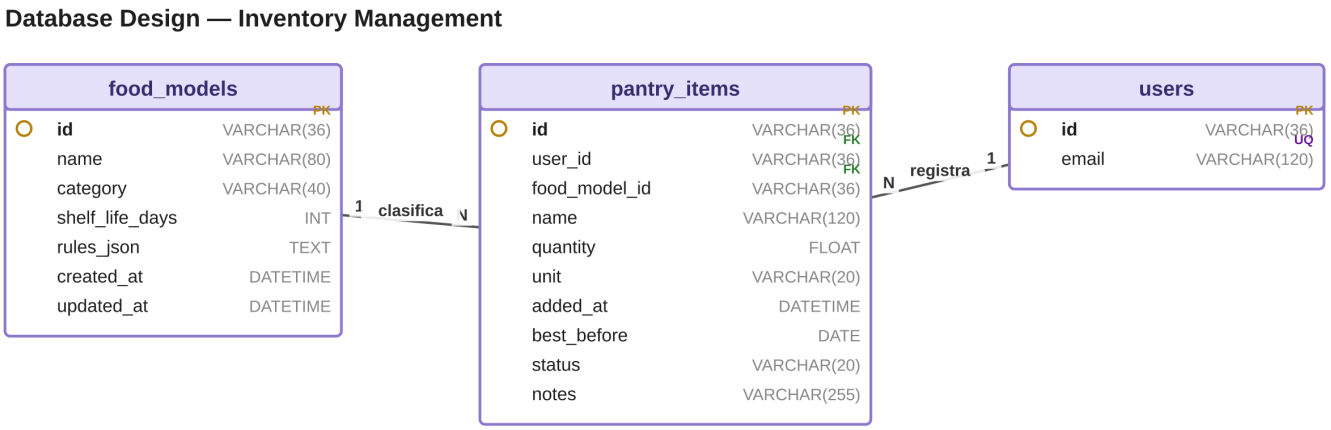
4.2.5.6. Bounded Context Software Architecture Code Level Diagrams

4.2.5.6.1. Bounded Context Domain Layer Class Diagrams



Class Diagram (Domain Layer) — Inventory Management.

4.2.5.6.2. Bounded Context Database Design Diagram



Database Design Diagram — Inventory Management.

4.2.6 Bounded Context: Consumption Management

El **Consumption Management Bounded Context** registra el consumo y la merma efectiva de los insumos y gestiona las **fichas técnicas y la rotación** (**EP05**). En el restaurante, la ficha técnica descuenta los insumos de cada platillo y prioriza los que están por vencer (**US13**), con costeo y merma por corte en el plan premium (**US19**). En la distribución en frío, sugiere la salida FIFO de los lotes (**US13**). También registra las incidencias de calidad de cada proveedor (**US40**). Su registro de consumo es la fuente de los reportes de ahorro e impacto ambiental.

4.2.6.1. Domain Layer

La Domain Layer del Consumption Management Bounded Context encapsula la lógica de negocio del contexto. En esta capa se definen los agregados, entidades, objetos de valor, servicios de dominio y políticas que representan los conceptos clave del sistema.

Aggregates

1. **Recipe**
   - *Propósito:* Agregado raíz que representa la ficha técnica estandarizada de un platillo del restaurante, con sus insumos y porciones; se usa para descontar insumos y priorizar los que están por vencer (US13).
   - *Atributos:*

Atributo	Tipo	Descripción
id	UUID	Identificador de la ficha técnica.
businessId	UUID	Restaurante al que pertenece la ficha.
title	String	Nombre del platillo.
prepMinutes	int	Tiempo de preparación en minutos.
portions	int	Porciones que rinde la ficha.
instructions	String	Pasos de preparación.
ingredients	List<RecipeIngredient>	Ingredientes requeridos.
isPremium	boolean	Indica si usa el costeo avanzado y la merma por corte (US19).

- *Métodos:*

Método	Descripción
matches(atRiskItems)	Evalúa cuántos insumos en riesgo usa la ficha técnica (US13).
addIngredient(name, qty, unit, optional)	Incorpora un insumo a la ficha técnica.
costPerPortion()	Calcula el costo por porción a partir de los insumos (US19).

Entities

1. **RecipeIngredient** — Insumo que forma parte de una ficha técnica.

Atributo	Tipo	Descripción
id	UUID	Identificador.
recipeId	UUID	Ficha técnica a la que pertenece.
name	String	Nombre del ingrediente.

Atributo	Tipo	Descripción
qty	Quantity (VO)	Cantidad requerida.
unit	Unit (VO)	Unidad de medida.
optional	boolean	Indica si el ingrediente es opcional.

Método	Descripción
isSatisfiedBy(inventoryItems)	Indica si el inventario cubre este ingrediente.

2. **ConsumptionRecord** — Registro histórico de cada alimento consumido o descartado, base de los reportes (US29, US34).

Atributo	Tipo	Descripción
id	UUID	Identificador del registro.
userId	UUID	Usuario que realizó la acción.
inventoryItemId	UUID	Alimento afectado.
action	ConsumptionAction (VO)	CONSUMED o DISCARDED.
reason	DiscardReason?	Causa de la merma, si aplica (US34).
supplier	String?	Proveedor del lote, usado en las incidencias de calidad (US40).
estimatedValue	Money (VO)	Valor económico del alimento.
recordedAt	DateTime	Fecha del registro.

Método	Descripción
isWaste()	Indica si el registro representa una merma.
carbonFootprint()	Estima el CO_2 asociado al alimento (US20).

Value Objects

Value Object	Descripción
ConsumptionAction	Enumeración: `CONSUMED`, `DISCARDED`.
SupplierIncident	Incidencia registrada contra un proveedor cuando su lote se malogra antes de tiempo (US40).

Domain Services

Servicio	Descripción
RecipeSuggestionService	Prioriza las fichas técnicas que usan insumos en estado AT_RISK del inventario del restaurante (US13).
RotationSuggestionService	Ordena la salida FIFO de los lotes de la distribuidora según su vencimiento (US13).
WasteValuationService	Calcula el valor económico y la huella de carbono del consumo y la merma (US18, US20).

Domain Events

Evento	Descripción
ConsumptionRegistered	Se registró el consumo de un alimento (ES: *alimento consumido*).
WasteRegistered	Se registró el descarte de un alimento con su causa (ES: *alimento desechado*).
RecipeSuggested	Se priorizaron fichas técnicas a partir de los insumos en riesgo (ES: *receta sugerida*).
SupplierIncidentRecorded	Se registró una incidencia de calidad contra un proveedor (US40).

Policies

- **Priorizar insumos en riesgo**: cuando existen insumos en estado AT_RISK, el restaurante ve primero las fichas técnicas que los usan y la distribuidora ve esos lotes al inicio de la salida FIFO (US13).
- **Registrar valor al consumir**: cuando ocurre `FoodItemConsumed`, se crea un `ConsumptionRecord` con el valor económico estimado.
- **Restringir costeo avanzado**: cuando un usuario sin plan PREMIUM consulta el costeo o la merma por corte, el acceso se bloquea (US19).

4.2.6.2. Interface Layer

La Interface Layer del Consumption Management expone los puntos de entrada del contexto mediante controladores REST, recursos y *assemblers* que traducen entre el modelo HTTP y el modelo de dominio.

Controllers

1. **RecipesController** — Expone las fichas técnicas del restaurante y su priorización.

Método	Endpoint	Descripción
GET	/api/v1/recipes	Lista las fichas técnicas del restaurante (US13).
GET	/api/v1/recipes/suggested	Devuelve las fichas priorizadas según los insumos en riesgo (US13).
GET	/api/v1/recipes/{id}	Obtiene el detalle, los insumos y el costo por porción de una ficha (US19).
POST	/api/v1/recipes	Crea una ficha técnica con sus insumos (US13).

- *Dependencias:* RecipeQueryService, RecipeCommandService

2. **ConsumptionController** — Expone el historial de consumo y descarte.

Método	Endpoint	Descripción
GET	/api/v1/consumption	Lista los registros de consumo y merma por periodo.
GET	/api/v1/consumption/export	Descarga el historial en CSV o Excel (US29).
GET	/api/v1/consumption/rotation	Devuelve el orden FIFO de salida de los lotes (US13).
POST	/api/v1/consumption/supplier-incidents	Registra una incidencia de calidad de un proveedor (US40).

- *Dependencias:* ConsumptionQueryService

Resources

Resource	Campos
RecipeResource	id, title, portions, prepMinutes, instructions, ingredients[], costPerPortion
RecipeIngredientResource	name, qty, unit, optional
SupplierIncidentResource	supplier, inventoryItemId, description
ConsumptionRecordResource	id, inventoryItemId, action, reason, estimatedValue, recordedAt

Transformers / Assemblers

Assembler	Transformación
RecipeResourceFromEntityAssembler	Convierte `Recipe` y sus ingredientes en `RecipeResource`.
ConsumptionRecordResourceFromEntityAssembler	Convierte `ConsumptionRecord` en su recurso.
SupplierIncidentCommandFromResourceAssembler	Convierte `SupplierIncidentResource` en `RecordSupplierIncidentCommand`.

4.2.6.3 Application Layer

La Application Layer del Consumption Management coordina las operaciones entre la Interface Layer y la Domain Layer, orquestando comandos, consultas y la reacción a eventos de otros contextos.

Command Services

Servicio	Comandos que maneja	Descripción
RecipeCommandServiceImpl	CreateRecipeCommand, UpdateRecipeCommand	Crea y mantiene las fichas técnicas del restaurante (US13, US19).
ConsumptionCommandServiceImpl	RegisterConsumptionCommand, RegisterWasteCommand, RecordSupplierIncidentCommand	Crea los registros de consumo y merma con su valoración económica y ambiental, y las incidencias de proveedores (US40).

Query Services / Facades

Servicio	Consultas	Descripción
RecipeQueryServiceImpl	GetRecipesQuery, GetSuggestedRecipesQuery, GetRecipeByIdQuery	Recupera las fichas técnicas y su priorización por riesgo (US13).
ConsumptionQueryServiceImpl	GetConsumptionByPeriodQuery, ExportConsumptionQuery, GetRotationQuery	Recupera el historial, genera la exportación en CSV/Excel (US29) y el orden FIFO de salida (US13).
ConsumptionContextFacade	getConsumptionSummary(userId, period)	Open Host Service consultado por Report Management para construir los reportes.

Event Handlers

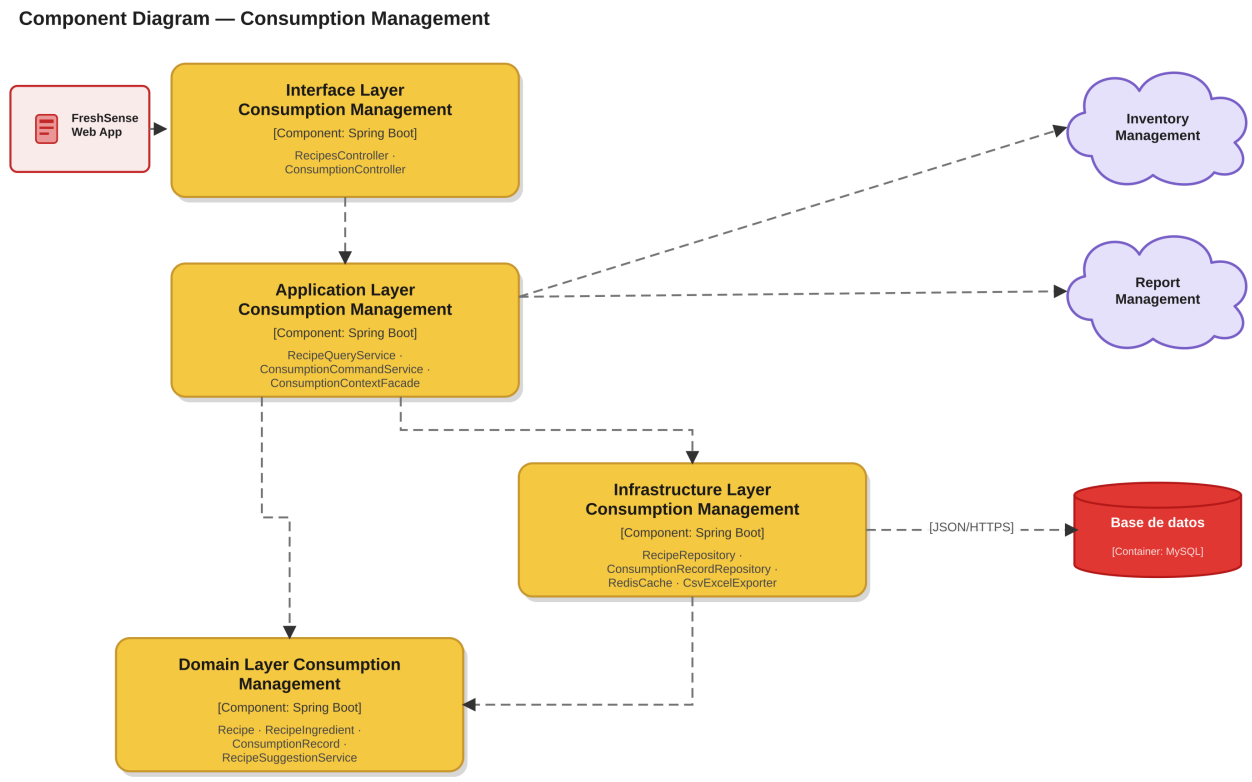
Handler	Evento que escucha	Reacción
FoodItemConsumedEventHandler	FoodItemConsumed (Inventory Management)	Crea un `ConsumptionRecord` de tipo CONSUMED y calcula el valor económico rescatado.
FoodItemDiscardedEventHandler	FoodItemDiscarded (Inventory Management)	Crea un `ConsumptionRecord` de tipo DISCARDED con su causa y estima la huella de carbono.
FoodConditionUpdatedEventHandler	FoodConditionUpdated (IoT Monitoring)	Recalcula la priorización de fichas y la rotación FIFO cuando un insumo entra en estado AT_RISK (US13).

4.2.6.4. Infrastructure Layer

La Infrastructure Layer del Consumption Management implementa la persistencia y las integraciones técnicas (ACL) hacia otros contextos y sistemas externos.

Componente	Tipo	Responsabilidad
RecipeRepository	JPA Repository	Persistencia de fichas técnicas; `findByBusinessId`, `findByIngredientName`.
RecipeIngredientRepository	JPA Repository	Persistencia de los insumos de cada ficha técnica.
ConsumptionRecordRepository	JPA Repository	Persistencia del historial de consumo y merma.
RedisCacheAdapter	Caché	Sirve las fichas técnicas y el inventario más consultados en menos de 50 ms (TS45).
CsvExcelExporter	Servicio técnico	Genera el archivo CSV/Excel del historial con Apache POI (US29).
ExternalInventoryService	ACL entrante	Consulta el `InventoryContextFacade` para obtener los alimentos en riesgo.

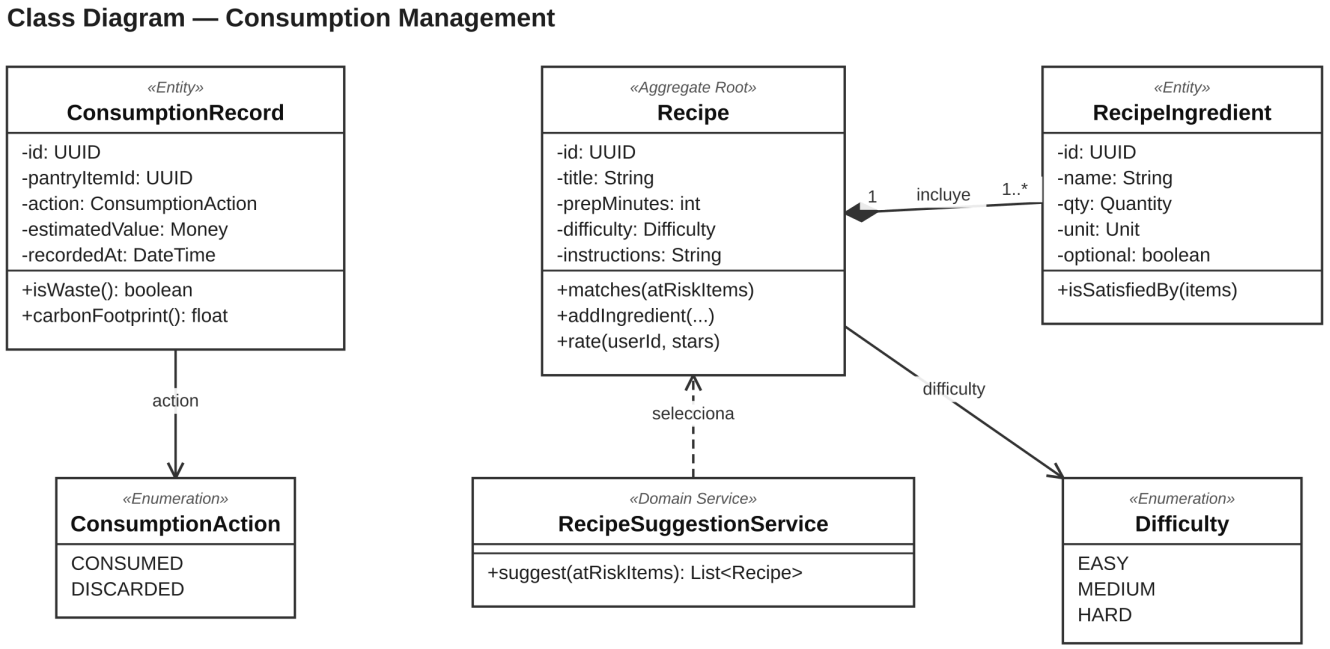
4.2.6.5. Bounded Context Software Architecture Component Level Diagrams



Component Level Diagram — Consumption Management.

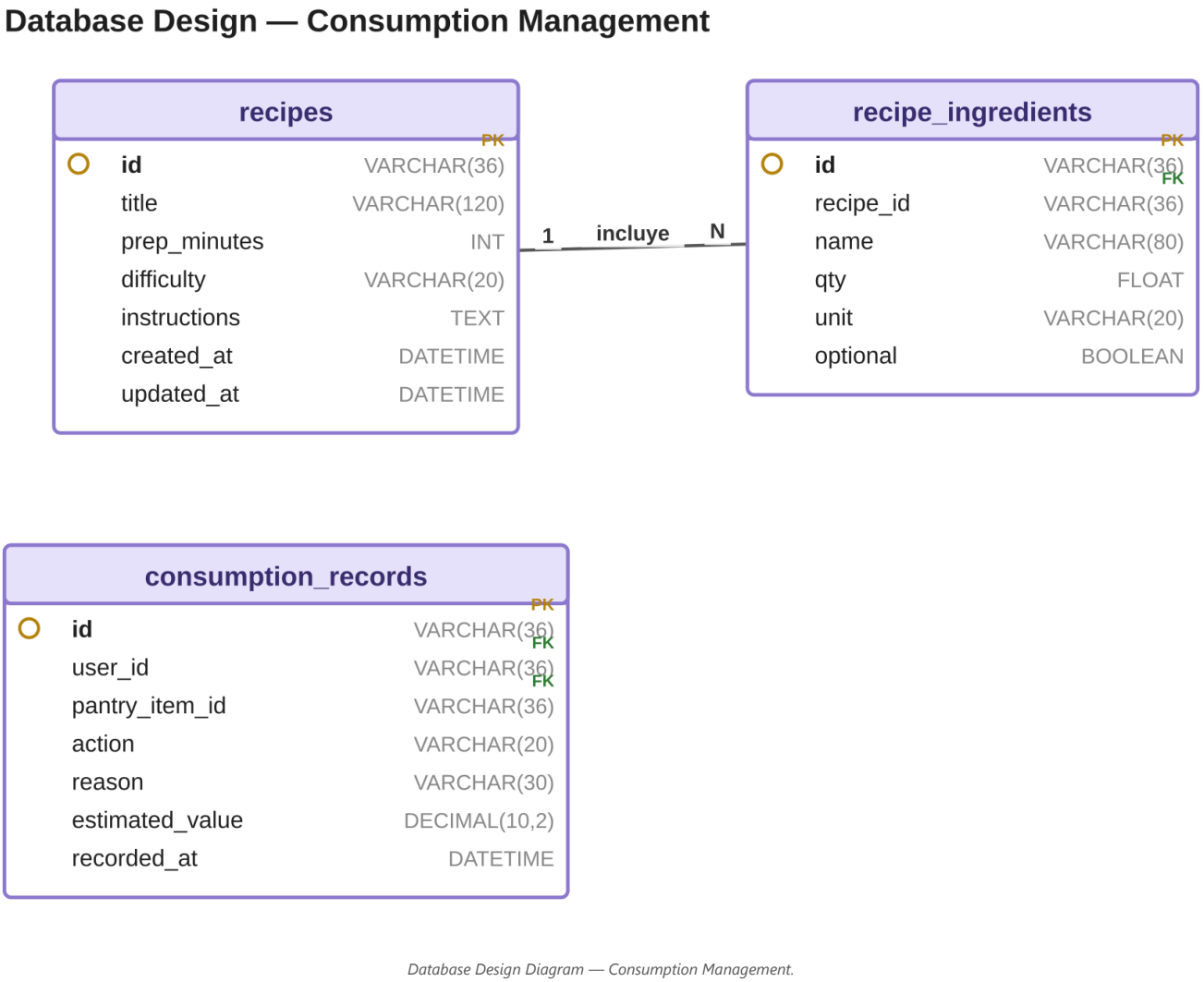
4.2.6.6. Bounded Context Software Architecture Code Level Diagrams

4.2.6.6.1. Bounded Context Domain Layer Class Diagrams



Class Diagram (Domain Layer) — Consumption Management.

4.2.6.6.2. Bounded Context Database Design Diagram



Database Design Diagram — Consumption Management.

4.2.7 Bounded Context: IoT Monitoring

El **IoT Monitoring Bounded Context** es el núcleo de la integración IoT del producto. Recibe la telemetría del dispositivo FreshSense mediante el endpoint REST seguro definido en la **TS41**, evalúa las lecturas contra los umbrales de cada alimento a través del `FreshnessService` del modelo de dominio, y emite las alertas preventivas de la **US08**, respetando el **Modo Mantenimiento** de la **US32**. Este contexto traduce la señal física (temperatura, humedad, etileno) en conocimiento de negocio: el estado de frescura del alimento.

4.2.7.1. Domain Layer

La Domain Layer del IoT Monitoring Bounded Context encapsula la lógica de negocio del contexto. En esta capa se definen los agregados, entidades, objetos de valor, servicios de dominio y políticas que representan los conceptos clave del sistema.

Aggregates

1. **SensorReading**
    - *Propósito:* Agregado raíz que representa una lectura de telemetría enviada por el dispositivo FreshSense.
    - *Atributos:*

Atributo	Tipo	Descripción
id	UUID	Identificador de la lectura.
deviceId	UUID	Dispositivo que la originó.
timestamp	DateTime	Instante de la medición.
temperatureC	float	Temperatura en grados Celsius.
humidityPct	float	Humedad relativa en porcentaje.
ethylenePpm	float	Concentración de gas etileno en ppm.
meta	String	Metadatos del envío (versión de firmware, calidad de señal).

- *Métodos:*

Método	Descripción
isValid()	Valida que los rangos de las tres magnitudes sean físicamente posibles.
exceeds(ShelfLifeRules)	Indica si la lectura supera los umbrales de la categoría del alimento.
toQualityRecord()	Proyecta la lectura al histórico de calidad consultado en los reportes.

Entities

1. **ExpirationAlert** — Alerta generada cuando un alimento entra en riesgo de deterioro (US08).

Atributo	Tipo	Descripción
id	UUID	Identificador de la alerta.
inventoryItemId	UUID	Alimento afectado.
raisedAt	DateTime	Momento en que se emitió.
type	AlertType (VO)	NEARING_EXPIRY, HIGH_ETHYLENE o TEMP_RISK.
status	AlertStatus (VO)	OPEN, SENT, SNOOZED o RESOLVED.
meta	String	Valores que motivaron la alerta.

Método	Descripción
resolve()	Marca la alerta como resuelta cuando el alimento se consume o descarta.
snooze(until)	Pospone la alerta según las preferencias del usuario (US09).
markSent()	Registra que la notificación fue despachada.

2. **ThresholdProfile** — Perfil de umbrales aplicado a una zona, incluido el Modo Mantenimiento (US32).

Atributo	Tipo	Descripción
id	UUID	Identificador del perfil.
zoneId	UUID	Zona de almacenamiento a la que aplica.
maxTemperatureC	float	Temperatura máxima tolerada.
maxEthylenePpm	float	Concentración máxima de etileno tolerada.
maintenanceMode	boolean	Indica si el modo de mantenimiento está activo (US32).

Método	Descripción
applyMaintenanceMode()	Pausa las alarmas de la zona mientras la cámara se limpia o se repara (US32).
evaluate(SensorReading)	Determina si la lectura constituye una desviación.

Value Objects

Value Object	Descripción
AlertType	Enumeración: `NEARING_EXPIRY`, `HIGH_ETHYLENE`, `TEMP_RISK`.
AlertStatus	Enumeración: `OPEN`, `SENT`, `SNOOZED`, `RESOLVED`.
FreshnessStatus	Enumeración compartida con Inventory: `FRESH`, `AT_RISK`, `SPOILED` (Shared Kernel).
Severity	Nivel de la alerta empleado para decidir el canal de entrega (US24).

Domain Services

Servicio	Descripción
FreshnessService	Servicio de dominio central del modelo: `evaluate(InventoryItem, SensorReading*)` determina el `FreshnessStatus` y `estimateBestBefore(InventoryItem, FoodModel, SensorReading*)` ajusta la fecha de consumo preferente con la telemetría real.
ThresholdEvaluationService	Compara cada lectura con el `ThresholdProfile` vigente y decide si corresponde emitir una alerta.

Domain Events

Evento	Descripción
SensorReadingRegistered	Se persistió una lectura del dispositivo (ES: *registrar humedad / temperatura / etileno*).
FoodConditionUpdated	El `FreshnessService` recalculó el estado del alimento (ES: *estado del alimento actualizado*).
EthyleneAlertIssued	La concentración de etileno superó el umbral y se emitió la alerta (ES: *alerta emitida*).
TemperatureAlertIssued	La temperatura salió del rango tolerado de la zona.
AlertResolved	La alerta se cerró al consumirse o descartarse el alimento.
MaintenanceModeActivated	Se pausaron las alarmas de la cámara por limpieza o reparación (US32).

Policies

- **Emitir alerta al superar umbrales**: cuando los valores del sensor superan los umbrales configurados, se emite una alerta de expiración (US08).
- **Evitar falsas alarmas**: cuando no existen desviaciones en los parámetros, el chequeo periódico no emite ninguna notificación (US08).
- **Pausar alarmas en Modo Mantenimiento**: mientras una cámara está en mantenimiento, sus lecturas se registran pero no generan alertas hasta que se reactive (US32).
- **Respetar el turno operativo**: cuando una alerta de baja severidad cae fuera del turno, se retiene hasta el siguiente; las críticas se envían siempre al responsable de turno (US09, US24).

4.2.7.2. Interface Layer

La Interface Layer del IoT Monitoring expone los puntos de entrada del contexto mediante controladores REST, recursos y *assemblers* que traducen entre el modelo HTTP y el modelo de dominio.

Controllers

1. **SensorReadingsController** — Endpoint de ingesta de telemetría del dispositivo IoT (TS41).

Método	Endpoint	Descripción
POST	/api/v1/sensor-readings	Recibe un JSON con temperatura, humedad y etileno; valida el token del dispositivo y responde 201.
GET	/api/v1/sensor-readings	Consulta el histórico de lecturas por dispositivo y periodo.

- *Dependencias:* MonitoringCommandService, MonitoringQueryService

2. **AlertsController** — Expone y gestiona las alertas de expiración (US08).

Método	Endpoint	Descripción
GET	/api/v1/alerts	Lista las alertas abiertas del usuario.
POST	/api/v1/alerts/{id}/snooze	Pospone la alerta según las preferencias (US09).
POST	/api/v1/alerts/{id}/resolve	Marca la alerta como resuelta.

- *Dependencias:* AlertCommandService, AlertQueryService

3. **ThresholdsController** — Configura los umbrales y el Modo Mantenimiento (US32).

Método	Endpoint	Descripción
GET	/api/v1/thresholds/{zoneId}	Obtiene el perfil de umbrales de la zona.
PUT	/api/v1/thresholds/{zoneId}	Actualiza los umbrales o activa el Modo Mantenimiento.

- *Dependencias:* ThresholdCommandService

Resources

Resource	Campos
SensorReadingResource	deviceId, timestamp, temperatureC, humidityPct, ethylenePpm, meta
AlertResource	id, inventoryItemId, type, status, raisedAt, meta
ThresholdProfileResource	zoneId, maxTemperatureC, maxEthylenePpm, maintenanceMode
QualityHistoryResource	deviceId, period, avgTemperature, avgHumidity, maxEthylene

Transformers / Assemblers

Assembler	Transformación
SensorReadingCommandFromResourceAssembler	Convierte el JSON entrante del dispositivo en `RegisterSensorReadingCommand` (TS41).
AlertResourceFromEntityAssembler	Convierte `ExpirationAlert` en `AlertResource`.
ThresholdProfileResourceFromEntityAssembler	Convierte `ThresholdProfile` en su recurso.

4.2.7.3 Application Layer

La Application Layer del IoT Monitoring coordina las operaciones entre la Interface Layer y la Domain Layer, orquestando comandos, consultas y la reacción a eventos de otros contextos.

Command Services

Servicio	Comandos que maneja	Descripción
MonitoringCommandServiceImpl	RegisterSensorReadingCommand	Valida el token del dispositivo, persiste la lectura y dispara la evaluación de frescura (TS41).
AlertCommandServiceImpl	IssueAlertCommand, SnoozeAlertCommand, ResolveAlertCommand	Emite, pospone y cierra las alertas de expiración (US08, US09).
ThresholdCommandServiceImpl	UpdateThresholdsCommand, ActivateMaintenanceModeCommand	Ajusta los umbrales por zona y activa el Modo Mantenimiento (US32).

Query Services / Facades

Servicio	Consultas	Descripción
MonitoringQueryServiceImpl	GetReadingsByDeviceQuery, GetQualityHistoryQuery	Recupera el histórico de telemetría y las métricas agregadas de calidad.
AlertQueryServiceImpl	GetOpenAlertsQuery, GetAlertsByItemQuery	Recupera las alertas vigentes del usuario.
MonitoringContextFacade	getQualityHistory(businessId, period), getOpenAlerts(userId)	Open Host Service consultado por Report Management e Inventory Management.

Event Handlers

Handler	Evento que escucha	Reacción
SensorConfiguredEventHandler	SensorConfigured (Sensor Management)	Crea el `ThresholdProfile` inicial de la zona con los umbrales por defecto de sus alimentos.
FoodItemRegisteredEventHandler	FoodItemRegistered (Inventory Management)	Asocia el nuevo alimento a la zona monitoreada y comienza a evaluarlo.
InventoryItemResolvedEventHandler	FoodItemConsumed / FoodItemDiscarded (Inventory Management)	Cierra automáticamente las alertas abiertas del alimento (`AlertResolved`).

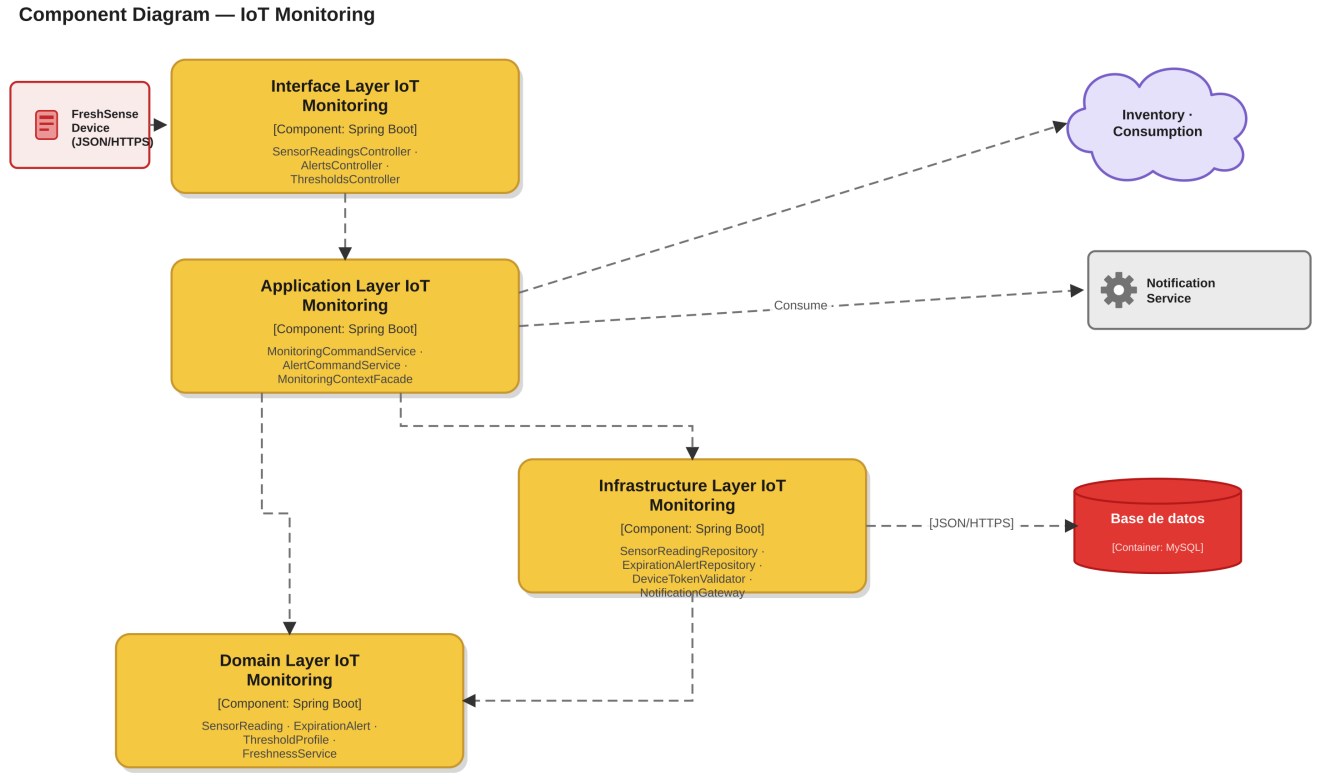
4.2.7.4. Infrastructure Layer

La Infrastructure Layer del IoT Monitoring implementa la persistencia y las integraciones técnicas (ACL) hacia otros contextos y sistemas externos.

Componente	Tipo	Responsabilidad
SensorReadingRepository	JPA Repository	Persistencia de la telemetría; `findByDeviceIdAndTsBetween`.
ExpirationAlertRepository	JPA Repository	Persistencia de alertas; `findByStatus`, `findByInventoryItemId`.

Componente	Tipo	Responsabilidad
ThresholdProfileRepository	JPA Repository	Persistencia de los perfiles de umbrales por zona.
DeviceTokenValidator	ACL entrante	Valida el token del dispositivo antes de aceptar la lectura (TS41).
NotificationGatewayAdapter	ACL saliente	Despacha la alerta a la cola de eventos del Notification Service (TS42).
FreshnessEvaluationScheduler	Spring Scheduler	Ejecuta el chequeo periódico de frescura sobre los alimentos activos.

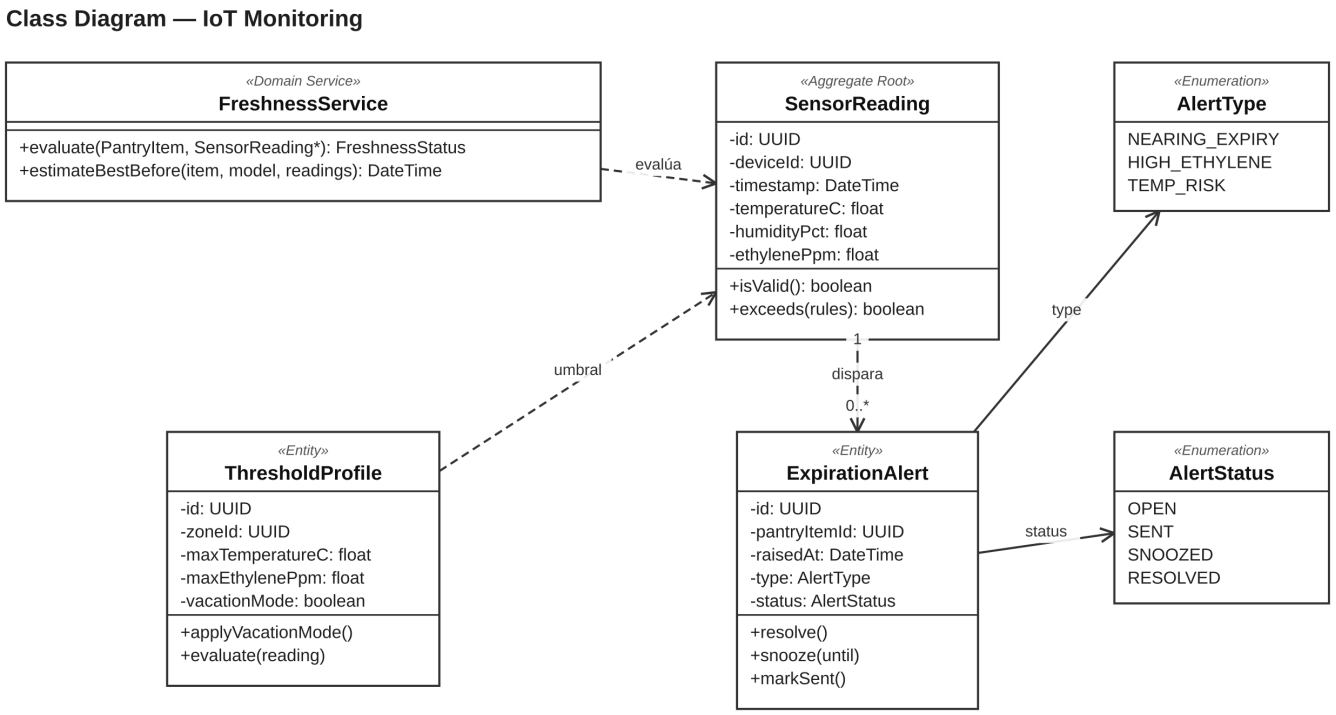
4.2.7.5. Bounded Context Software Architecture Component Level Diagrams



Component Level Diagram — IoT Monitoring.

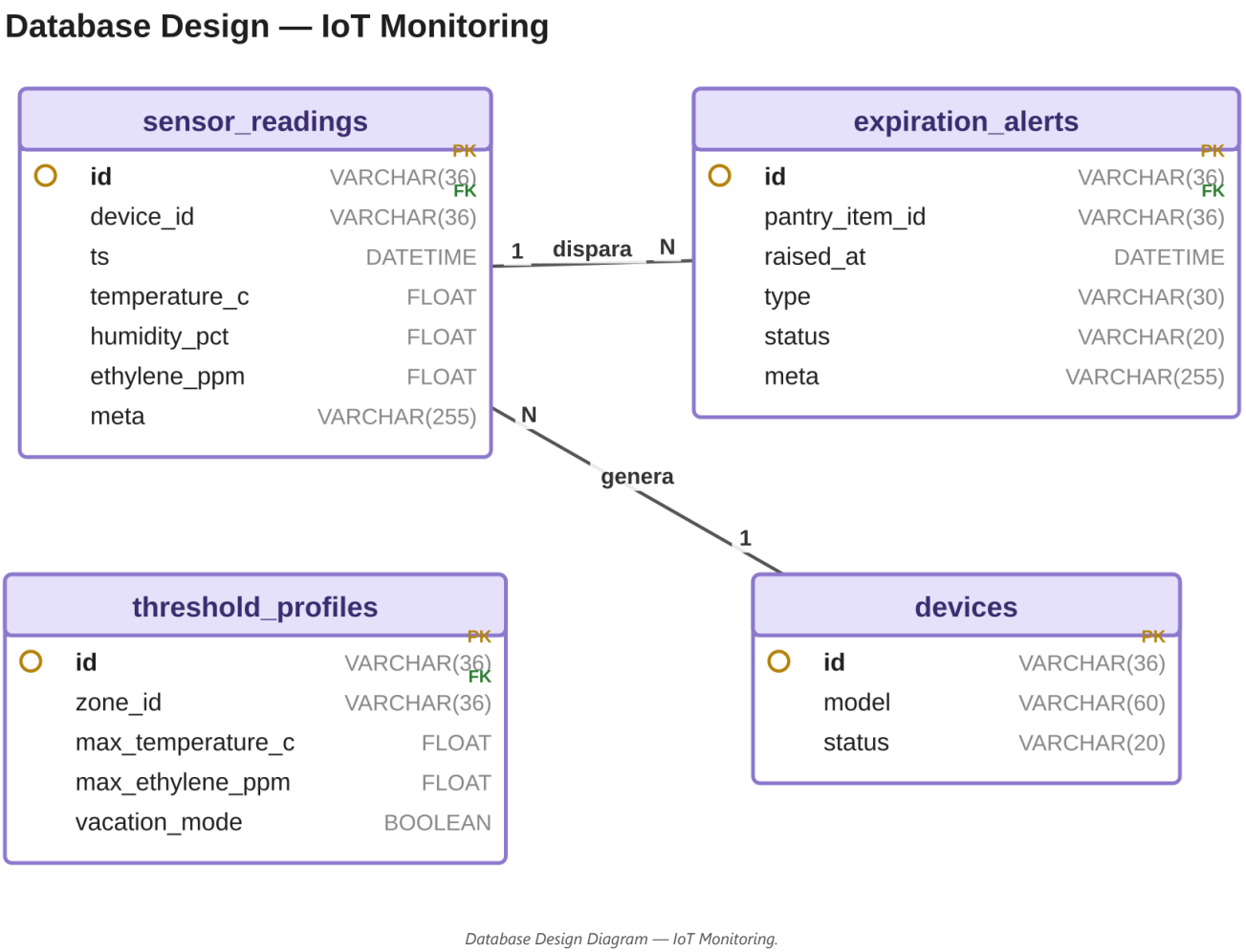
4.2.7.6. Bounded Context Software Architecture Code Level Diagrams

4.2.7.6.1. Bounded Context Domain Layer Class Diagrams



Class Diagram (Domain Layer) — IoT Monitoring.

4.2.7.6.2. Bounded Context Database Design Diagram



Database Design Diagram — IoT Monitoring.

4.2.8 Bounded Context: Report Management

El **Report Management Bounded Context** consolida la información de los demás contextos para producir los **reportes de consumo y sostenibilidad**: el reporte semanal de mermas (**US12**), la analítica avanzada de rotación (**US17**), el panel de ahorro económico (**US18**) y las métricas de impacto ecológico en kg de CO_2 evitado

(**US20**). Genera los archivos descargables en PDF y Excel con JasperReports y Apache POI, y expone la información al Analytics Service externo.

4.2.8.1. Domain Layer

La Domain Layer del Report Management Bounded Context encapsula la lógica de negocio del contexto. En esta capa se definen los agregados, entidades, objetos de valor, servicios de dominio y políticas que representan los conceptos clave del sistema.

Aggregates

1. **ConsumptionReport**
    - *Propósito:* Agregado raíz que representa un reporte consolidado de consumo, merma y ahorro para un periodo determinado.
    - *Atributos:*

Atributo	Tipo	Descripción
id	UUID	Identificador del reporte.
userId	UUID	Usuario o negocio destinatario.
period	Period (VO)	Rango de fechas cubierto (periodStart–periodEnd).
savedAmount	Money (VO)	Valor económico rescatado al consumir a tiempo (US18).
wasteReducedKgCO2	float	Kilogramos de CO_2 evitados (US20).
sections	List<ReportSection>	Bloques del reporte: inventario, consumo, calidad.
createdAt	DateTime	Fecha de generación.

- *Métodos:*

Método	Descripción
generate(inventory, consumption, quality)	Construye el reporte a partir de los tres contextos fuente.
exportTo(format)	Exporta el reporte a PDF o Excel (US29).
isPremiumOnly()	Indica si el nivel de detalle requiere plan PREMIUM (US17).

Entities

1. **ReportSection** — Bloque temático del reporte con sus métricas calculadas.

Atributo	Tipo	Descripción
id	UUID	Identificador de la sección.
reportId	UUID	Reporte al que pertenece.
type	SectionType (VO)	INVENTORY, CONSUMPTION, QUALITY o SUSTAINABILITY.
metricsJson	String	Métricas calculadas serializadas.
order	int	Posición dentro del reporte.

Método	Descripción
render(format)	Produce la representación de la sección en el formato solicitado.

Value Objects

Value Object	Descripción
Period	Rango de fechas con inicio y fin; valida que el fin sea posterior al inicio.
Money	Importe con moneda, usado para el ahorro estimado (US18).
SectionType	Enumeración: `INVENTORY`, `CONSUMPTION`, `QUALITY`, `SUSTAINABILITY`.
ExportFormat	Enumeración: `PDF`, `XLSX`, `CSV` (US29).

Domain Services

Servicio	Descripción
ReportAssemblyService	Orquesta la consulta a Inventory, Consumption e IoT Monitoring y arma las secciones del reporte.
SustainabilityMetricsService	Convierte los kilogramos de alimento salvado en CO_2 evitado y litros de agua preservados (US20).

Domain Events

Evento	Descripción
ReportRequested	El usuario solicitó un reporte (ES: *solicitar reporte*).

Evento	Descripción
ReportGenerated	El reporte se construyó y quedó disponible (ES: *reporte generado*).
WeeklyReportSent	Se despachó el resumen semanal al correo del usuario (US12).
ReportExported	El usuario descargó el reporte en PDF, Excel o CSV (US29).

Policies

- **Emitir reporte semanal**: cuando se cumple el ciclo de 7 días, el servidor consolida los datos y envía un resumen gráfico al correo y al panel (US12).
- **Verificar plan antes de la analítica avanzada**: cuando un usuario sin PREMIUM solicita la rotación detallada, el acceso se bloquea (US17).
- **Calcular impacto ambiental**: cuando se genera el reporte, se estima el CO_2 evitado a partir de los alimentos consumidos a tiempo (US20).

4.2.8.2. Interface Layer

La Interface Layer del Report Management expone los puntos de entrada del contexto mediante controladores REST, recursos y *assemblers* que traducen entre el modelo HTTP y el modelo de dominio.

Controllers

1. **ReportsController** — Genera y expone los reportes de consumo y sostenibilidad.

Método	Endpoint	Descripción
GET	/api/v1/reports	Lista los reportes generados del usuario.
POST	/api/v1/reports	Genera un reporte para el periodo indicado (US12).
GET	/api/v1/reports/{id}	Obtiene el detalle del reporte y sus secciones.
GET	/api/v1/reports/{id}/export	Descarga el reporte en PDF, Excel o CSV (US29).

- *Dependencias:* ReportCommandService, ReportQueryService

2. **AnalyticsController** — Expone la analítica avanzada y el panel de ahorro (US17, US18).

Método	Endpoint	Descripción
GET	/api/v1/analytics/rotation	Gráficos de rotación de inventario (solo PREMIUM).
GET	/api/v1/analytics/savings	Ahorro acumulado estimado en moneda local.
GET	/api/v1/analytics/sustainability	Métricas de CO_2 evitado y agua preservada (US20).

- *Dependencias:* ReportQueryService

Resources

Resource	Campos
ReportResource	id, period, savedAmount, wasteReducedKgCO2, sections[], createdAt
ReportSectionResource	type, metrics, order
CreateReportResource	periodStart, periodEnd, scope
SustainabilityResource	kgCO2Avoided, litersWaterSaved, itemsSaved

Transformers / Assemblers

Assembler	Transformación
ReportResourceFromEntityAssembler	Convierte `ConsumptionReport` y sus secciones en `ReportResource`.
SustainabilityResourceFromEntityAssembler	Convierte las métricas ambientales en su recurso.
CreateReportCommandFromResourceAssembler	Convierte `CreateReportResource` en `GenerateReportCommand`.

4.2.8.3 Application Layer

La Application Layer del Report Management coordina las operaciones entre la Interface Layer y la Domain Layer, orquestando comandos, consultas y la reacción a eventos de otros contextos.

Command Services

Servicio	Comandos que maneja	Descripción
ReportCommandServiceImpl	GenerateReportCommand, ExportReportCommand	Construye el reporte consultando los contextos fuente y genera el archivo descargable.
WeeklyReportSchedulerImpl	SendWeeklyReportCommand	Consolida y despacha el reporte semanal de mermas al correo del negocio (US12).

Query Services / Facades

Servicio	Consultas	Descripción
ReportQueryServiceImpl	GetReportsByUserQuery, GetReportByIdQuery, GetSustainabilityMetricsQuery	Recupera los reportes generados y las métricas de impacto ambiental.
AnalyticsQueryServiceImpl	GetRotationAnalyticsQuery, GetSavingsQuery	Recupera la analítica avanzada y el ahorro acumulado, previa verificación del plan (US17, US18).
ReportContextFacade	getLatestReport(userId)	Open Host Service consultado por el Analytics Service externo.

Event Handlers

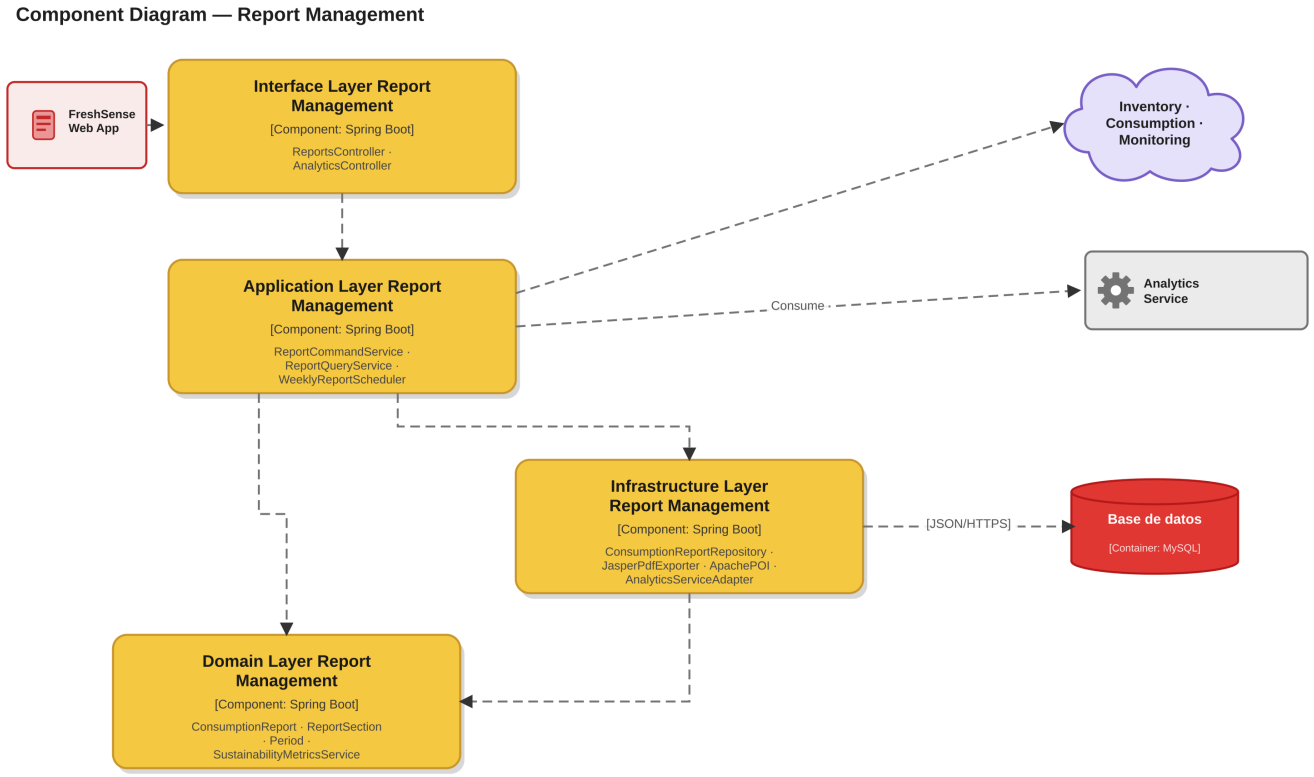
Handler	Evento que escucha	Reacción
ConsumptionRegisteredEventHandler	ConsumptionRegistered (Consumption Management)	Acumula el valor rescatado y actualiza las métricas del periodo en curso.
WasteRegisteredEventHandler	WasteRegistered (Consumption Management)	Acumula la merma y recalcula la huella de carbono del periodo (US20).
WeeklyCycleEventHandler	WeeklyCycleCompleted (Spring Scheduler)	Dispara `SendWeeklyReportCommand` para generar y enviar el resumen (US12).

4.2.8.4. Infrastructure Layer

La Infrastructure Layer del Report Management implementa la persistencia y las integraciones técnicas (ACL) hacia otros contextos y sistemas externos.

Componente	Tipo	Responsabilidad
ConsumptionReportRepository	JPA Repository	Persistencia de reportes; `findByUserIdAndPeriod`.
ReportSectionRepository	JPA Repository	Persistencia de las secciones de cada reporte.
JasperPdfExporter	Servicio técnico	Genera el reporte en PDF con JasperReports.
ApachePoiExcelExporter	Servicio técnico	Genera el reporte en Excel con Apache POI (US29).
ExternalInventoryService	ACL entrante	Consulta el `InventoryContextFacade` para el snapshot de inventario.
ExternalConsumptionService	ACL entrante	Consulta el `ConsumptionContextFacade` para el resumen de consumo.
ExternalMonitoringService	ACL entrante	Consulta el `MonitoringContextFacade` para el histórico de calidad.
AnalyticsServiceAdapter	ACL saliente	Publica las métricas consolidadas al Analytics Service externo.

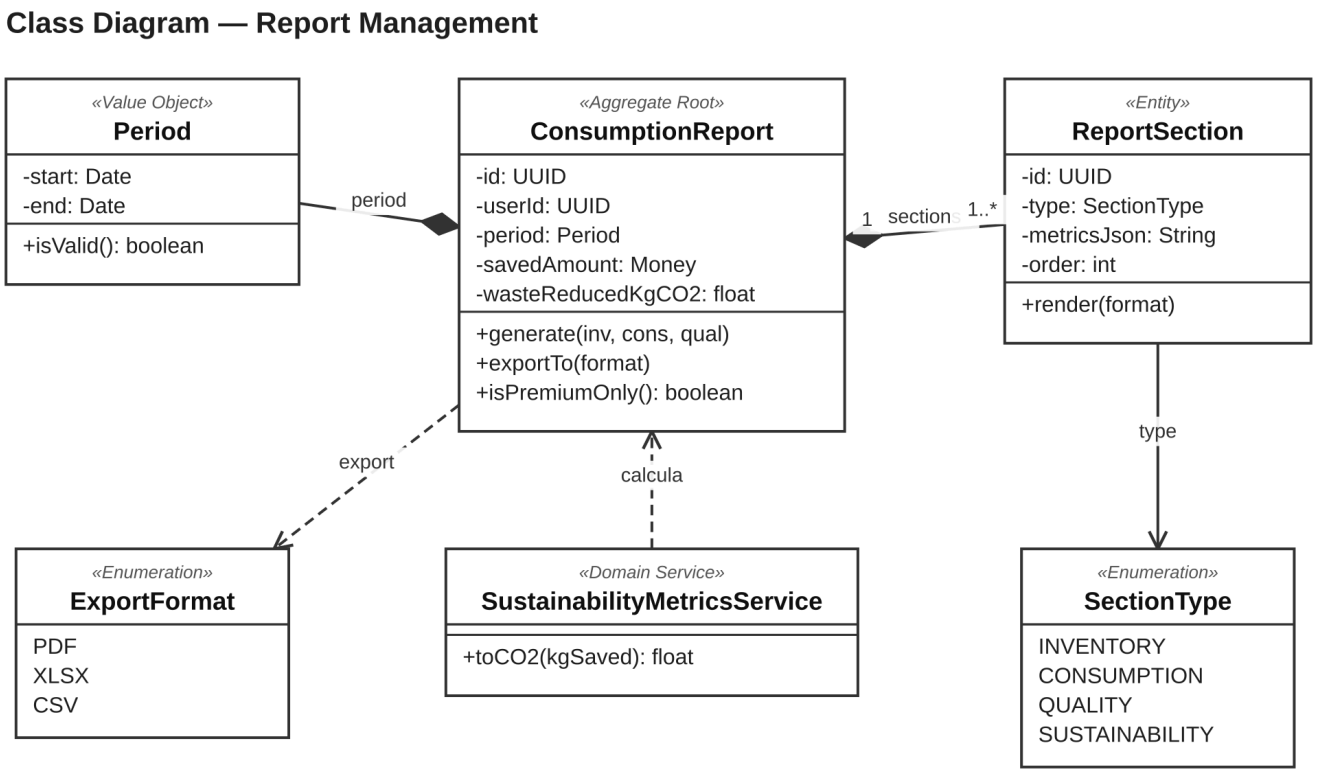
4.2.8.5. Bounded Context Software Architecture Component Level Diagrams



Component Level Diagram — Report Management.

4.2.8.6. Bounded Context Software Architecture Code Level Diagrams

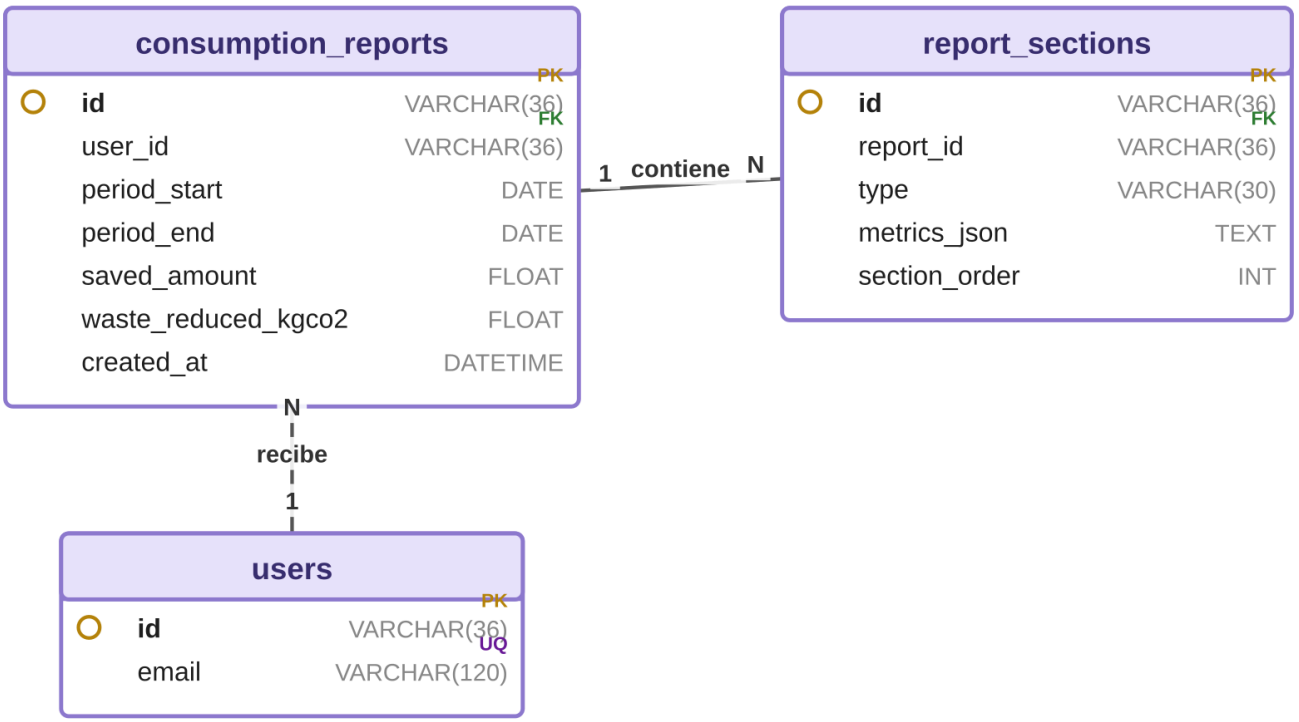
4.2.8.6.1. Bounded Context Domain Layer Class Diagrams



Class Diagram (Domain Layer) — Report Management.

4.2.8.6.2. Bounded Context Database Design Diagram

Database Design — Report Management



Database Design Diagram — Report Management.

Capítulo V: Solution UI/UX Design

5.1. Style Guidelines

FreshSense establece un conjunto de lineamientos visuales y de interacción orientados a mantener una experiencia consistente entre el Landing Page, la aplicación web, la aplicación móvil y la experiencia asociada al dispositivo IoT.

La solución está dirigida principalmente a **empresas de distribución y cadena de frío** y a **empresas productoras y comercializadoras de alimentos perecibles**. Por ello, la experiencia visual prioriza claridad, confiabilidad y rápida interpretación de información relacionada con monitoreo ambiental, lotes, dispositivos, alertas y trazabilidad.

Los recursos visuales del producto, como logotipos, imágenes, tipografías y demás elementos gráficos, se centralizarán en la carpeta `Assets/` del repositorio para mantener una referencia común entre todos los integrantes del equipo.

5.1.1. General Style Guidelines

Los lineamientos generales de FreshSense definen la identidad visual y comunicacional utilizada en los diferentes productos digitales que forman parte de la solución.

Branding

La identidad de FreshSense está orientada a transmitir **frescura, control, trazabilidad, tecnología y confiabilidad**.

La marca busca representar una solución tecnológica que permita a las empresas monitorear las condiciones de conservación de productos perecibles durante su almacenamiento y distribución, facilitando la detección de desviaciones y reduciendo las pérdidas asociadas al deterioro de productos.

El diseño visual mantiene una apariencia limpia y profesional, evitando interfaces excesivamente cargadas. Los elementos relacionados con monitoreo, conservación, alertas, dispositivos y trazabilidad deben utilizarse de manera consistente en todos los productos digitales.

Typography

FreshSense utiliza la familia tipográfica **Poppins** debido a su apariencia moderna, limpia y legible en interfaces digitales.

Se establece la siguiente jerarquía tipográfica:

- **H1:** títulos principales de páginas y mensajes de mayor importancia.
- **H2:** títulos de secciones.
- **H3:** subtítulos y encabezados de componentes.
- **Body:** contenido descriptivo, datos y textos de apoyo.
- **Labels:** nombres de campos, indicadores, filtros y estados.

La jerarquía debe mantenerse de manera consistente en las experiencias Web y Mobile para facilitar la lectura y comprensión de la información.

Colors

La paleta cromática de FreshSense está compuesta principalmente por verde, azul, tonos neutros y blanco.

Color	Uso principal
Green - Primary	Acciones principales, indicadores de condiciones adecuadas y elementos asociados a conservación.
Blue - Secondary	Monitoreo, información tecnológica, gráficos y componentes secundarios.
Gray - Neutral	Textos secundarios, etiquetas, íconos y divisores.
White - Background	Fondos principales, tarjetas y espacios de contenido.
Text - Base	Información principal y contenido de alta prioridad.

Los estados que requieran atención podrán utilizar indicadores visuales diferenciados según su nivel de severidad. Sin embargo, el color no será el único mecanismo de comunicación; cada estado deberá estar acompañado por texto o iconografía que permita comprender claramente la situación.

Spacing & Layout

FreshSense utiliza una estructura modular para mantener consistencia entre páginas y componentes.

Base Unit

- Size: `8 px`
- Uso: unidad base para márgenes, paddings y separación entre elementos.

Grid System

- Grid: `12 columnas`
- Gutter: `22 px`
- Margins: proporcionales a la unidad base.

Section Spacing

- Standard section: `56 px`
- Hero section: `72 px`
- Footer: `36–56 px`

Cards & Components

- Internal padding: `18–22 px`
- Border radius: `16 px`
- Elevation: `0 10px 25px rgba(0,0,0,.08)`

Alignment

- Contenido principal dentro de un contenedor máximo de `1120 px` o `92%` del ancho disponible.
- El contenido textual se alinea principalmente a la izquierda para facilitar su lectura.
- Los indicadores críticos y métricas principales deberán tener mayor jerarquía visual.
- Se utilizarán espacios amplios entre grupos de información para diferenciar claramente cada sección.

Tone of Voice

El tono de comunicación de FreshSense busca transmitir profesionalismo, confianza y claridad, debido a que la solución presenta información utilizada para supervisar las condiciones de conservación de productos perecibles.

Dimensión	Orientación de FreshSense	Justificación
Divertido / Serio	**Serio**	Los datos de monitoreo y las alertas requieren una comunicación clara y confiable.
Formal / Casual	**Formal**	La solución está orientada a organizaciones y procesos empresariales.
Respetuoso / Irreverente	**Respetuoso**	Los mensajes deben orientar al usuario sin generar confusión.
Entusiasta / Sereno	**Sereno**	Las incidencias deben comunicarse con claridad sin utilizar mensajes alarmistas.

Los mensajes del sistema deben ser breves, precisos y orientados a una acción concreta.

Ejemplos:

- `Temperature above allowed range`
- `Device disconnected`
- `Cold chain deviation detected`
- `Reading updated successfully`
- `Lot requires attention`

5.1.2. Web, Mobile and IoT Style Guidelines

FreshSense mantiene una identidad visual consistente entre sus diferentes interfaces. Los usuarios deben reconocer los mismos colores, tipografía, iconografía, indicadores y terminología independientemente de si utilizan la aplicación web o móvil.

Web Style Guidelines

El Landing Page y la aplicación web utilizarán principios de **Material Design**, manteniendo consistencia en componentes como botones, formularios, tarjetas, tablas, menús, indicadores y mensajes de estado.

Para la versión Desktop del Landing Page se utilizará principalmente el **patrón de lectura Z**, dirigiendo inicialmente la atención hacia la marca, propuesta de valor y Call-to-Action principal.

Posteriormente, el contenido presentará el funcionamiento de FreshSense, sus principales beneficios y la solución específica para cada segmento empresarial.

En pantallas de menor tamaño, el contenido adoptará una estructura principalmente vertical.

La aplicación web priorizará la visualización de información relacionada con:

- Dashboard.
- Monitoring.
- Inventory.
- Lots.
- Devices.
- Alerts.
- Traceability.
- Reports.

Los datos provenientes del dispositivo IoT, como temperatura, humedad, última lectura y estado de conexión, se presentarán mediante tarjetas, tablas, indicadores y gráficos que permitan identificar rápidamente desviaciones o situaciones que requieran atención.

Mobile Style Guidelines

La aplicación móvil mantendrá los mismos principios visuales definidos para la aplicación web, adaptando la distribución a pantallas de menor tamaño.

Se priorizarán las funcionalidades que requieren consulta rápida:

- Estado general del monitoreo.
- Alertas activas.
- Lecturas recientes.
- Estado de dispositivos.
- Estado de lotes.

La información se organizará principalmente en una sola columna y se priorizarán los eventos o condiciones que requieran atención inmediata.

Los nombres, colores, iconos y estados serán equivalentes a los utilizados en Web para reducir la curva de aprendizaje entre plataformas.

IoT Style Guidelines

El dispositivo IoT actual de FreshSense funciona como un nodo de monitoreo encargado de registrar las condiciones ambientales relacionadas con la conservación de productos perecibles.

El prototipo utiliza un microcontrolador **ESP32** junto con un sensor **DHT22** para obtener periódicamente información de temperatura y humedad.

Las mediciones son transmitidas mediante Wi-Fi hacia el Edge API y posteriormente enviadas al backend de FreshSense para su almacenamiento, procesamiento y visualización.

El prototipo físico actual no incorpora una pantalla ni controles de interacción directa documentados. Por ello, la interacción del usuario con el dispositivo se realiza principalmente mediante las aplicaciones digitales de FreshSense.

Los principales elementos relacionados con el dispositivo utilizarán etiquetas simples y consistentes:

Elemento	Label
Estado del dispositivo	`Connected` / `Disconnected`
Temperatura	`Temperature`
Humedad	`Humidity`
Última medición	`Last Reading`
Estado del monitoreo	`Monitoring Status`
Identificador	`Device ID`

La interfaz debe permitir que el usuario comprenda el estado del dispositivo y de las condiciones monitoreadas sin necesidad de conocer detalles técnicos como el funcionamiento del ESP32, DHT22, JSON o Edge API.

Internationalization & Accessibility

FreshSense considera dos locales principales:

- `en_US` - English.
- `es_419` - Latin American Spanish.

El idioma predeterminado de las interfaces será **English**, manteniendo disponible la estructura necesaria para presentar los mismos contenidos en español latinoamericano.

En las experiencias Web se utilizarán atributos ARIA para facilitar el uso de tecnologías de asistencia.

Asimismo, los estados importantes no serán representados únicamente mediante colores, sino también mediante texto, iconos u otros indicadores reconocibles.

La estructura visual, las etiquetas y los componentes mantendrán consistencia entre ambos idiomas.

5.2. Information Architecture

La arquitectura de información de FreshSense está diseñada para que los visitantes y usuarios puedan comprender rápidamente la propuesta del producto y acceder a información relacionada con monitoreo, dispositivos, productos perecibles, alertas y trazabilidad sin recorrer estructuras complejas.

La organización considera las necesidades de los dos segmentos principales:

- **Empresas de distribución y cadena de frío:** organizaciones encargadas del transporte, distribución o almacenamiento de productos perecibles que requieren supervisar continuamente las condiciones de conservación.
- **Empresas productoras y comercializadoras de alimentos perecibles:** organizaciones que producen, almacenan o comercializan alimentos y necesitan controlar las condiciones de sus productos y reducir pérdidas asociadas al deterioro.

La arquitectura considera el Landing Page, la aplicación web, la aplicación móvil y las funcionalidades asociadas al monitoreo mediante dispositivos IoT.

5.2.1. Organization Systems

FreshSense utiliza distintos sistemas de organización dependiendo del tipo de información y de la tarea que realiza el usuario.

Hierarchical Organization

El Landing Page utiliza una organización jerárquica que presenta primero la información de mayor relevancia:

1. Propuesta de valor.
2. Funcionamiento de FreshSense.
3. Beneficios.
4. Solución para distribución y cadena de frío.
5. Solución para productores y comercializadores.
6. Call-to-Action.
7. Contacto.

En las aplicaciones, la información se organiza priorizando inicialmente indicadores generales, alertas activas y posibles desviaciones antes de mostrar información secundaria.

Sequential Organization

Los procesos que requieren completar varias acciones utilizan una organización secuencial.

Ejemplos:

```
Register → Add Device → Configure Device → Start Monitoring
```

```
Register Lot → Add Product Information → Associate Device → Start Tracking
```

De esta manera, el usuario recibe únicamente la información necesaria para completar cada etapa del proceso.

Matrix Organization

Las vistas de monitoreo utilizan estructuras matriciales que permiten comparar diferentes dispositivos, lotes o variables.

Monitoring

Device	Temperature	Humidity	Status	Last Reading
Device ID	Value	Value	Connection Status	Date & Time

Lots

Lot	Product	Location	Monitoring Status	Last Update
Lot ID	Product Name	Location	Status	Date & Time

Este sistema facilita la supervisión simultánea de múltiples productos y dispositivos.

Alphabetical Organization

Los productos, dispositivos y ubicaciones podrán organizarse alfabéticamente cuando sea necesario localizar un elemento específico.

Chronological Organization

Las lecturas, alertas, incidencias y registros de trazabilidad se organizan principalmente de forma cronológica, mostrando primero los eventos más recientes.

Topic-Based Organization

Las principales funcionalidades se agrupan según el tópico al que pertenecen:

- Dashboard
- Monitoring
- Inventory
- Lots
- Devices
- Alerts
- Traceability
- Reports

Audience-Based Organization

En el Landing Page se diferencia el contenido según los segmentos objetivo:

- **For Cold Chain:** contenido orientado a empresas de distribución y operadores de cadena de frío.
- **For Producers:** contenido orientado a empresas productoras y comercializadoras de alimentos perecibles.

Esto permite presentar beneficios específicos para cada segmento sin modificar la identidad general de FreshSense.

5.2.2. Labeling Systems

El sistema de etiquetado de FreshSense utiliza términos breves, consistentes y fáciles de reconocer.

Se evita mostrar terminología técnica relacionada con la implementación cuando no aporta valor directo al usuario.

Landing Page

Label	Propósito
`Home`	Regresar al inicio.
`Solution`	Presentar la solución FreshSense.
`How It Works`	Explicar el funcionamiento general del sistema.
`Benefits`	Presentar los principales beneficios.
`For Cold Chain`	Información dirigida a empresas de distribución y cadena de frío.
`For Producers`	Información dirigida a productores y comercializadores.
`Contact`	Presentar los medios de contacto.
`Sign In`	Acceder a la plataforma.
`Get Started`	Iniciar el proceso de acceso o registro.

Web and Mobile Applications

Label	Información asociada
`Dashboard`	Resumen general del sistema e indicadores principales.
`Monitoring`	Visualización de las condiciones registradas por los dispositivos.
`Inventory`	Información de los productos registrados.
`Lots`	Gestión y seguimiento de lotes.
`Devices`	Gestión de dispositivos IoT asociados.
`Alerts`	Eventos o desviaciones que requieren atención.
`Traceability`	Historial de eventos y condiciones asociadas a productos o lotes.
`Reports`	Información consolidada y resultados de monitoreo.

IoT Monitoring

Label	Información asociada
`Device`	Dispositivo IoT registrado.
`Device ID`	Identificador único del dispositivo.
`Connected`	Dispositivo comunicándose correctamente.
`Disconnected`	Dispositivo sin comunicación con el sistema.
`Temperature`	Temperatura obtenida mediante el sensor.
`Humidity`	Humedad obtenida mediante el sensor.
`Last Reading`	Fecha y hora de la lectura más reciente.
`Monitoring Status`	Estado actual del proceso de monitoreo.

Las etiquetas se mantendrán equivalentes entre las experiencias Web y Mobile para evitar que un mismo concepto tenga diferentes nombres dependiendo de la plataforma.

5.2.3. SEO Tags and Meta Tags

FreshSense utilizará SEO Tags y Meta Tags para representar adecuadamente el contenido del Landing Page y de la aplicación web.

Landing Page

Element	Value
Title	`FreshSense
Description	`FreshSense helps companies monitor temperature and humidity conditions during the storage and distribution of perishable food products using IoT technology.`
Keywords	`cold chain monitoring, perishable food, IoT monitoring, temperature monitoring, humidity monitoring, food traceability, cold storage`
Author	`FreshSense Team`

Web Application

Element	Value
Title	`FreshSense Platform
Description	`Monitor devices, environmental conditions, lots, alerts and traceability information for perishable food products with FreshSense.`
Keywords	`FreshSense, cold chain, IoT monitoring, food traceability, temperature, humidity, logistics`
Author	`FreshSense Team`

Mobile Application - ASO

En caso de publicación de la aplicación móvil mediante un App Store, se utilizarán los siguientes elementos:

Element	Value
App Title	`FreshSense`
App Subtitle	`Smart Cold Chain Monitoring`
App Keywords	`cold chain, food, monitoring, temperature, humidity, traceability, IoT`
App Description	`FreshSense helps companies monitor environmental conditions, connected devices and alerts related to the storage and distribution of perishable food products.`

5.2.4. Searching Systems

El sistema de búsqueda se concentra principalmente en las aplicaciones Web y Mobile, donde el volumen de información puede aumentar debido al registro de dispositivos, lotes, productos, lecturas y alertas.

El Landing Page no requiere un buscador interno debido a que su contenido está organizado en un número reducido de secciones accesibles mediante navegación directa.

Lots Search

El usuario podrá buscar lotes mediante:

- Lot ID.
- Producto.
- Ubicación.

Los resultados podrán filtrarse según:

- Monitoring Status.
- Fecha.
- Ubicación.
- Producto.

Cada resultado mostrará información relevante del lote y su estado actual.

Device Search

Los dispositivos podrán buscarse mediante:

- Device ID.
- Ubicación.

Los resultados podrán filtrarse según:

- Connected.
- Disconnected.
- Fecha de última lectura.

Cada resultado mostrará como mínimo:

- Device ID.
- Connection Status.
- Temperature.
- Humidity.
- Last Reading.

Monitoring Search

La información de monitoreo podrá consultarse utilizando:

- Device.
- Lot.
- Rango de fechas.

- Ubicación.

Los resultados mostrarán las principales mediciones registradas durante el período seleccionado.

Alerts Search

Las alertas podrán filtrarse según:

- Estado.
- Severidad.
- Fecha.
- Tipo de evento.
- Device.
- Lot.

Por defecto, se mostrarán primero las alertas más recientes y aquellas que requieran mayor atención.

Traceability Search

La información de trazabilidad podrá consultarse mediante:

- Lot ID.
- Producto.
- Device.
- Período.

Los resultados se mostrarán cronológicamente para facilitar la revisión de los eventos registrados durante el almacenamiento o distribución del producto.

Cuando una búsqueda no presente coincidencias, la interfaz mostrará un mensaje claro y permitirá modificar o eliminar los filtros aplicados.

5.2.5. Navigation Systems

La navegación de FreshSense busca mantener recorridos simples y consistentes entre el Landing Page y las diferentes aplicaciones.

Landing Page - Desktop

La versión Desktop utilizará una barra de navegación superior con acceso a las principales secciones:

```
Home | Solution | How It Works | Benefits | For Cold Chain | For Producers | Contact
```

Además, se mostrarán las principales acciones:

```
Sign In | Get Started
```

Los Call-to-Action permitirán dirigir a los usuarios hacia el acceso a la plataforma o hacia información específica relacionada con su segmento.

Landing Page - Mobile

En pantallas móviles, las mismas opciones estarán agrupadas en un menú compacto para reducir el espacio utilizado y priorizar el contenido principal.

El orden y significado de las secciones serán equivalentes a los utilizados en Desktop.

Web Application

Una vez autenticado, el usuario tendrá acceso a los principales módulos de FreshSense:

```
Dashboard | Monitoring | Inventory | Lots | Devices | Alerts | Traceability | Reports
```

El **Dashboard** funcionará como punto inicial de la experiencia y permitirá visualizar información relevante como:

- Estado general del monitoreo.
- Dispositivos conectados.
- Alertas activas.
- Lecturas recientes.
- Lotes que requieren atención.

Mobile Application

La navegación móvil priorizará las funcionalidades que requieren consulta frecuente:

- Dashboard.
- Monitoring.
- Alerts.
- Devices.
- Lots.

Las funcionalidades complementarias, como Inventory, Traceability y Reports, permanecerán disponibles desde la navegación secundaria.

IoT Device Navigation

La interacción con los dispositivos IoT se realiza principalmente mediante las aplicaciones Web y Mobile.

El recorrido principal será:

```
Devices → Select Device → Monitoring → Reading Details
```

Para el seguimiento de productos, se utilizará:

```
Lots → Select Lot → Traceability → Event Details
```

Desde estas vistas, el usuario podrá conocer el estado de conexión de los dispositivos, consultar las mediciones de temperatura y humedad y revisar los eventos asociados al monitoreo de cada lote.

Esta organización evita que el usuario necesite interactuar directamente con componentes técnicos como el ESP32 o el sensor DHT22 para utilizar las funciones principales de FreshSense.

5.3. Landing Page UI Design

5.3.1. Landing Page Wireframe

A continuación se realizaron los wireframes de la landing page de FreshSense, siguiendo los user stories como referencia, para conocer las necesidades y preferencias de los usuarios visitantes:

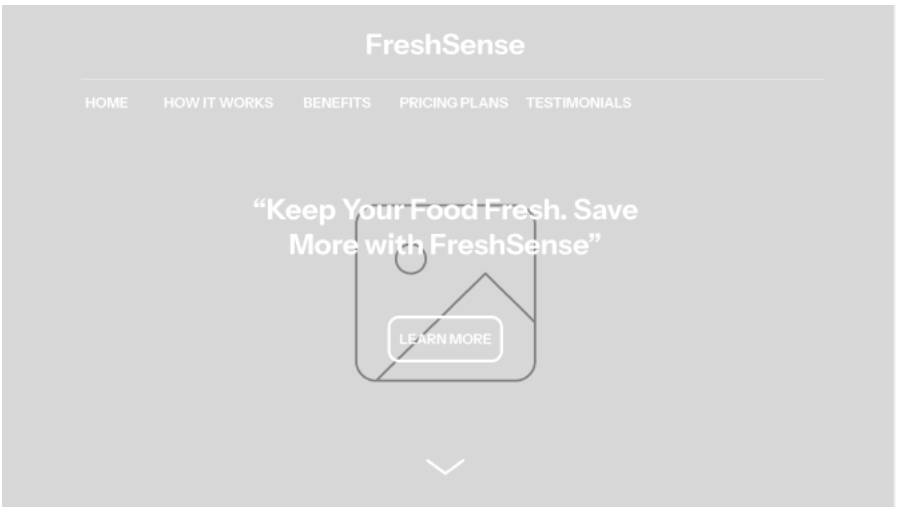


Figura 1. Wireframe de la página principal

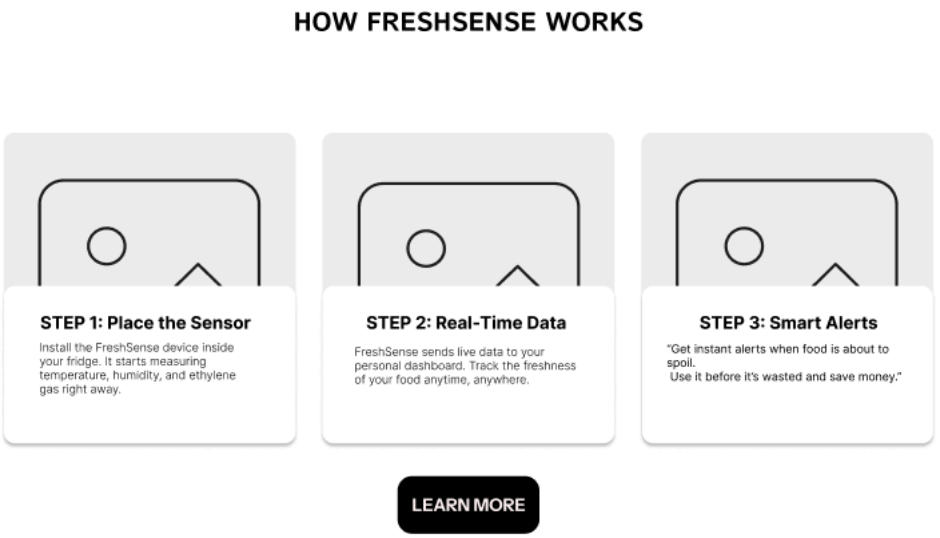
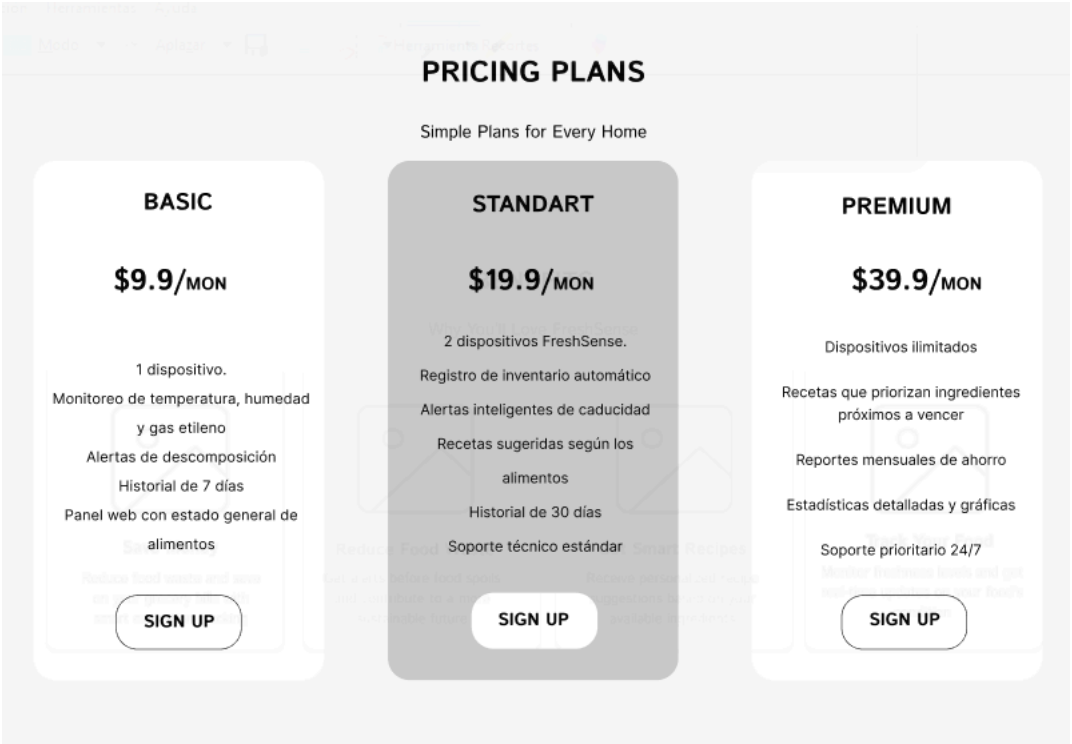


Figura 2. Como funciona FreshSense

Figura 3. Vistazo inicial a los beneficios

Figura 4. Modelo inicial para los planes de subscripción.



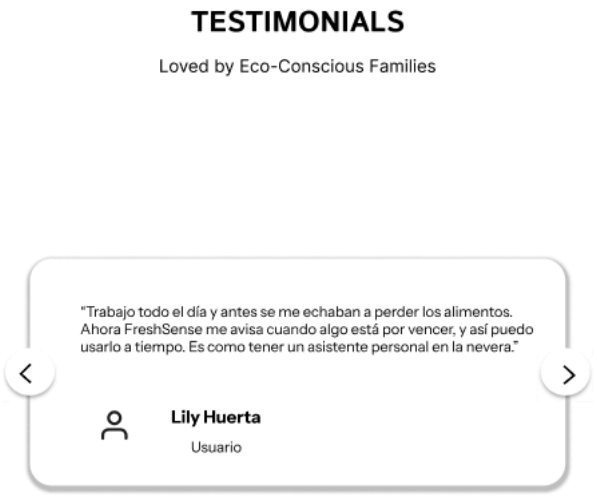


Figura 5. Wireframe para los testimonios

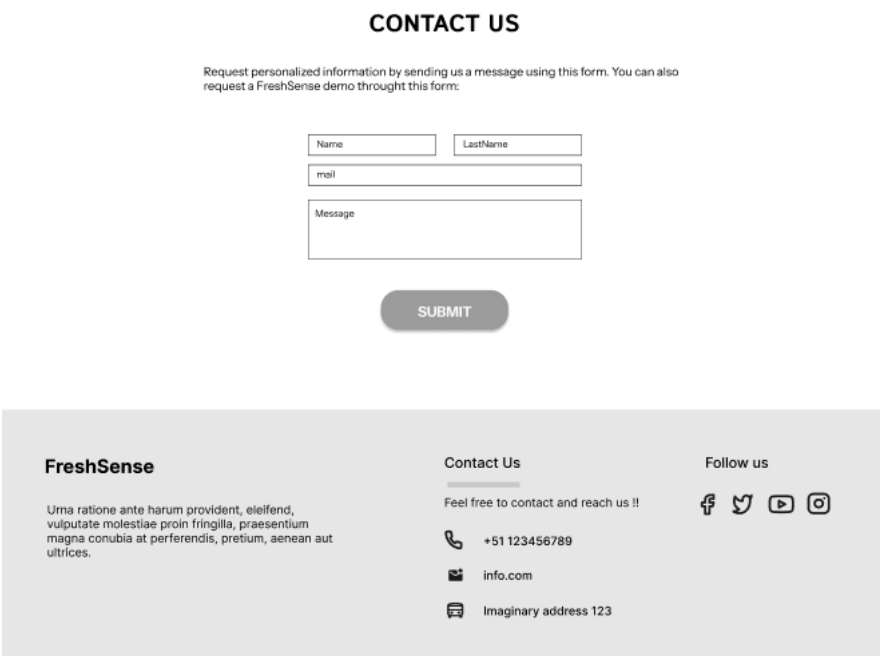


Figura 6. Wireframe para el formulario y se incluye el footer

5.3.2. Landing Page Mock-up

Una vez se realizaron los wireframes, usamos los Style Guidelines, para desarrollar el siguiente paso, los mock ups, utilizamos los colores y modelos referidos en los guidelines, los colores verdes y azules predominantes en el diseño, aluden a la escencia de la aplicación:

Figura 7. Mock-Up de la página principal

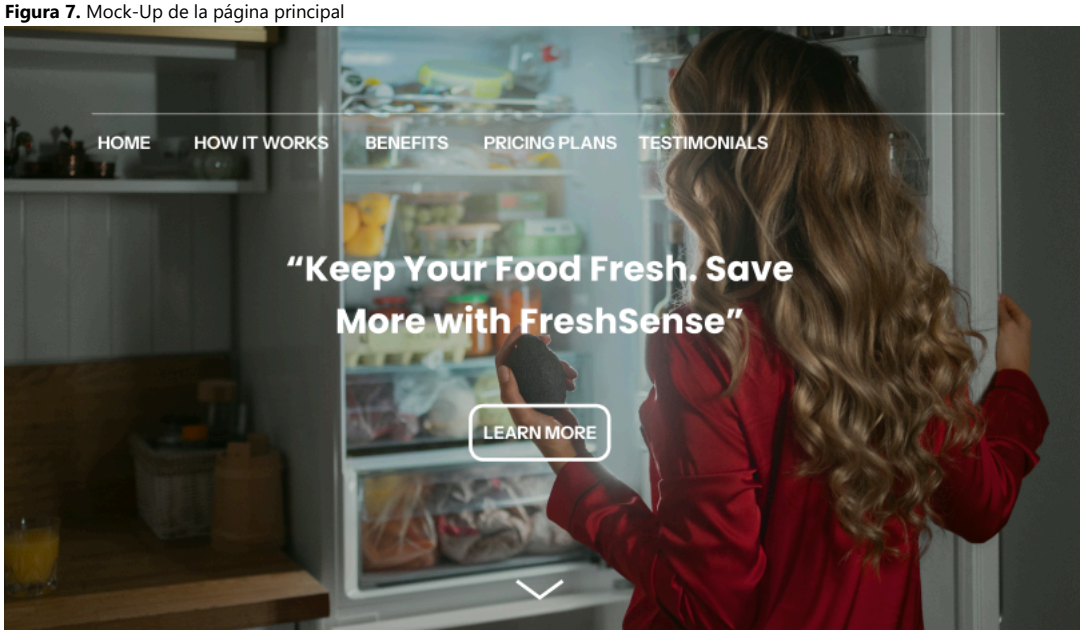


Figura 8. Mock-Up se muestra las funciones de FreshSense

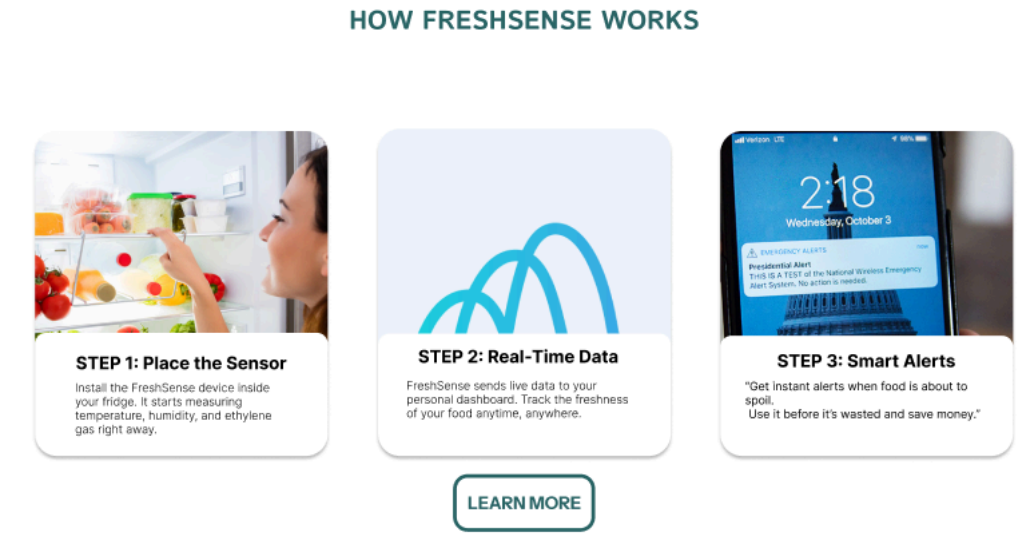


Figura 9. Vistazo inicial a los beneficios

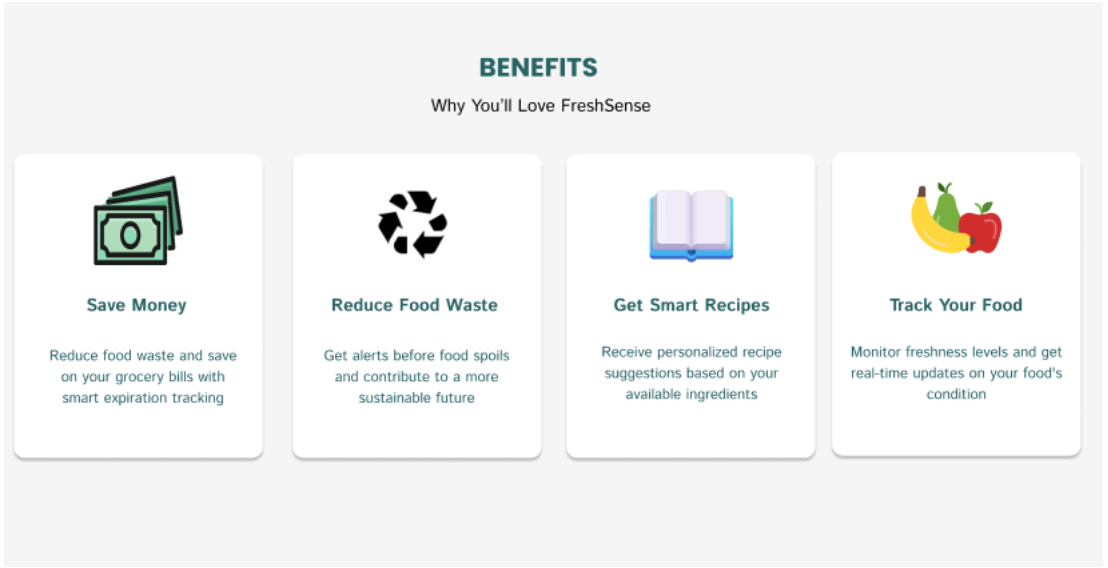
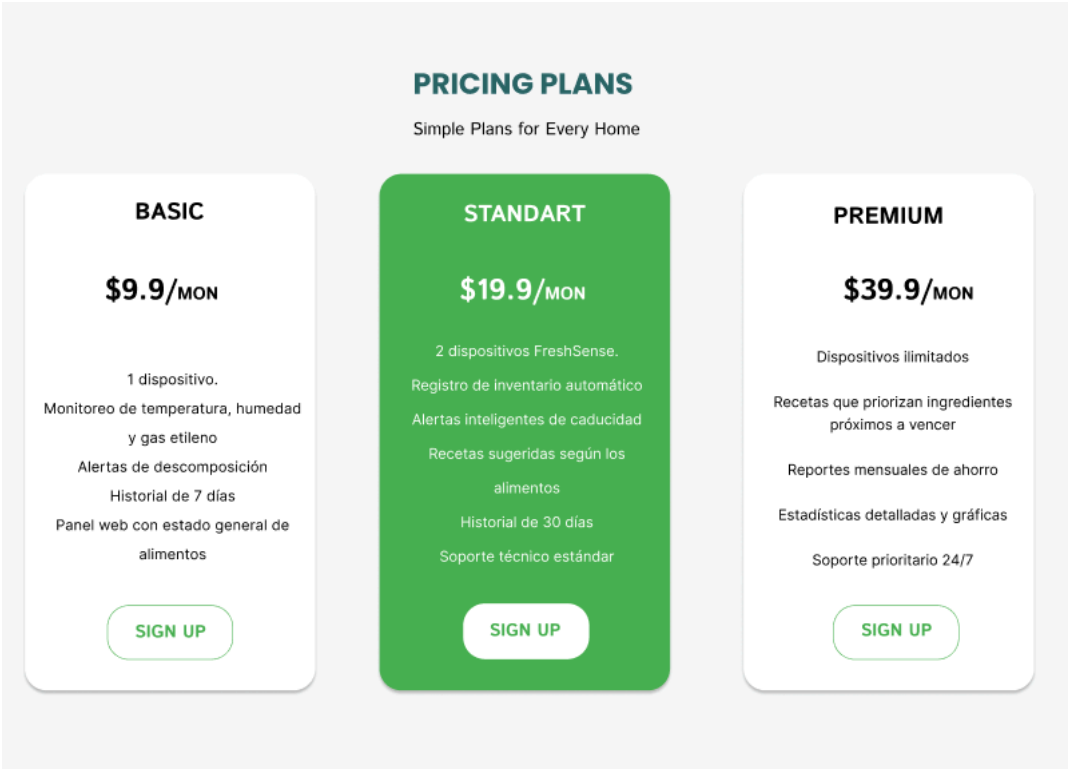


Figura 10. Mock-Up para los planes de subscripción.



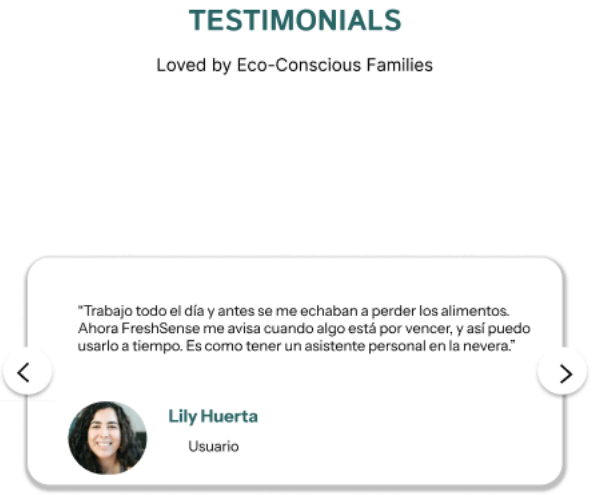


Figura 11. Mock-Up para los testimonios

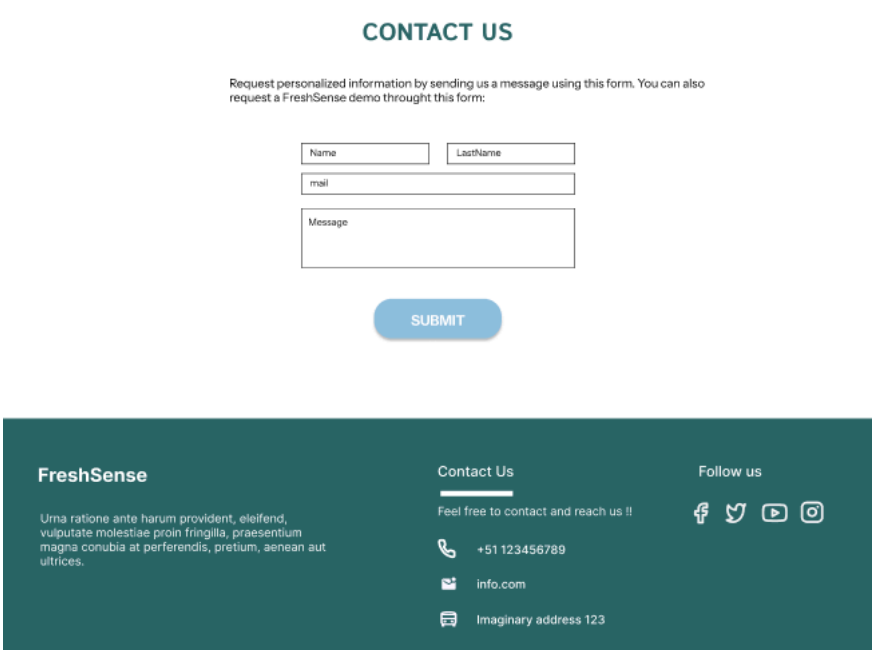
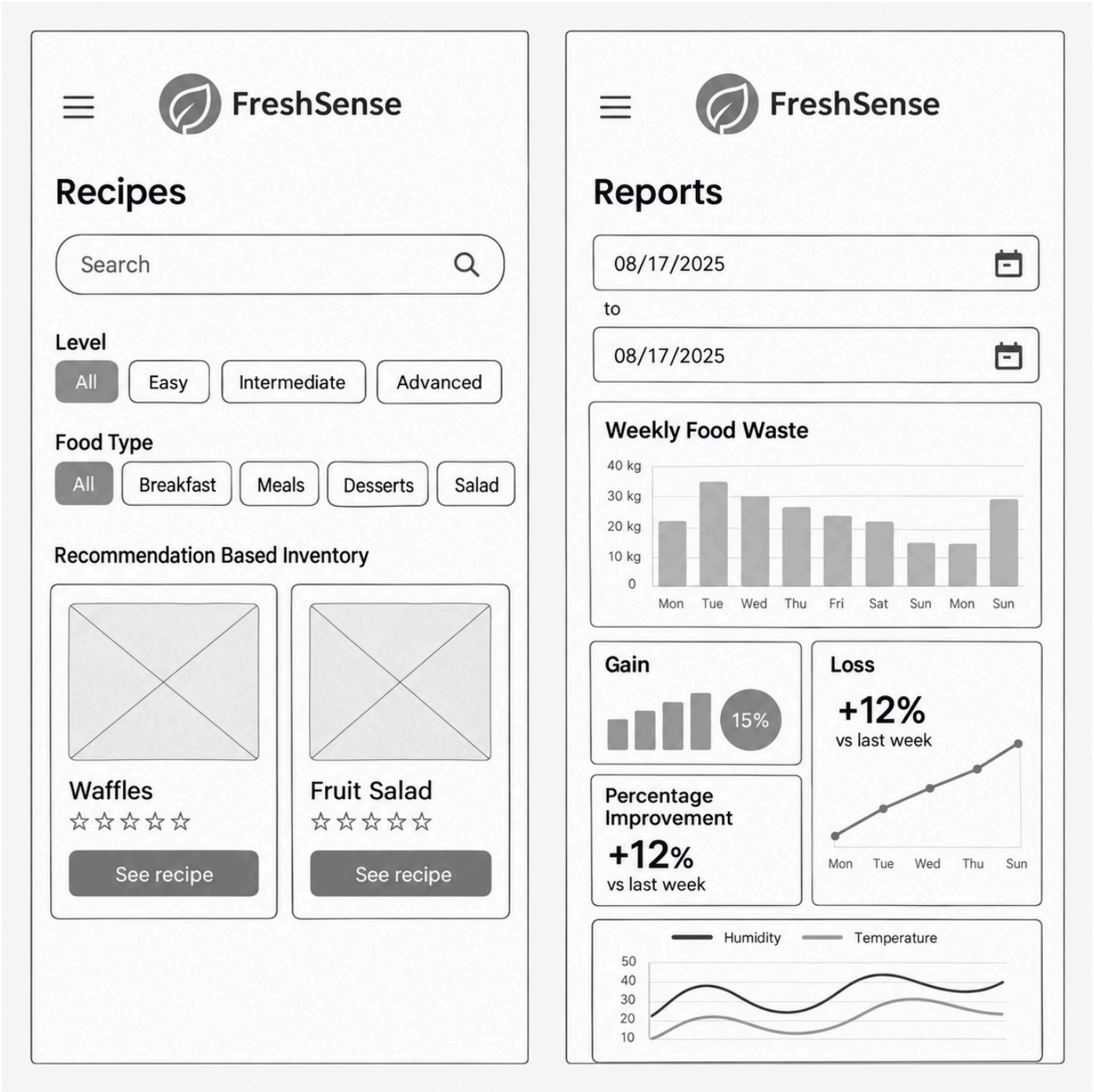


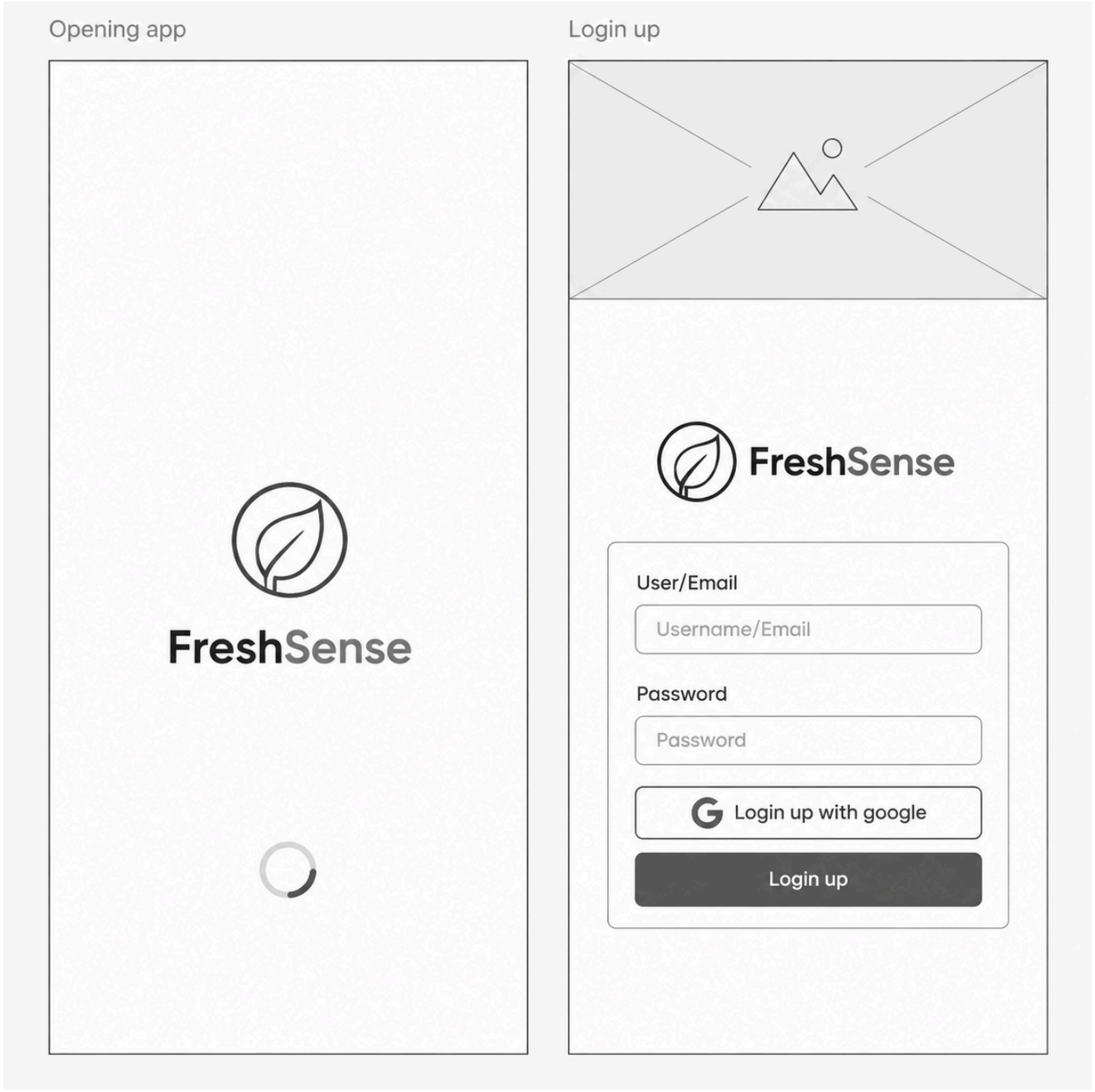
Figura 12. Mock-Up para el formulario y se incluye el footer

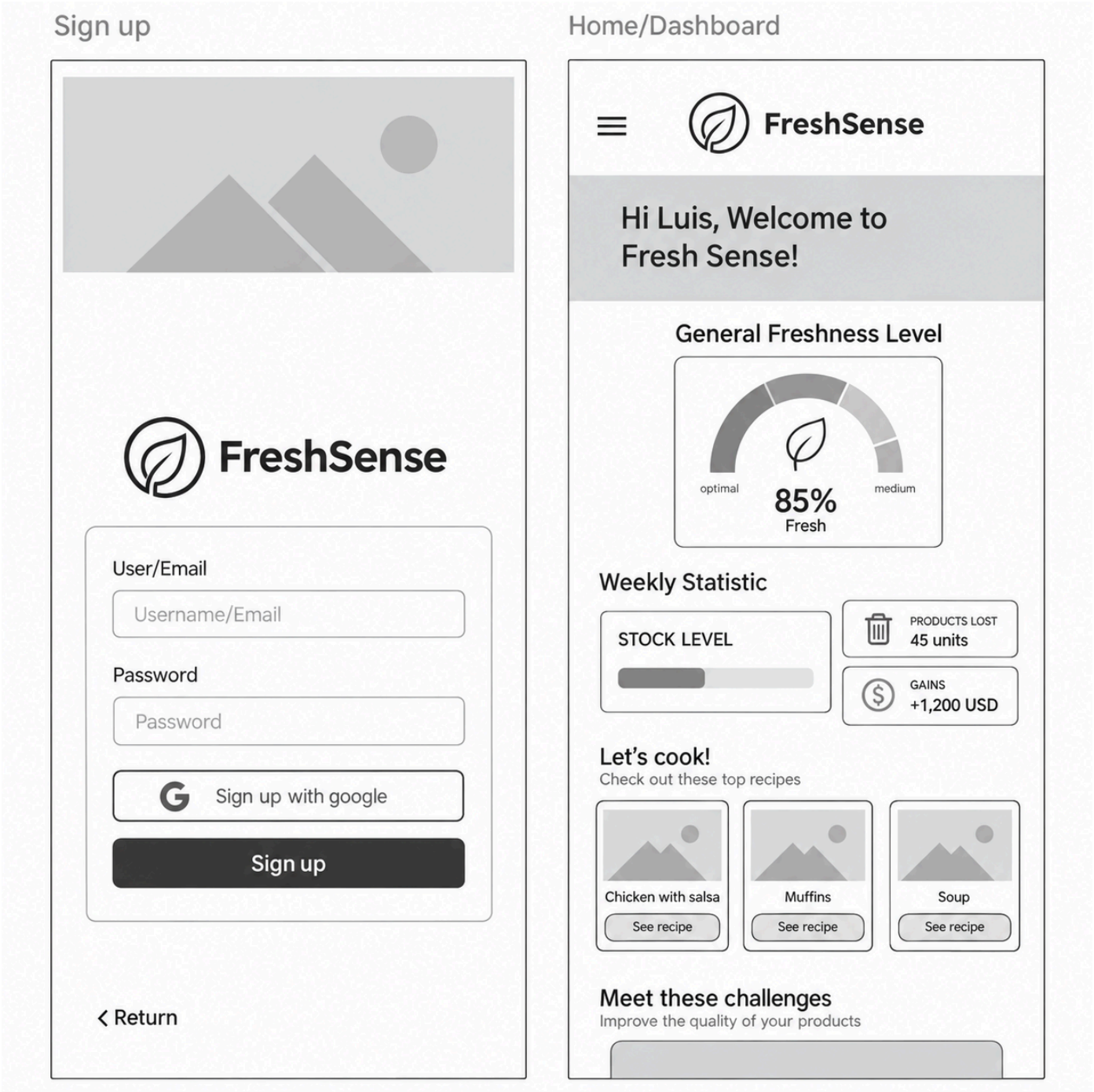
5.4. Applications UX/UI Design

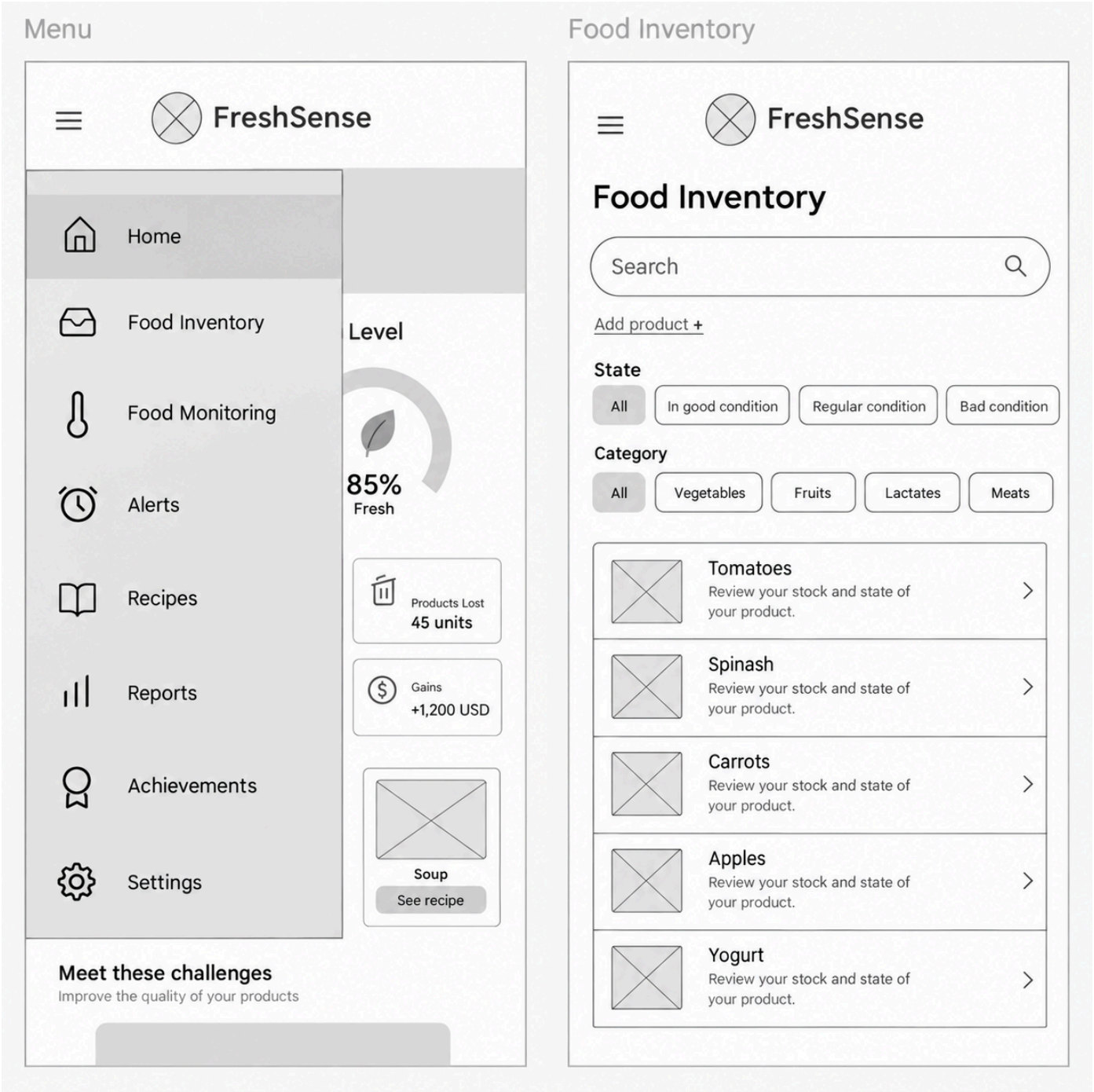
En la sección de Applications UX/UI Design nos enfocamos en el diseño de la interfaz y la experiencia de usuario de la aplicación web de FreshSenser, donde incluimos una visualización funcional por cada parte del aplicativo con sus flujos de interacción completos. Se elaboraron wireframes en formato mobile que facilitan la disposición de las funciones de la plataforma a través de su dispositivo móvil frecuente, con elementos en pantallas que son la introducción al app, el login up, el sign up, el home o dashboard, el menú, el inventario de insumos, el detalle de cada insumo, el monitoreo de insumos, alertas, recetas, reportes, logros y soporte. En base a estos esquemas se diseñaron los mockups con alta fidelidad. En los siguientes sprints se muestra el desarrollo de cada vista de la app y cómo estas interactúan.

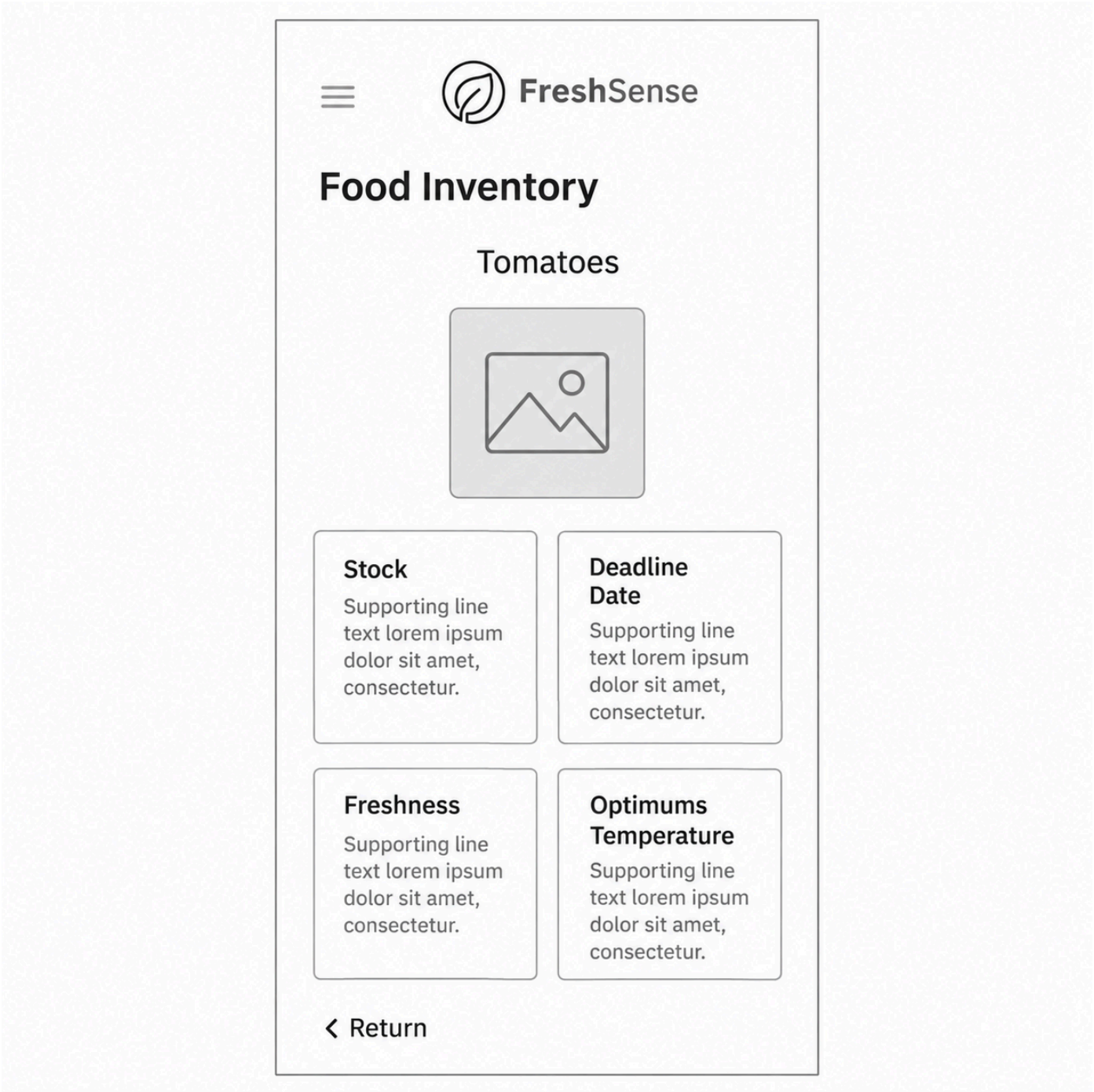
5.4.1. Applications Wireframes

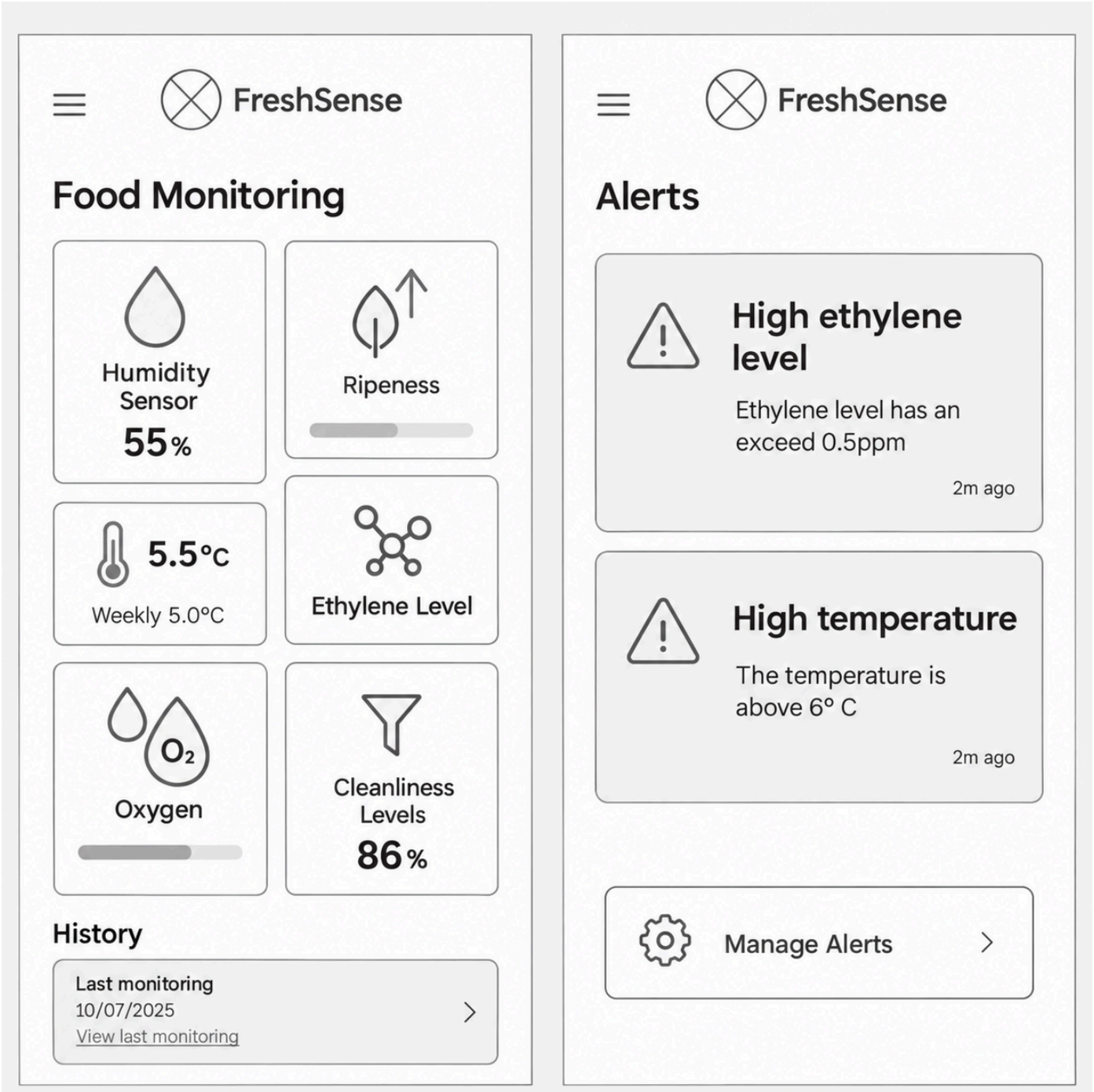

FreshSense
Recipes
Search
Level
All
Easy
Intermediate
Advanced
Food Type
All
Breakfast
Meals
Desserts
Salad
Recommendation Based Inventory
Waffles
Fruit Salad
See recipe
See recipe
FreshSense
Reports
08/17/2025
to
08/17/2025
Weekly Food Waste
40 kg
30 kg
20 kg
10 kg
0
Mon Tue Wed Thu Fri Sat Sun Mon Sun
Gain
15%
Loss
+12%
vs last week
Mon Tue Wed Thu Sun
Percentage Improvement
+12%
vs last week
Humidity
Temperature
50
40
30
20
10

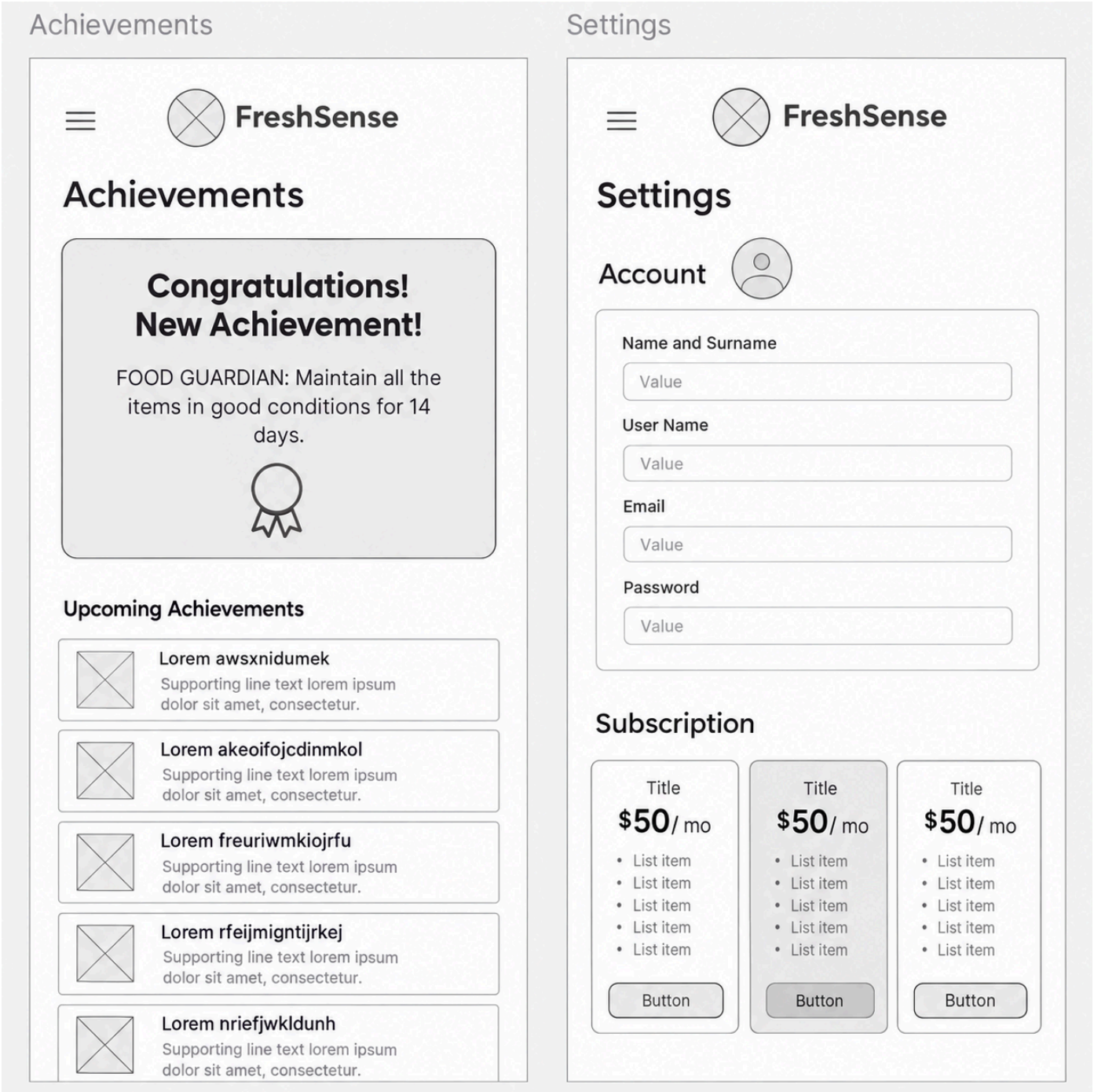

Opening app
FreshSense
Login up
FreshSense
User/Email
Username/Email
Password
Password
Login up with google
Login up


Sign up
FreshSense
User/Email
Username/Email
Password
Password
Sign up with google
Sign up
Return
Home/Dashboard
FreshSense
Hi Luis, Welcome to Fresh Sense!
General Freshness Level
optimal
85%
Fresh
medium
Weekly Statistic
STOCK LEVEL
PRODUCTS LOST
45 units
GAINS
+1,200 USD
Let's cook!
Check out these top recipes
Chicken with salsa
See recipe
Muffins
See recipe
Soup
See recipe
Meet these challenges
Improve the quality of your products


Menu
FreshSense
Home
Food Inventory
Food Monitoring
Alerts
Recipes
Reports
Achievements
Settings
Level
85%
Fresh
Products Lost
45 units
Gains
+1,200 USD
Soup
See recipe
Meet these challenges
Improve the quality of your products
Food Inventory
FreshSense
Food Inventory
Search
Add product +
State
All
In good condition
Regular condition
Bad condition
Category
All
Vegetables
Fruits
Lactates
Meats
Tomatoes
Review your stock and state of your product.
Spinash
Review your stock and state of your product.
Carrots
Review your stock and state of your product.
Apples
Review your stock and state of your product.
Yogurt
Review your stock and state of your product.

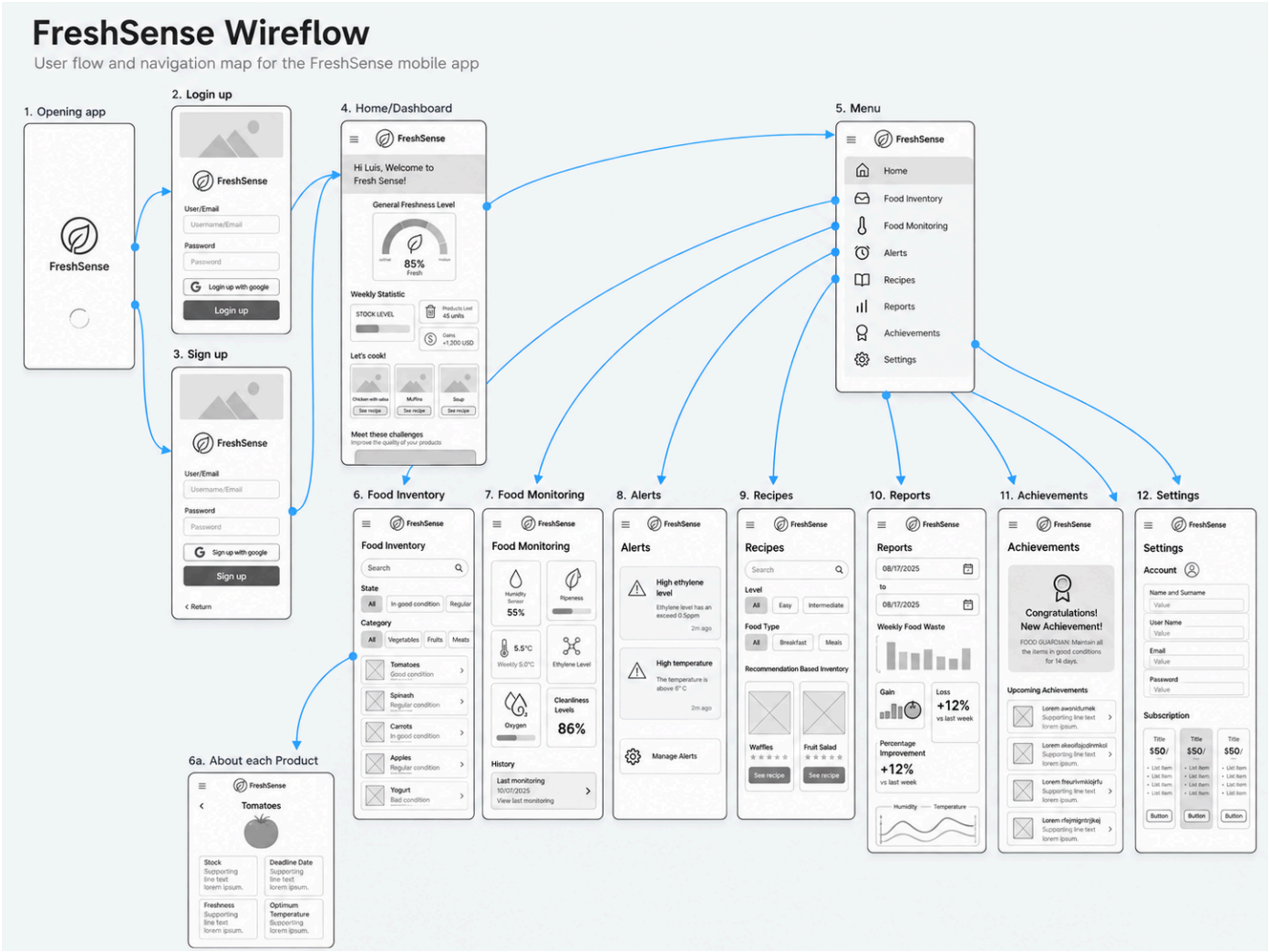

FreshSense
Food Inventory
Tomatoes
Stock
Supporting line text lorem ipsum dolor sit amet, consectetur.
Deadline Date
Supporting line text lorem ipsum dolor sit amet, consectetur.
Freshness
Supporting line text lorem ipsum dolor sit amet, consectetur.
Optimums Temperature
Supporting line text lorem ipsum dolor sit amet, consectetur.
Return


FreshSense
Food Monitoring
Humidity Sensor
55%
Ripeness
5.5°C
Weekly 5.0°C
Ethylene Level
O2
Oxygen
Cleanliness Levels
86%
History
Last monitoring
10/07/2025
View last monitoring
FreshSense
Alerts
High ethylene level
Ethylene level has an exceed 0.5ppm
2m ago
High temperature
The temperature is above 6° C
2m ago
Manage Alerts

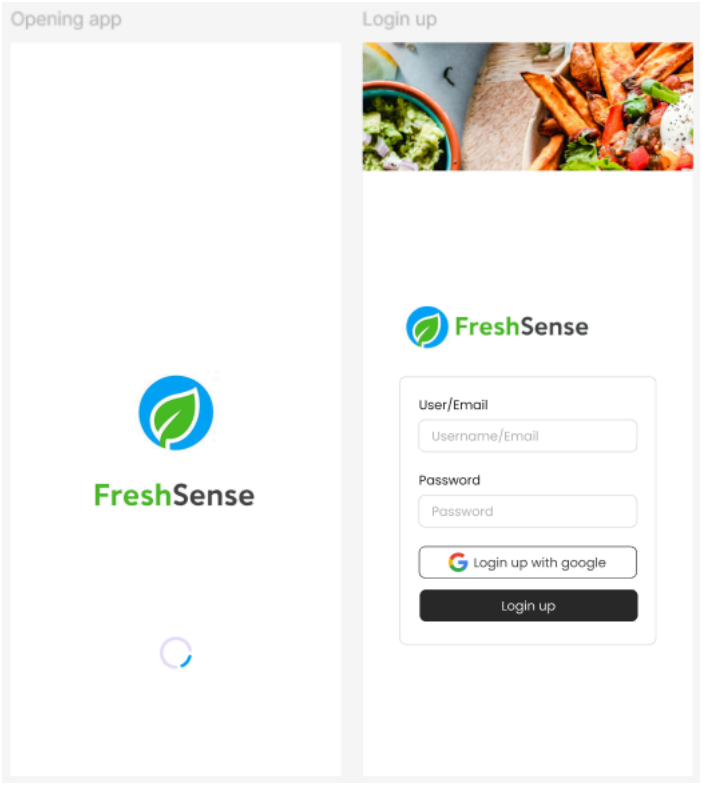


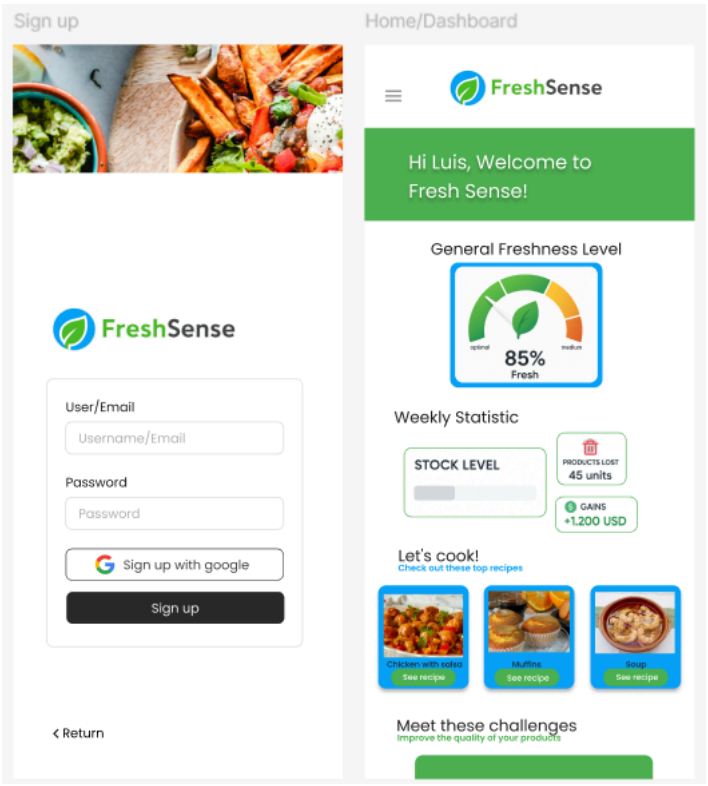
5.4.2. Applications Wireflow Diagrams

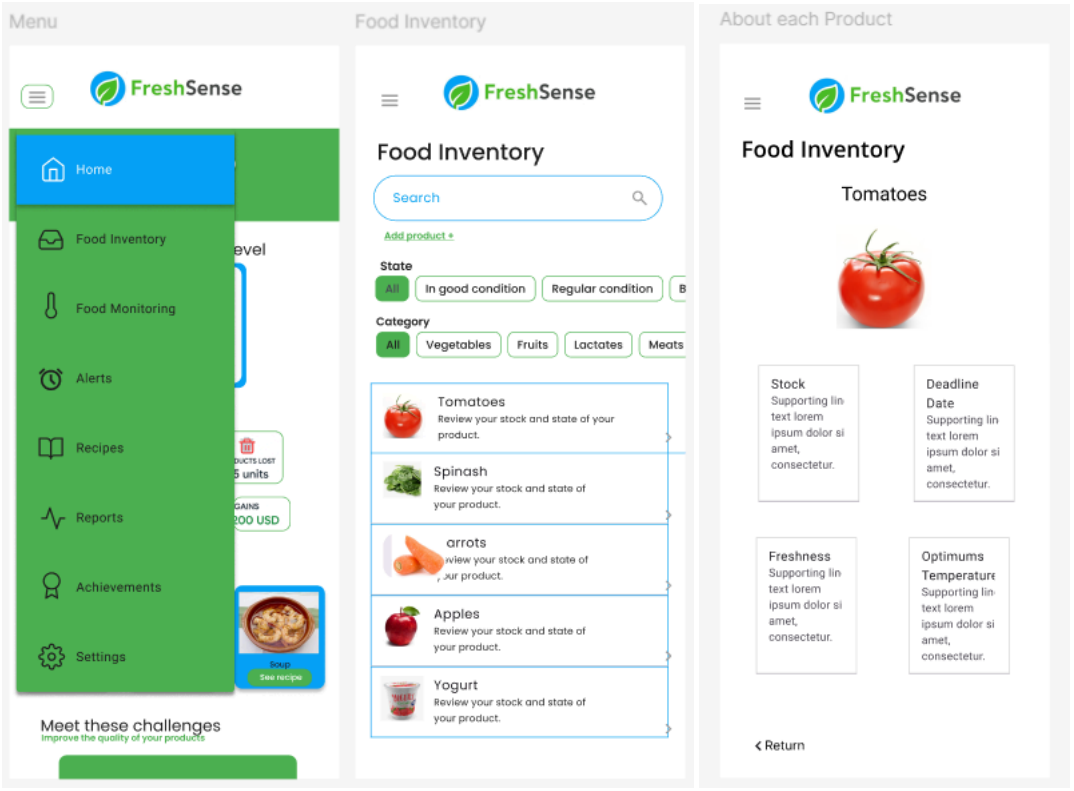
Para este apartado, el wireflow se diseñó para representar de forma detallada el proceso de uso desde el inicio de sesión hasta las funcionalidades principales, como la gestión del inventario de alimentos, el monitoreo en tiempo real, la recepción de alertas, la consulta de recetas, el seguimiento de logros y la personalización de ajustes. De esta manera, se asegura que la navegación sea coherente, intuitiva y centrada en mejorar la experiencia del usuario final.

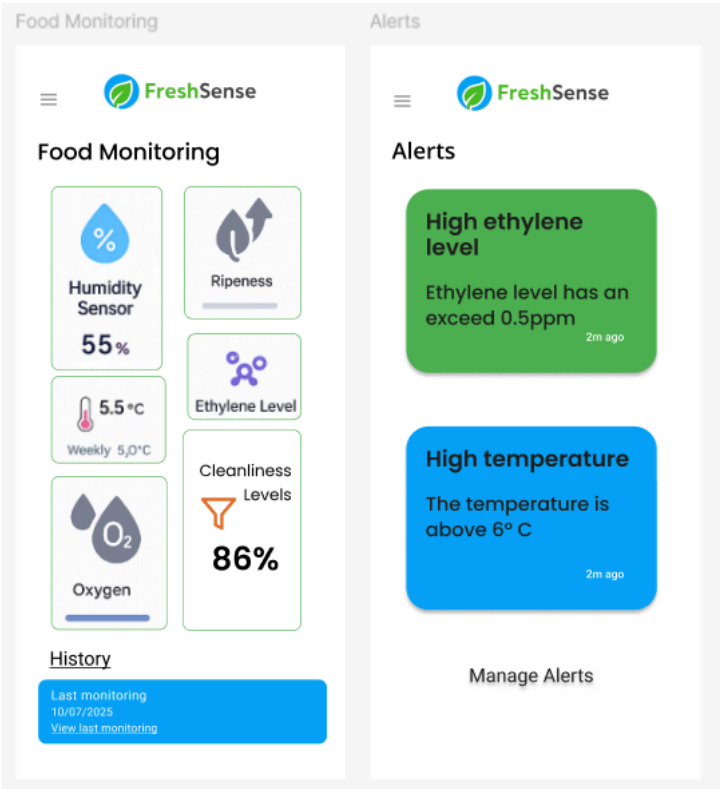


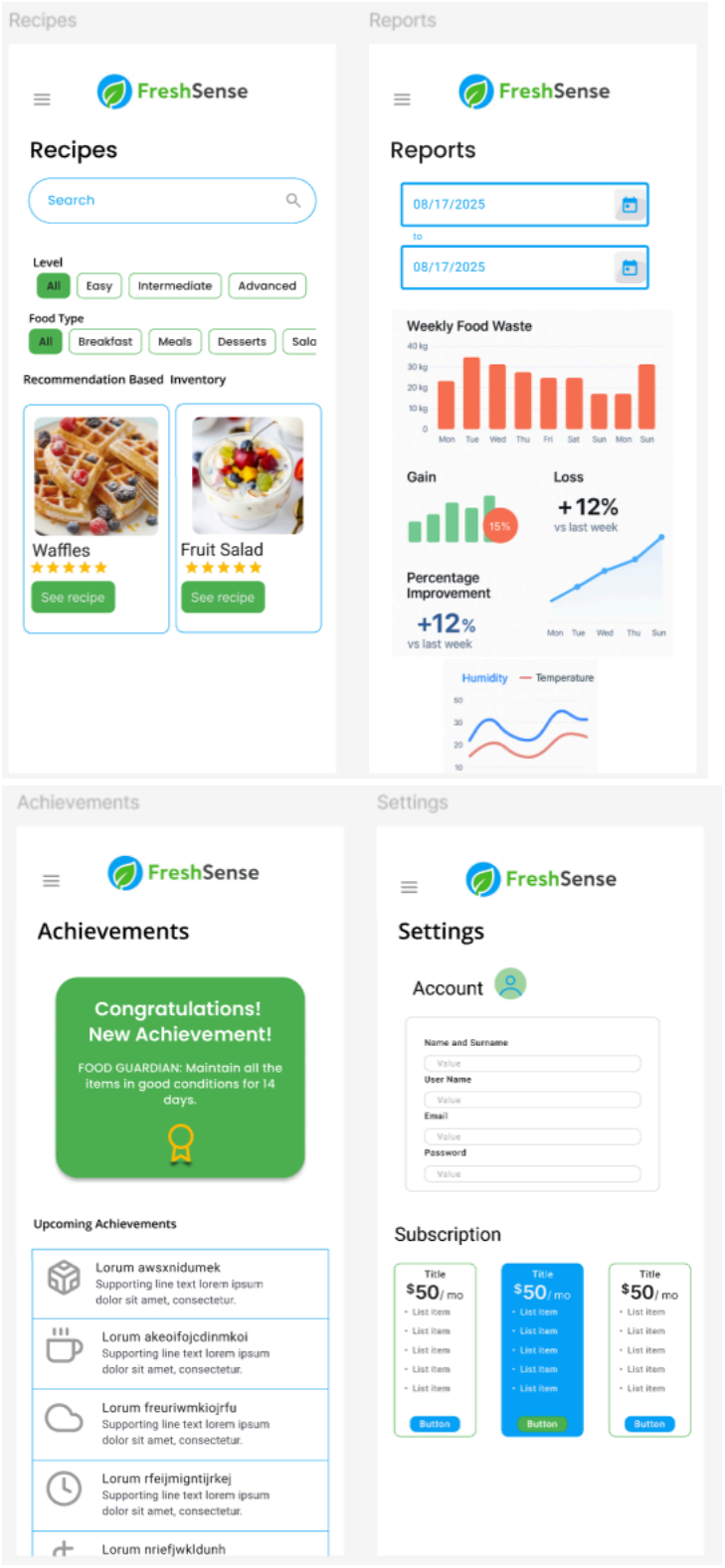
5.4.2. Applications Mock-ups


Opening app
FreshSense
Login up
FreshSense
User/Email
Username/Email
Password
Password
Login up with google
Login up

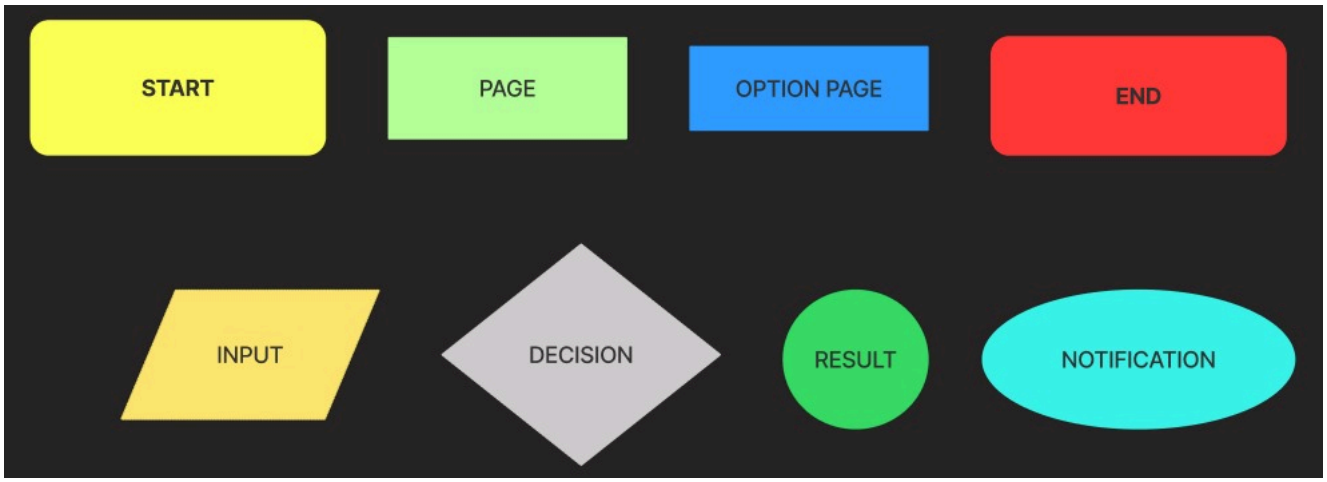

Sign up
FreshSense
User/Email
Username/Email
Password
Password
Sign up with google
Sign up
Return
Home/Dashboard
FreshSense
Hi Luis, Welcome to Fresh Sense!
General Freshness Level
85%
Fresh
Weekly Statistic
STOCK LEVEL
PRODUCTS LOST
45 units
GAINS
+1,200 USD
Let's cook!
Check out these top recipes
Chicken with salsa
See recipe
Muffins
See recipe
Soup
See recipe
Meet these challenges
Improve the quality of your products


Menu
FreshSense
Home
Food Inventory
Food Monitoring
Alerts
Recipes
Reports
Achievements
Settings
Meet these challenges
Improve the quality of your products
Food Inventory
FreshSense
Food Inventory
Search
Add product +
State
All
In good condition
Regular condition
Category
All
Vegetables
Fruits
Lactates
Meats
Tomatoes
Review your stock and state of your product.
Spinash
Review your stock and state of your product.
Apples
Review your stock and state of your product.
Yogurt
Review your stock and state of your product.
About each Product
FreshSense
Food Inventory
Tomatoes
Stock
Deadline Date
Freshness
Optimums Temperature
Return


Food Monitoring
FreshSense
Food Monitoring
Humidity Sensor
55%
Ripeness
5.5 °C
Weekly 5,0°C
Ethylene Level
Cleanliness Levels
86%
Oxygen
History
Last monitoring
10/07/2025
View last monitoring
Alerts
FreshSense
Alerts
High ethylene level
Ethylene level has an exceed 0.5ppm
2m ago
High temperature
The temperature is above 6° C
2m ago
Manage Alerts

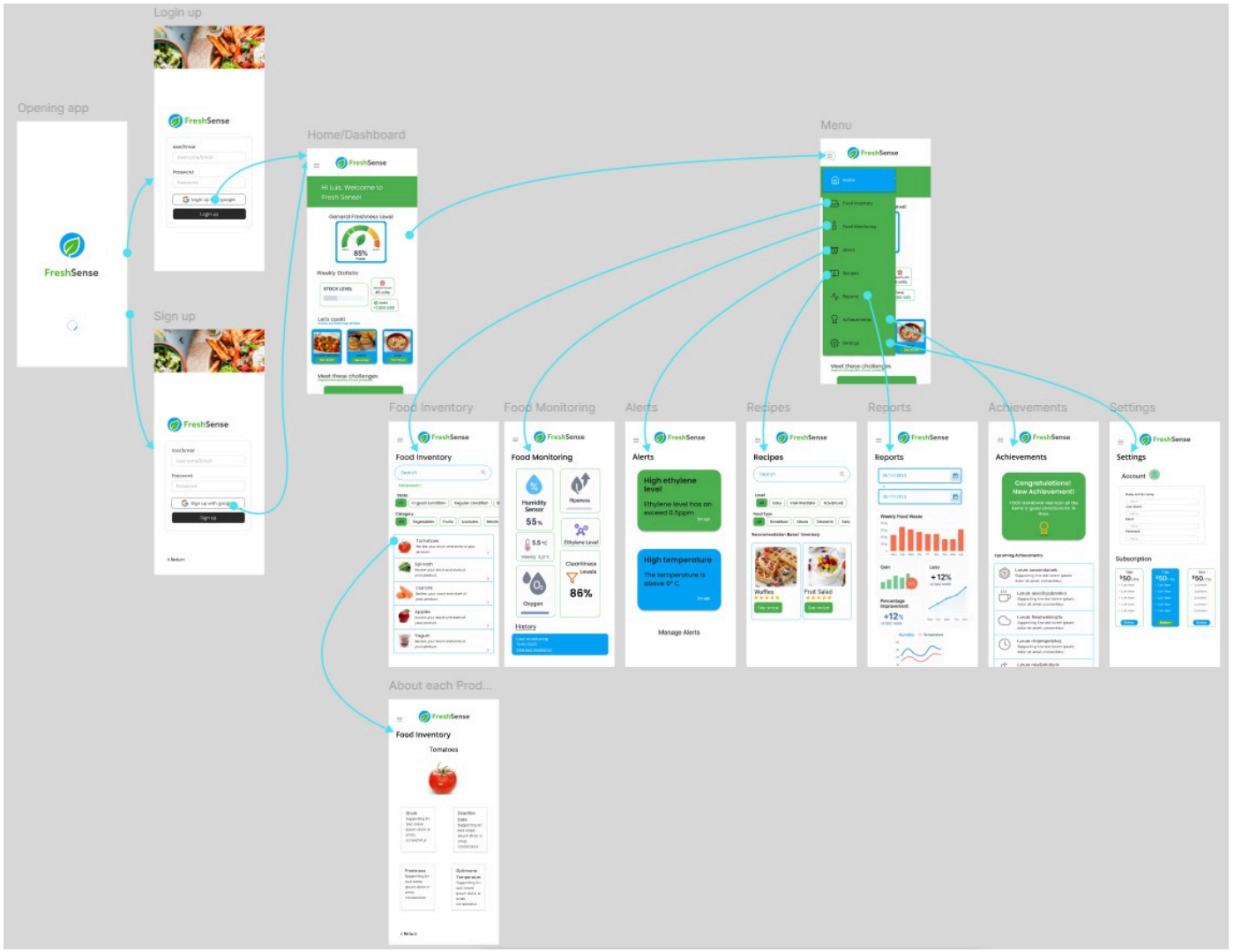


5.4.3. Applications User Flow Diagrams



Cada figura del diagrama tiene un significado específico dentro del flujo de usuario:

- Start: punto de inicio del recorrido.
- Page: pantalla normal de la aplicación.
- Option Page: menú o sección con varias opciones.
- End: final del flujo o salida de la app.
- Input: ingreso de datos por parte del usuario.
- Decision: condición que define diferentes caminos.
- Result: acción realizada con éxito.
- Notification: mensaje o alerta mostrado al usuario.



Ahora representamos los User Flow Diagrams de la aplicación web FreshSense, los cuales permiten visualizar de manera clara el recorrido que realiza el usuario dentro del sistema, desde que abre la aplicación hasta que cierra sesión. Este diagrama utiliza convenciones gráficas específicas para identificar los distintos tipos de pantallas, acciones, decisiones, resultados y notificaciones que intervienen en la experiencia del usuario. Gracias a esta representación, se facilita el análisis de la interacción, la detección de posibles mejoras en la navegación y la validación de que todos los escenarios de uso estén contemplados.

5.5. Applications Prototyping

Para validar la navegación y la interacción de los usuarios con FreshSense se desarrolló un prototipo interactivo de la aplicación. Este prototipo permite recorrer las principales vistas y funcionalidades definidas durante el proceso de diseño UX/UI, simulando el comportamiento esperado de la solución antes de su implementación completa.

El prototipo facilita la validación de los flujos de navegación, la organización de las pantallas y las interacciones entre las diferentes funcionalidades de la aplicación.

El prototipo interactivo de FreshSense puede consultarse en el siguiente enlace:

Prototipo interactivo de FreshSense en Figma

5.6. IoT Device Design

El dispositivo IoT de FreshSense funciona como un nodo de monitoreo ambiental diseñado para ser colocado dentro de un refrigerador. Su propósito es capturar periódicamente información acerca de las condiciones en las que se encuentran almacenados los alimentos y transmitir dichas lecturas al sistema FreshSense.

La solución utiliza un microcontrolador **ESP32 DevKit C v4** junto con un sensor digital **DHT22 (AM2302)** para obtener información de temperatura y humedad. El ESP32 proporciona la capacidad de procesamiento y conectividad Wi-Fi necesaria para transmitir las mediciones hacia los servicios de la plataforma.

Hardware del dispositivo

Componente	Especificación	Función
Microcontrolador	ESP32 DevKit C v4	Procesamiento de datos y conectividad Wi-Fi.
Sensor	DHT22 (AM2302)	Medición digital de temperatura y humedad.
Pin de datos	GPIO 12	Recepción de la señal digital proveniente del DHT22.
Alimentación	3V3 y GND	Alimentación eléctrica del sensor.
Conectividad	Wi-Fi 802.11	Comunicación del dispositivo con el Edge API.
Entorno de simulación	Wokwi	Simulación y validación del firmware antes de utilizar hardware físico.

La conexión principal entre el ESP32 y el DHT22 se realiza utilizando el pin **GPIO 12** para la señal de datos. El sensor se alimenta mediante los pines **3V3** y **GND** del microcontrolador.

Firmware

El firmware del dispositivo se encarga de obtener las lecturas del sensor, estructurar la información y transmitirla hacia la plataforma.

Las principales librerías utilizadas son:

- **DHT sensor library**, para obtener las lecturas del sensor DHT22.
- **WiFi**, para establecer la conexión inalámbrica del ESP32.
- **ArduinoJson**, para serializar las lecturas utilizando formato JSON.
- **HTTPClient**, para enviar las solicitudes HTTP al Edge API.

Las principales variables generadas por el dispositivo son:

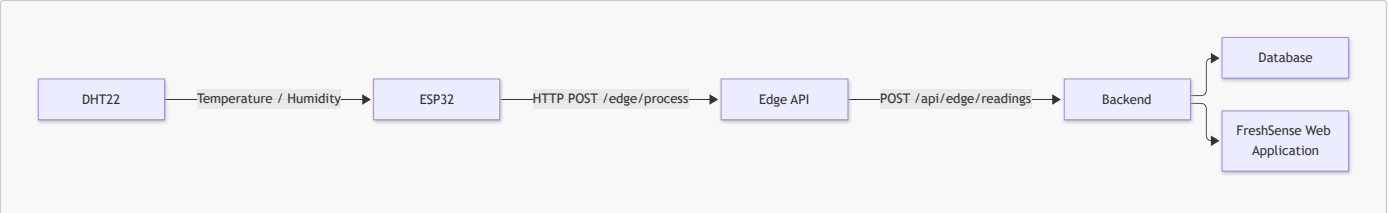
Variable	Unidad	Descripción
`temperature`	°C	Temperatura medida por el sensor DHT22.
`humidity`	%	Humedad relativa medida por el sensor DHT22.
`deviceId`	-	Identificador del dispositivo que genera la lectura.
`id`	-	Identificador único de la medición.
`time`	`dd/MM/yyyy HH:mm`	Fecha y hora asociadas a la lectura.

Un ejemplo de la información enviada por el dispositivo es:

```
{
  "deviceId": "esp32-cocina-01",
  "id": "rd-000123",
  "temperature": 6.4,
  "humidity": 82.0,
  "time": "07/07/2026 14:35"
}
```

Flujo de comunicación

El dispositivo forma parte de un flujo de comunicación que conecta el hardware IoT con la aplicación web de FreshSense.



El flujo funciona de la siguiente manera:

1. El sensor **DHT22** obtiene las mediciones de temperatura y humedad.
2. El **ESP32** procesa las lecturas y las serializa en formato JSON.
3. El ESP32 realiza una solicitud HTTP `POST` al endpoint `/edge/process` del **Edge API**.
4. El Edge API valida la información recibida y procesa la lectura.
5. La información es reenviada al backend mediante el endpoint `/api/edge/readings`.
6. El backend persiste la información y permite que los resultados sean utilizados por la aplicación web.

El Edge API funciona como una capa intermedia entre el dispositivo y el backend principal, permitiendo validar y procesar las lecturas antes de incorporarlas al resto de la solución.

Simulación y prototipo físico

Durante el desarrollo de FreshSense se utilizó **Wokwi** para simular el circuito formado por el ESP32 y el sensor DHT22. Esta simulación permitió validar el firmware, la lectura de temperatura y humedad, la conexión Wi-Fi y la generación del mensaje JSON sin depender inicialmente del hardware físico.

Posteriormente, el mismo diseño fue llevado a un prototipo físico utilizando un **ESP32 y un sensor DHT22**, permitiendo obtener mediciones reales de temperatura y humedad y completar la comunicación entre el dispositivo, el Edge API y el backend de FreshSense.

Capítulo VI: Product Implementation, Validation & Deployment

6.1. Software Configuration Management

FreshSense se configura como un solo producto para los dos segmentos: propietarios y administradores de restaurantes, y encargados de negocios de distribución de alimentos en fríos. No hay una rama ni un repositorio por rubro. Los dos usan el mismo frontend y el mismo API. En el restaurante la cámara es la de cocina. En la distribución es la cámara o la vitrina del lote.

El código de esa aplicación y este informe se versionan aparte, en la organización `1ASI0572-2620-8725`. Así un cambio del informe no mezcla el historial de la app. El entorno de desarrollo está en 6.1.1, el manejo de ramas y módulos en 6.1.2, el estilo en 6.1.3 y el despliegue en 6.1.4.

6.1.1. Software Development Environment Configuration

En esta sección se describen los productos de software que el equipo utiliza en cada etapa del ciclo de vida de FreshSense: gestión del proyecto, requisitos y diseño, desarrollo del software, desarrollo del dispositivo IoT, pruebas, documentación y despliegue. Es el mismo entorno para los dos segmentos objetivo (propietarios y administradores de restaurantes, y encargados de negocios de distribución de alimentos en frío), porque ambos usan la misma Landing Page, la misma aplicación web y el mismo API.

Project Management y colaboración

Producto	Tipo	Propósito en el proyecto	Ruta de referencia
GitHub (organización `1ASI0572-2620-8725`)	SaaS	Alojamiento de los repositorios del informe, la Landing Page, el frontend y el backend; control de ramas y *pull requests*.	https://github.com/1ASI0572-2620-8725
Git	Software de escritorio	Control de versiones local de cada integrante, con el flujo de ramas descrito en 6.1.2.	https://git-scm.com/downloads
Markdown	Lenguaje de marcado	Redacción del informe del proyecto (`README.md`) directamente en el repositorio.	https://www.markdownguide.org

Requirements Management y diseño

Producto	Tipo	Propósito en el proyecto	Ruta de referencia
Visual Paradigm Online	SaaS	Diagramas C4 (contexto, contenedores y despliegue) del Capítulo IV.	https://online.visual-paradigm.com
Figma	SaaS	Wireframes, mock-ups y prototipo interactivo de la Landing Page y de la aplicación web (Capítulo V).	https://www.figma.com

Software Development

Producto	Tipo	Propósito en el proyecto	Ruta de referencia
Zed	Software de escritorio	Editor usado para revisar el código del frontend y del backend.	https://zed.dev/download
Node.js 20 LTS y npm 10	Entorno de ejecución	Instalación de dependencias y ejecución del servidor de desarrollo y del *build* del frontend.	https://nodejs.org
Angular CLI 20	Framework y CLI	Creación, ejecución (`npm start`, puerto 4200) y compilación (`npm run build`) de la aplicación web.	https://angular.dev/tools/cli
JDK 17	Entorno de ejecución	Compilación y ejecución del backend. En el servidor se ejecuta con Java 21, compatible con el código compilado para Java 17.	https://adoptium.net
Maven Wrapper (`mvnw`)	Herramienta de *build*	Compila y empaqueta los tres servicios del backend en archivos `.jar` sin instalar Maven globalmente.	https://maven.apache.org/wrapper
Spring Boot 3.5.7	Framework	Monolito modular (puerto 8080) y los servicios de alertas (8083) y recetas (8082).	https://spring.io/projects/spring-boot
MySQL Community Server 8	Gestor de base de datos	Bases `freshsense_db`, `alerts_db` y `recipes_db`, una por servicio.	https://dev.mysql.com/downloads/mysql
Docker y Docker Compose	Contenedores	Levantar en local el frontend, el backend y la base de datos juntos para pruebas de integración.	https://docs.docker.com/get-docker

IoT Development

Producto	Tipo	Propósito en el proyecto	Ruta de referencia
Arduino IDE 2 (núcleo ESP32)	Software de escritorio	Escritura, compilación y carga del firmware C++ del ESP32 DevKit C v4 con las librerías DHT sensor library, WiFi, ArduinoJson y HTTPClient.	https://www.arduino.cc/en/software
Wokwi	SaaS	Simulación del circuito ESP32 + DHT22 y validación del firmware antes de usar el hardware físico.	https://wokwi.com

Software Testing y documentación

Producto	Tipo	Propósito en el proyecto	Ruta de referencia
Swagger UI (springdoc-openapi)	Librería	Documentación y prueba de los endpoints REST del backend.	https://springdoc.org
Navegador web (Firefox / Chrome) con DevTools	Software de escritorio	Pruebas de la aplicación web, revisión de peticiones al API y de errores en consola.	https://www.mozilla.org/firefox

Software Deployment

Producto	Tipo	Propósito en el proyecto	Ruta de referencia
GitHub Pages	SaaS	Publicación de la Landing Page.	https://pages.github.com
Google Cloud Compute Engine	IaaS	Máquina virtual donde se ejecutan la aplicación web, el API y la base de datos.	https://cloud.google.com/compute
Nginx	Servidor web	Sirve la aplicación web y actúa como proxy inverso hacia el API.	https://nginx.org
Certbot (Let's Encrypt)	Herramienta CLI	Emisión y renovación automática del certificado HTTPS.	https://certbot.eff.org

Para ejecutar el proyecto en local, cada integrante sigue este orden:

```
# 1. Backend: un terminal por servicio (requiere MySQL en localhost:3306)
cd alerts-service  && ../mvnw spring-boot:run -Dspring-boot.run.profiles=local
cd recipes-service && ../mvnw spring-boot:run -Dspring-boot.run.profiles=local
./mvnw spring-boot:run -Dspring-boot.run.profiles=local      # monolito en :8080

# 2. Frontend
npm install
npm start                                                    # http://localhost:4200
```

En local, el frontend consume el API en `http://localhost:8080/api`, configurado en `src/environments/environment.ts`. Las credenciales de la base de datos y las claves (`JWT_SECRET`, `AES_SECRET`, `OPENAI_API_KEY`) se pasan como variables de entorno o en `application-local.properties`, que no se sube al repositorio.

6.1.2. Source Code Management

El código de la aplicación y el informe no están en el mismo repositorio, para poder versionar cada uno sin pisar al otro.

Repositorio	Visibilidad	Rama estable	Qué guarda
Frontend	Privado	`main`	Aplicación web que usan los dos segmentos
report	Público	`main`	Este informe y las figuras de `Assets/`

En el informe usamos estas ramas:

Rama	Para qué la usamos	Ejemplo
`main`	Versión que ya se puede mostrar	`main`
`develop`	Juntar avances antes de pasarlos a `main`	`develop`
`chapterN-vN`	Cierre de un capítulo	`chapter2-v3`
`feature/...`	Una entrega concreta, sin tocar la versión cerrada	`feature/chapter-6.1-6.1.2-6.2.1.3-6.2.1.6`

El flujo del informe es corto. Se saca la rama `feature` desde la versión del capítulo que estamos usando, se modifica `README.md`, el commit lleva el prefijo `docs:` y se sube esa rama. Recién después de la revisión se integra a `main`.

El frontend está dividido según lo que hace la aplicación, y esos módulos son los mismos para los dos segmentos. En `src/app` están `accounts`, `inventory`, `monitoring`, `alerts`, `recipes`, `reports`, `notifications` y `billing`. `shared` y `core` son soporte, no un rubro distinto. Las rutas de las pantallas están en `src/app/app.routes.ts`: al entrar redirige a `/login`, y con sesión se abre `/inventory`, `/monitoring`, `/devices`, `/alerts`, `/recipes`, `/reports`, `/notifications` y `/settings`.

Dentro de cada módulo de negocio van `domain`, `application`, `infrastructure` y `presentation`. Los textos de pantalla están en `public/i18n/es.json` y `public/i18n/en.json`.

No subimos contraseñas ni llaves. La URL del API va en `src/environments/environment.ts` cuando se corre en local (`http://localhost:8080/api`), en `environment.production.ts` para la app de Azure y en `environment.docker.ts` cuando Nginx publica `/api`.

6.1.3. Source Code Style Guide & Conventions

En esta sección se definen las convenciones de nomenclatura y estilo de código que el equipo aplica en todos los productos de FreshSense: Landing Page, aplicación web, backend (monolito y microservicios de alertas y recetas), Edge API y firmware del dispositivo. Su objetivo es que el código sea legible, consistente y fácil de mantener sin importar qué integrante lo haya escrito. Como regla general, **todos los elementos de código (variables, funciones, clases, archivos, ramas, commits y escenarios de prueba) se nombran en inglés**. El español se reserva para los textos visibles por el usuario, que se gestionan mediante archivos de traducción.

Convenciones generales

Aspecto	Convención	Ejemplo
Idioma del código	Inglés	`pantryItem`, `getFreshnessStatus()`
Indentación	2 espacios (HTML, CSS, JS, TS, C++); 4 espacios (Java)	—
Codificación de archivos	UTF-8, salto de línea LF	—
Longitud de línea	Máximo 100 caracteres (TS/JS/Java); 80 caracteres (C++)	—
Comentarios	Solo donde aportan contexto; en inglés	`// Threshold based on food category`
Formateo automático	Prettier (frontend), formateador del IDE (Java) y *Auto Format* de Arduino IDE, basado en clang-format (C++)	`.prettierrc`, `.clang-format`

HTML (Landing Page y templates de Angular)

Referencias: *HTML Style Guide and Coding Conventions* (W3Schools) y *Google HTML/CSS Style Guide*.

- Elementos y atributos en minúsculas; los valores de atributos siempre entre comillas dobles: `<section class="hero-section">`.
- Uso de etiquetas semánticas (`header`, `nav`, `main`, `section`, `article`, `footer`) en lugar de `div` genéricos.
- Toda imagen incluye el atributo `alt` con una descripción significativa.
- Se declara `<!DOCTYPE html>`, `lang="es"` y las meta tags de `charset` y `viewport` en el Landing Page.
- No se usan estilos en línea (`style="..."`); los estilos van en hojas CSS.
- Identificadores y clases en *kebab-case*: `id="contact-form"`, `class="plan-card"`.

CSS

Referencia: *Google HTML/CSS Style Guide*.

- Nombres de clases en *kebab-case* y descriptivos según su función, no su apariencia: `.alert-banner` en lugar de `.red-box`.
- Se adopta la metodología **BEM** para componentes del Landing Page: `.pricing-card`, `.pricing-card__title`, `.pricing-card--featured`.
- Colores, tipografías y espaciados se definen como variables CSS (`--color-primary`, `--spacing-md`) siguiendo la guía de estilos del Capítulo V.
- Diseño responsive con enfoque *mobile-first* y media queries en orden ascendente.
- Se evita el uso de `!important` y de selectores por ID para estilos.

JavaScript (Landing Page)

Referencia: *Google JavaScript Style Guide*.

- Variables y funciones en *camelCase*: `submitContactForm()`, `isFormValid`.
- Constantes globales en *UPPER_SNAKE_CASE*: `MAX_MESSAGE_LENGTH`.
- Uso de `const` por defecto y `let` solo cuando el valor cambia; no se usa `var`.
- Siempre se usa punto y coma y comparación estricta (`===`).
- Archivos en *kebab-case*: `contact-form.js`, `language-switcher.js`.

TypeScript y Angular (Frontend Web Application)

Referencias: *Angular Coding Style Guide* y *Google TypeScript Style Guide*.

- **Archivos** en *kebab-case* con sufijo según su tipo: `product-list.component.ts`, `monitoring.service.ts`, `auth.guard.ts`.
- **Clases, interfaces y enums** en *PascalCase*: `ProductListComponent`, `MonitoringService`, `FreshnessStatus`.
- **Propiedades y métodos** en *camelCase*: `loadProducts()`, `latestReading`.
- **Selectores de componentes** con el prefijo `app-`: `<app-monitoring-panel>`.
- **Una responsabilidad por archivo**: un componente, servicio o modelo por archivo.
- **Organización por bounded context y por capa**, alineada con el diseño táctico del Capítulo IV. Los módulos de negocio son `accounts`, `inventory`, `monitoring`, `alerts`, `recipes`, `reports`, `notifications` y `billing`; `core` y `shared` contienen lo transversal:

```
src/app/
├── core/                   # interceptores, guards, configuración global
├── shared/                 # componentes y utilidades reutilizables
└── <bounded-context>/      # accounts, inventory, monitoring, alerts, recipes...
    ├── domain/             # modelos y reglas del contexto
    ├── application/        # servicios de aplicación y casos de uso
    ├── infrastructure/     # clientes HTTP hacia el API
    └── presentation/       # componentes y páginas
```

- Las rutas de la aplicación se declaran en `src/app/app.routes.ts`, en *kebab-case* y en inglés: `/login`, `/monitoring`, `/alerts`, `/inventory`, `/devices`.
- Uso de **Angular Material** como única librería de componentes de UI, según las restricciones del proyecto.
- Los textos visibles se externalizan en `public/i18n/es.json` y `public/i18n/en.json`; las claves se escriben en inglés y en *dot.case*: `"inventory.list.title"`.
- La URL del API nunca se escribe en los componentes: se lee de `src/environments/` (`environment.ts`, `environment.production.ts`, `environment.docker.ts`).
- Se usa tipado explícito y se evita `any`.

Java y Spring Boot (monolito, alerts-service y recipes-service)

Referencias: *Google Java Style Guide* y *Spring Boot Features*.

- Paquetes en minúsculas, organizados por bounded context y capa: `<paquete-base>.inventory.domain.model`, `<paquete-base>.monitoring.interfaces.rest`.
- Clases en *PascalCase* con sufijo según su rol: `Product`, `ProductCommandService`, `ProductsController`, `ProductResource`.
- Métodos y atributos en *camelCase*; constantes en *UPPER_SNAKE_CASE*.
- Endpoints REST en inglés, con recursos en plural y en *kebab-case* bajo el prefijo `/api`: `/api/products`, `/api/alerts`, `/api/edge/readings`.
- Tablas de base de datos en *snake_case* y en plural: `products`, `sensor_readings`. Cada servicio tiene su propia base (`freshsense_db`, `alerts_db`, `recipes_db`).
- Los clientes Feign entre servicios se nombran con el sufijo `Client`: `AlertsServiceClient`, `RecipesServiceClient`.
- Las claves y credenciales (`JWT_SECRET`, `AES_SECRET`, `OPENAI_API_KEY`, datos de la base) se leen de variables de entorno; nunca se escriben en el código ni en `application.properties` versionado.

C++ (firmware del dispositivo FreshSense – ESP32 + DHT22)

Referencias: *Google C++ Style Guide* y *Barr Group Embedded C Coding Standard* (BARR-C:2018).

- **Archivos** en *snake_case*, con un `.h` y un `.cpp` por módulo; el sketch principal es `freshsense_device.ino`: `dht_sensor.h` / `dht_sensor.cpp`, `edge_client.cpp`.
- **Clases, structs y enums** en *PascalCase*: `DhtSensor`, `SensorReading`, `DeviceState`.
- **Funciones y métodos** en *camelCase*, empezando con un verbo: `readTemperature()`, `sendReading()`.
- **Variables locales y parámetros** en *camelCase*: `humidityPct`, `lastReadingMs`. Los **miembros privados** llevan el sufijo `_`: `pin_`, `intervalMs_`.
- **Constantes y configuración** en *UPPER_SNAKE_CASE*, con `constexpr` en un archivo `config.h`: `READING_INTERVAL_MS`, `EDGE_API_URL`. Los pines se nombran por su función: `DHT_SENSOR_PIN`.
- Las **unidades de medida van en el nombre** de la variable: `temperatureC`, `humidityPct`, `timeoutMs`.
- **Guardas de inclusión** con `#pragma once` en todos los headers.
- **Tipos de ancho fijo** (`uint8_t`, `int16_t`, `uint32_t`) en lugar de `int` o `long`, para que el tamaño de los datos no dependa de la placa.
- No se usan `delay()` bloqueantes en el `loop()`; la temporización se maneja con `millis()`. Se evita la memoria dinámica (`new`, `malloc`) y los objetos `String` en el ciclo principal.
- El JSON que se envía al Edge API se arma con **ArduinoJson**, con claves en *camelCase* iguales a las del backend: `temperatureC`, `humidityPct`.
- Las credenciales (Wi-Fi y `X-Device-Key`) van en `secrets.h`, excluido con `.gitignore`; solo se versiona `secrets.example.h` como plantilla.

```
// dht_sensor.h
#pragma once
#include <stdint.h>

constexpr uint32_t READING_INTERVAL_MS = 5000;

class DhtSensor {
 public:
  explicit DhtSensor(uint8_t pin);
  float readTemperatureC();
  float readHumidityPct();

 private:
  uint8_t pin_;
};
```

Gherkin (Acceptance Tests)

Referencia: *Gherkin Conventions for Readable Specifications*.

- Archivos `.feature` en inglés y en *kebab-case*, uno por User Story: `us07-freshness-dashboard.feature`.
- Cada `Feature` lleva el ID de la User Story en su título.
- Los escenarios se escriben desde la perspectiva del usuario con la estructura `Given` / `When` / `Then`, y `And` para pasos adicionales.
- Se usa `Scenario Outline` con `Examples` cuando un escenario se repite con distintos datos.

```
Feature: US07 - Freshness traffic-light dashboard
  As a restaurant manager or cold-food distribution manager
  I want to see the cold room status by color
  So that I can act before the products spoil

  Scenario: Cold room above the safe temperature is shown in red
    Given the user is logged in
    And the latest reading of the cold room is above the safe temperature
    When the user opens the monitoring panel
    Then the cold room is displayed with a red indicator
```

Convenciones de commits

Los mensajes de commit siguen **Conventional Commits** (detallado en la sección 6.1.2): `feat:`, `fix:`, `docs:`, `style:`, `refactor:`, `test:`, `chore:`. Ejemplo: `feat(monitoring): add traffic-light status to latest reading`.

6.1.4. Software Deployment Configuration

En esta sección se describe la configuración con la que se publica cada producto de FreshSense. La Landing Page se publica en GitHub Pages. La aplicación web, el API y la base de datos se publican en una máquina virtual de Google Cloud, detrás de Nginx y con HTTPS. Así, el navegador y el dispositivo IoT se comunican con el sistema por un único dominio seguro.

Resumen de despliegue

Producto	Plataforma	URL pública
Landing Page	GitHub Pages	https://1asi0572-2620-8725.github.io/landing-page/
Aplicación web (Angular)	Google Cloud Compute Engine + Nginx	https://35-224-123-160.sslip.io
API REST (Spring Boot)	Google Cloud Compute Engine + Nginx (proxy inverso)	https://35-224-123-160.sslip.io/api
Documentación del API	Swagger UI	https://35-224-123-160.sslip.io/swagger-ui/index.html
Base de datos	MySQL 8 en la misma máquina virtual (sin acceso público)	—

Landing Page

1. El repositorio `landing-page` contiene el sitio estático (HTML, CSS y JavaScript).
2. En *Settings* → *Pages* se configuró la publicación desde la rama principal del repositorio.
3. Cada *push* a `main` vuelve a publicar el sitio automáticamente en la URL indicada.

Infraestructura en Google Cloud

Elemento	Configuración
Servicio	Compute Engine, máquina e2-micro (2 vCPU compartidas, 1 GB de RAM), zona `us-central1-f`.
Sistema operativo	Rocky Linux 9, con 2 GB de swap para compensar la memoria limitada.

Elemento	Configuración
Software instalado	Java 21 (OpenJDK), MySQL 8, Nginx y Certbot.
Firewall	Reglas de la VPC y `firewalld` que abren los puertos 80 (redirige a HTTPS) y 443 (HTTPS). El puerto 8080 se mantiene abierto temporalmente para probar el API y Swagger de forma directa durante el desarrollo; MySQL solo escucha en la propia máquina.
Dominio	`35-224-123-160.sslip.io`, un nombre gratuito que resuelve a la IP externa de la máquina y permite emitir un certificado sin comprar dominio.
Certificado	Let's Encrypt emitido con Certbot (plugin de Nginx), con renovación automática programada.

Backend (API REST)

1. Se compilan los tres servicios en local con `./mvnw clean package -DskipTests`, que genera `backend.jar`, `alerts-service.jar` y `recipes-service.jar`.
2. Los archivos se suben a `/opt/freshsense` en la máquina virtual.
3. Las variables de entorno se definen en `/opt/freshsense/env.sh`, con permisos solo para el usuario del servicio. No se suben al repositorio.

Variable	Uso
`DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`	Conexión a MySQL local.
`JWT_SECRET`, `AES_SECRET`	Firma de tokens JWT y cifrado AES-256 de datos sensibles.
`ALERTS_SERVICE_URL`, `RECIPES_SERVICE_URL`	URL directa de los servicios de alertas (`http://localhost:8083`) y recetas (`http://localhost:8082`) que consume el monolito mediante Feign.
`CORS_EXTRA_ORIGINS`	Orígenes adicionales permitidos por CORS (`https://35-224-123-160.sslip.io`).
`OPENAI_API_KEY`	Clave del servicio de recetas para la generación con IA. Solo vive en el servidor, nunca en el frontend.

4. Cada servicio se inicia con un *script* que carga `env.sh` y ejecuta `java -jar` con un límite de memoria (`-Xmx`) acorde a la máquina: el monolito en el puerto 8080, alertas en el 8083 y recetas en el 8082.
5. Hibernate (`ddl-auto=update`) crea o actualiza las tablas al arrancar, y el servicio publica su documentación en Swagger UI.

Aplicación web (Angular)

1. En `src/environments/environment.production.ts` se configura `apiBaseUrl: 'https://35-224-123-160.sslip.io/api'`.
2. Se compila con `npm run build -- --configuration=production`, que genera `dist/frontend/browser`.
3. El contenido de esa carpeta se copia a `/var/www/freshsense` en la máquina virtual.

Nginx (servidor web y proxy inverso)

Nginx publica en un solo dominio la aplicación web y el API, lo que evita problemas de contenido mixto y de cookies entre dominios:

Ruta	Destino
`/`	Archivos estáticos de Angular, con `try_files ... /index.html` para que funcionen las rutas de la SPA.
`/api/`, `/swagger-ui/`, `/v3/api-docs`	Proxy inverso a `http://127.0.0.1:8080`, reenviando las cabeceras `Host`, `X-Forwarded-For` y `X-Forwarded-Proto`.
Puerto 80	Redirección permanente (301) a HTTPS.

Dispositivo IoT

1. El firmware del ESP32 se configura con la red Wi-Fi del local y la dirección del Edge API (sección 5.6).
2. Se carga en la placa desde Arduino IDE por USB, después de validarlo en Wokwi.
3. El Edge API reenvía cada lectura al backend mediante `POST https://35-224-123-160.sslip.io/api/edge/readings`, autenticándose con la cabecera `X-Device-Key` que se obtiene al registrar el dispositivo en la aplicación.

6.2. Landing Page, Services & Applications Implementation

En esta sección se presenta la evidencia del proceso de implementación, pruebas, documentación y despliegue de los productos digitales de FreshSense, organizada por Sprint. Cada Sprint incluye su planificación, la distribución de liderazgo entre los integrantes, el backlog trabajado y la evidencia de desarrollo, ejecución, documentación de servicios, despliegue y colaboración del equipo.

En el Sprint 1 se construyó el primer recorrido completo de la solución, el mismo para los dos segmentos objetivo:

- **Landing Page** (HTML5, CSS3 y JavaScript), publicada en GitHub Pages, con la propuesta de valor, el formulario de demo y el acceso a la aplicación.
- **Frontend Web Application** (Angular 20 con Angular Material), con registro, ingreso, monitoreo, alertas, inventario y dispositivos.
- **Web Services** en Spring Boot: un monolito modular (puerto 8080) y dos microservicios, **alerts-service** (8083) y **recipes-service** (8082), cada uno con su propia base de datos MySQL. El recipes-service consume un servicio externo de inteligencia artificial (OpenAI) para generar recetas a partir de los insumos disponibles.
- **Prototipo del dispositivo IoT simulado en Wokwi** (ESP32 + DHT22, firmware en C++), que junto con el **Edge API** envía las lecturas de temperatura y humedad al backend mediante `POST /api/edge/readings`. El prototipo físico se implementará en un Sprint posterior.

La aplicación web, el API y la documentación en Swagger UI están desplegados en Google Cloud Compute Engine detrás de Nginx con HTTPS (sección 6.1.4).

6.2.1. Sprint 1

En el sprint 1 queremos que una persona de cualquiera de los dos segmentos pueda entender FreshSense, pedir una demo y entrar a ver la cámara: la lectura, el color y la alerta. Los segmentos son propietarios y administradores de restaurantes, y encargados de negocios de distribución de alimentos en fríos.

6.2.1.1. Sprint Planning 1

El equipo FreshEat decidió que el primer Sprint debía validar de punta a punta la propuesta central de FreshSense: que la información de la cámara de frío llegue desde el dispositivo hasta el usuario y lo alerte a tiempo. Para lograrlo se incluyó en el alcance el Landing Page, que presenta la solución a los visitantes y los dirige a la aplicación; el registro e ingreso a la aplicación web; un panel de monitoreo con semáforo de frescura y alertas preventivas; un conjunto de endpoints para autenticación, ingesta de telemetría y alertas, que los desarrolladores del frontend y del Edge API pueden usar; y una primera versión del prototipo físico (ESP32 + DHT22) integrada con el Edge API. Con este criterio se tomaron, en orden de prioridad, las 11 primeras historias del Product Backlog.

Sprint #	**Sprint 1**
Sprint Planning Background	
Date	20/09/2026
Time	10:00 pm
Location	Reunión virtual vía Discord
Prepared By	Romina Tuesta Marin
Attendees (to planning meeting)	Chavez Viera Joseph Manuel / Juan Carlos Pastor Napa / Mostajo Orosco Maria Fernanda / Cossar Sanchez Eduardo Jose / Tuesta Marin Romina Alejandra
Previous Sprint Review Summary	No aplica. El Sprint 1 es el primer Sprint de implementación del proyecto, por lo que no existe un incremento previo que revisar.
Previous Sprint Retrospective Summary	No aplica. Al ser el primer Sprint, no existe una retrospectiva previa. Para este Sprint el equipo acordó trabajar con GitFlow, Conventional Commits y Pull Requests revisados por al menos un integrante
Sprint Goal & User Stories	
Sprint 1 Goal	Nuestro enfoque es proporcionar a los propietarios y administradores de restaurantes, así como a los responsables de distribución de productos refrigerados que visiten el sitio web, información clara sobre FreshSense y un acceso directo a la plataforma; proporcionarles una aplicación web donde puedan registrarse, iniciar sesión y monitorear la temperatura y humedad de sus cámaras frigoríficas mediante un estado tipo semáforo y alertas preventivas; proporcionar a los desarrolladores de frontend y edge endpoints seguros para autenticación, recepción de telemetría y alertas; y proporcionar a los fabricantes de dispositivos un primer prototipo físico (ESP32 + DHT22) integrado con la API Edge.

Creemos que esto proporciona mayor confianza a los visitantes para solicitar una demostración o crear una cuenta; una visibilidad temprana y confiable de las condiciones de la cadena de frío que reduzca las verificaciones manuales de temperatura para los responsables de restaurantes y distribución; una base documentada para que los desarrolladores puedan construir nuevas funcionalidades sobre la API; y un flujo de datos de extremo a extremo validado, desde el dispositivo hasta la plataforma.

Esto se confirmará cuando un visitante llegue a la página de registro desde el llamado a la acción (CTA) de la Landing Page en no más de dos clics; una lectura enviada por el dispositivo físico se muestre en el panel de monitoreo con su color correspondiente al semáforo en menos de 10 segundos; una lectura fuera del rango seguro genere una alerta visible en la vista de alertas; los endpoints de autenticación, telemetría y alertas respondan correctamente desde la interfaz Swagger desplegada; y las 11 historias de usuario del Sprint cumplan con sus escenarios de aceptación en el entorno desplegado. | | Sprint 1 Velocity | 37 Story Points | | Sum of Story Points | 37 Story Points |

User Stories incluidos en el Sprint 1

#	ID	Título	Story Points	Aporte al Sprint Goal
1	US01	Visualización de propuesta B2B	2	Información clara de la solución para los visitantes.
2	US02	Sección para Restaurantes y Negocios en Frío	2	Información específica para cada segmento objetivo.
3	US03	Formulario de contacto y demos B2B	2	Conversión del visitante en un contacto comercial.
4	US04	Call to Action (CTA) Corporativo	2	Camino directo del Landing Page a la aplicación.
5	US05	Adaptabilidad en móviles y tablets	3	Acceso desde tablets de cocina y teléfonos.
6	TS43	API Gestor de Autenticación y RBAC B2B	5	Registro, ingreso y endpoints seguros.
7	TS41	API Ingesta de Sensores IoT B2B	5	Endpoint de telemetría para el Edge API.
8	US06	Telemetría y monitoreo IoT de cámaras frías	5	Flujo de datos del prototipo físico a la plataforma.
9	US07	Dashboard con semáforo comercial	3	Estado de la cámara visible con un color.
10	US08	Alertas preventivas de temperatura	5	Aviso oportuno cuando se pierde la cadena de frío.
11	TS42	Microservicio de Notificaciones de Emergencia	3	Envío de las alertas a los usuarios.
	Total		**37**	

La velocidad de 37 Story Points se fijó en función de la capacidad del equipo de cinco integrantes durante las dos semanas del Sprint. El detalle de las tareas de cada historia está en el Sprint Backlog 1 (sección 6.2.1.3).

6.2.1.2. Aspect Leaders and Collaborators

Para el Sprint 1, el equipo organizó el trabajo en aspectos que siguen el recorrido del usuario y los productos del Sprint: el Landing Page (US01 a US05); la gestión de cuentas, con registro, ingreso y roles (TS43); la ingesta IoT, que abarca el firmware, el Edge API y el endpoint de lecturas (TS41, US06); el monitoreo y las alertas, con el semáforo, las alertas preventivas y el servicio de notificaciones (US07, US08, TS42); y el despliegue de todos los productos. Cada aspecto tiene un líder (L), responsable de coordinarlo y asegurar su calidad, y colaboradores (C), que participan en su implementación. Esta distribución sirvió de base para asignar las tareas del Sprint Backlog 1.

Team Member (Last Name, First Name)	GitHub Username	Landing Page	Accounts (IAM)	IoT (Device prototype)	Monitoring & Alerts	Deployment
Tuesta Marin, Romina	Romimi1	L	C	L	C	C
Chavez Viera, Joseph	u202314019-MrOsoPanda	C	L	C	C	C
Pastor Napa, Juan Carlos	ElKiwi1271	C	C	C	C	L
Mostajo Orosco, Maria Fernanda	Mafer-m30	C	C	C	L	C
Cossar Sanchez, Eduardo	coleeeee-dev	C	C	C	C	C

6.2.1.3. Sprint Backlog 1

Del Product Backlog se tomaron las primeras 11 historias. Suman 37 puntos. El sprint 1 no arma dos productos. Arma el mismo recorrido para propietarios y administradores de restaurantes y para encargados de negocios de distribución de alimentos en fríos:

1. Entrar a la misma aplicación, por registro o por login (US01 a US05 y TS43). En el código eso es `/register` y `/login`.
2. Recibir la lectura del sensor de la cámara (TS41). En el API es `POST /api/edge/readings` y `POST /api/monitoring`.
3. Ver esa lectura con color, y la alerta si el frío se pierde (US06, US07, US08 y TS42). En la app son `/monitoring` y `/alerts`.

En el restaurante la cámara es la de cocina y el producto es un insumo. En la distribución la cámara es la de la mercadería y el producto es un lote. Las fichas de platillo y el costeo no entran aquí: son del restaurante y van después en el backlog.

Orden	ID	Título	Puntos	A quién le sirve
1	US01	Visualización de propuesta B2B	2	Los dos segmentos
2	US02	Sección para Restaurantes y Negocios en Frío	2	Los dos segmentos
3	US03	Formulario de contacto y demos B2B	2	Los dos segmentos
4	US04	Call to Action (CTA) Corporativo	2	Los dos segmentos
5	US05	Adaptabilidad en móviles y tablets	3	Los dos segmentos
6	TS43	API Gestor de Autenticación y RBAC B2B	5	Los dos segmentos
7	TS41	API Ingesta de Sensores IoT B2B	5	Los dos segmentos
8	US06	Telemetría y monitoreo IoT de cámaras frías	5	Los dos segmentos
9	US07	Dashboard con semáforo comercial	3	Los dos segmentos
10	US08	Alertas preventivas de temperatura	5	Los dos segmentos
11	TS42	Microservicio de Notificaciones de Emergencia	3	Los dos segmentos

Una historia de esta lista se cierra cuando se cumplen sus tres escenarios. Si en la revisión falta uno, se deja anotado cuál y por qué no se pudo probar.

6.2.1.4. Development Evidence for Sprint Review

Durante el Sprint 1 de FreshSense, el equipo trabajó en la implementación de los productos digitales necesarios para establecer las funcionalidades iniciales de la solución. Los avances comprenden el Landing Page, desarrollado con HTML, CSS y JavaScript, y la aplicación web, desarrollada con Angular. Asimismo, se contempló la integración con los servicios Backend y los componentes IoT encargados de la recepción y procesamiento de lecturas de temperatura y humedad.

En el Landing Page se incorporaron las secciones informativas de FreshSense, recursos visuales, elementos de navegación, botones de acceso a la plataforma y archivos de traducción. Estas funcionalidades se relacionan con las historias US01, US02, US03, US04 y US05 del Sprint Backlog.

En la aplicación web se desarrollaron componentes para el registro e inicio de sesión, gestión de inventario, monitoreo de sensores IoT y visualización de alertas. También se incorporaron servicios para la comunicación con el API REST, configuraciones de entorno y mecanismos de navegación. Estos avances se relacionan principalmente con las historias TS43, US06, US07 y US08.

Para mantener la trazabilidad del desarrollo, se presenta la siguiente tabla con los commits registrados en los repositorios de código fuente. Algunos corresponden a componentes preparados antes de la reunión de planificación del Sprint 1, que fueron incorporados como base del incremento desarrollado durante esta iteración.

Tabla. Commits relacionados con la implementación del Sprint 1

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
1ASI0572-2620-8725/Frontend	main	5615ceb	feat: initial files	This are the initial files for the frontend - first commit	02/09/2026
1ASI0572-2620-8725/Frontend	main	a0684d8	feat: app file	Sin cuerpo de mensaje	02/09/2026
1ASI0572-2620-8725/Frontend	main	9814e44	feat: rest of the app	Sin cuerpo de mensaje	02/09/2026
1ASI0572-2620-8725/landing-page	main	477bd20	first commit	Sin cuerpo de mensaje	07/10/2026
1ASI0572-2620-8725/BackEnd	main	cd732db	first commit	Sin cuerpo de mensaje	08/10/2026

Los commits del Frontend evidencian la incorporación de la estructura inicial del proyecto Angular, los componentes de interfaz, los servicios de comunicación con el Backend y los archivos de configuración necesarios para su ejecución. Por su parte, el commit del Landing Page registra la incorporación de los archivos principales del sitio web, incluyendo su estructura HTML, hojas de estilo, scripts y recursos gráficos.

Evidencia de desarrollo del Backend y componentes IoT

En el repositorio Backend de FreshSense se registró la incorporación del código fuente de los servicios desarrollados con Spring Boot, incluyendo módulos de autenticación, gestión de dispositivos IoT, recepción y almacenamiento de lecturas, monitoreo ambiental y administración de alertas y notificaciones. Asimismo, se incorporaron los servicios complementarios de alertas, recetas y descubrimiento de servicios mediante Eureka. Este commit permite evidenciar la estructura y las funcionalidades implementadas en el Backend, relacionadas principalmente con las historias técnicas TS41, TS42 y TS43, así como con las historias US06, US07 y US08 del Sprint 1.

6.2.1.5. Testing Suite Evidence for Sprint Review

Durante el Sprint 1 de FreshSense, el equipo inició la especificación de pruebas para evaluar las funcionalidades principales de la solución, considerando las historias de usuario y las historias técnicas establecidas en el Sprint Backlog. Las pruebas están orientadas a verificar el comportamiento del Landing Page, la autenticación de usuarios, la recepción de datos provenientes de sensores IoT, el monitoreo de condiciones ambientales y la gestión de alertas.

Para definir los escenarios de aceptación se utilizó el enfoque Behavior-Driven Development (BDD), empleando el lenguaje Gherkin y la estructura Given-When-Then. Esta aproximación permite describir el comportamiento esperado de las funcionalidades y mantener la trazabilidad entre los requisitos y sus correspondientes pruebas.

Las especificaciones se encuentran almacenadas en un repositorio independiente de GitHub, organizado mediante archivos `.feature`. Actualmente se dispone de 14 archivos de especificación. Algunos de ellos corresponden a funcionalidades de otros incrementos o utilizan identificadores anteriores del Product Backlog, por lo que requieren ajustes para mantener coherencia con el alcance vigente del Sprint 1.

Repositorio de pruebas: FreshSense Acceptance Tests

Archivos del Sprint: Sprint 01

Relación de pruebas de aceptación

La siguiente tabla presenta las especificaciones existentes relacionadas con el Sprint 1 y su correspondencia con las historias definidas en el Sprint Backlog.

Story ID	Funcionalidad evaluada	Archivo de prueba existente
US01	Visualización de propuesta B2B	`us01-visualizacion-propuesta.feature`
US02	Sección para Restaurantes y Negocios en Frío	`us02-seccion-pequenos-negocios.feature`
US03	Formulario de contacto y demos B2B	`us03-formulario-contacto.feature`
US04	Call to Action Corporativo	`us04-cta-suscripcion.feature`
US05	Adaptabilidad en móviles y tablets	`us05-compatibilidad-movil.feature`
TS43	API Gestor de Autenticación y RBAC B2B	`ts33-api-usuarios.feature`
TS41	API Ingesta de Sensores IoT B2B	`ts31-api-sensores.feature`
US07	Dashboard con semáforo comercial	`us07-visualizacion-inventario.feature`
US08	Alertas preventivas de temperatura	`us08-alertas-anticipadas.feature`

Los escenarios publicados permiten establecer una base para la validación funcional del producto. Sin embargo, las especificaciones que no coinciden con las historias actuales deberán actualizarse antes de considerarse evidencia definitiva de aceptación del Sprint.

Especificaciones de pruebas mediante Gherkin

A continuación, se presentan ejemplos del código Gherkin publicado en el repositorio de pruebas, que describen comportamientos esperados de las funcionalidades del Sprint 1.

US01 – Visualización de la propuesta de FreshSense

Archivo: us01-visualizacion-propuesta.feature

```
Feature: US01 - Visualizacion de la propuesta de FreshSense

    Scenario: Visitante entiende la propuesta de valor
      Given un visitante accede a la landing page de FreshSense
      When visualiza la seccion principal
      Then ve el mensaje principal con la propuesta de valor de la solucion
      And puede identificar claramente el beneficio del producto
```

Esta especificación describe el comportamiento esperado del Landing Page cuando un visitante accede a la sección principal e identifica la propuesta de valor de FreshSense.

US03 – Formulario de contacto

Archivo: us03-formulario-contacto.feature

```
Feature: US03 - Formulario de contacto

    Scenario: Visitante envia formulario de contacto
      Given un visitante esta en la landing page de FreshSense
      When completa el formulario con nombre, email y mensaje
      And hace clic en el boton de enviar
      Then el sistema registra la solicitud de contacto
      And muestra un mensaje de confirmacion al usuario
```

El escenario define la validación del formulario de contacto, verificando que el visitante pueda completar sus datos y que el sistema registre la solicitud y presente una confirmación. Su ejecución satisfactoria deberá comprobarse sobre el sitio implementado.

US05 – Compatibilidad móvil

Archivo: us05-compatibilidad-movil.feature

```
Feature: US05 - Compatibilidad movil

    Scenario: Usuario accede desde un dispositivo movil
      Given un usuario accede a la landing page desde un smartphone
      When carga la pagina en pantalla pequena
      Then todos los elementos se adaptan correctamente al tamaño de pantalla
      And el contenido es legible sin necesidad de zoom horizontal

    Scenario: Usuario accede desde una tablet
      Given un usuario accede a la landing page desde una tablet
      When carga la pagina
      Then el layout se adapta correctamente al tamaño de pantalla
```

Estos escenarios describen la adaptación de la interfaz del Landing Page a dispositivos móviles y tablets, buscando garantizar la legibilidad y correcta distribución de sus componentes visuales.

Evidencia de commits relacionados con Testing

La siguiente tabla presenta los commits registrados en el repositorio de pruebas de aceptación, permitiendo identificar los avances asociados a la preparación de especificaciones del Sprint 1.

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
1ASI0572-2620-8725/Acceptance-Tests	main	fbf1a89	upload: gherkin tests	Sin cuerpo de mensaje	08/10/2026

El commit presentado evidencia la incorporación de 14 archivos `.feature` organizados dentro del directorio `Sprint 01`. Estos archivos documentan escenarios de aceptación de diferentes funcionalidades del producto.

6.2.1.6. Execution Evidence for Sprint Review

La aplicación es una sola para los dos segmentos. No hay una pantalla de restaurante y otra de distribución. El propietario o administrador del restaurante y el encargado del negocio de distribución de alimentos en fríos entran al mismo registro, al mismo monitoreo, a las mismas alertas y al mismo inventario. Lo que cambia es la lectura: cocina e insumos en el restaurante, cámara o vitrina y lote por entregar en la distribución.

En esta parte van tres enlaces. El back, la landing y el front ya están publicados.

Dónde	Enlace
Back	https://35-224-123-160.sslip.io/swagger-ui/index.html
Landing	https://1asi0572-2620-8725.github.io/landing-page/

Dónde	Enlace
Front	https://35-224-123-160.sslip.io/login

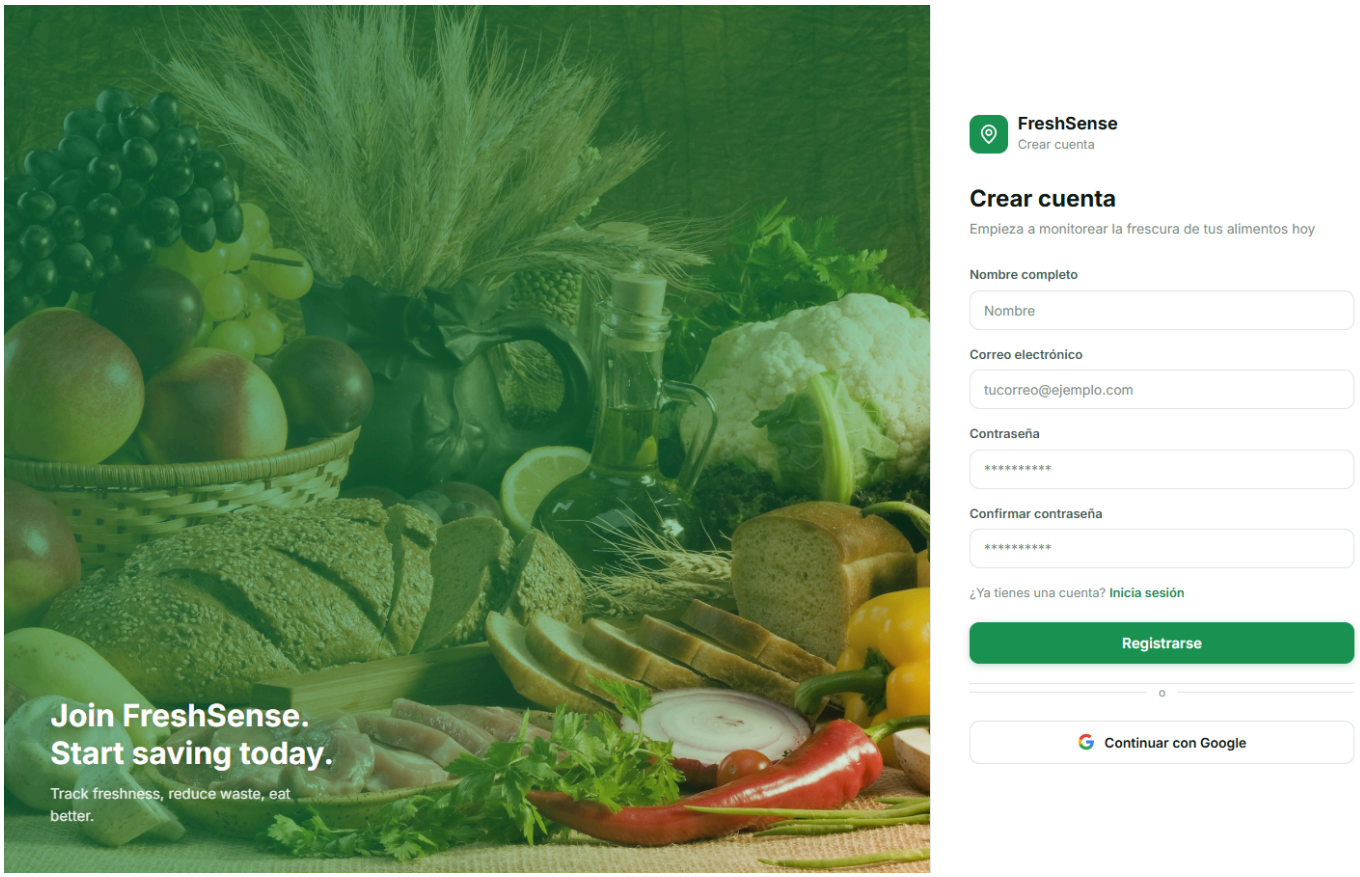
Landing ya desplegada. Desde ahí se entra a la aplicación. La misma página es la puerta para el propietario o administrador del restaurante y para el encargado de la distribución de alimentos en fríos.

El recorrido de la aplicación es uno solo. Se hace igual si quien entra es propietario o administrador de un restaurante, o encargado de un negocio de distribución de alimentos en fríos.

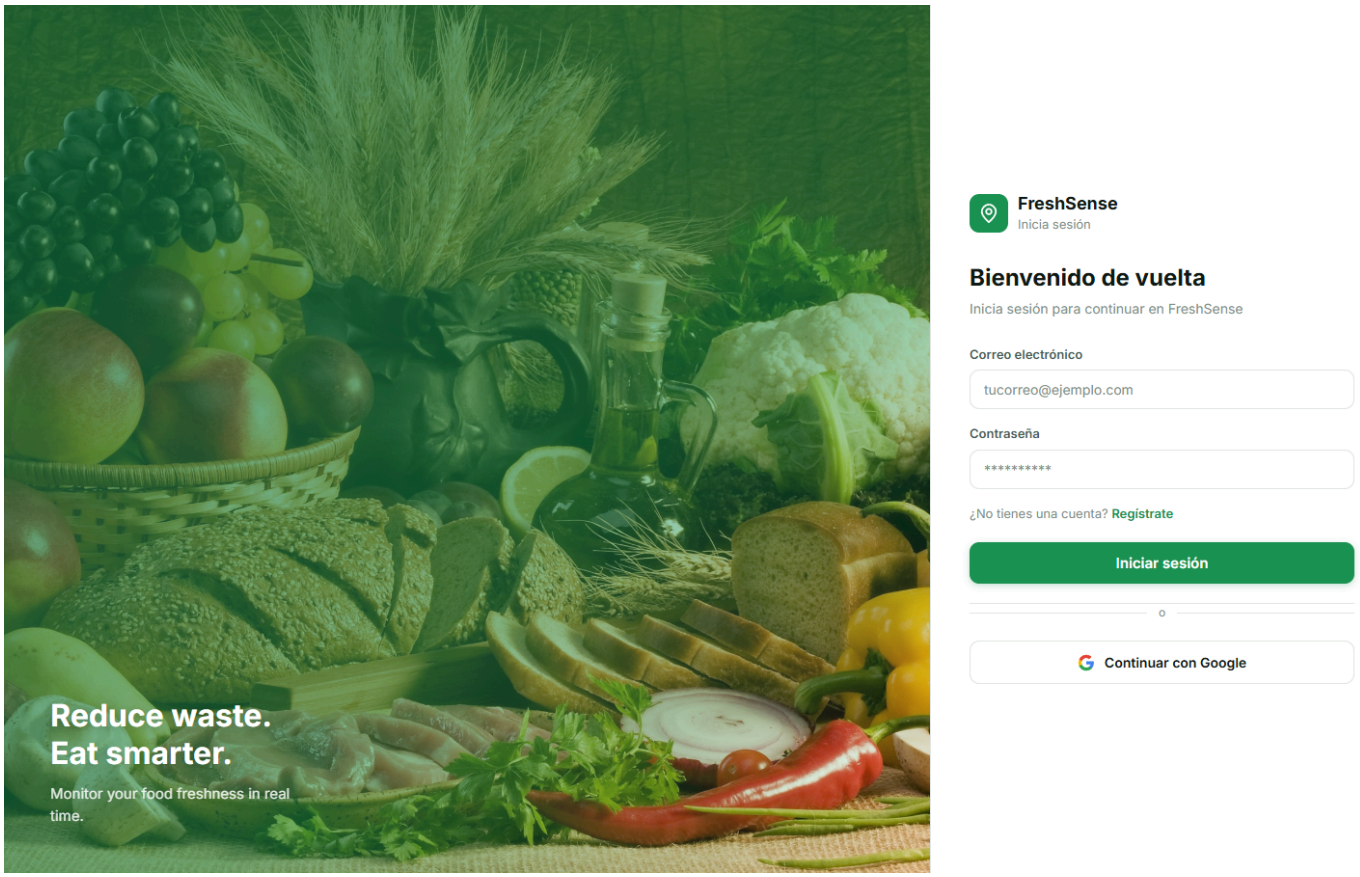
1. `/register` crea la cuenta. Si ya existe, `/login`. En Swagger: `POST /api/accounts/register` y `POST /api/accounts/login`.
2. `/monitoring` muestra temperatura, humedad y el color verde, amarillo o rojo. En Swagger: `GET /api/monitoring/latest` y `POST /api/monitoring`. El sensor manda la lectura con `POST /api/edge/readings`.
3. `/alerts` lista el aviso si la cámara está en rojo. En Swagger: `GET /api/alerts` y `POST /api/alerts`.
4. `/inventory` muestra el producto, la cantidad y el vencimiento. En Swagger: `GET /api/products` y `POST /api/products`.
5. `/devices` empareja el sensor de esa cámara. En Swagger: `POST /api/devices`.

Pantalla	Ruta	Operación	Cómo lo lee el restaurante	Cómo lo lee la distribución en fríos
Registro	`/register`	`POST /api/accounts/register`	Alta del administrador del local	Alta del encargado del negocio
Ingreso	`/login`	`POST /api/accounts/login`	Entra al turno de cocina	Entra a revisar la mercadería
Monitoreo	`/monitoring`	`GET /api/monitoring/latest`	Cámara de cocina	Cámara o vitrina del lote
Alertas	`/alerts`	`GET /api/alerts`	El frío de los insumos se perdió	El lote no está para salir
Inventario	`/inventory`	`GET /api/products`	Insumo, cantidad y vencimiento	Lote, cantidad y vencimiento
Dispositivos	`/devices`	`POST /api/devices`	Sensor de la cocina	Sensor de la cámara de distribución

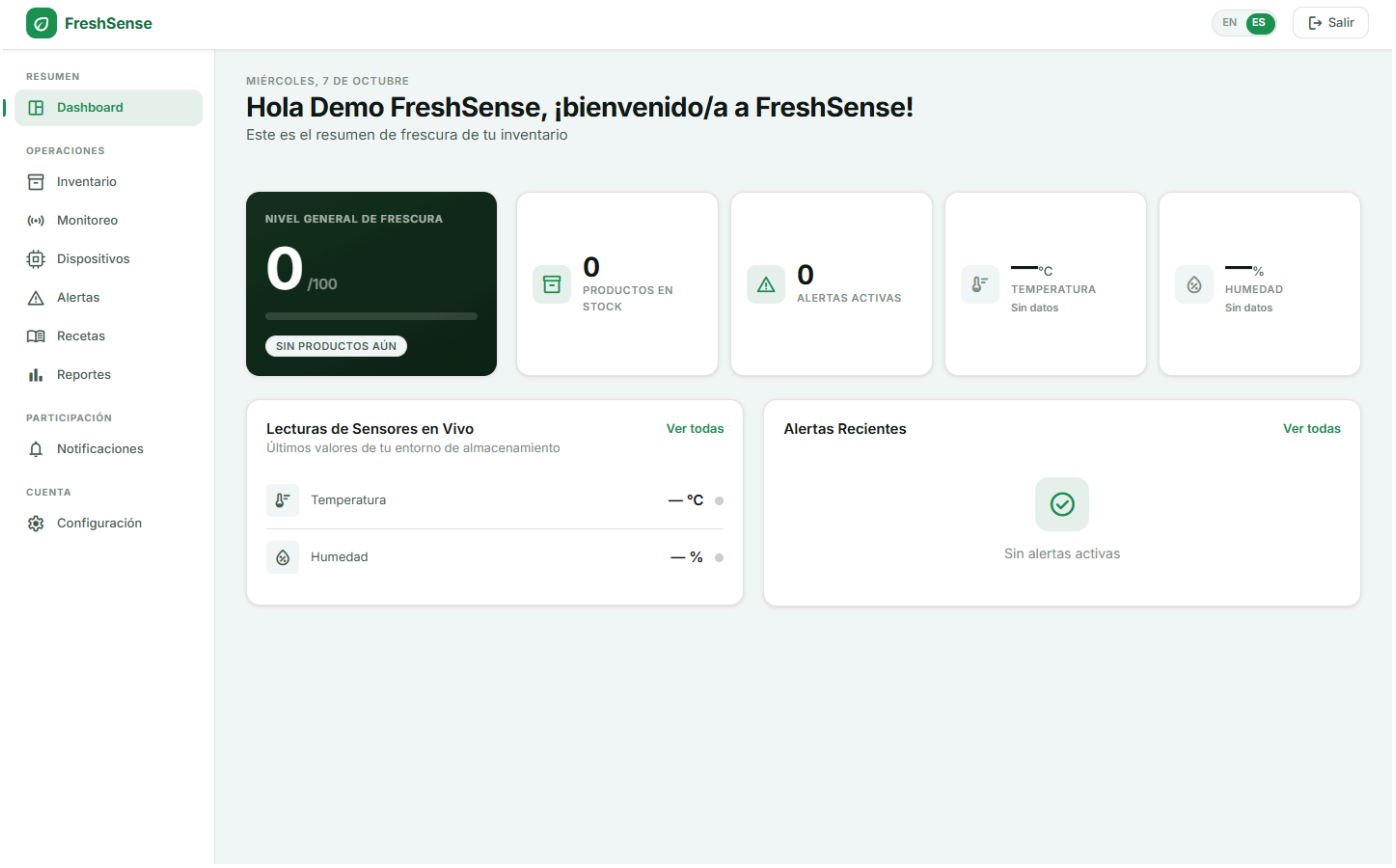
Las capturas de abajo son del recorrido de la aplicación. El front publicado está en https://35-224-123-160.sslip.io/login. La cuenta demo no tenía productos ni una lectura reciente: monitoreo, alertas e inventario se ven vacíos, y sin lectura reciente esa fila no queda en verde.



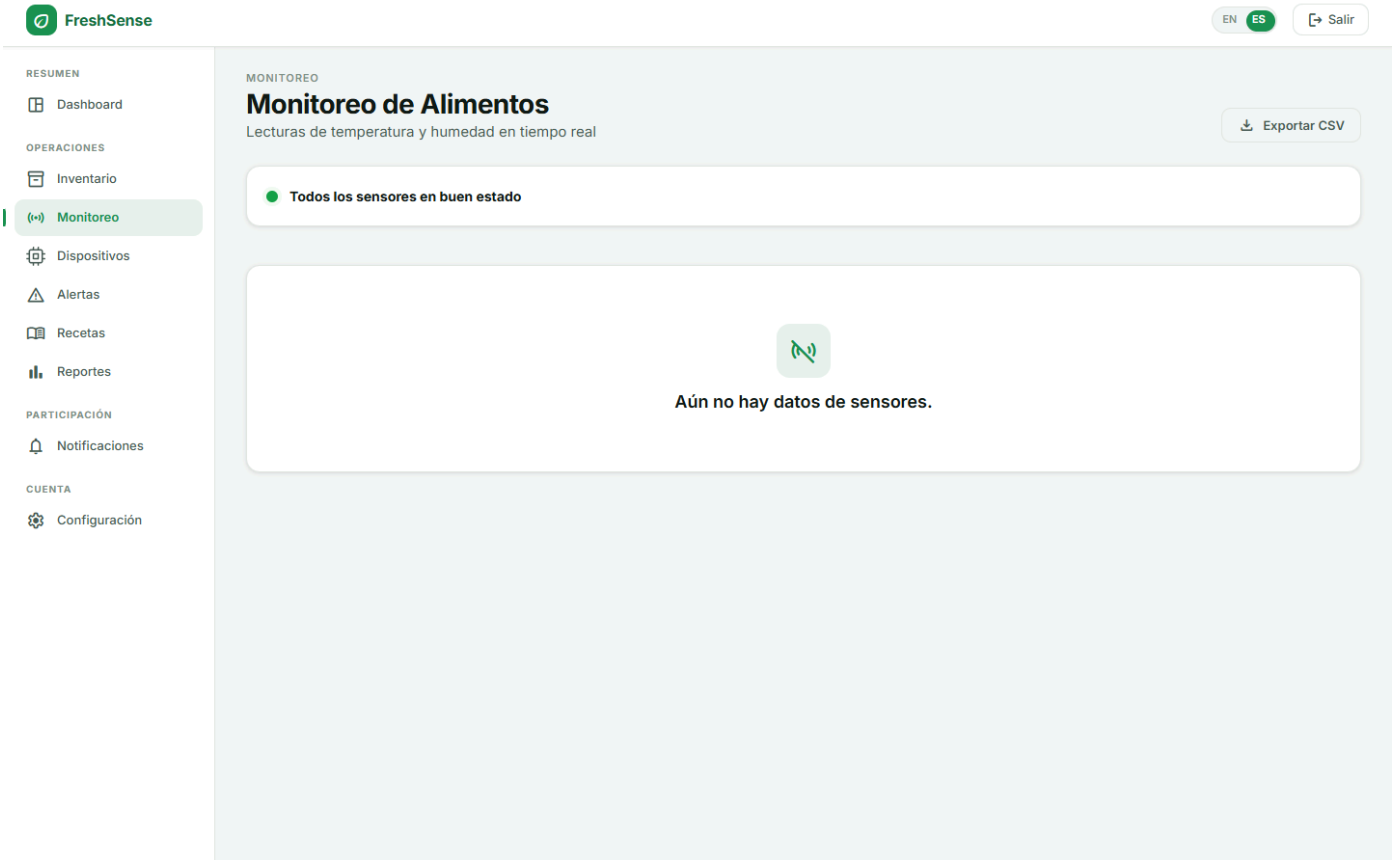
Registro. La misma pantalla sirve para el administrador del restaurante y para el encargado de la distribución de alimentos en fríos.



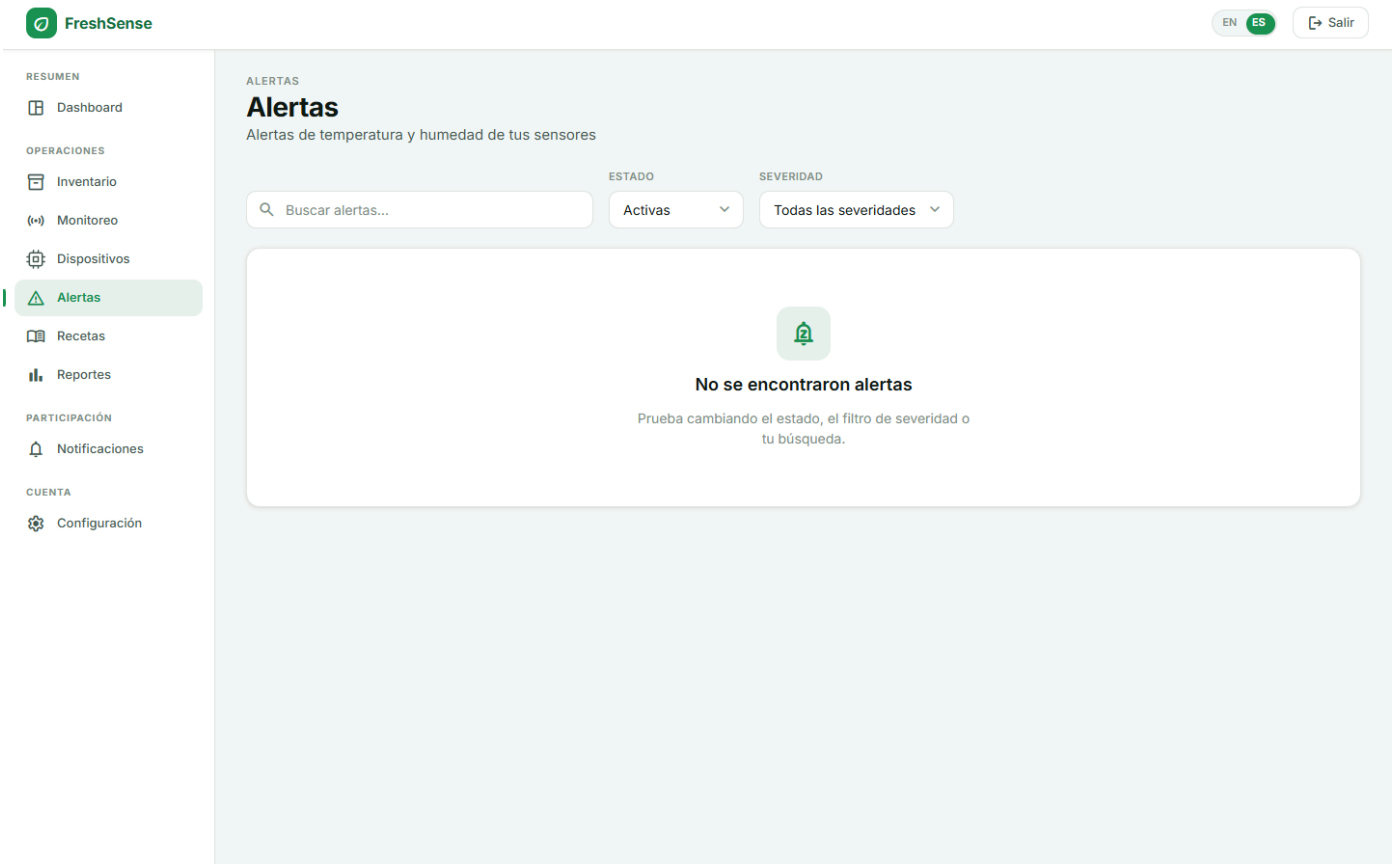
Ingreso. Después de entrar, el panel es el mismo para los dos segmentos.



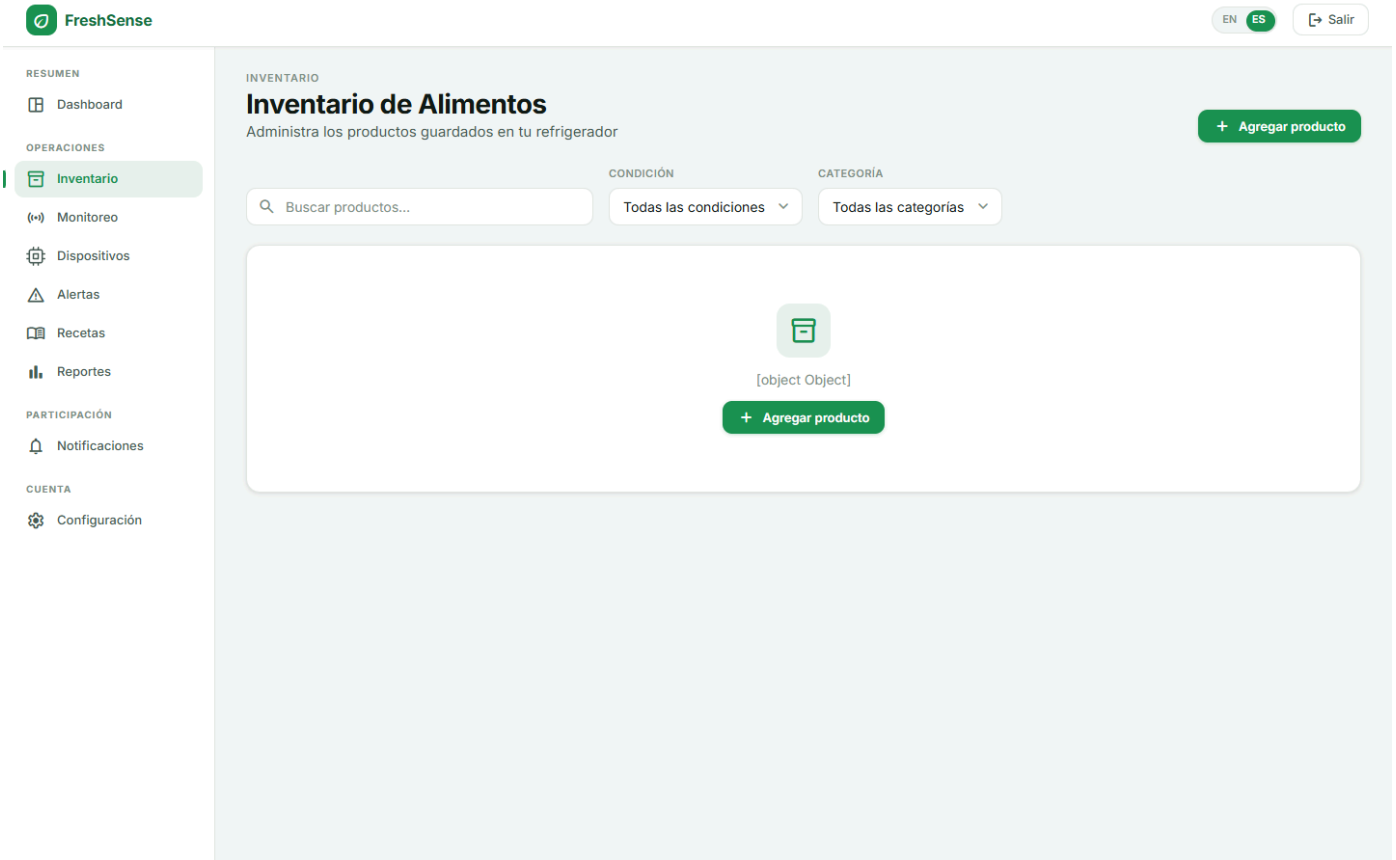
Panel de la cuenta demo. Temperatura y humedad salen sin datos porque no había una lectura reciente.



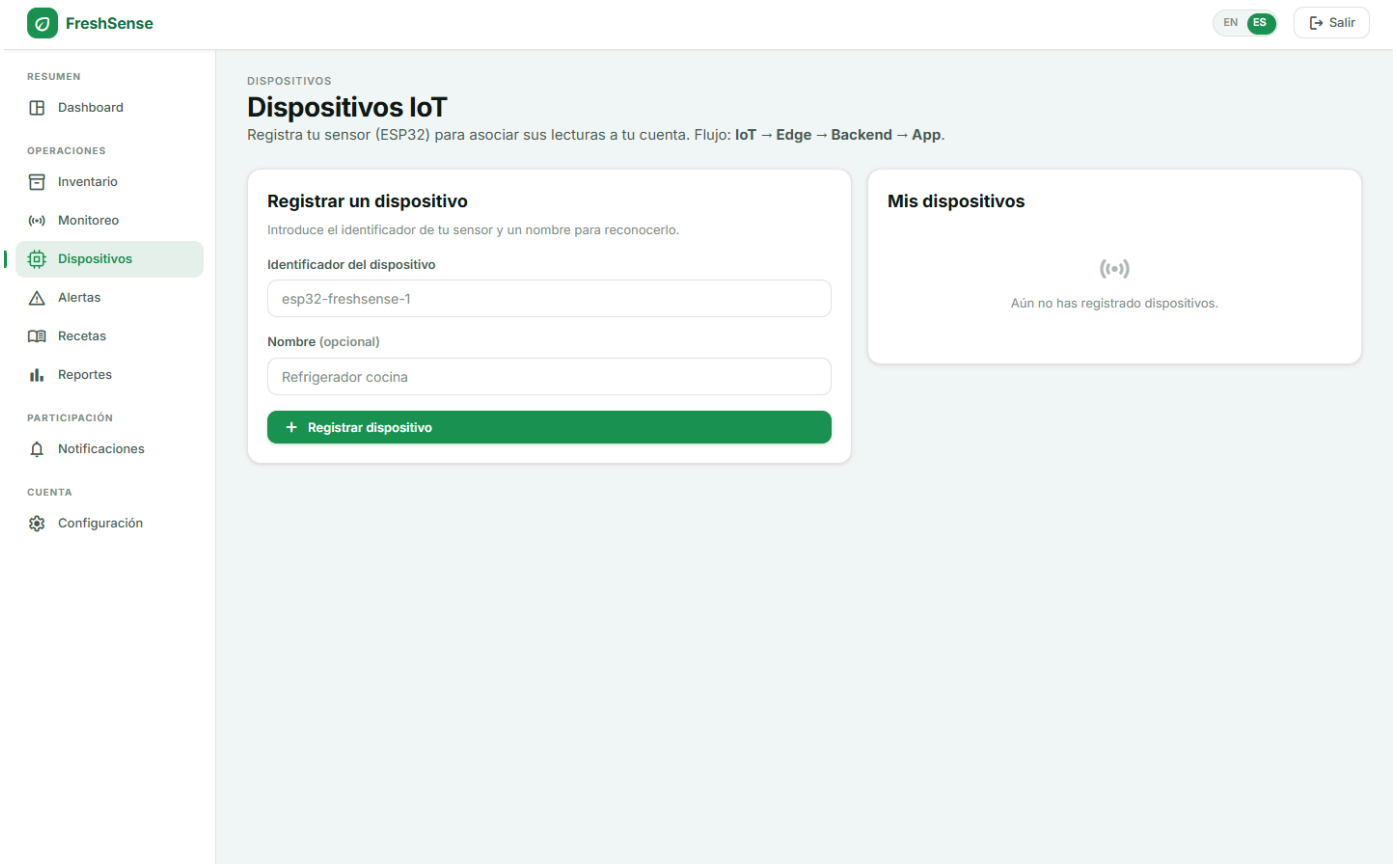
Monitoreo. En el restaurante se lee como la cámara de cocina; en la distribución, como la cámara o vitrina del lote.



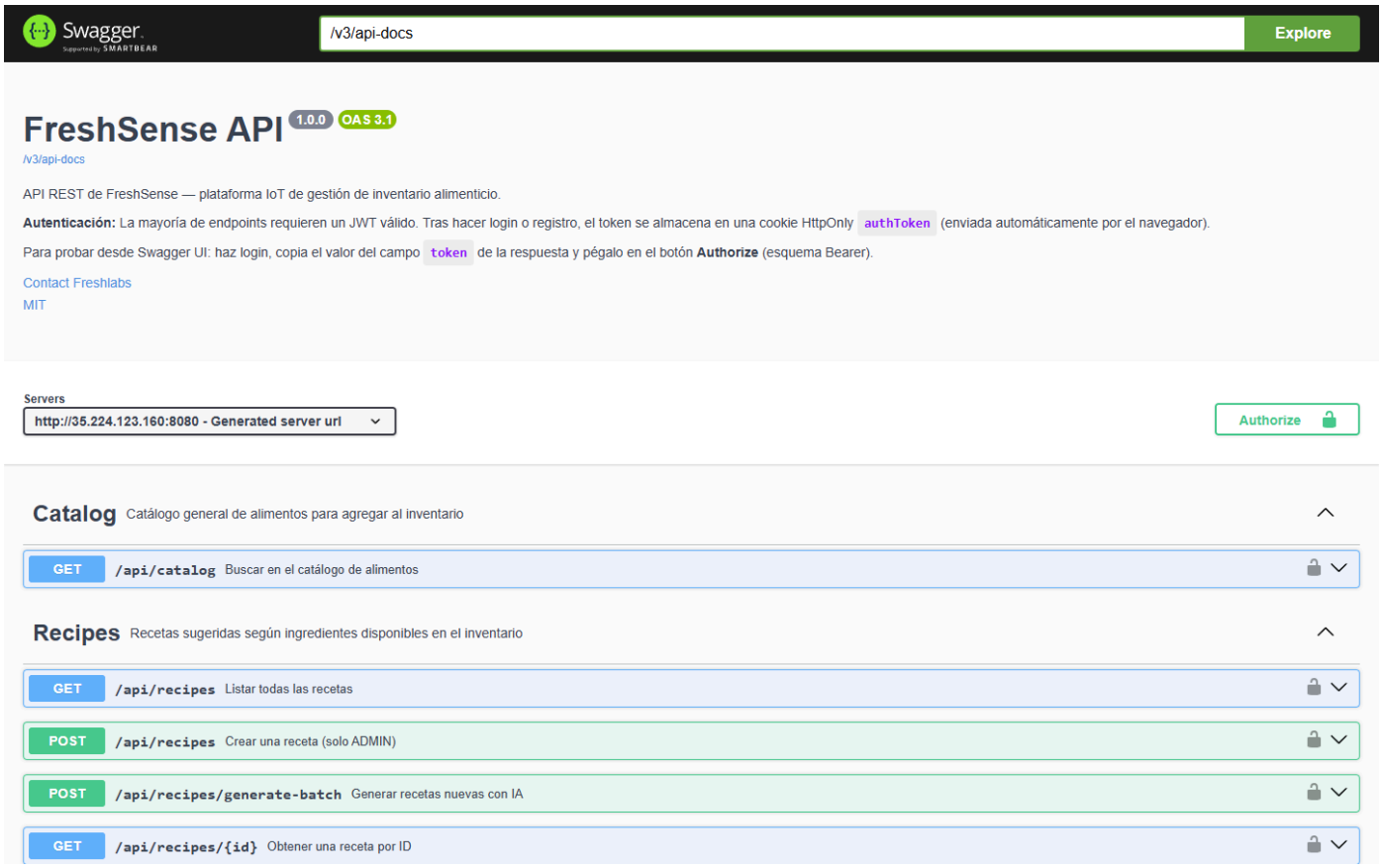
Alertas. Sin aviso activo en la cuenta demo.



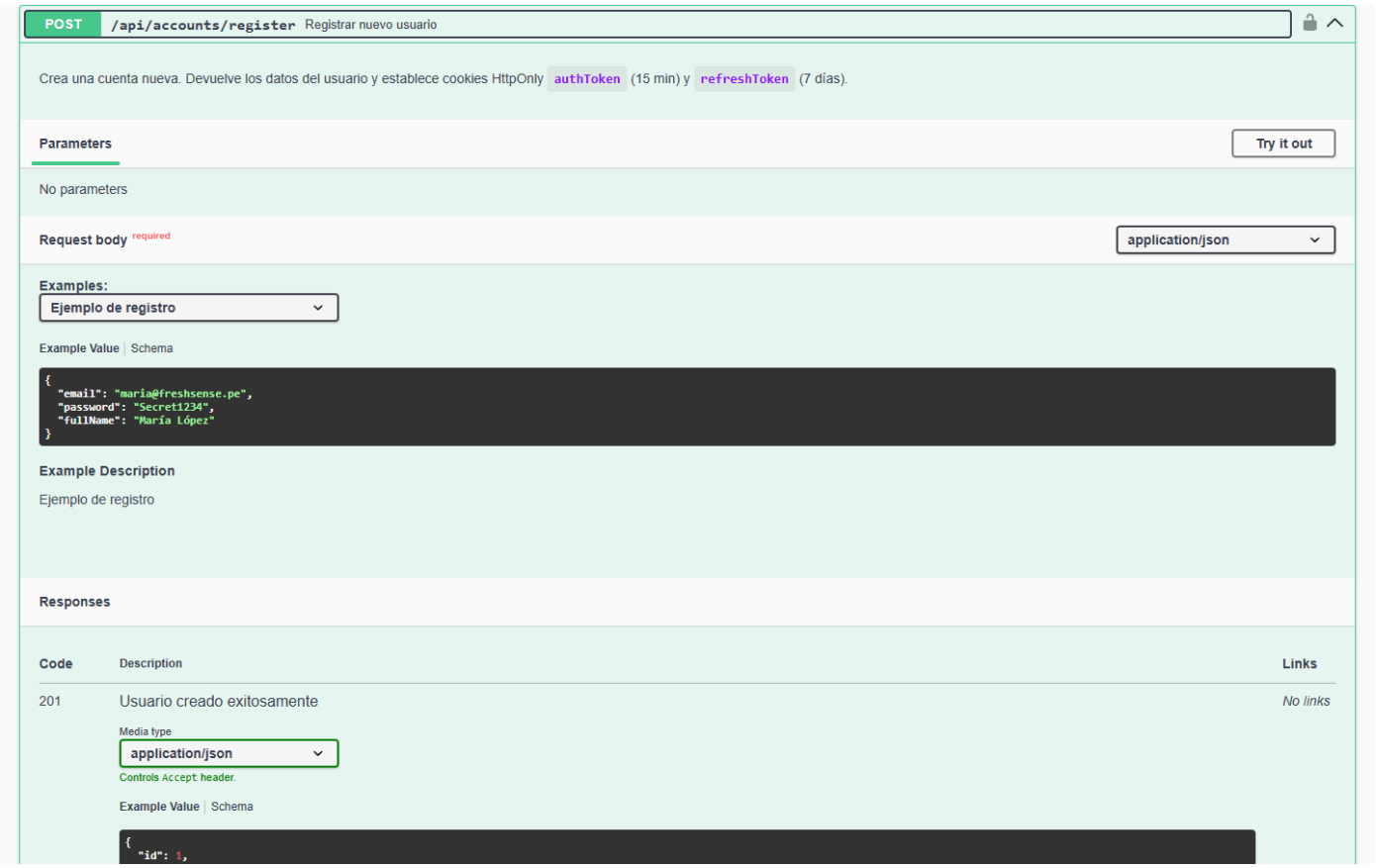
Inventario. Sin productos cargados: insumo en el restaurante, lote en la distribución.



Dispositivos. Aquí se registra el sensor de la cocina o el de la cámara de distribución.



Swagger del API en https://35-224-123-160.sslip.io/swagger-ui/index.html.



Operación de registro en Swagger, la misma que usa `/register`.

Si no llega una lectura reciente, esa fila no se marca en verde. Quien no tiene permiso de facturación igual entra por `/login` al panel. `/plan` queda para la suscripción, no para esta revisión de la cámara.

6.2.1.7. Services Documentation Evidence for Sprint Review

En el Sprint 1 se subió al repositorio de la organización el backend existente de FreshSense como versión base, en un único commit inicial. Este código ya contiene los endpoints documentados con OpenAPI 3 (Swagger) que se describen en esta sección, así que en este sprint no hubo cambios sobre los servicios. Las modificaciones y nuevos endpoints (Reports, Billing y Catalog, entre otros) se registrarán como commits propios en los sprints en que se trabajen sus historias de usuario.

- **Swagger UI:** https://35-224-123-160.sslip.io/swagger-ui/index.html
- **Especificación OpenAPI (JSON):** https://35-224-123-160.sslip.io/v3/api-docs

Todas las operaciones intercambian JSON y están protegidas con un token JWT (`Authorization: Bearer <token>`), salvo el registro, el inicio de sesión y la vinculación del dispositivo (`/api/edge/claim`). Los errores siguen los códigos HTTP estándar: `400` (datos inválidos), `401` (sin token o token inválido), `403` (requiere plan premium), `404` (recurso no encontrado) y `409` (correo ya registrado).

Auth – registro, inicio de sesión y gestión de sesión (TS43)

Endpoint	Acción	Verbo	Sintaxis de llamada	Parámetros	Respuesta
`/api/accounts/register`	Registrar nuevo usuario	POST	`POST /api/accounts/register`	Body `UserRegistrationRequest`: `email`, `password` (mín. 8 caracteres, una mayúscula y un dígito), `fullName`	`201` `UserResponse`. `400` datos inválidos; `409` correo ya registrado.
`/api/accounts/login`	Iniciar sesión	POST	`POST /api/accounts/login`	Body `LoginRequest`: `email`, `password`	`200` `UserResponse` con `token` y `refreshToken`. `401` credenciales incorrectas.
`/api/accounts/refresh`	Renovar el access token	POST	`POST /api/accounts/refresh`	—	`200` `UserResponse` con un nuevo `token`.
`/api/accounts/logout`	Cerrar sesión	POST	`POST /api/accounts/logout`	Header `Authorization`	`204` sin contenido.
`/api/accounts/me`	Obtener el usuario autenticado	GET	`GET /api/accounts/me`	Header `Authorization`	`200` `UserResponse` (`token` en `null`).
`/api/accounts/me`	Actualizar el perfil	PUT	`PUT /api/accounts/me`	Body `UpdateProfileRequest`: `fullName`	`200` `UserResponse`.

Endpoint	Acción	Verbo	Sintaxis de llamada	Parámetros	Respuesta
`/api/accounts/change-password`	Cambiar la contraseña	POST	`POST /api/accounts/change-password`	Body `ChangePasswordRequest`: `currentPassword`, `newPassword`	`204` sin contenido. `400` / `401`.

Devices y Edge/IoT – registro del dispositivo e ingesta de lecturas (TS41, US06)

Endpoint	Acción	Verbo	Sintaxis de llamada	Parámetros	Respuesta
`/api/devices`	Registrar un dispositivo en la cuenta	POST	`POST /api/devices`	Body `RegisterDeviceRequest`: `deviceId`, `name`	`200` `DeviceRegistrationResponse`: `deviceId`, `name`, `pairingCode`, `pairingExpiresAt`.
`/api/devices`	Listar los dispositivos de la cuenta	GET	`GET /api/devices`	Header `Authorization`	`200` lista de `DeviceResponse`: `id`, `deviceId`, `name`, `registeredAt`.
`/api/edge/claim`	Vincular el dispositivo con el código de emparejamiento	POST	`POST /api/edge/claim`	Body `ClaimRequest`: `code`	`200` `ClaimResponse`: `deviceId`, `secretKey`. La `secretKey` se usa como `X-Device-Key`.
`/api/edge/readings`	Recibir una lectura enviada por el Edge API	POST	`POST /api/edge/readings`	Header `X-Device-Key`. Body `EdgeReadingRequest`: `deviceId`, `temperature`, `humidity`, `time` (`dd/MM/yyyy HH:mm`), `status`, `category`	`200` `MonitoringReadingDto` con el estado calculado.

Monitoring – lecturas y semáforo (US06, US07)

Endpoint	Acción	Verbo	Sintaxis de llamada	Parámetros	Respuesta
`/api/monitoring`	Listar las lecturas registradas	GET	`GET /api/monitoring`	Header `Authorization`	`200` lista de `MonitoringReadingDto`.
`/api/monitoring`	Registrar una lectura de monitoreo	POST	`POST /api/monitoring`	Body `MonitoringReadingRequest`: `temperature`, `humidity`, `ethyleneLevel`, `oxygenLevel`, `ripeness`, `cleanliness`	`201` `MonitoringReadingDto`. `400` datos inválidos.
`/api/monitoring/latest`	Obtener la última lectura con su color	GET	`GET /api/monitoring/latest`	Header `Authorization`	`200` `MonitoringReadingDto` con `status` (`GREEN`, `YELLOW`, `RED`, `UNKNOWN`). `204` si aún no hay lecturas.

Alerts – alertas de la cadena de frío, vencimiento y stock (US08)

Endpoint	Acción	Verbo	Sintaxis de llamada	Parámetros	Respuesta
`/api/alerts`	Listar todas las alertas	GET	`GET /api/alerts`	Header `Authorization`	`200` lista de `AlertResponse`.
`/api/alerts`	Crear una alerta	POST	`POST /api/alerts`	Body `AlertRequest`: `title`, `message`, `severity` (`INFO`, `WARNING`, `CRITICAL`), `source`, `state`, `timeAgo`	`200` `AlertResponse`.
`/api/alerts/{id}`	Actualizar una alerta (por ejemplo, marcarla como resuelta)	PUT	`PUT /api/alerts/{id}`	Path `id` (int64). Body `AlertRequest`	`200` `AlertResponse`. `404` si la alerta no existe.

Notifications – envío y gestión de notificaciones (TS42)

Endpoint	Acción	Verbo	Sintaxis de llamada	Parámetros	Respuesta
`/api/notifications/send`	Enviar una notificación a un usuario	POST	`POST /api/notifications/send`	Body `SendNotificationRequest`: `userId`, `title`, `message`, `alertType`, `channelType` (`IN_APP`, `EMAIL`, `PUSH`, `VOICE`)	`200` objeto con el resu del envío.
`/api/notifications`	Obtener la bandeja de notificaciones in-app	GET	`GET /api/notifications`	Header `Authorization`	`200` lista de `NotificationRespon` `id`, `title`, `message`, `alertType`, `read`, `createdAt`.

Endpoint	Acción	Verbo	Sintaxis de llamada	Parámetros	Respuesta
`/api/notifications/{id}/read`	Marcar una notificación como leída	PATCH	`PATCH /api/notifications/{id}/read`	Path `id` (int64)	`200`.
`/api/notifications/preferences`	Obtener las preferencias de notificación	GET	`GET /api/notifications/preferences`	Header `Authorization`	`200` `NotificationPrefer`
`/api/notifications/preferences`	Actualizar las preferencias de notificación	PUT	`PUT /api/notifications/preferences`	Body `NotificationPreferenceRequest`: `inAppEnabled`, `emailEnabled`, `pushEnabled`, `quietStart`, `quietEnd`	`200` `NotificationPrefer`

Inventory – productos del inventario

Endpoint	Acción	Verbo	Sintaxis de llamada	Parámetros	Respuesta
`/api/products`	Listar los productos	GET	`GET /api/products`	Header `Authorization`	`200` lista de `ProductResponse`.
`/api/products`	Registrar un producto	POST	`POST /api/products`	Body `ProductRequest`: `name`, `description`, `category`, `quantity`, `imageUrl`, `expirationDate` (`yyyy-MM-dd`)	`201` `ProductResponse`. `400` datos inválidos.
`/api/products/{id}`	Actualizar parcialmente un producto	PATCH	`PATCH /api/products/{id}`	Path `id` (int64). Body `UpdateProductRequest` (todos los campos opcionales)	`200` `ProductResponse`. `404`.
`/api/products/{id}`	Eliminar un producto	DELETE	`DELETE /api/products/{id}`	Path `id` (int64)	`204` sin contenido. `404`.

Recipes – recetas y generación con IA (recipes-service)

Endpoint	Acción	Verbo	Sintaxis de llamada	Parámetros	Respuesta
`/api/recipes/generate-batch`	Generar un lote de recetas con IA (OpenAI) a partir de los insumos	POST	`POST /api/recipes/generate-batch`	Header `Authorization`	`201` recetas generadas.
`/api/recipes`	Listar las recetas	GET	`GET /api/recipes`	Header `Authorization`	`200` lista de `RecipeResponse`.
`/api/recipes`	Crear una receta	POST	`POST /api/recipes`	Body `CreateRecipeRequest`: `title`, `description`, `image`, `rating` (1–5), `level` (`Easy`, `Medium`, `Hard`), `type` (`Vegetarian`, `Vegan`, `Omnivore`), `time`, `ingredients[]`, `steps[]`	`201` `RecipeResponse`. `403` si el plan no lo permite.
`/api/recipes/{id}`	Obtener el detalle de una receta	GET	`GET /api/recipes/{id}`	Path `id` (int64)	`200` `RecipeResponse`. `404`.
`/api/recipes/premium`	Listar recetas premium	GET	`GET /api/recipes/premium`	Header `Authorization`	`200` recetas premium. `403` si el usuario no es premium.

Ejemplos de request y response con datos de muestra

Registro – `POST /api/accounts/register`

```
// Request
{ "email": "jose.jimenez@restaurante.pe", "password": "Fresh2026", "fullName": "José Jiménez" }

// Response 201
{ "id": 12, "email": "jose.jimenez@restaurante.pe", "fullName": "José Jiménez",
  "role": "USER_STANDARD", "token": "eyJhbGciOi...", "refreshToken": "eyJhbGciOi..." }
```

El API crea la cuenta con el rol `USER_STANDARD` y devuelve el token, así que el usuario entra a la aplicación sin volver a iniciar sesión.

Lectura del dispositivo – `POST /api/edge/readings`

```
// Header: X-Device-Key: <secretKey obtenida en /api/edge/claim>
// Request
{ "deviceId": "FS-ESP32-001", "temperature": 7.8, "humidity": 82.5,
  "time": "08/10/2026 14:30", "category": "Lácteos" }

// Response 200
{ "id": 245, "temperature": 7.8, "humidity": 82.5, "recordedAt": "2026-10-08T14:30:00",
  "deviceId": "FS-ESP32-001", "status": "YELLOW", "category": "Lácteos" }
```

El Edge API reenvía la lectura del ESP32 simulado en Wokwi. El backend la guarda y le asigna el color del semáforo (`status`): en este ejemplo, `YELLOW` porque la temperatura se acerca al límite seguro.

Commits relacionados con la documentación

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
1ASI0572-2620-8725/BackEnd	main	cd732db	first commit	Carga inicial del backend existente de FreshSense (API REST Spring Boot con documentación OpenAPI) como base del proyecto.	08/10/2026

6.2.1.8. Software Deployment Evidence for Sprint Review

Durante el Sprint 1 de FreshSense, el equipo realizó actividades de configuración y despliegue de los productos digitales que conforman la primera versión de la solución. Estas actividades comprendieron la publicación del Landing Page mediante GitHub Pages, la preparación de infraestructura en Google Cloud Compute Engine y el despliegue de la aplicación web Angular y los servicios Backend desarrollados con Spring Boot.

Asimismo, se configuró Nginx como servidor web y proxy inverso para facilitar la comunicación entre los componentes de la solución. El proceso contempló la compilación de los servicios, la transferencia de archivos ejecutables al servidor y la verificación de accesibilidad de los componentes desplegados.

Resumen de los componentes de despliegue

Producto	Plataforma	Configuración
Landing Page	GitHub Pages	Publicación del sitio web estático desde el repositorio de GitHub
Frontend Web Application	Google Cloud Compute Engine	Aplicación Angular servida mediante Nginx
Backend REST API	Google Cloud Compute Engine	Servicios Spring Boot ejecutados en la máquina virtual
Base de datos	MySQL 8	Almacenamiento de información en el entorno del servidor
Documentación de servicios	Swagger UI	Acceso a la documentación de los endpoints REST

1. Despliegue del Landing Page

El Landing Page de FreshSense se publicó utilizando GitHub Pages, a partir del repositorio que contiene los archivos HTML, CSS y JavaScript del sitio web. Esta plataforma permite poner a disposición de los visitantes la información sobre la propuesta de valor, las funcionalidades y los servicios ofrecidos por FreshSense.

La publicación se realizó desde la rama `main`, utilizando la configuración de GitHub Pages descrita en la sección 6.1.4. El sitio web se encuentra documentado en la siguiente dirección:

URL: https://1asi0572-2620-8725.github.io/landing-page/

Figura 1. Landing Page de FreshSense publicada mediante GitHub Pages.

2. Preparación y compilación del Backend

Para el despliegue de los servicios Backend, el equipo utilizó una máquina virtual de Google Cloud Compute Engine con Rocky Linux 9. De acuerdo con la configuración definida en la sección 6.1.4, el entorno incorpora Java 21, MySQL 8 y Nginx.

Como parte del proceso, se realizó la compilación de los proyectos Spring Boot mediante Maven, generando los archivos ejecutables JAR correspondientes al Backend principal y a los servicios complementarios de alertas y recetas.

La evidencia presentada muestra el resultado `BUILD SUCCESS`, que confirma que el proceso de compilación mostrado finalizó satisfactoriamente.

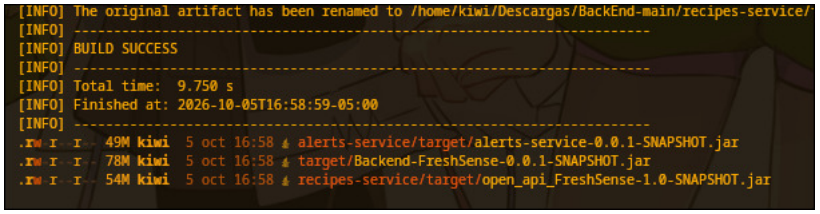


Figura 2. Compilación satisfactoria de los servicios Backend mediante Maven y generación de archivos JAR.

3. Transferencia de archivos y preparación del servidor

Después de la compilación, se preparó el directorio `/opt/freshsense` de la máquina virtual para almacenar los archivos ejecutables de los servicios.

Los archivos JAR fueron transferidos al entorno de despliegue, donde se realiza su ejecución y configuración según los puertos establecidos para cada servicio. Esta organización permite mantener los componentes Backend en una ubicación definida dentro del servidor.

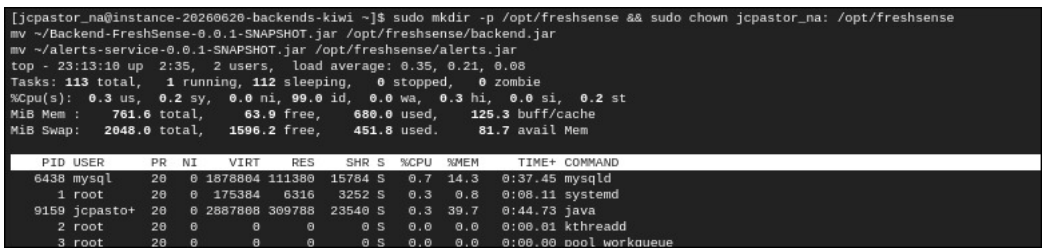


Figura 3. Preparación del directorio de despliegue y transferencia de archivos ejecutables a Google Cloud Compute Engine.

4. Configuración del servidor web y verificación del despliegue

Para publicar la aplicación web y permitir su comunicación con los servicios Backend, se utilizó Nginx como servidor web y proxy inverso. Esta configuración permite atender las solicitudes de la aplicación Angular y redirigir las peticiones del API REST hacia los servicios correspondientes.

Adicionalmente, la configuración de despliegue contempla el acceso mediante HTTPS, utilizando el dominio `35-224-123-160.sslip.io`, según lo establecido en la sección 6.1.4.

Durante la verificación del entorno se ejecutaron comprobaciones de configuración de Nginx y solicitudes HTTP a los componentes del sistema. La evidencia muestra una validación satisfactoria de la configuración del servidor y respuestas HTTP 200 en las comprobaciones realizadas para Backend, Frontend y API.

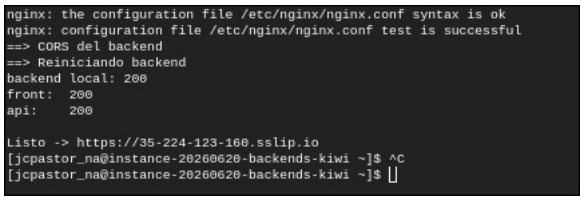


Figura 4. Validación de la configuración de Nginx y comprobación de accesibilidad de los componentes desplegados.

5. Acceso a los productos desplegados

Las siguientes direcciones corresponden a los servicios publicados e identificados en la configuración del proyecto.

Producto	URL
Landing Page	https://1asi0572-2620-8725.github.io/landing-page/
Aplicación web	https://35-224-123-160.sslip.io/login
Documentación Swagger	https://35-224-123-160.sslip.io/swagger-ui/index.html
Especificación OpenAPI	https://35-224-123-160.sslip.io/v3/api-docs

Estas direcciones permiten acceder a los productos principales de la solución y verificar la disponibilidad de sus interfaces. Las comprobaciones HTTP presentadas constituyen evidencia inicial de accesibilidad, mientras que la validación funcional de los recorridos de usuario y endpoints se documenta en las secciones de Testing y Execution Evidence del Sprint.

6. Estado de los componentes IoT

La arquitectura de FreshSense contempla el uso de un ESP32 con sensor DHT22 y un Edge API encargado de transmitir las lecturas ambientales hacia el Backend. La configuración prevista para estos componentes se describe en la sección 6.1.4.

La evidencia de instalación, ejecución e integración de estos componentes se incorporará cuando se disponga de las capturas correspondientes al entorno IoT utilizado durante el Sprint.

En conjunto, las evidencias presentadas documentan los procesos de compilación, preparación del servidor, transferencia de ejecutables y comprobación básica de accesibilidad de los componentes web y Backend. Estos avances constituyen la base técnica para continuar con la integración y validación de las funcionalidades de FreshSense en los siguientes incrementos.

6.2.1.9. Team Collaboration Insights during Sprint

Durante el Sprint 1, el equipo trabajó en la organización de GitHub `1ASI0572-2620-8725`, con un repositorio por producto. Cada integrante desarrolló sus tareas en ramas `feature/*`, las integró a `develop` mediante Pull Requests y, al cerrar el Sprint, se pasaron a `main` para el despliegue.

Repositorios del Sprint 1:

6.3. Validation Interviews

6.3.1. Diseño de Entrevistas

6.3.2. Registro de Entrevistas

6.3.3. Evaluaciones según heurísticas

6.4. Video About-the-Product

Conclusiones

El análisis realizado permitió identificar que los propietarios y administradores de restaurantes, y los encargados de negocios de distribución de alimentos en fríos, enfrentan dificultades relacionadas con el control constante de sus productos perecibles y las condiciones necesarias para su adecuada conservación. El deterioro de los alimentos, las fallas en la cadena de frío y la falta de información oportuna pueden generar mermas y pérdidas económicas que afectan directamente la rentabilidad del negocio. Asimismo, se identificó la necesidad de contar con una solución tecnológica sencilla que facilite el monitoreo del inventario, permita recibir alertas ante posibles riesgos y proporcione información útil para tomar decisiones oportunas. En este contexto, FreshSense busca responder a estas necesidades mediante una herramienta que contribuya a reducir las pérdidas, mejorar el control de los productos y optimizar la gestión del negocio.

Video About-the-Team

Bibliografía

Anexos

Link a repositorio de github: https://github.com/1ASI0572-2620-8725/report